

Cryptnox SDK for C++ Manual

Release 1.0.0

June 20, 2026

1 cryptnox-sdk-cpp **1**

1.0.0.1 cryptnox-sdk-cpp . . . 1

1.0.1 Used by . . . 1

1.0.2 Porting to a new platform . . . 2

1.0.3 What's inside . . . 2

1.0.4 Integrating the core . . . 2

1.0.5 Building standalone . . . 3

1.0.6 Documentation . . . 3

1.0.7 License . . . 3

2 Topic Documentation **4**

2.1 Public API . . . 4

2.1.1 Detailed Description . . . 4

2.2 Secure channel protocol . . . 4

2.2.1 Detailed Description . . . 4

2.3 Adapter interfaces . . . 4

2.3.1 Detailed Description . . . 5

2.4 Utilities & shared definitions . . . 5

2.4.1 Detailed Description . . . 5

2.4.2 Enumeration Type Documentation . . . 5

2.4.2.1 CW_Curve . . . 5

2.4.3 Variable Documentation . . . 6

2.4.3.1 CW_TRON_DERIVE_PATH . . . 6

3 Directory Documentation **7**

3.1 tests Directory Reference . . . 7

4 Class Documentation **8**

4.1 __FlashStringHelper Class Reference . . . 8

4.1.1 Detailed Description . . . 8

4.2 CryptnoxWallet Class Reference . . . 8

4.2.1 Detailed Description . . . 10

4.2.2 Constructor & Destructor Documentation . . . 10

4.2.2.1 CryptnoxWallet() [1/2] . . . 10

4.2.2.2 CryptnoxWallet() [2/2] . . . 10

4.2.3 Member Function Documentation . . . 10

4.2.3.1 begin() . . . 10

4.2.3.2 buildSignPayload() . . . 11

4.2.3.3 connect() . . . 11

4.2.3.4 debugPrintSignature() . . . 11

4.2.3.5 disconnect() . . . 12

4.2.3.6 establishSecureChannel() . . . 12

© 2026 Cryptnox SA

4.2.3.7 extractRawSignature() . . . 12
4.2.3.8 getCardInfo() . . . 13
4.2.3.9 getPublicKey() [1/2] . . . 13
4.2.3.10 getPublicKey() [2/2] . . . 14
4.2.3.11 isSecureChannelOpen() . . . 14
4.2.3.12 operator=() . . . 15
4.2.3.13 parseDerSignature() . . . 15
4.2.3.14 printPN532FirmwareVersion() . . . 15
4.2.3.15 sendSignApdu() . . . 15
4.2.3.16 sign() . . . 16
4.2.3.17 validateSignRequest() . . . 17
4.2.3.18 verifyPin() . . . 17
4.2.3.19 writeUserData() . . . 17
4.2.4 Member Data Documentation . . . 18
4.2.4.1 _logger . . . 18
4.2.4.2 _platform . . . 18
4.2.4.3 _secure . . . 18
4.3 CW_CardInfo Struct Reference . . . 19
4.3.1 Detailed Description . . . 19
4.3.2 Constructor & Destructor Documentation . . . 19
4.3.2.1 CW_CardInfo() . . . 19
4.3.3 Member Data Documentation . . . 19
4.3.3.1 email . . . 19
4.3.3.2 name . . . 19
4.4 CW_CryptoProvider Class Reference . . . 19
4.4.1 Detailed Description . . . 20
4.4.2 Constructor & Destructor Documentation . . . 20
4.4.2.1 ~CW_CryptoProvider() . . . 20
4.4.3 Member Function Documentation . . . 21
4.4.3.1 aesCbcDecrypt() . . . 21
4.4.3.2 aesCbcEncrypt() . . . 21
4.4.3.3 ecdh() . . . 22
4.4.3.4 ecdsaVerify() . . . 22
4.4.3.5 makeKey() . . . 22
4.4.3.6 random() . . . 23
4.4.3.7 sha256() . . . 23
4.4.3.8 sha512() . . . 24
4.5 CW_Logger Class Reference . . . 24
4.5.1 Detailed Description . . . 25
4.5.2 Constructor & Destructor Documentation . . . 25
4.5.2.1 ~CW_Logger() . . . 25

© 2026 Cryptnox SA

4.5.3 Member Function Documentation . . . 25
4.5.3.1 begin() . . . 25
4.5.3.2 print() [1/7] . . . 25
4.5.3.3 print() [2/7] . . . 25
4.5.3.4 print() [3/7] . . . 26
4.5.3.5 print() [4/7] . . . 26
4.5.3.6 print() [5/7] . . . 26
4.5.3.7 print() [6/7] . . . 26
4.5.3.8 print() [7/7] . . . 26
4.5.3.9 println() [1/8] . . . 26
4.5.3.10 println() [2/8] . . . 26
4.5.3.11 println() [3/8] . . . 26
4.5.3.12 println() [4/8] . . . 26
4.5.3.13 println() [5/8] . . . 27
4.5.3.14 println() [6/8] . . . 27
4.5.3.15 println() [7/8] . . . 27
4.5.3.16 println() [8/8] . . . 27
4.6 CW_NfcTransport Class Reference . . . 27
4.6.1 Detailed Description . . . 28
4.6.2 Constructor & Destructor Documentation . . . 28
4.6.2.1 ~CW_NfcTransport() . . . 28
4.6.3 Member Function Documentation . . . 28
4.6.3.1 begin() . . . 28
4.6.3.2 inListPassiveTarget() . . . 28
4.6.3.3 printFirmwareVersion() . . . 28
4.6.3.4 resetReader() . . . 29
4.6.3.5 sendAPDU() . . . 29
4.6.3.6 sendAPDULarge() . . . 29
4.7 CW_Platform Class Reference . . . 30
4.7.1 Detailed Description . . . 30
4.7.2 Constructor & Destructor Documentation . . . 30
4.7.2.1 ~CW_Platform() . . . 30
4.7.3 Member Function Documentation . . . 30
4.7.3.1 sleep_ms() . . . 30
4.8 CW_SecureChannel Class Reference . . . 31
4.8.1 Detailed Description . . . 33
4.8.2 Constructor & Destructor Documentation . . . 33
4.8.2.1 CW_SecureChannel() [1/2] . . . 33
4.8.2.2 CW_SecureChannel() [2/2] . . . 33
4.8.3 Member Function Documentation . . . 33
4.8.3.1 aesCbcDecrypt() . . . 33

© 2026 Cryptnox SA

4.8.3.2 aesCbcEncrypt() [1/2] . . . 34
4.8.3.3 aesCbcEncrypt() [2/2] . . . 35
4.8.3.4 appletMajor() . . . 36
4.8.3.5 begin() . . . 36
4.8.3.6 checkStatusWord() . . . 36
4.8.3.7 extractCardEphemeralKey() . . . 37
4.8.3.8 getCardCertificate() . . . 37
4.8.3.9 getManufacturerCertificate() . . . 37
4.8.3.10 inListPassiveTarget() . . . 38
4.8.3.11 macIvForCounter() . . . 38
4.8.3.12 mutuallyAuthenticate() . . . 38
4.8.3.13 openSecureChannel() . . . 39
4.8.3.14 operator=() . . . 40
4.8.3.15 parseDerSigToRaw() . . . 40
4.8.3.16 preFetchManufacturerCert() . . . 40
4.8.3.17 printFirmwareVersion() . . . 41
4.8.3.18 resetReader() . . . 41
4.8.3.19 selectApdu() . . . 41
4.8.3.20 verifyCertificateChain() . . . 41
4.8.3.21 verifyEcdsaSha256() . . . 42
4.8.4 Member Data Documentation . . . 42
4.8.4.1 _appletMajor . . . 42
4.8.4.2 _cachedMfCertLen . . . 43
4.8.4.3 _crypto . . . 43
4.8.4.4 _driver . . . 43
4.8.4.5 _lastNonce . . . 43
4.8.4.6 _logger . . . 43
4.8.4.7 _platform . . . 43
4.9 CW_SecureSession Struct Reference . . . 43
4.9.1 Detailed Description . . . 44
4.9.2 Constructor & Destructor Documentation . . . 44
4.9.2.1 CW_SecureSession() . . . 44
4.9.3 Member Function Documentation . . . 44
4.9.3.1 clear() . . . 44
4.9.4 Member Data Documentation . . . 44
4.9.4.1 aesKey . . . 44
4.9.4.2 iv . . . 44
4.9.4.3 macCounter . . . 45
4.9.4.4 macKey . . . 45
4.10 CW_SignRequest Struct Reference . . . 45
4.10.1 Detailed Description . . . 46

© 2026 Cryptnox SA

4.10.2 Constructor & Destructor Documentation . . . 46
4.10.2.1 CW_SignRequest() . . . 46
4.10.2.2 ~CW_SignRequest() . . . 46
4.10.3 Member Data Documentation . . . 46
4.10.3.1 derivePath . . . 46
4.10.3.2 derivePathLength . . . 46
4.10.3.3 hash . . . 47
4.10.3.4 hashLength . . . 47
4.10.3.5 keyType . . . 47
4.10.3.6 pin . . . 47
4.10.3.7 pinLessMode . . . 47
4.10.3.8 session . . . 47
4.10.3.9 signatureType . . . 48
4.11 CW_SignResult Struct Reference . . . 48
4.11.1 Detailed Description . . . 48
4.11.2 Constructor & Destructor Documentation . . . 48
4.11.2.1 CW_SignResult() . . . 48
4.11.3 Member Data Documentation . . . 48
4.11.3.1 errorCode . . . 48
4.11.3.2 signature . . . 48
4.12 CW_Tron Class Reference . . . 49
4.12.1 Detailed Description . . . 49
4.12.2 Member Function Documentation . . . 49
4.12.2.1 addressBytesFromPublicKey() . . . 49
4.12.2.2 addressFromPublicKey() . . . 50
4.12.2.3 decodeAddress() . . . 50
4.12.2.4 encodeAddress() . . . 50
4.12.2.5 keccak256() . . . 51
4.13 CW_Utils Class Reference . . . 51
4.13.1 Detailed Description . . . 52
4.13.2 Member Function Documentation . . . 52
4.13.2.1 fill_secure_random() . . . 52
4.13.2.2 safe_memcpy() . . . 52
4.13.2.3 secure_compare() . . . 53
4.13.2.4 secure_wipe() . . . 53
4.14 FakeCrypto Class Reference . . . 54
4.14.1 Detailed Description . . . 55
4.14.2 Member Function Documentation . . . 55
4.14.2.1 aesCbcDecrypt() . . . 55
4.14.2.2 aesCbcEncrypt() . . . 56
4.14.2.3 ecdh() . . . 56

© 2026 Cryptnox SA

4.14.2.4 ecdsaVerify() . . . 57
4.14.2.5 makeKey() . . . 57
4.14.2.6 random() . . . 57
4.14.2.7 sha256() . . . 58
4.14.2.8 sha512() . . . 58
4.14.3 Member Data Documentation . . . 58
4.14.3.1 failMac . . . 59
4.14.3.2 failMacIv . . . 59
4.15 FakeNfc Class Reference . . . 59
4.15.1 Detailed Description . . . 60
4.15.2 Member Function Documentation . . . 60
4.15.2.1 begin() . . . 60
4.15.2.2 inListPassiveTarget() . . . 60
4.15.2.3 printFirmwareVersion() . . . 61
4.15.2.4 resetReader() . . . 61
4.15.2.5 sendAPDU() . . . 61
4.15.3 Member Data Documentation . . . 61
4.15.3.1 appletMajor . . . 61
4.15.3.2 last . . . 61
4.15.3.3 resp . . . 62
4.15.3.4 respLen . . . 62
4.15.3.5 sendOk . . . 62
4.15.3.6 sent . . . 62
4.16 NullLogger Class Reference . . . 62
4.16.1 Detailed Description . . . 63
4.16.2 Member Function Documentation . . . 63
4.16.2.1 begin() . . . 63
4.16.2.2 print() [1/7] . . . 64
4.16.2.3 print() [2/7] . . . 64
4.16.2.4 print() [3/7] . . . 64
4.16.2.5 print() [4/7] . . . 64
4.16.2.6 print() [5/7] . . . 64
4.16.2.7 print() [6/7] . . . 64
4.16.2.8 print() [7/7] . . . 64
4.16.2.9 println() [1/8] . . . 65
4.16.2.10 println() [2/8] . . . 65
4.16.2.11 println() [3/8] . . . 65
4.16.2.12 println() [4/8] . . . 65
4.16.2.13 println() [5/8] . . . 65
4.16.2.14 println() [6/8] . . . 65
4.16.2.15 println() [7/8] . . . 65

© 2026 Cryptnox SA

4.16.2.16 println() [8/8] . . . 65
4.17 NullPlatform Class Reference . . . 66
4.17.1 Detailed Description . . . 66
4.17.2 Member Function Documentation . . . 66
4.17.2.1 sleep_ms() . . . 66
4.18 Rig Struct Reference . . . 67
4.18.1 Detailed Description . . . 67
4.18.2 Constructor & Destructor Documentation . . . 67
4.18.2.1 Rig() . . . 67
4.18.3 Member Function Documentation . . . 68
4.18.3.1 send() . . . 68
4.18.4 Member Data Documentation . . . 68
4.18.4.1 ch . . . 68
4.18.4.2 crypto . . . 68
4.18.4.3 nfc . . . 68
4.18.4.4 s . . . 68

5 File Documentation **69**

5.1 CryptnoxWallet.cpp File Reference . . . 69
5.1.1 Detailed Description . . . 69
5.1.2 Function Documentation . . . 70
5.1.2.1 cwIsEd25519() . . . 70
5.1.2.2 cwMaxEd25519Message() . . . 70
5.1.2.3 cwPubKeyTypeAllowed() . . . 70
5.1.2.4 cwSignNeedsPath() . . . 70
5.2 CryptnoxWallet.cpp . . . 70
5.3 CryptnoxWallet.h File Reference . . . 79
5.3.1 Detailed Description . . . 81
5.3.2 Macro Definition Documentation . . . 81
5.3.2.1 CW_CARD_EMAIL_MAX_LEN . . . 81
5.3.2.2 CW_CARD_NAME_MAX_LEN . . . 81
5.4 CryptnoxWallet.h . . . 81
5.5 CW_CryptoProvider.h File Reference . . . 83
5.5.1 Detailed Description . . . 84
5.6 CW_CryptoProvider.h . . . 84
5.7 CW_Defs.h File Reference . . . 85
5.7.1 Detailed Description . . . 87
5.7.2 Macro Definition Documentation . . . 88
5.7.2.1 CW_AESKEY_SIZE . . . 88
5.7.2.2 CW_CERT_CARD_SIG_INVALID . . . 88
5.7.2.3 CW_CERT_FORMAT_ERROR . . . 88

© 2026 Cryptnox SA

5.7.2.4 CW_CERT_KEY_NOT_FOUND 88
5.7.2.5 CW_CERT_MANUF_SIG_INVALID 88
5.7.2.6 CW_CERT_NONCE_MISMATCH 88
5.7.2.7 CW_CERT_NONCE_SIZE 88
5.7.2.8 CW_CERT_OK 88
5.7.2.9 CW_CONNECT_MAX_ATTEMPTS 89
5.7.2.10 CW_DEBUG_LOGGING 89
5.7.2.11 CW_DER_TAG_INTEGER 89
5.7.2.12 CW_DER_TAG_SEQUENCE 89
5.7.2.13 CW_ED25519_PUBKEY_SIZE 89
5.7.2.14 CW_HASH_SIZE 89
5.7.2.15 CW_INVALID_SESSION 89
5.7.2.16 CW_IV_SIZE 89
5.7.2.17 CW_MACKEY_SIZE 90
5.7.2.18 CW_MANUF_CERT_MAX_BYTES 90
5.7.2.19 CW_MAX_DERIVE_PATH_LENGTH 90
5.7.2.20 CW_MAX_ED25519_MESSAGE_LENGTH 90
5.7.2.21 CW_MAX_ED25519_MESSAGE_LENGTH_CURR 90
5.7.2.22 CW_MAX_PIN_LENGTH 90
5.7.2.23 CW_MIN_APPLET_MAJOR 90
5.7.2.24 CW_MIN_PIN_LENGTH 90
5.7.2.25 CW_NOK 91
5.7.2.26 CW_OK 91
5.7.2.27 CW_RAW_PUBKEY_SIZE 91
5.7.2.28 CW_RAW_SIGNATURE_SIZE 91
5.7.2.29 CW_SEC1_PUBKEY_SIZE 91
5.7.2.30 CW_SEC1_UNCOMPRESSED_TAG 91
5.7.2.31 CW_SECURE_PAYLOAD_MAX 91
5.7.2.32 CW_SIG_R_OFFSET 91
5.7.2.33 CW_SIG_S_OFFSET 92
5.7.2.34 CW_SIGN_CURR_ED25519 92
5.7.2.35 CW_SIGN_CURR_K1 92
5.7.2.36 CW_SIGN_CURR_R1 92
5.7.2.37 CW_SIGN_DERIVE_ED25519 92
5.7.2.38 CW_SIGN_DERIVE_K1 92
5.7.2.39 CW_SIGN_DERIVE_R1 92
5.7.2.40 CW_SIGN_INVALID_PATH 92
5.7.2.41 CW_SIGN_KEY_TOO_SHORT 93
5.7.2.42 CW_SIGN_KEY_TOO_SHORT_WITH_PINLESS_MODE 93
5.7.2.43 CW_SIGN_MESSAGE_TOO_LONG 93
5.7.2.44 CW_SIGN_NO_KEY_LOADED 93

© 2026 Cryptnox SA

5.7.2.45 CW_SIGN_PIN_INCORRECT . . . 93
5.7.2.46 CW_SIGN_PINLESS . . . 93
5.7.2.47 CW_SIGN_PINLESS_K1 . . . 93
5.7.2.48 CW_SIGN_SIG_ECDSA_EOSIO . . . 93
5.7.2.49 CW_SIGN_SIG_ECDSA_LOW_S . . . 93
5.7.2.50 CW_SIGN_SIG_EDDSA . . . 94
5.7.2.51 CW_SIGN_SIG_SCHNORR_BIP340 . . . 94
5.7.2.52 CW_SIGN_UNSUPPORTED_KEY_TYPE . . . 94
5.7.2.53 CW_SIGN_WITH_PIN . . . 94
5.7.2.54 CW_USER_DATA_PAGE_SIZE . . . 94
5.7.2.55 CW_VERIFY_CERT . . . 94
5.8 CW_Defs.h . . . 94
5.9 CW_Logger.h File Reference . . . 97
5.9.1 Detailed Description . . . 98
5.10 CW_Logger.h . . . 98
5.11 CW_NfcTransport.h File Reference . . . 99
5.11.1 Detailed Description . . . 100
5.12 CW_NfcTransport.h . . . 100
5.13 CW_Platform.h File Reference . . . 101
5.13.1 Detailed Description . . . 102
5.14 CW_Platform.h . . . 102
5.15 CW_SecureChannel.cpp File Reference . . . 103
5.15.1 Detailed Description . . . 104
5.15.2 Macro Definition Documentation . . . 104
5.15.2.1 AES_BLOCK_SIZE . . . 104
5.15.2.2 APDU_HEADER_LEN . . . 104
5.15.2.3 APDU_LC_LEN . . . 105
5.15.2.4 APPLET_MAJOR_V2 . . . 105
5.15.2.5 CARDEPHEMERALPUBKEY_SIZE . . . 105
5.15.2.6 CLIENT_PRIVATE_KEY_SIZE . . . 105
5.15.2.7 CLIENT_PUBLIC_KEY_SIZE . . . 105
5.15.2.8 COMMON_PAIRING_DATA . . . 105
5.15.2.9 DER_BIT_UNUSED_ZERO . . . 105
5.15.2.10 DER_EC_POINT_BYTES . . . 105
5.15.2.11 DER_EC_UNCOMPRESSED . . . 105
5.15.2.12 DER_LEN_LONG_1 . . . 105
5.15.2.13 DER_LEN_LONG_2 . . . 106
5.15.2.14 DER_LEN_LONG_FLAG . . . 106
5.15.2.15 DER_TAG_BIT_STRING . . . 106
5.15.2.16 DER_TAG_CTX0 . . . 106
5.15.2.17 DER_TAG_SEQUENCE . . . 106

© 2026 Cryptnox SA

5.15.2.18 ENC_BUF_MAX_LEN . . . 106
5.15.2.19 GETCARDCERTIFICATE_IN_BYTES . . . 106
5.15.2.20 INPUT_BUFFER_LIMIT . . . 106
5.15.2.21 MAC_APDU_LEN . . . 106
5.15.2.22 MAX_MAC_DATA_LEN . . . 106
5.15.2.23 MUTUALAUTH_CHALLENGE_IN_BYTES . . . 107
5.15.2.24 OPENSECURECHANNEL_SALT_IN_BYTES . . . 107
5.15.2.25 RANDOM_BYTES . . . 107
5.15.2.26 RESPONSE_GETCARDCERTIFICATE_IN_BYTES . . . 107
5.15.2.27 RESPONSE_GETMANUFACTURERCERT_PAGE_IN_BYTES . . . 107
5.15.2.28 RESPONSE_OPENSECURECHANNEL_IN_BYTES . . . 107
5.15.2.29 RESPONSE_SELECT_IN_BYTES . . . 107
5.15.2.30 RESPONSE_STATUS_WORDS_IN_BYTES . . . 107
5.15.2.31 SEND_APDU_MAX_LEN . . . 107
5.15.3 Function Documentation . . . 108
5.15.3.1 derReadLength() . . . 108
5.15.3.2 derSkipField() . . . 108
5.15.3.3 derWalkMfCert() . . . 108
5.15.4 Variable Documentation . . . 108
5.15.4.1 s_apduBuf . . . 108
5.15.4.2 s_dataBuf . . . 108
5.15.4.3 s_macBuf . . . 108
5.15.4.4 s_mfCertBuf . . . 109
5.16 CW_SecureChannel.cpp . . . 109
5.17 CW_SecureChannel.h File Reference . . . 125
5.17.1 Detailed Description . . . 126
5.17.2 Macro Definition Documentation . . . 126
5.17.2.1 CW_PAIRING_DATA . . . 126
5.17.2.2 CW_PAIRING_DATA_BYTES . . . 126
5.18 CW_SecureChannel.h . . . 126
5.19 CW_Tron.cpp File Reference . . . 128
5.19.1 Detailed Description . . . 129
5.19.2 Macro Definition Documentation . . . 129
5.19.2.1 BASE58_BASE . . . 129
5.19.2.2 ETH_ADDRESS_BYTES . . . 129
5.19.2.3 KECCAK_DIGEST_BYTES . . . 130
5.19.2.4 KECCAK_RATE_BYTES . . . 130
5.19.2.5 KECCAK_ROUNDS . . . 130
5.19.2.6 KECCAK_STATE_LANES . . . 130
5.19.2.7 PUBLIC_KEY_BYTES . . . 130
5.19.2.8 ROTL64 . . . 130

© 2026 Cryptnox SA

5.19.2.9 TRON_CHECKSUM_BYTES . . . 130
5.19.2.10 TRON_PAYLOAD_BYTES . . . 130
5.19.3 Function Documentation . . . 130
5.19.3.1 base58_decode_payload() . . . 130
5.19.3.2 base58_encode_payload() . . . 131
5.19.3.3 keccak_absorb() . . . 131
5.19.3.4 keccak_f() . . . 131
5.19.3.5 tron_checksum() . . . 132
5.19.4 Variable Documentation . . . 132
5.19.4.1 BASE58_ALPHABET . . . 132
5.19.4.2 KECCAK_RC . . . 132
5.19.4.3 KECCAK_ROT . . . 132
5.20 CW_Tron.cpp . . . 133
5.21 CW_Tron.h File Reference . . . 137
5.21.1 Detailed Description . . . 139
5.21.2 Macro Definition Documentation . . . 139
5.21.2.1 CW_TRON_ADDRESS_BYTES . . . 139
5.21.2.2 CW_TRON_ADDRESS_PREFIX . . . 139
5.21.2.3 CW_TRON_ADDRESS_STR_LEN . . . 140
5.21.2.4 CW_TRON_ADDRESS_STR_SIZE . . . 140
5.21.2.5 CW_TRON_PATH_LENGTH . . . 140
5.22 CW_Tron.h . . . 140
5.23 CW_TrustedKeys.h File Reference . . . 141
5.23.1 Detailed Description . . . 142
5.23.2 Macro Definition Documentation . . . 142
5.23.2.1 CW_TRUSTED_CA_COUNT . . . 142
5.23.3 Variable Documentation . . . 142
5.23.3.1 CW_CA_DLT_PUBKEY . . . 142
5.23.3.2 CW_TRUSTED_CA_KEYS . . . 142
5.24 CW_TrustedKeys.h . . . 142
5.25 CW_Utils.cpp File Reference . . . 143
5.25.1 Detailed Description . . . 143
5.26 CW_Utils.cpp . . . 144
5.27 CW_Utils.h File Reference . . . 144
5.27.1 Detailed Description . . . 145
5.28 CW_Utils.h . . . 145
5.29 platform_compat.h File Reference . . . 146
5.29.1 Detailed Description . . . 147
5.29.2 Macro Definition Documentation . . . 147
5.29.2.1 BIN . . . 147
5.29.2.2 DEC . . . 147

© 2026 Cryptnox SA

5.29.2.3 F . . . 147
5.29.2.4 HEX . . . 147
5.29.2.5 OCT . . . 147
5.30 platform_compat.h . . . 148
5.31 README.md File Reference . . . 148
5.32 tests/test_pr28.cpp File Reference . . . 148
5.32.1 Macro Definition Documentation . . . 150
5.32.1.1 CHECK . . . 150
5.32.2 Function Documentation . . . 150
5.32.2.1 check_ed25519_applet_gate() . . . 150
5.32.2.2 check_pubkey_allow_list() . . . 150
5.32.2.3 check_session_clearing() . . . 150
5.32.2.4 isCleared() . . . 150
5.32.2.5 main() . . . 151
5.32.2.6 openSession() . . . 151
5.32.2.7 pathFor() . . . 151
5.32.2.8 selectOnly() . . . 151
5.32.2.9 signWith() . . . 151
5.32.3 Variable Documentation . . . 151
5.32.3.1 DATA . . . 151
5.32.3.2 HDR . . . 151
5.32.3.3 logger . . . 152
5.32.3.4 PATH . . . 152
5.32.3.5 platform . . . 152
5.33 test_pr28.cpp . . . 152

© 2026 Cryptnox SA

Chapter 1

cryptnox-sdk-cpp

1.0.0.1 cryptnox-sdk-cpp

Platform-independent C++ core SDK for Cryptnox Hardware Wallet

`cryptnox-sdk-cpp` is the **shared C++ core SDK** for the **Cryptnox Hardware Wallet**. It implements the card-side protocol — secure channel establishment (SELECT → certificate → ECDH → mutual auth), APDU framing, PIN verification, signing, and user-data writing — independently of any target platform, NFC reader, or crypto library.

Important

> **This SDK is not usable on its own.** It exposes three abstract interfaces (`CW_NfcTransport`, `CW_CryptoProvider`, `CW_Logger`) that **must be implemented by a host integration**. It ships no transport driver, no crypto backend, and no logging output.

1.0.1 Used by

This core is consumed as a submodule by the platform-specific SDKs:

Integration	Repository
ESP32-S3 (ESP-IDF v5.5)	`embarquech/cryptnox-sdk-esp32`
Arduino R4 (Renesas RA4M1)	`embarquech/cryptnox-sdk-arduino`

If you want to talk to a Cryptnox card on real hardware, **start from one of those repositories**.

© 2026 Cryptnox SA

1.0.2 Porting to a new platform

This repository is intended as the **starting point for porting the SDK to a new platform** (another MCU family, a desktop OS, a different NFC reader, a different crypto backend, etc.).
A port consists of providing concrete implementations of the three adapter interfaces:

Interface	What you must provide
CW_NfcTransport	Driver for your NFC reader (PN532, PN7150, PC/SC, ...)
CW_CryptoProvider	SHA-256/512, AES-CBC, ECDH, EC key generation, RNG (mbedTLS, BearSSL, OpenSSL, hardware peripheral, ...)
CW_Logger	Output sink (UART, stdout, syslog, network, ...)

Then drop this repository in as a submodule (or copy of its sources) inside your project, build it together with your adapters, and instantiate CryptnoxWallet with the three injected dependencies. The existing platform SDKs (cryptnox-sdk-esp32, cryptnox-sdk-arduino) are useful references for a complete port.

1.0.3 What's inside

File	Role
CryptnoxWallet.{h,cpp}	High-level API: begin, connect, verifyPin, sign, get↩ PublicKey, writeUserData, disconnect
CW_SecureChannel.{h,cpp}	Secure channel protocol (mutual auth, session keys, encrypted APDU exchange)
CW_NfcTransport.h	**Adapter interface** — NFC reader contract
CW_CryptoProvider.h	**Adapter interface** — SHA-256/512, AES-CBC, ECDH, EC keygen, RNG
CW_Logger.h	**Adapter interface** — debug/serial output
CW_TrustedKeys.h	Cryptnox CA public keys used to verify card certificates
CW_Tron.{h,cpp}	TRON network: m/44'/195'/0'/0/0 path, Keccak-256, Base58Check address encode/decode
CW_Defs.h, CW_Utils.{h,cpp}	Constants, error codes, small helpers
platform_compat.h	Shim for non-Arduino targets

The three adapter interfaces are the only contract a host must satisfy. Everything else is self-contained.

1.0.4 Integrating the core

A host integration injects its three adapters into CryptnoxWallet:

```
#include "CryptnoxWallet.h"

// MyXxx = concrete adapters provided by the platform SDK:
//   - MyNfcTransport    : public CW_NfcTransport
//   - MyCryptoProvider : public CW_CryptoProvider
//   - MyLogger          : public CW_Logger

MyLogger          logger;
MyCryptoProvider  crypto;
MyNfcTransport    transport(/* platform-specific args */);
CryptnoxWallet    wallet(transport, logger, crypto);

if (!wallet.begin()) { /* reader init failed */ }

CW_SecureSession session;
if (wallet.connect(session)) {
    // wallet.verifyPin(...), wallet.sign(...), wallet.writeUserData(...)
    wallet.disconnect(session);
```

© 2026 Cryptnox SA

```
}
```

Full runnable examples (PIN verify, transaction signing, full ESP-IDF / Arduino boilerplate) live in the platform SDKs linked above.

1.0.5 Building standalone

There is **no standalone build target** in this repository. The CI workflow runs `cppcheck` static analysis only (`.github/workflows/static_analysis.yml`); host-supplied headers (`uECC.h`, `mbedtls/*`, platform logger) are not vendored, so `missingInclude` is suppressed.
To compile and exercise the code, use one of the platform SDKs.

1.0.6 Documentation

The generated documentation for this project is available here.

1.0.7 License

`cryptnox-sdk-cpp` is dual-licensed:

- **LGPL-3.0** for open-source projects and proprietary projects that comply with LGPL requirements
- **Commercial license** for projects that require a proprietary license without LGPL obligations

For commercial inquiries, contact: contact@cryptnox.com

© 2026 Cryptnox SA

Chapter 2

Topic Documentation

2.1 Public API

Types and classes application code interacts with directly.

Classes

- struct CW_CardInfo

  Subset of the Cryptnox card info returned by APDU 0x80FA0000.
- struct CW_SignRequest

  Request parameters for CryptnoxWallet::sign.
- class CryptnoxWallet

  High-level interface for interacting with a Cryptnox Hardware Wallet over NFC.
- struct CW_SecureSession

  Holds cryptographic session state for reentrant secure channel operations.

2.1.1 Detailed Description

Types and classes application code interacts with directly.
Includes CryptnoxWallet (the main entry point), the sign request / result structs, CW_CardInfo, and CW_SecureSession.

2.2 Secure channel protocol

Low-level secure messaging implementation.

Classes

- class CW_SecureChannel

  Implements the Cryptnox secure channel protocol over NFC.

2.2.1 Detailed Description

Low-level secure messaging implementation.
CW_SecureChannel is composed inside CryptnoxWallet and is not normally used directly. Documented here for callers that need to drive the activation sequence manually (e.g. custom retry policies, fuzzing).

2.3 Adapter interfaces

Abstract contracts a host integration must implement.

© 2026 Cryptnox SA

Classes

- class CW_CryptoProvider

  Abstract interface for cryptographic operations used by CW_SecureChannel.
- class CW_Logger

  Abstract interface for serial/debug output.
- class CW_NfcTransport

  Abstract interface for NFC transport operations.
- class CW_Platform

  Abstract interface for platform-specific operations used by the SDK.

2.3.1 Detailed Description

Abstract contracts a host integration must implement.
Provide concrete implementations of CW_NfcTransport, CW_CryptoProvider, CW_Logger, and CW_Platform when porting the SDK to a new MCU or operating system.

2.4 Utilities & shared definitions

Platform-independent helpers and shared constants.

Classes

- class CW_Tron

  TRON address derivation and Base58Check encoding.
- class CW_Utils

  Portable utility functions for cryptographic and security operations.

Enumerations

- enum CW_Curve { CW_CURVE_SECP256R1 = 0 , CW_CURVE_SECP256K1 = 1 }

  Portable curve identifier used throughout the SDK.

Variables

- const uint8_t CW_TRON_DERIVE_PATH [CW_TRON_PATH_LENGTH]

  BIP44 derivation path for TRON: `m/44'/195'/0'/0/0`.

2.4.1 Detailed Description

Platform-independent helpers and shared constants.
Includes CW_Utils (constant-time compare, secure wipe, safe memcpy, RNG), the `CW_CERT_*` / `CW_SIGN_*` error codes, and the trusted-CA key table (CW_TrustedKeys.h).

2.4.2 Enumeration Type Documentation

2.4.2.1 CW_Curve

`enum CW_Curve`
Portable curve identifier used throughout the SDK.
Replaces direct references to uECC_Curve_t at every API boundary so the abstract interfaces (CW_CryptoProvider, CW_SecureChannel) remain decoupled from any specific ECC back-end.

© 2026 Cryptnox SA

Enumerator

CW_CURVE_SECP256R1	NIST P-256 / secp256r1
CW_CURVE_SECP256K1	Koblitz secp256k1

Definition at line 183 of file CW_Defs.h.

2.4.3 Variable Documentation

2.4.3.1 CW_TRON_DERIVE_PATH

```
const uint8_t CW_TRON_DERIVE_PATH[CW_TRON_PATH_LENGTH]  [extern]
```

BIP44 derivation path for TRON: `m/44'/195'/0'/0/0`.
Wire format expected by the card's SIGN command: five big-endian uint32 levels, hardened levels carrying the high bit. Pass as `CW_SignRequest::derivePath` with `keyType` == CW_SIGN_DERIVE_K1.
Definition at line 31 of file CW_Tron.cpp.

© 2026 Cryptnox SA

Chapter 3

Directory Documentation

3.1 tests Directory Reference

Files

- file test_pr28.cpp

© 2026 Cryptnox SA

Chapter 4

Class Documentation

4.1 __FlashStringHelper Class Reference

`#include <platform_compat.h>`

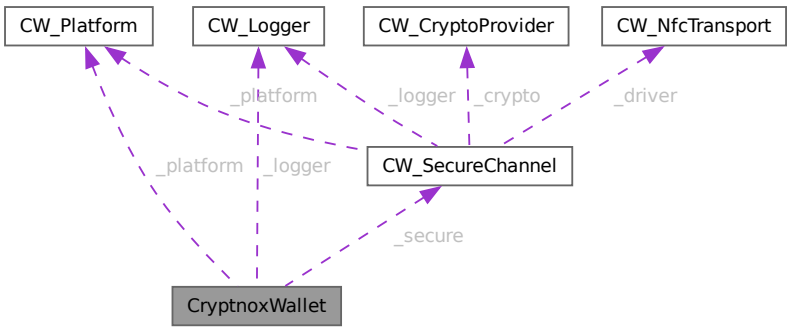
4.1.1 Detailed Description

Definition at line 41 of file platform_compat.h.
The documentation for this class was generated from the following file:

- platform_compat.h

4.2 CryptnoxWallet Class Reference

High-level interface for interacting with a Cryptnox Hardware Wallet over NFC.
`#include <CryptnoxWallet.h>`
Collaboration diagram for CryptnoxWallet:



Public Member Functions

- CryptnoxWallet (CW_NfcTransport &driver, CW_Logger &logger, CW_CryptoProvider &crypto, CW_Platform &platform)

  Construct a CryptnoxWallet.
- CryptnoxWallet (const CryptnoxWallet &)=delete
- CryptnoxWallet & operator= (const CryptnoxWallet &)=delete
- bool begin ()

© 2026 Cryptnox SA

Initialize the NFC module via the underlying transport driver.

- bool connect (CW_SecureSession &session)

  Connect to the Cryptnox card and establish a secure channel.
- bool establishSecureChannel (CW_SecureSession &session)

  Establish a secure channel (SELECT → certificate → ECDH → mutual auth).
- void disconnect (CW_SecureSession &session)

  Disconnect and securely clear the session.
- bool getCardInfo (CW_SecureSession &session, CW_CardInfo *info=NULL)

  Send a secured GET CARD INFO APDU (0x80FA0000) and optionally decode the owner name/email from the response.
- bool verifyPin (CW_SecureSession &session, const uint8_t *pin, uint8_t pinLength)

  Verify the PIN code on the card.
- bool getPublicKey (CW_SecureSession &session, const uint8_t *derivePath, uint8_t derivePath↩ Length, uint8_t *publicKey64)

  Read a public key from the card for a BIP32 derivation path.
- CW_SignResult sign (CW_SignRequest &request)

  Sign a digest or message using a card-resident key.
- bool getPublicKey (CW_SecureSession &session, uint8_t keyType, const uint8_t *derivePath, uint8_t derivePathLength, uint8_t *pubKey, uint16_t pubKeyBufSize, uint16_t &pubKeyLength)

  Read a card-resident public key (GET PUBLIC KEY, APDU 0x80C2).
- bool writeUserData (CW_SecureSession &session, const uint8_t *data, uint16_t dataLength)

  Write user data (WRITE APDU 0x80FC, P1=0 user-data slot, single page).

Static Public Member Functions

- static bool parseDerSignature (const uint8_t *der, uint8_t derLength, uint8_t *r, uint8_t &rLength, uint8_t *s, uint8_t &sLength)

  Parse a DER-encoded ECDSA signature to extract raw r and s values.

Private Member Functions

- bool printPN532FirmwareVersion ()
- bool validateSignRequest (const CW_SignRequest &request, CW_SignResult &result)
- bool sendSignApdu (CW_SignRequest &request, const uint8_t *data, uint16_t dataLength, uint8_t *derResponse, uint16_t derResponseSize, uint16_t &derLength, CW_SignResult &result)
- bool extractRawSignature (const uint8_t *derResponse, uint16_t derLength, CW_SignResult &result)
- void debugPrintSignature (const uint8_t *signature)

Static Private Member Functions

- static bool isSecureChannelOpen (const CW_SecureSession &session)
- static void buildSignPayload (const CW_SignRequest &request, uint8_t *data, uint16_t &data↩ Length)

Private Attributes

- CW_Logger & _logger

  Logging interface.
- CW_Platform & _platform

  Platform abstraction (sleep_ms).
- CW_SecureChannel _secure

  Owned secure channel.

© 2026 Cryptnox SA

4.2.1 Detailed Description

High-level interface for interacting with a Cryptnox Hardware Wallet over NFC.
Manages card connection, secure channel establishment (delegated to CW_SecureChannel), PIN verification, transaction signing, user-data writing, and card-info retrieval.
Dependencies are injected by the caller via the constructor. The class itself only talks to the four abstract adapters, keeping the implementation platform-independent.

Typical lifecycle

```
CryptnoxWallet wallet(transport, logger, crypto, platform);
wallet.begin();

CW_SecureSession session;
if (wallet.connect(session)) {
    wallet.verifyPin(session, pin, pinLen);
    wallet.sign(req);
}
wallet.disconnect(session);   // mandatory -- even on connect() failure
```

Note

Single-shot use — the class is non-copyable. Reuse the same CryptnoxWallet instance across multiple card sessions; do not construct one per APDU.

Definition at line 155 of file CryptnoxWallet.h.

4.2.2 Constructor & Destructor Documentation

4.2.2.1 CryptnoxWallet() [1/2]

```
CryptnoxWallet::CryptnoxWallet (
            CW_NfcTransport & driver,
            CW_Logger & logger,
            CW_CryptoProvider & crypto,
            CW_Platform & platform)
```

Construct a CryptnoxWallet.

Parameters

driver	Reference to the NFC transport implementation.
logger	Reference to the logging implementation.
crypto	Reference to the crypto provider implementation.
platform	Reference to the platform abstraction (for sleep_ms).

Definition at line 79 of file CryptnoxWallet.cpp.
References _logger, _platform, _secure, logger, and platform.
Referenced by CryptnoxWallet(), and operator=().

4.2.2.2 CryptnoxWallet() [2/2]

```
CryptnoxWallet::CryptnoxWallet (
            const CryptnoxWallet & )  [delete]
```

References CryptnoxWallet().

4.2.3 Member Function Documentation

4.2.3.1 begin()

```
bool CryptnoxWallet::begin ()
```

Initialize the NFC module via the underlying transport driver.

© 2026 Cryptnox SA

Returns

true if the module was successfully initialised, false otherwise.

Definition at line 88 of file CryptnoxWallet.cpp.
References _secure, and printPN532FirmwareVersion().

4.2.3.2 buildSignPayload()

```
void CryptnoxWallet::buildSignPayload (
            const CW_SignRequest & request,
            uint8_t * data,
            uint16_t & dataLength)  [static], [private]
```

Definition at line 623 of file CryptnoxWallet.cpp.
References CW_MAX_PIN_LENGTH, CW_USER_DATA_PAGE_SIZE, cwIsEd25519(), cwSignNeedsPath(), CW_SignRequest::derivePath, CW_SignRequest::derivePathLength, CW_SignRequest::hash, CW_SignRequest::hashLength, CW_SignRequest::keyType, CW_SignRequest::pin, CW_SignRequest::pinLessMode, and CW_Utils::safe_memcpy().
Referenced by sign().

4.2.3.3 connect()

```
bool CryptnoxWallet::connect (
            CW_SecureSession & session)
```

Connect to the Cryptnox card and establish a secure channel.
Retries the full card activation sequence up to `CW_CONNECT_MAX_ATTEMPTS` times. On any failure (including transient transport errors) the session is securely wiped before the next attempt so no partial key material can survive a retry (CRIT-04).

Parameters

`out`	*session*	Secure session to populate with derived keys and IV on success; left zero-wiped on failure.

Returns

true if the secure channel was established and `session` is ready for use, false otherwise.

Postcondition

On true: `session` holds valid Kenc / Kmac / IV.

On false: `session` is zero-wiped.

Warning

Always call disconnect() after this — even on failure — to release the reader for the next card cycle.

Definition at line 96 of file CryptnoxWallet.cpp.
References _logger, _platform, _secure, CW_SecureSession::clear(), CW_CONNECT_MAX_ATTEMPTS, establishSecureChannel(), and F.
Referenced by selectOnly().

4.2.3.4 debugPrintSignature()

```
void CryptnoxWallet::debugPrintSignature (
            const uint8_t * signature)  [private]
```

Definition at line 754 of file CryptnoxWallet.cpp.
References _logger, CW_RAW_SIGNATURE_SIZE, F, and HEX.
Referenced by sign().

© 2026 Cryptnox SA

4.2.3.5 disconnect()

```
void CryptnoxWallet::disconnect (
            CW_SecureSession & session)
```

Disconnect and securely clear the session.
Wipes any session keys and resets the NFC reader so the next card detection cycle starts from a clean state.

Parameters

in,out	*session*	Session to clear. Safe to pass a never-connected or partially-connected session.

Precondition

Must be called at the end of every card-processing iteration — including iterations where connect() failed — otherwise the NFC reader may remain in an unresponsive state.

Definition at line 218 of file CryptnoxWallet.cpp.
References _secure, CW_SecureSession::clear(), and isSecureChannelOpen().

4.2.3.6 establishSecureChannel()

```
bool CryptnoxWallet::establishSecureChannel (
            CW_SecureSession & session)
```

Establish a secure channel (SELECT → certificate → ECDH → mutual auth).
Lower-level than connect(): runs the full activation sequence once, without the retry loop. Used internally by connect(); exposed for advanced callers that handle retry policy themselves.

Parameters

out	*session*	Secure session to populate.

Returns

true if mutual authentication succeeded, false if any step of the activation sequence (SELECT, certificate chain verification, ECDH, MAC check) failed.

Warning

All sensitive ephemeral key material is wiped from the stack on every exit path (H-01, M-02).

Definition at line 127 of file CryptnoxWallet.cpp.
References _logger, _secure, CW_CERT_OK, CW_CURVE_SECP256R1, F, HEX, and CW_Utils::secure_wipe().
Referenced by connect().

4.2.3.7 extractRawSignature()

```
bool CryptnoxWallet::extractRawSignature (
            const uint8_t * derResponse,
            uint16_t derLength,
            CW_SignResult & result)  [private]
```

Definition at line 691 of file CryptnoxWallet.cpp.
References _logger, CW_DER_TAG_SEQUENCE, CW_NOK, CW_RAW_SIGNATURE_SIZE, CW_SignResult::errorCode, F, parseDerSignature(), CW_Utils::safe_memcpy(), CW_Utils::secure_wipe(), and CW_SignResult::signature.
Referenced by sign().

© 2026 Cryptnox SA

4.2.3.8 getCardInfo()

```
bool CryptnoxWallet::getCardInfo (
            CW_SecureSession & session,
            CW_CardInfo * info = NULL)
```

Send a secured GET CARD INFO APDU (0x80FA0000) and optionally decode the owner name/email from the response.

Parameters

in,out	*session*	Valid secure session.
out	*info*	Optional output. When non-NULL and the call succeeds, populated with the card's owner name and email (ASCII, NUL-terminated).

Returns

true if the secure exchange completed and (when `info` is non-NULL) parsing the name/email fields succeeded.

Definition at line 225 of file CryptnoxWallet.cpp.
References _logger, _secure, CW_CARD_EMAIL_MAX_LEN, CW_CARD_NAME_MAX_LEN, CW_CardInfo::email, F, isSecureChannelOpen(), CW_CardInfo::name, CW_Utils::safe_memcpy(), and CW_Utils::secure_wipe().

4.2.3.9 getPublicKey() [1/2]

```
bool CryptnoxWallet::getPublicKey (
            CW_SecureSession & session,
            const uint8_t * derivePath,
            uint8_t derivePathLength,
            uint8_t * publicKey64)
```

Read a public key from the card for a BIP32 derivation path.
Sends an encrypted GET PUBLIC KEY APDU (`80` C2 P1 01) with the wire-format path as data, and returns the uncompressed key with the `0x04` SEC1 prefix stripped — the form CW_Tron::addressFromPublicKey and the Ethereum address derivation both expect.
Without this a terminal cannot learn which account the card in front of it owns, so the account has to be compiled in; with it, any card works.

Parameters

in,out	*session*	Valid secure session.
in	*derivePath*	Wire-format BIP32 path (5 big-endian uint32 levels, hardened levels carrying the high bit), or `NULL` to read the card's current key.
in	*derivePathLength*	Length of `derivePath` in bytes; must be a multiple of 4 and at most CW_MAX_DERIVE_PATH_LENGTH. Pass 0 with a `NULL` path.
out	*publicKey64*	64-byte output: X || Y.

Returns

true on success; false on bad arguments, a closed session, a card refusal, or a response that is not an uncompressed key.

© 2026 Cryptnox SA

Precondition

The PIN must already have been verified for this session (see verifyPin) — the card refuses the export otherwise, except on the PIN-less path.

Definition at line 342 of file CryptnoxWallet.cpp.
References _logger, CW_RAW_PUBKEY_SIZE, CW_SEC1_PUBKEY_SIZE, CW_SEC1_UNCOMPRESSED_TAG, CW_SIGN_CURR_K1, CW_SIGN_DERIVE_K1, F, getPublicKey(), CW_Utils::safe_memcpy(), and CW_Utils::secure_wipe().
Referenced by check_ed25519_applet_gate(), check_pubkey_allow_list(), and getPublicKey().

4.2.3.10 getPublicKey() [2/2]

```
bool CryptnoxWallet::getPublicKey (
            CW_SecureSession & session,
            uint8_t keyType,
            const uint8_t * derivePath,
            uint8_t derivePathLength,
            uint8_t * pubKey,
            uint16_t pubKeyBufSize,
            uint16_t & pubKeyLength)
```

Read a card-resident public key (GET PUBLIC KEY, APDU 0x80C2).
Returns the key exactly as the card sends it: for CW_SIGN_CURR_ED25519 / CW_SIGN_DERIVE_ED25519 a raw 32-byte Ed25519 key (the Solana address bytes); for k1/r1 key types the 65-byte uncompressed point (0x04 || X[32] || Y[32]).

Parameters

in, out	*session*	Valid secure session.
in	*keyType*	One of the CW_SIGN_CURR_* / CW_SIGN_DERIVE_* constants; anything else (derive-and-make-current, PIN-less, unknown curve) is refused.
in	*derivePath*	BIP32 path bytes for DERIVE key types; NULL for current-key.
in	*derivePathLength*	Length of derivePath (multiple of 4); 0 for current-key.
out	*pubKey*	Buffer to receive the raw key bytes.
in	*pubKeyBufSize*	Size of pubKey ($\geq$ 32 for Ed25519, $\geq$ 65 for k1/r1).
out	*pubKeyLength*	Actual number of key bytes written on success.

Returns

true on success, false on closed/invalid session, bad parameters, a buffer too small for the returned key, or transport / MAC failure.

Note

Ed25519 key types require applet v2.0+. On an older applet this returns false without sending anything.

Definition at line 426 of file CryptnoxWallet.cpp.
References _logger, _secure, CW_MAX_DERIVE_PATH_LENGTH, cwIsEd25519(), cwPubKeyTypeAllowed(), cwSignNeedsPath(), F, isSecureChannelOpen(), CW_Utils::safe_memcpy(), and CW_Utils::secure_wipe().

4.2.3.11 isSecureChannelOpen()

```
bool CryptnoxWallet::isSecureChannelOpen (
            const CW_SecureSession & session)  [static], [private]
```

Definition at line 542 of file CryptnoxWallet.cpp.
References CW_SecureSession::aesKey, and CW_AESKEY_SIZE.
Referenced by disconnect(), getCardInfo(), getPublicKey(), validateSignRequest(), verifyPin(), and writeUserData().

© 2026 Cryptnox SA

4.2.3.12 operator=()

```
CryptnoxWallet & CryptnoxWallet::operator= (
            const CryptnoxWallet & )  [delete]
```

References CryptnoxWallet().

4.2.3.13 parseDerSignature()

```
bool CryptnoxWallet::parseDerSignature (
            const uint8_t * der,
            uint8_t derLength,
            uint8_t * r,
            uint8_t & rLength,
            uint8_t * s,
            uint8_t & sLength)  [static]
```

Parse a DER-encoded ECDSA signature to extract raw r and s values.

Parameters

in	*der*	DER-encoded signature bytes.
in	*derLength*	DER length.
out	*r*	Buffer for r (at least 33 bytes).
out	*rLength*	Actual r length written.
out	*s*	Buffer for s (at least 33 bytes).
out	*sLength*	Actual s length written.

Returns

true on success, false on malformed DER.

Definition at line 494 of file CryptnoxWallet.cpp.
References CW_DER_TAG_INTEGER, CW_DER_TAG_SEQUENCE, and CW_Utils::safe_memcpy().
Referenced by extractRawSignature().

4.2.3.14 printPN532FirmwareVersion()

```
bool CryptnoxWallet::printPN532FirmwareVersion ()  [private]
```

Definition at line 550 of file CryptnoxWallet.cpp.
References _secure.
Referenced by begin().

4.2.3.15 sendSignApdu()

```
bool CryptnoxWallet::sendSignApdu (
            CW_SignRequest & request,
            const uint8_t * data,
            uint16_t dataLength,
            uint8_t * derResponse,
            uint16_t derResponseSize,
            uint16_t & derLength,
            CW_SignResult & result)  [private]
```

Definition at line 664 of file CryptnoxWallet.cpp.
References _logger, _secure, CW_SIGN_NO_KEY_LOADED, CW_SIGN_SIG_EDDSA, cwIsEd25519(), CW_SignResult::errorCode, F, CW_SignRequest::keyType, CW_SignRequest::session, and CW_SignRequest::signatureType.
Referenced by sign().

© 2026 Cryptnox SA

4.2.3.16 sign()

```
CW_SignResult CryptnoxWallet::sign (
            CW_SignRequest & request)
```

Sign a digest or message using a card-resident key.
Builds the SIGN payload, sends it through the secure channel and returns the signature in its 64-byte raw form. Depending on the key and signature type:

- **ECDSA** (CW_SIGN_SIG_ECDSA_LOW_S / CW_SIGN_SIG_ECDSA_EOSIO): `hash` is a 32-byte digest. Payload: hash || optional BIP32 path || optional PIN. The card's DER signature is unpacked to r[32] || s[32].
- **Ed25519** (`CW_SIGN_*_ED25519`, applet 2.0+): `hash` is the RAW message, the card hashes it. Payload: len(2, big-endian) || message || optional path || optional PIN. P2 is forced to CW_SIGN_SIG_EDDSA. The card returns a raw 64-byte R[32] || S[32]. The maximum message length depends on the path sent: 197 bytes with the current key, 181 bytes with a 4-level path.
- **Schnorr BIP340** (CW_SIGN_SIG_SCHNORR_BIP340): `hash` is a 32-byte digest; the card returns a raw 64-byte signature, no DER.

Parameters

`in`	*request*	Sign parameters — must reference a valid secure session, the digest or message, the desired key/signature type, and the PIN (unless `pinLessMode` is set).

Returns

CW_SignResult. On success `errorCode` is CW_OK and `signature` holds the 64-byte raw signature. On failure the signature is zeroed and `errorCode` indicates the cause:

Return values

CW_OK	Signature valid.
CW_INVALID_SESSION	Secure channel not open.
CW_SIGN_KEY_TOO_SHORT	Missing hash buffer / zero length.
CW_SIGN_INVALID_PATH	Bad derivation path.
CW_SIGN_MESSAGE_TOO_LONG	Hash/message too long for the key type.
CW_SIGN_UNSUPPORTED_KEY_TYPE	Ed25519 key type on an applet older than 2.0.
CW_SIGN_NO_KEY_LOADED	Card rejected the SIGN APDU.
CW_SIGN_PIN_INCORRECT	PIN length out of range.
CW_SIGN_KEY_TOO_SHORT_WITH_PINLESS_MODE	PIN-less mode requested but `keyType` is not CW_SIGN_PINLESS_K1.

Warning

When `pinLessMode` is false the `request.pin` field must be populated. The destructor of CW_SignRequest securely wipes the PIN, but the caller must zero any other copy.

Definition at line 381 of file CryptnoxWallet.cpp.
References _logger, buildSignPayload(), CW_NOK, CW_OK, CW_RAW_SIGNATURE_SIZE, CW_SIGN_SIG_SCHNORR_BIP340, CW_USER_DATA_PAGE_SIZE, cwIsEd25519(), debugPrintSignature(),

© 2026 Cryptnox SA

CW_SignResult::errorCode, extractRawSignature(), F, CW_SignRequest::keyType, CW_Utils::safe_memcpy(), CW_Utils::secure_wipe(), sendSignApdu(), CW_SignResult::signature, CW_SignRequest::signatureType, and validateSignRequest().
Referenced by signWith().

4.2.3.17 validateSignRequest()

```
bool CryptnoxWallet::validateSignRequest (
            const CW_SignRequest & request,
            CW_SignResult & result)  [private]
```

Definition at line 554 of file CryptnoxWallet.cpp.
References _logger, _secure, CW_HASH_SIZE, CW_INVALID_SESSION, CW_MAX_DERIVE_PATH_LENGTH, CW_MAX_PIN_LENGTH, CW_MIN_PIN_LENGTH, CW_SIGN_INVALID_PATH, CW_SIGN_KEY_TOO_SHORT, CW_SIGN_KEY_TOO_SHORT_WITH_PINLESS_MODE, CW_SIGN_MESSAGE_TOO_LONG, CW_SIGN_PIN_INCORRECT, CW_SIGN_PINLESS_K1, CW_SIGN_UNSUPPORTED_KEY_TYPE, cwIsEd25519(), cwMaxEd25519Message(), cwSignNeedsPath(), CW_SignRequest::derivePath, CW_SignRequest::derivePathLength, CW_SignResult::errorCode, F, CW_SignRequest::hash, CW_SignRequest::hashLength, isSecureChannelOpen(), CW_SignRequest::keyType, CW_SignRequest::pin, CW_SignRequest::pinLessMode, and CW_SignRequest::session.
Referenced by sign().

4.2.3.18 verifyPin()

```
bool CryptnoxWallet::verifyPin (
            CW_SecureSession & session,
            const uint8_t * pin,
            uint8_t pinLength)
```

Verify the PIN code on the card.
Sends an encrypted VERIFY PIN APDU. The card maintains a try counter: every wrong attempt decrements it, and reaching zero locks the PIN permanently until a successful PUK / re-initialisation flow.

Parameters

in,out	*session*	Valid secure session.
in	*pin*	PIN bytes (ASCII digits, 4–9 characters).
in	*pinLength*	Length of the PIN (must be in [CW_MIN_PIN_LENGTH, CW_MAX_PIN_LENGTH]).

Returns

true if the card accepted the PIN, false on wrong PIN, closed or invalid session, length out of range, or transport / MAC failure.

Warning

Each failed attempt decrements the card's PIN counter. Treat a false return as "wrong PIN" only after confirming session validity — a transport glitch should not be retried with a new PIN. A wrong PIN or any card / transport failure clears `session`: call connect again before the next command.

Definition at line 279 of file CryptnoxWallet.cpp.
References _logger, _secure, CW_MAX_PIN_LENGTH, CW_MIN_PIN_LENGTH, F, isSecureChannelOpen(), CW_Utils::safe_memcpy(), and CW_Utils::secure_wipe().

4.2.3.19 writeUserData()

```
bool CryptnoxWallet::writeUserData (
            CW_SecureSession & session,
```

© 2026 Cryptnox SA

```
const uint8_t * data,
uint16_t dataLength)
```

Write user data (WRITE APDU 0x80FC, P1=0 user-data slot, single page).
Writes `data` in one APDU to page 0 of the card's single 3600-byte user-data area (three 1200-byte pages, P2 = page index).

- At most CW_USER_DATA_PAGE_SIZE (208) bytes, page 0 only.
- Each write replaces the previous content: the card sets the stored length to this write's length. Nothing is appended.
- The limit is the transport's, not the card's. A page is one APDU on the card (up to 1239 bytes), but this SDK caps APDUs at 255 bytes (PN532 frame size; `sendAPDU` takes a `uint8_t` length), and pages 1 and 2 are refused until page 0 holds a full 1200 bytes. A full page would also need a ~1240-byte buffer, ruling out AVR targets.

Writing more needs extended APDUs in the SDK (readers such as PN7150 or PC/SC, targets such as ESP32) or an applet change (offset/append on P2).

Parameters

`in,out`	*session*	Valid secure session.
`in`	*data*	Data to write.
`in`	*dataLength*	Bytes to write (1..CW_USER_DATA_PAGE_SIZE).

Returns

true on success, false on closed session, bad parameters, `dataLength` above one page, or transport / MAC failure.

Definition at line 302 of file CryptnoxWallet.cpp.
References _logger, _secure, CW_USER_DATA_PAGE_SIZE, F, and isSecureChannelOpen().

4.2.4 Member Data Documentation

4.2.4.1 _logger

```
CW_Logger& CryptnoxWallet::_logger  [private]
```

Logging interface.
Definition at line 412 of file CryptnoxWallet.h.
Referenced by connect(), CryptnoxWallet(), debugPrintSignature(), establishSecureChannel(), extractRawSignature(), getCardInfo(), getPublicKey(), getPublicKey(), sendSignApdu(), sign(), validateSignRequest(), verifyPin(), and writeUserData().

4.2.4.2 _platform

```
CW_Platform& CryptnoxWallet::_platform  [private]
```

Platform abstraction (sleep_ms).
Definition at line 413 of file CryptnoxWallet.h.
Referenced by connect(), and CryptnoxWallet().

4.2.4.3 _secure

```
CW_SecureChannel CryptnoxWallet::_secure  [private]
```

Owned secure channel.
Definition at line 414 of file CryptnoxWallet.h.
Referenced by begin(), connect(), CryptnoxWallet(), disconnect(), establishSecureChannel(), getCardInfo(), getPublicKey(), printPN532FirmwareVersion(), sendSignApdu(), validateSignRequest(), verifyPin(), and writeUserData().

© 2026 Cryptnox SA

The documentation for this class was generated from the following files:

- CryptnoxWallet.h
- CryptnoxWallet.cpp

4.3 CW_CardInfo Struct Reference

Subset of the Cryptnox card info returned by APDU 0x80FA0000.
`#include <CryptnoxWallet.h>`

Public Member Functions

- CW_CardInfo ()=default

Public Attributes

- char name [CW_CARD_NAME_MAX_LEN+1U] {}
- char email [CW_CARD_EMAIL_MAX_LEN+1U] {}

4.3.1 Detailed Description

Subset of the Cryptnox card info returned by APDU 0x80FA0000.
Mirrors the fields the Python SDK exposes as `card._owner`: ASCII name and email programmed when the card was initialised. NUL-terminated.
Definition at line 55 of file CryptnoxWallet.h.

4.3.2 Constructor & Destructor Documentation

4.3.2.1 CW_CardInfo()

```
CW_CardInfo::CW_CardInfo () [default]
```

4.3.3 Member Data Documentation

4.3.3.1 email

```
char CW_CardInfo::email[CW_CARD_EMAIL_MAX_LEN+1U] {}
```
NUL-terminated ASCII email.
Definition at line 57 of file CryptnoxWallet.h.
Referenced by CryptnoxWallet::getCardInfo().

4.3.3.2 name

```
char CW_CardInfo::name[CW_CARD_NAME_MAX_LEN+1U] {}
```
NUL-terminated ASCII name.
Definition at line 56 of file CryptnoxWallet.h.
Referenced by CryptnoxWallet::getCardInfo().
The documentation for this struct was generated from the following file:

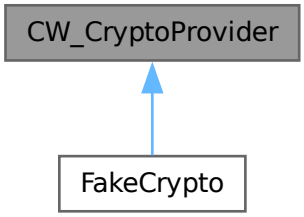
- CryptnoxWallet.h

4.4 CW_CryptoProvider Class Reference

Abstract interface for cryptographic operations used by CW_SecureChannel.
`#include <CW_CryptoProvider.h>`

© 2026 Cryptnox SA

Inheritance diagram for CW_CryptoProvider:



Public Member Functions

- virtual bool sha256 (const uint8_t *data, size_t len, uint8_t *out)=0
  Compute SHA-256 over a contiguous data buffer.
- virtual bool sha512 (const uint8_t *data, size_t len, uint8_t *out)=0
  Compute SHA-512 over a contiguous data buffer.
- virtual uint16_t aesCbcEncrypt (const uint8_t *in, uint16_t len, uint8_t *out, const uint8_t *key, uint8_t keyLen, uint8_t *iv, bool bitPadding)=0
  AES-CBC encrypt.
- virtual uint16_t aesCbcDecrypt (uint8_t *in, uint16_t len, uint8_t *out, const uint8_t *key, uint8_t keyLen, uint8_t *iv, bool bitPadding)=0
  AES-CBC decrypt.
- virtual bool ecdh (const uint8_t *pubKey, const uint8_t *privKey, uint8_t *secret, CW_Curve curve)=0
  ECDH shared secret computation.
- virtual bool makeKey (uint8_t *pubKey, uint8_t *privKey, CW_Curve curve)=0
  Generate a new EC key pair.
- virtual bool random (uint8_t *dest, unsigned size)=0
  Fill a buffer with cryptographically random bytes.
- virtual bool ecdsaVerify (const uint8_t *pubKey64, const uint8_t *hash, size_t hashLen, const uint8_t *sig, CW_Curve curve)=0
  Verify an ECDSA signature (raw r||s, 64 bytes) against a message hash.
- virtual ~CW_CryptoProvider ()

4.4.1 Detailed Description

Abstract interface for cryptographic operations used by CW_SecureChannel.
Decouples the secure channel protocol from any specific crypto library. The concrete ESP32 implementation (ESP32CryptoProvider) wraps mbedTLS (SHA-256/SHA-512/AES-CBC hardware-accelerated on ESP32-S3) and the ESP32 hardware TRNG for random number generation.
Definition at line 45 of file CW_CryptoProvider.h.

4.4.2 Constructor & Destructor Documentation

4.4.2.1 ~CW_CryptoProvider()

```
virtual CW_CryptoProvider::~CW_CryptoProvider () [inline], [virtual]
```

Definition at line 146 of file CW_CryptoProvider.h.

© 2026 Cryptnox SA

4.4.3 Member Function Documentation

4.4.3.1 aesCbcDecrypt()

```
virtual uint16_t CW_CryptoProvider::aesCbcDecrypt (
            uint8_t * in,
            uint16_t len,
            uint8_t * out,
            const uint8_t * key,
            uint8_t keyLen,
            uint8_t * iv,
            bool bitPadding)  [pure virtual]
```

AES-CBC decrypt.

Parameters

in	*in*	Ciphertext input buffer (non-const; may be modified internally).
in	*len*	Length of the ciphertext.
out	*out*	Plaintext output buffer.
in	*key*	AES key bytes.
in	*keyLen*	AES key length in bytes.
in,out	*iv*	16-byte IV used as decrypt IV.
in	*bitPadding*	true = Bit padding removal; false = Null padding (no removal).

Returns

Length of the plaintext written to `out`.

Implemented in FakeCrypto.

4.4.3.2 aesCbcEncrypt()

```
virtual uint16_t CW_CryptoProvider::aesCbcEncrypt (
            const uint8_t * in,
            uint16_t len,
            uint8_t * out,
            const uint8_t * key,
            uint8_t keyLen,
            uint8_t * iv,
            bool bitPadding)  [pure virtual]
```

AES-CBC encrypt.

Parameters

in	*in*	Plaintext input buffer.
in	*len*	Length of the plaintext.
out	*out*	Ciphertext output buffer (must be large enough for padding).
in	*key*	AES key bytes.
in	*keyLen*	AES key length in bytes (16, 24, or 32).
in,out	*iv*	16-byte IV; updated to last cipher block on return.
in	*bitPadding*	true = ISO/IEC 9797-1 Method 2 (Bit) padding; false = Null padding (no padding added).

© 2026 Cryptnox SA

Returns

Length of the ciphertext written to `out`.

Implemented in FakeCrypto.

4.4.3.3 ecdh()

```
virtual bool CW_CryptoProvider::ecdh (
            const uint8_t * pubKey,
            const uint8_t * privKey,
            uint8_t * secret,
            CW_Curve curve)  [pure virtual]
```

ECDH shared secret computation.

Parameters

in	*pubKey*	Remote public key (64 bytes, X||Y, no 0x04 prefix).
in	*privKey*	Local private key (32 bytes).
out	*secret*	32-byte shared secret output.
in	*curve*	Curve identifier (CW_CURVE_SECP256R1 or CW_CURVE_SECP256K1).

Returns

true on success, false otherwise.

Implemented in FakeCrypto.

4.4.3.4 ecdsaVerify()

```
virtual bool CW_CryptoProvider::ecdsaVerify (
            const uint8_t * pubKey64,
            const uint8_t * hash,
            size_t hashLen,
            const uint8_t * sig,
            CW_Curve curve)  [pure virtual]
```

Verify an ECDSA signature (raw r||s, 64 bytes) against a message hash.

Parameters

in	*pubKey64*	64-byte public key (X||Y, no 0x04 prefix).
in	*hash*	Message hash buffer.
in	*hashLen*	Length of the hash in bytes.
in	*sig*	64-byte raw signature (r[32]||s[32]).
in	*curve*	Curve identifier for the verification operation.

Returns

true if the signature is valid, false otherwise.

Implemented in FakeCrypto.

4.4.3.5 makeKey()

```
virtual bool CW_CryptoProvider::makeKey (
            uint8_t * pubKey,
            uint8_t * privKey,
```

© 2026 Cryptnox SA

```
            CW_Curve curve)  [pure virtual]
```

Generate a new EC key pair.

Parameters

out	*pubKey*	64-byte public key output (X‖Y, no prefix).
out	*privKey*	32-byte private key output.
in	*curve*	Curve identifier (CW_CURVE_SECP256R1 or CW_CURVE_SECP256K1).

Returns

true on success, false otherwise.

Implemented in FakeCrypto.

4.4.3.6 random()

```
virtual bool CW_CryptoProvider::random (
            uint8_t * dest,
            unsigned size)  [pure virtual]
```

Fill a buffer with cryptographically random bytes.

Parameters

out	*dest*	Buffer to fill.
in	*size*	Number of bytes to generate.

Returns

true on success, false otherwise.

Implemented in FakeCrypto.

4.4.3.7 sha256()

```
virtual bool CW_CryptoProvider::sha256 (
            const uint8_t * data,
            size_t len,
            uint8_t * out)  [pure virtual]
```

Compute SHA-256 over a contiguous data buffer.

Parameters

in	*data*	Input buffer.
in	*len*	Number of bytes to hash.
out	*out*	32-byte output buffer.

Returns

true on success, false if the underlying hash accelerator faults.

Implemented in FakeCrypto.
Referenced by tron_checksum().

© 2026 Cryptnox SA

4.4.3.8 sha512()

```
virtual bool CW_CryptoProvider::sha512 (
            const uint8_t * data,
            size_t len,
            uint8_t * out)  [pure virtual]
```

Compute SHA-512 over a contiguous data buffer.

Parameters

in	*data*	Input buffer.
in	*len*	Number of bytes to hash.
out	*out*	64-byte output buffer.

Returns

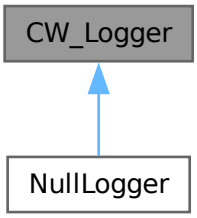
true on success, false if the underlying hash accelerator faults.

Implemented in FakeCrypto.
The documentation for this class was generated from the following file:

- CW_CryptoProvider.h

4.5 CW_Logger Class Reference

Abstract interface for serial/debug output.
`#include <CW_Logger.h>`
Inheritance diagram for CW_Logger:



Public Member Functions

- virtual bool begin (unsigned long baudRate=115200UL)=0

  Initialize the logging interface.
- virtual ~CW_Logger ()

Print methods (no newline)

- virtual void print (const __FlashStringHelper *str)=0
- virtual void print (const char *str)=0
- virtual void print (char c)=0
- virtual void print (uint8_t value, int base=DEC)=0
- virtual void print (uint16_t value, int base=DEC)=0

© 2026 Cryptnox SA

- virtual void print (uint32_t value, int base=DEC)=0
- virtual void print (int value, int base=DEC)=0

Println methods (with newline)

- virtual void println ()=0
- virtual void println (const __FlashStringHelper *str)=0
- virtual void println (const char *str)=0
- virtual void println (char c)=0
- virtual void println (uint8_t value, int base=DEC)=0
- virtual void println (uint16_t value, int base=DEC)=0
- virtual void println (uint32_t value, int base=DEC)=0
- virtual void println (int value, int base=DEC)=0

4.5.1 Detailed Description

Abstract interface for serial/debug output.
Provides a hardware-agnostic logging contract so that higher-level components (CryptnoxWallet, CW_SecureChannel) remain independent of the physical output device (UART, LCD, network, etc.). On Arduino, the F() macro returns a __FlashStringHelper* so the dedicated overloads are called, keeping string literals in flash. On non-Arduino, F() is the identity macro (returns const char*), so the print(const char*) overload is called instead.
Definition at line 48 of file CW_Logger.h.

4.5.2 Constructor & Destructor Documentation

4.5.2.1 ~CW_Logger()

```
virtual CW_Logger::~CW_Logger ()  [inline], [virtual]
```

Definition at line 80 of file CW_Logger.h.

4.5.3 Member Function Documentation

4.5.3.1 begin()

```
virtual bool CW_Logger::begin (
            unsigned long baudRate = 115200UL)  [pure virtual]
```

Initialize the logging interface.

Parameters

baudRate	Baud rate (relevant for UART implementations).

Returns

true if initialization succeeded, false otherwise.

Implemented in NullLogger.

4.5.3.2 print() [1/7]

```
virtual void CW_Logger::print (
            char c)  [pure virtual]
```

Implemented in NullLogger.

4.5.3.3 print() [2/7]

```
virtual void CW_Logger::print (
            const __FlashStringHelper * str)  [pure virtual]
```

Implemented in NullLogger.

© 2026 Cryptnox SA

4.5.3.4 print() [3/7]

```
virtual void CW_Logger::print (
            const char * str)  [pure virtual]
```

Implemented in NullLogger.

4.5.3.5 print() [4/7]

```
virtual void CW_Logger::print (
            int value,
            int base = DEC)  [pure virtual]
```

Implemented in NullLogger.
References DEC.

4.5.3.6 print() [5/7]

```
virtual void CW_Logger::print (
            uint16_t value,
            int base = DEC)  [pure virtual]
```

Implemented in NullLogger.
References DEC.

4.5.3.7 print() [6/7]

```
virtual void CW_Logger::print (
            uint32_t value,
            int base = DEC)  [pure virtual]
```

Implemented in NullLogger.
References DEC.

4.5.3.8 print() [7/7]

```
virtual void CW_Logger::print (
            uint8_t value,
            int base = DEC)  [pure virtual]
```

Implemented in NullLogger.
References DEC.

4.5.3.9 println() [1/8]

```
virtual void CW_Logger::println ()  [pure virtual]
```

Implemented in NullLogger.

4.5.3.10 println() [2/8]

```
virtual void CW_Logger::println (
            char c)  [pure virtual]
```

Implemented in NullLogger.

4.5.3.11 println() [3/8]

```
virtual void CW_Logger::println (
            const __FlashStringHelper * str)  [pure virtual]
```

Implemented in NullLogger.

4.5.3.12 println() [4/8]

```
virtual void CW_Logger::println (
            const char * str)  [pure virtual]
```

Implemented in NullLogger.

© 2026 Cryptnox SA

4.5.3.13 println() [5/8]

```
virtual void CW_Logger::println (
            int value,
            int base = DEC)  [pure virtual]
```

Implemented in NullLogger.
References DEC.

4.5.3.14 println() [6/8]

```
virtual void CW_Logger::println (
            uint16_t value,
            int base = DEC)  [pure virtual]
```

Implemented in NullLogger.
References DEC.

4.5.3.15 println() [7/8]

```
virtual void CW_Logger::println (
            uint32_t value,
            int base = DEC)  [pure virtual]
```

Implemented in NullLogger.
References DEC.

4.5.3.16 println() [8/8]

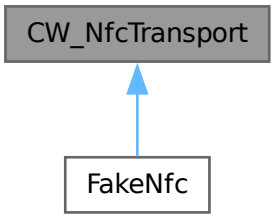
```
virtual void CW_Logger::println (
            uint8_t value,
            int base = DEC)  [pure virtual]
```

Implemented in NullLogger.
References DEC.
The documentation for this class was generated from the following file:

- CW_Logger.h

4.6 CW_NfcTransport Class Reference

Abstract interface for NFC transport operations.
`#include <CW_NfcTransport.h>`
Inheritance diagram for CW_NfcTransport:



Public Member Functions

- virtual bool begin ()=0

© 2026 Cryptnox SA

Initialize the NFC transport hardware.

- virtual bool inListPassiveTarget ()=0

  Detect the presence of a passive ISO-DEP NFC target.
- virtual bool sendAPDU (const uint8_t *apdu, uint8_t apduLen, uint8_t *response, uint8_t &responseLen)=0

  Send an APDU command to the card and receive the response.
- virtual bool sendAPDULarge (const uint8_t *apdu, uint8_t apduLen, uint8_t *response, uint16_t &responseLen)

  Send an APDU and receive a response that may exceed 255 bytes.
- virtual void resetReader ()=0

  Reset the NFC reader/field for the next card detection cycle.
- virtual bool printFirmwareVersion ()=0

  Print NFC module firmware version information to the logger.
- virtual ~CW_NfcTransport ()

4.6.1 Detailed Description

Abstract interface for NFC transport operations.
Defines the hardware-agnostic contract for NFC communication so that CW_SecureChannel and CryptnoxWallet remain independent of the physical NFC module (PN532, PN7150, etc.).
Definition at line 40 of file CW_NfcTransport.h.

4.6.2 Constructor & Destructor Documentation

4.6.2.1 ~CW_NfcTransport()

```
virtual CW_NfcTransport::~CW_NfcTransport () [inline], [virtual]
```

Definition at line 103 of file CW_NfcTransport.h.

4.6.3 Member Function Documentation

4.6.3.1 begin()

```
virtual bool CW_NfcTransport::begin () [pure virtual]
```

Initialize the NFC transport hardware.

Returns

true if initialization succeeded, false otherwise.

Implemented in FakeNfc.

4.6.3.2 inListPassiveTarget()

```
virtual bool CW_NfcTransport::inListPassiveTarget () [pure virtual]
```

Detect the presence of a passive ISO-DEP NFC target.

Returns

true if a card is detected, false otherwise.

Implemented in FakeNfc.

4.6.3.3 printFirmwareVersion()

```
virtual bool CW_NfcTransport::printFirmwareVersion () [pure virtual]
```

Print NFC module firmware version information to the logger.

Returns

true if firmware info was retrieved successfully, false otherwise.

Implemented in FakeNfc.

© 2026 Cryptnox SA

4.6.3.4 resetReader()

```
virtual void CW_NfcTransport::resetReader ()  [pure virtual]
```

Reset the NFC reader/field for the next card detection cycle.
Implemented in FakeNfc.

4.6.3.5 sendAPDU()

```
virtual bool CW_NfcTransport::sendAPDU (
            const uint8_t * apdu,
            uint8_t apduLen,
            uint8_t * response,
            uint8_t & responseLen)  [pure virtual]
```

Send an APDU command to the card and receive the response.

Parameters

in	*apdu*	APDU command bytes.
in	*apduLen*	Length of the APDU command.
out	*response*	Buffer to receive the card response.
out	*responseLen*	Actual number of bytes written to response.

Returns

true if the exchange succeeded, false otherwise.

Implemented in FakeNfc.
Referenced by sendAPDULarge().

4.6.3.6 sendAPDULarge()

```
virtual bool CW_NfcTransport::sendAPDULarge (
            const uint8_t * apdu,
            uint8_t apduLen,
            uint8_t * response,
            uint16_t & responseLen)  [inline], [virtual]
```

Send an APDU and receive a response that may exceed 255 bytes.
Used for APDUs whose DataOut can be larger than a uint8_t can express (e.g. GET_↩ MANUFACTURER_CERTIFICATE returns up to 415 bytes). Implementations that cannot deliver more than 255 bytes may delegate to sendAPDU; the default below does exactly that.

Parameters

in	*apdu*	APDU command bytes.
in	*apduLen*	Length of the APDU command.
out	*response*	Buffer to receive the card response.
in,out	*responseLen*	On entry: capacity of response. On exit: actual bytes written.

Returns

true if the exchange succeeded, false otherwise.

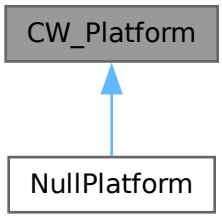
Definition at line 81 of file CW_NfcTransport.h.
References sendAPDU().
The documentation for this class was generated from the following file:

- CW_NfcTransport.h

© 2026 Cryptnox SA

4.7 CW_Platform Class Reference

Abstract interface for platform-specific operations used by the SDK.
`#include <CW_Platform.h>`
Inheritance diagram for CW_Platform:



Public Member Functions

- virtual void sleep_ms (uint32_t ms)=0
  Block for at least `ms` milliseconds.
- virtual ~CW_Platform ()

4.7.1 Detailed Description

Abstract interface for platform-specific operations used by the SDK.
Decouples the SDK core from any specific RTOS or bare-metal delay mechanism, allowing the same SDK to run on ESP32 (FreeRTOS), Arduino, and hosted (Linux/macOS) test environments.
Definition at line 39 of file CW_Platform.h.

4.7.2 Constructor & Destructor Documentation

4.7.2.1 ~CW_Platform()

```
virtual CW_Platform::~CW_Platform () [inline], [virtual]
```
Definition at line 48 of file CW_Platform.h.

4.7.3 Member Function Documentation

4.7.3.1 sleep_ms()

```
virtual void CW_Platform::sleep_ms (
            uint32_t ms) [pure virtual]
```
Block for at least `ms` milliseconds.

Parameters

`in`	*ms*	Duration to sleep in milliseconds.

Implemented in NullPlatform.
The documentation for this class was generated from the following file:

- CW_Platform.h

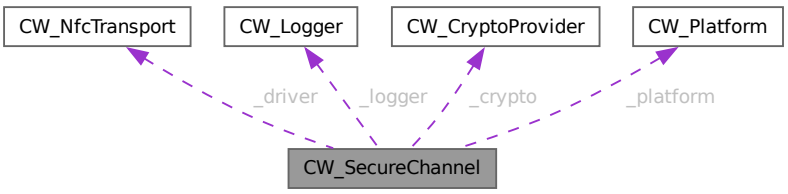
© 2026 Cryptnox SA

4.8 CW_SecureChannel Class Reference

Implements the Cryptnox secure channel protocol over NFC.

`#include <CW_SecureChannel.h>`

Collaboration diagram for CW_SecureChannel:



Public Member Functions

- CW_SecureChannel (CW_NfcTransport &driver, CW_Logger &logger, CW_CryptoProvider &crypto, CW_Platform &platform)

  Construct a CW_SecureChannel.
- CW_SecureChannel (const CW_SecureChannel &)=delete
- CW_SecureChannel & operator= (const CW_SecureChannel &)=delete
- bool begin ()

  Initialize the NFC transport module.
- bool inListPassiveTarget ()

  Detect a passive NFC target (ISO-DEP card).
- void resetReader ()

  Reset the NFC reader hardware.
- bool printFirmwareVersion ()

  Print the NFC reader firmware version to the logger.
- uint8_t appletMajor () const

  Applet major version read at the last SELECT.
- bool selectApdu ()

  Send the SELECT APDU to activate the Cryptnox application.
- bool getCardCertificate (uint8_t *cardCertificate, uint8_t &cardCertificateLength)

  Retrieve the card's ephemeral public key via GET CARD CERTIFICATE.
- bool openSecureChannel (uint8_t *salt, uint8_t *clientPublicKey, uint8_t *clientPrivateKey, CW_Curve sessionCurve)

  Send OPEN SECURE CHANNEL and retrieve the session salt.
- bool mutuallyAuthenticate (CW_SecureSession &session, const uint8_t *salt, uint8_t *client↩ PublicKey, const uint8_t *clientPrivateKey, CW_Curve sessionCurve, const uint8_t *card↩ EphemeralPubKey)

  Perform ECDH derivation and MUTUALLY AUTHENTICATE with the card.
- bool getManufacturerCertificate (uint8_t *cert, uint16_t &certLen)

  Retrieve the manufacturer certificate stored in card flash.
- bool preFetchManufacturerCert ()

  Fetch and cache the manufacturer certificate before getCardCertificate().
- uint8_t verifyCertificateChain (const uint8_t *cardCert, uint8_t cardCertLen)

  Verify the full card certificate chain against the trusted CA.

© 2026 Cryptnox SA

- bool aesCbcEncrypt (CW_SecureSession &session, const uint8_t apdu[], uint16_t apduLength, const uint8_t data[], uint16_t dataLength, uint8_t *decryptedOutput, uint16_t decryptedOutput↩ Size, uint16_t *decryptedOutputLength)

  AES-CBC encrypt + MAC, send APDU, and decrypt response.
- bool aesCbcEncrypt (CW_SecureSession &session, const uint8_t apdu[], uint16_t apduLength, const uint8_t data[], uint16_t dataLength)

  aesCbcEncrypt for a command whose response payload is discarded.
- bool aesCbcDecrypt (const CW_SecureSession &session, uint8_t *response, size_t response↩ Len, uint8_t *macValue, uint8_t *decryptedOutput, uint16_t decryptedOutputSize, uint16_↩ t *decryptedOutputLength)

  Verify MAC and decrypt an encrypted APDU response.
- bool checkStatusWord (const uint8_t *response, uint16_t responseLength, uint8_t sw1Expected, uint8_t sw2Expected)

  Verify the SW1/SW2 status word at the end of an APDU response.

Static Public Member Functions

- static bool extractCardEphemeralKey (const uint8_t *cardCertificate, uint8_t *cardEphemeral↩ PubKey, uint8_t *fullEphemeralPubKey65=NULL)

  Extract the card's ephemeral EC P-256 public key from a certificate.

Private Member Functions

- bool macIvForCounter (const CW_SecureSession &session, uint8_t *macIv)

  MAC IV for the current counter (zero on 1.6.x).
- bool verifyEcdsaSha256 (const uint8_t *pubKey64, const uint8_t *message, uint16_t msgLen, const uint8_t *derSig, uint8_t derSigLen)

Static Private Member Functions

- static bool parseDerSigToRaw (const uint8_t *der, uint8_t derLen, uint8_t *raw64)

Private Attributes

- CW_NfcTransport & _driver

  NFC transport for APDU exchange.
- CW_Logger & _logger

  Logging interface.
- CW_CryptoProvider & _crypto

  Crypto operations (AES, SHA, ECDH, RNG).
- CW_Platform & _platform

  Platform abstraction (sleep_ms).
- uint8_t _lastNonce [CW_CERT_NONCE_SIZE] {}

  Nonce sent in the last getCardCertificate() call; checked in verifyCertificateChain().
- uint16_t _cachedMfCertLen {}

  Non-zero when s_mfCertBuf holds a valid pre-fetched manufacturer certificate.
- uint8_t _appletMajor {}

  Applet major version from the last SELECT (0 = unknown → legacy channel).

© 2026 Cryptnox SA

4.8.1 Detailed Description

Implements the Cryptnox secure channel protocol over NFC.
Handles all low-level APDU exchanges required to establish and use a secure session with the Cryptnox smart card:

- Application selection (SELECT APDU)
- Card certificate retrieval and ephemeral key extraction
- ECDH-based session key derivation (OPEN SECURE CHANNEL + MUTUALLY AUTHENTICATE)
- AES-CBC-MAC secure messaging (encrypt / decrypt / MAC verify)
- Status word checking

CW_SecureChannel is composed inside CryptnoxWallet and is not intended to be used directly by application code.
Definition at line 64 of file CW_SecureChannel.h.

4.8.2 Constructor & Destructor Documentation

4.8.2.1 CW_SecureChannel() [1/2]

```
CW_SecureChannel::CW_SecureChannel (
            CW_NfcTransport & driver,
            CW_Logger & logger,
            CW_CryptoProvider & crypto,
            CW_Platform & platform)
```

Construct a CW_SecureChannel.

Parameters

driver	Reference to the NFC transport.
logger	Reference to the logging interface.
crypto	Reference to the crypto provider.
platform	Reference to the platform abstraction (for sleep_ms).

Definition at line 91 of file CW_SecureChannel.cpp.
References _appletMajor, _cachedMfCertLen, _crypto, _driver, _logger, _platform, logger, and platform.
Referenced by CW_SecureChannel(), and operator=().

4.8.2.2 CW_SecureChannel() [2/2]

```
CW_SecureChannel::CW_SecureChannel (
            const CW_SecureChannel & ) = [delete]
```

References CW_SecureChannel().

4.8.3 Member Function Documentation

4.8.3.1 aesCbcDecrypt()

```
bool CW_SecureChannel::aesCbcDecrypt (
            const CW_SecureSession & session,
            uint8_t * response,
            size_t responseLen,
            uint8_t * macValue,
            uint8_t * decryptedOutput,
            uint16_t decryptedOutputSize,
            uint16_t * decryptedOutputLength)
```

© 2026 Cryptnox SA

Verify MAC and decrypt an encrypted APDU response.
Internal helper called from aesCbcEncrypt — exposed for the fuzz harness. Verifies the response MAC against Kmac, then decrypts the payload with Kenc using the supplied IV (which is the MAC of the sent request, by protocol).

Parameters

in,out	*session*	Secure session.
in	*response*	Encrypted response buffer (MAC || cipher || SW).
in	*responseLen*	Response length.
in	*macValue*	MAC of the request — used as decrypt IV.
out	*decryptedOutput*	Plaintext output buffer, or NULL.
in	*decryptedOutputSize*	Capacity of decryptedOutput; see aesCbcEncrypt.
out	*decryptedOutputLength*	Plaintext output length, or NULL.

Returns

true if MAC matched and decryption succeeded, false otherwise.

Warning

A false return indicates either tampering or a corrupted channel — the session must not be reused without renegotiation.

Definition at line 660 of file CW_SecureChannel.cpp.
References _crypto, _logger, AES_BLOCK_SIZE, CW_SecureSession::aesKey, F, HEX, macIvForCounter(), CW_SecureSession::macKey, s_apduBuf, s_dataBuf, s_macBuf, CW_Utils::safe_memcpy(), CW_Utils::secure_compare(), and CW_Utils::secure_wipe().
Referenced by aesCbcEncrypt().

4.8.3.2 aesCbcEncrypt() [1/2]

```
bool CW_SecureChannel::aesCbcEncrypt (
            CW_SecureSession & session,
            const uint8_t apdu[],
            uint16_t apduLength,
            const uint8_t data[],
            uint16_t dataLength)
```

aesCbcEncrypt for a command whose response payload is discarded.
The output buffer and its capacity come as a pair in the full overload, so there is no way to pass one without the other.
One secure-messaging round-trip is built in five stages, reusing module- private scratch buffers (s_↩ apduBuf, s_macBuf, s_dataBuf) to keep the call-site stack frame small:

1. Plaintext padding — ISO/IEC 9797-1 Method 2 (bit padding) is appended to data so the length is a multiple of the AES block size.
2. Encryption — AES-256-CBC under Kenc with the current rolling IV.
3. MAC computation — AES-CMAC over (APDU header || Lc || ciphertext) under Kmac. The MAC's last 8 bytes are prepended to the ciphertext in the outgoing APDU.
4. Transport — the assembled APDU goes to the card; the response is structured as (MAC[8] || cipher || SW1 SW2).
5. Response decryption — delegated to aesCbcDecrypt, which verifies the response MAC against Kmac (using the request MAC as the IV per protocol) before decrypting under Kenc.

© 2026 Cryptnox SA

IV update — on success, the first 16 bytes of the response (the card's response MAC) become the new session IV (rolling IV). Every failure after the MAC counter advances clears the session, a non-9000 SW included.
Definition at line 483 of file CW_SecureChannel.cpp.
References aesCbcEncrypt().

4.8.3.3 aesCbcEncrypt() [2/2]

```
bool CW_SecureChannel::aesCbcEncrypt (
            CW_SecureSession & session,
            const uint8_t apdu[],
            uint16_t apduLength,
            const uint8_t data[],
            uint16_t dataLength,
            uint8_t * decryptedOutput,
            uint16_t decryptedOutputSize,
            uint16_t * decryptedOutputLength)
```

AES-CBC encrypt + MAC, send APDU, and decrypt response.
Performs one secure messaging round-trip: pads and encrypts `data` with Kenc using the current IV; computes a CMAC over (header || cipher) with Kmac; sends the wrapped APDU; on the response, verifies the MAC and decrypts the payload. The new IV for the next call is the first 16 bytes of the response, i.e. the card's response MAC (rolling IV).

Parameters

`in,out`	*session*	Secure session (Kenc / Kmac / IV).
`in`	*apdu*	APDU header (CLA, INS, P1, P2).
`in`	*apduLength*	Header length (must be 4).
`in`	*data*	Plaintext payload.
`in`	*dataLength*	Plaintext length (≤ CW_USER_DATA_PAGE_SIZE).
`out`	*decryptedOutput*	Buffer for the decrypted response payload, or NULL.
`in`	*decryptedOutputSize*	Capacity of `decryptedOutput`. Only the card decides how long its answer is, so pass the real size: a payload that does not fit is refused instead of overrunning the buffer.
`out`	*decryptedOutputLength*	Receives the decrypted payload length, or NULL.

Returns

true if the APDU was sent and the response MAC verified + decrypted successfully; false otherwise. The session is cleared (renegotiate) on an exhausted MAC counter, a MAC IV or MAC computation failure, a send-length failure, a transport failure, a bad response MAC, or any status word other than 9000. It is NOT cleared on bad parameters rejected before anything is sent.

Precondition

`session` must be the output of a successful mutuallyAuthenticate call.

Warning

Mutates `session.iv`. The failures listed above clear the session automatically. That includes a card error such as a wrong PIN (0x63Cn): a bare status word carries no MAC and may be forged in the NFC path, and keeping the old IV would let the card silently mis-decrypt the next command's first block. Reconnect before the next command.

Reuses module-private static buffers (`s_apduBuf`, `s_macBuf`, `s_dataBuf`). Not safe to call concurrently from multiple tasks; serialise at the application level.

© 2026 Cryptnox SA

Definition at line 489 of file CW_SecureChannel.cpp.
References _crypto, _driver, _logger, AES_BLOCK_SIZE, aesCbcDecrypt(), CW_SecureSession::aesKey, APDU_LC_LEN, checkStatusWord(), CW_SecureSession::clear(), CW_IV_SIZE, F, HEX, INPUT_BUFFER_LIMIT, CW_SecureSession::iv, MAC_APDU_LEN, CW_SecureSession::macCounter, macIvForCounter(), CW_SecureSession::macKey, MAX_MAC_DATA_LEN, s_apduBuf, s_dataBuf, s_macBuf, CW_Utils::safe_memcpy(), CW_Utils::secure_wipe(), and SEND_APDU_MAX_LEN.
Referenced by aesCbcEncrypt(), and mutuallyAuthenticate().

4.8.3.4 appletMajor()

```
uint8_t CW_SecureChannel::appletMajor () const  [inline]
```

Applet major version read at the last SELECT.

Returns

1 for 1.6.x, 2 for 2.x, 0 if unknown (no SELECT yet).

Definition at line 107 of file CW_SecureChannel.h.
References _appletMajor.

4.8.3.5 begin()

```
bool CW_SecureChannel::begin ()
```

Initialize the NFC transport module.

Returns

true if the module was successfully initialised, false otherwise.

Definition at line 103 of file CW_SecureChannel.cpp.
References _driver.

4.8.3.6 checkStatusWord()

```
bool CW_SecureChannel::checkStatusWord (
            const uint8_t * response,
            uint16_t responseLength,
            uint8_t sw1Expected,
            uint8_t sw2Expected)
```

Verify the SW1/SW2 status word at the end of an APDU response.

Parameters

response	APDU response buffer.
responseLength	Response length.
sw1Expected	Expected SW1 byte.
sw2Expected	Expected SW2 byte.

Returns

true if last two bytes match expectations, false otherwise.

Definition at line 123 of file CW_SecureChannel.cpp.
References _logger, F, and HEX.
Referenced by aesCbcEncrypt(), getCardCertificate(), getManufacturerCertificate(), openSecureChannel(), and selectApdu().

© 2026 Cryptnox SA

4.8.3.7 extractCardEphemeralKey()

```
bool CW_SecureChannel::extractCardEphemeralKey (
            const uint8_t * cardCertificate,
            uint8_t * cardEphemeralPubKey,
            uint8_t * fullEphemeralPubKey65 = NULL)  [static]
```

Extract the card's ephemeral EC P-256 public key from a certificate.

Parameters

in	*cardCertificate*	Raw certificate bytes.
out	*cardEphemeralPubKey*	64-byte key (X‖Y, no 0x04 prefix) for ECDH.
out	*fullEphemeralPubKey65*	Optional 65-byte key including 0x04 prefix.

Returns

true on success, false otherwise.

Definition at line 248 of file CW_SecureChannel.cpp.

4.8.3.8 getCardCertificate()

```
bool CW_SecureChannel::getCardCertificate (
            uint8_t * cardCertificate,
            uint8_t & cardCertificateLength)
```

Retrieve the card's ephemeral public key via GET CARD CERTIFICATE.
Sends a random challenge nonce to the card and stores it internally. The nonce echo check is performed inside verifyCertificateChain() to ensure replay protection is coupled with signature verification.

Parameters

out	*cardCertificate*	Buffer to receive the raw certificate bytes.
out	*cardCertificateLength*	Actual certificate length (bytes).

Returns

true on success, false otherwise.

Definition at line 200 of file CW_SecureChannel.cpp.
References _crypto, _driver, _lastNonce, _logger, checkStatusWord(), F, GETCARDCERTIFICATE_IN_BYTES, RANDOM_BYTES, RESPONSE_GETCARDCERTIFICATE_IN_BYTES, RESPONSE_STATUS_WORDS_IN_BYTES, and CW_Utils::safe_memcpy().

4.8.3.9 getManufacturerCertificate()

```
bool CW_SecureChannel::getManufacturerCertificate (
            uint8_t * cert,
            uint16_t & certLen)
```

Retrieve the manufacturer certificate stored in card flash.

Parameters

out	*cert*	Buffer to receive the raw DER certificate bytes.
out	*certLen*	Actual certificate length written.

© 2026 Cryptnox SA

Returns

true on success, false if APDU fails or buffer too small.

Definition at line 1164 of file CW_SecureChannel.cpp.
References _driver, _logger, checkStatusWord(), CW_MANUF_CERT_MAX_BYTES, F, RESPONSE_GETMANUFACTURERCERT_PAGE_IN_BYTES, RESPONSE_STATUS_WORDS_IN_BYTES, and CW_Utils::safe_memcpy().
Referenced by preFetchManufacturerCert(), and verifyCertificateChain().

4.8.3.10 inListPassiveTarget()

```
bool CW_SecureChannel::inListPassiveTarget ()
```

Detect a passive NFC target (ISO-DEP card).

Returns

true if a card was found, false otherwise.

Definition at line 107 of file CW_SecureChannel.cpp.
References _driver.

4.8.3.11 macIvForCounter()

```
bool CW_SecureChannel::macIvForCounter (
            const CW_SecureSession & session,
            uint8_t * macIv)  [private]
```

MAC IV for the current counter (zero on 1.6.x).

Returns

false if AES failed.

Definition at line 444 of file CW_SecureChannel.cpp.
References _appletMajor, _crypto, AES_BLOCK_SIZE, APPLET_MAJOR_V2, CW_SecureSession::macCounter, and CW_SecureSession::macKey.
Referenced by aesCbcDecrypt(), and aesCbcEncrypt().

4.8.3.12 mutuallyAuthenticate()

```
bool CW_SecureChannel::mutuallyAuthenticate (
            CW_SecureSession & session,
            const uint8_t * salt,
            uint8_t * clientPublicKey,
            const uint8_t * clientPrivateKey,
            CW_Curve sessionCurve,
            const uint8_t * cardEphemeralPubKey)
```

Perform ECDH derivation and MUTUALLY AUTHENTICATE with the card.
Final step of the secure channel handshake:

1. ECDH shared secret = clientPrivateKey cardEphemeralPubKey
2. (Kenc || Kmac) ← SHA-512(sharedSecret || pairingKey || salt); initial IV = Kenc[:16], MAC counter = 0
3. Sends a 32-byte random challenge as the first wrapped command (MUTUALLY AUTHENTICATE, MAC counter 1); the response MAC is verified
4. Applet 2.0+: checks the response equals SHA256(Kenc || challenge), proving the card holds this session's Kenc. 1.6.x replies random bytes and only the length is checked.

Parameters

© 2026 Cryptnox SA

out	*session*	Secure session populated with derived keys + initial IV.
in	*salt*	32-byte salt from openSecureChannel.
in	*clientPublicKey*	64-byte client public key.
in	*clientPrivateKey*	32-byte client private key.
in	*sessionCurve*	ECC curve.
in	*cardEphemeralPubKey*	64-byte card ephemeral public key.

Returns

true on success, false if ECDH failed, the card's challenge response did not match, or any APDU exchange failed.

Precondition

openSecureChannel must have been called and returned true.

Postcondition

On true: `session` has Kenc, Kmac, and rolling IV ready for aesCbcEncrypt. On false: `session` is left untouched and must not be used.

Warning

All ephemeral key material in the caller's stack (`clientPrivateKey`, `salt`) must be wiped after this call.

Cryptographic flow:

1. Compute the ECDH shared secret S = ECDH(clientPrivateKey, cardEphemeralPubKey) on the negotiated curve.
2. Derive 64 bytes of keying material via SHA-512 over (S || pairingData || salt) and split into:
   - Kenc[32]: AES-256 session encryption key
   - Kmac[32]: AES-256 session MAC key The initial encryption IV is Kenc[:16] (the card does the same), and the anti-replay MAC counter starts at 0.
3. Send MUTUALLY AUTHENTICATE as the first wrapped command (counter 1) with a random 32-byte challenge. aesCbcEncrypt verifies the response MAC.
4. Applet 2.0+: the response must equal SHA256(Kenc || challenge), proving the card holds this session's Kenc. 1.6.x answers random bytes, so only the length is checked there.

Any failure leaves `session` cleared.
Definition at line 348 of file CW_SecureChannel.cpp.
References _appletMajor, _crypto, _logger, aesCbcEncrypt(), CW_SecureSession::aesKey, APPLET_MAJOR_V2, CW_SecureSession::clear(), COMMON_PAIRING_DATA, CW_AESKEY_SIZE, CW_IV_SIZE, CW_MACKEY_SIZE, ENC_BUF_MAX_LEN, F, CW_SecureSession::iv, CW_SecureSession::macCount CW_SecureSession::macKey, MUTUALAUTH_CHALLENGE_IN_BYTES, CW_Utils::safe_memcpy(), CW_Utils::secure_compare(), and CW_Utils::secure_wipe().

4.8.3.13 openSecureChannel()

```
bool CW_SecureChannel::openSecureChannel (
            uint8_t * salt,
            uint8_t * clientPublicKey,
            uint8_t * clientPrivateKey,
            CW_Curve sessionCurve)
```

Send OPEN SECURE CHANNEL and retrieve the session salt.

© 2026 Cryptnox SA

Generates a client EC key pair via the crypto provider and sends the public key to the card; the card responds with a 32-byte salt that later feeds into Kenc / Kmac derivation in mutuallyAuthenticate.

Parameters

out	*salt*	32-byte session salt returned by the card.
out	*clientPublicKey*	64-byte freshly generated client public key.
out	*clientPrivateKey*	32-byte freshly generated client private key.
in	*sessionCurve*	ECC curve for key generation (secp256r1).

Returns

true on success, false otherwise.

Precondition

selectApdu must have been called successfully.

Warning

`clientPrivateKey` is sensitive — the caller MUST wipe it via CW_Utils::secure_wipe on every exit path.

Definition at line 281 of file CW_SecureChannel.cpp.
References _crypto, _driver, _logger, checkStatusWord(), CLIENT_PUBLIC_KEY_SIZE, F, OPENSECURECHANNEL_SALT_IN_BYTES, RESPONSE_OPENSECURECHANNEL_IN_BYTES, and CW_Utils::safe_memcpy().

4.8.3.14 operator=()

```
CW_SecureChannel & CW_SecureChannel::operator= (
            const CW_SecureChannel & )  [delete]
```

References CW_SecureChannel().

4.8.3.15 parseDerSigToRaw()

```
bool CW_SecureChannel::parseDerSigToRaw (
            const uint8_t * der,
            uint8_t derLen,
            uint8_t * raw64)  [static], [private]
```

Definition at line 1084 of file CW_SecureChannel.cpp.
References CW_Utils::safe_memcpy().
Referenced by verifyEcdsaSha256().

4.8.3.16 preFetchManufacturerCert()

```
bool CW_SecureChannel::preFetchManufacturerCert ()
```

Fetch and cache the manufacturer certificate before getCardCertificate().
The Cryptnox card state machine advances after GET_CARD_CERTIFICATE (INS=F8) and will not respond to GET_MANUFACTURER_CERTIFICATE (INS=F7) after that point. Call this method immediately after selectApdu() and before getCardCertificate() so that verifyCertificateChain() can use the cached copy without an APDU.

Returns

true if the certificate was fetched and cached, false otherwise.

Definition at line 1260 of file CW_SecureChannel.cpp.
References _cachedMfCertLen, getManufacturerCertificate(), and s_mfCertBuf.

© 2026 Cryptnox SA

4.8.3.17 printFirmwareVersion()

```
bool CW_SecureChannel::printFirmwareVersion ()
```

Print the NFC reader firmware version to the logger.

Returns

true on success, false otherwise.

Definition at line 115 of file CW_SecureChannel.cpp.
References _driver.

4.8.3.18 resetReader()

```
void CW_SecureChannel::resetReader ()
```

Reset the NFC reader hardware.
Definition at line 111 of file CW_SecureChannel.cpp.
References _driver.

4.8.3.19 selectApdu()

```
bool CW_SecureChannel::selectApdu ()
```

Send the SELECT APDU to activate the Cryptnox application.
Also records the applet major version (SELECT byte 1), which gates the 2.0 channel features: the anti-replay MAC counter and the mutual-auth proof.

Returns

true on success, false otherwise.

Definition at line 158 of file CW_SecureChannel.cpp.
References _appletMajor, _driver, _logger, checkStatusWord(), CW_MIN_APPLET_MAJOR, F, RESPONSE_SELECT_IN_BYTES, and RESPONSE_STATUS_WORDS_IN_BYTES.

4.8.3.20 verifyCertificateChain()

```
uint8_t CW_SecureChannel::verifyCertificateChain (
            const uint8_t * cardCert,
            uint8_t cardCertLen)
```

Verify the full card certificate chain against the trusted CA.
Walks the cached manufacturer certificate (fetched earlier by preFetchManufacturerCert), verifies its ECDSA signature against each entry in CW_TRUSTED_CA_KEYS, then verifies the card's ephemeral certificate against the manufacturer public key. Also checks that the challenge nonce sent in getCardCertificate was echoed back inside the card certificate.

Parameters

in	*cardCert*	Raw card certificate bytes (typically 146 bytes).
in	*cardCertLen*	Length of `cardCert`.

Returns

One of the `CW_CERT_*` result codes:

Return values

CW_CERT_OK	Chain verified end-to-end.

© 2026 Cryptnox SA

CW_CERT_FORMAT_ERROR	Malformed certificate / unexpected TLV.
CW_CERT_NONCE_MISMATCH	Card did not echo the challenge nonce.
CW_CERT_CARD_SIG_INVALID	Card cert ECDSA signature failed verification.
CW_CERT_MANUF_SIG_INVALID	Manufacturer cert signature does not match any trusted CA key.
CW_CERT_KEY_NOT_FOUND	Device public-key OID not found in the certificate.

Precondition

preFetchManufacturerCert must have been called and returned true (the manufacturer certificate is cached internally and not re-fetchable after getCardCertificate).

getCardCertificate must have been called so the challenge nonce is recorded internally.

Two-step ECDSA chain walk against the pinned trusted CAs in CW_TRUSTED_CA_KEYS (currently a single secp256r1 key, CW_CA_DLT_PUBKEY):

1. Manufacturer certificate — the cached DER blob fetched by preFetchManufacturerCert is parsed to extract its EC public key and its ECDSA signature. The signature is verified against every entry in the trusted-CA table; the first match wins. The extracted manufacturer public key becomes the trusted issuer for step 2.
2. Card certificate — `cardCert` is parsed for the device's ephemeral public key, the manufacturer ECDSA signature over that key, and the echoed challenge nonce. The nonce is compared (constant-time) against the value stored by getCardCertificate; mismatch immediately returns CW_CERT_NONCE_MISMATCH to defeat replay. The signature is then verified against the manufacturer public key from step 1.

Both signatures are SHA-256-then-ECDSA over the relevant TBS bytes. Any TLV parsing error short-circuits with CW_CERT_FORMAT_ERROR rather than touching the verifier — DER parsing is the largest attack surface here and is independently fuzzed (see `fuzz/fuzz_der.cpp`).
Definition at line 1293 of file CW_SecureChannel.cpp.
References _cachedMfCertLen, _lastNonce, _logger, CW_CERT_CARD_SIG_INVALID, CW_CERT_FORMAT_ERROR, CW_CERT_KEY_NOT_FOUND, CW_CERT_MANUF_SIG_INVALID, CW_CERT_NONCE_MISMATCH, CW_CERT_NONCE_SIZE, CW_CERT_OK, CW_TRUSTED_CA_COUNT, CW_TRUSTED_CA_KEYS, derWalkMfCert(), F, getManufacturerCertificate(), s_mfCertBuf, CW_Utils::secure_compare(), CW_Utils::secure_wipe(), and verifyEcdsaSha256().

4.8.3.21 verifyEcdsaSha256()

```
bool CW_SecureChannel::verifyEcdsaSha256 (
            const uint8_t * pubKey64,
            const uint8_t * message,
            uint16_t msgLen,
            const uint8_t * derSig,
            uint8_t derSigLen)  [private]
```

Definition at line 1148 of file CW_SecureChannel.cpp.
References _crypto, CW_CURVE_SECP256R1, and parseDerSigToRaw().
Referenced by verifyCertificateChain().

4.8.4 Member Data Documentation

4.8.4.1 _appletMajor

```
uint8_t CW_SecureChannel::_appletMajor {}  [private]
```

Applet major version from the last SELECT (0 = unknown → legacy channel).
Definition at line 356 of file CW_SecureChannel.h.
Referenced by appletMajor(), CW_SecureChannel(), macIvForCounter(), mutuallyAuthenticate(), and selectApdu().

© 2026 Cryptnox SA

4.8.4.2 _cachedMfCertLen

`uint16_t CW_SecureChannel::_cachedMfCertLen {} [private]`
Non-zero when s_mfCertBuf holds a valid pre-fetched manufacturer certificate.
Definition at line 353 of file CW_SecureChannel.h.
Referenced by CW_SecureChannel(), preFetchManufacturerCert(), and verifyCertificateChain().

4.8.4.3 _crypto

`CW_CryptoProvider& CW_SecureChannel::_crypto [private]`
Crypto operations (AES, SHA, ECDH, RNG).
Definition at line 346 of file CW_SecureChannel.h.
Referenced by aesCbcDecrypt(), aesCbcEncrypt(), CW_SecureChannel(), getCardCertificate(), macIvForCounter(), mutuallyAuthenticate(), openSecureChannel(), and verifyEcdsaSha256().

4.8.4.4 _driver

`CW_NfcTransport& CW_SecureChannel::_driver [private]`
NFC transport for APDU exchange.
Definition at line 344 of file CW_SecureChannel.h.
Referenced by aesCbcEncrypt(), begin(), CW_SecureChannel(), getCardCertificate(), getManufacturerCertificate(), inListPassiveTarget(), openSecureChannel(), printFirmwareVersion(), resetReader(), and selectApdu().

4.8.4.5 _lastNonce

`uint8_t CW_SecureChannel::_lastNonce[CW_CERT_NONCE_SIZE] {} [private]`
Nonce sent in the last getCardCertificate() call; checked in verifyCertificateChain().
Definition at line 350 of file CW_SecureChannel.h.
Referenced by getCardCertificate(), and verifyCertificateChain().

4.8.4.6 _logger

`CW_Logger& CW_SecureChannel::_logger [private]`
Logging interface.
Definition at line 345 of file CW_SecureChannel.h.
Referenced by aesCbcDecrypt(), aesCbcEncrypt(), checkStatusWord(), CW_SecureChannel(), getCardCertificate(), getManufacturerCertificate(), mutuallyAuthenticate(), openSecureChannel(), selectApdu(), and verifyCertificateChain().

4.8.4.7 _platform

`CW_Platform& CW_SecureChannel::_platform [private]`
Platform abstraction (sleep_ms).
Definition at line 347 of file CW_SecureChannel.h.
Referenced by CW_SecureChannel().
The documentation for this class was generated from the following files:

- CW_SecureChannel.h
- CW_SecureChannel.cpp

4.9 CW_SecureSession Struct Reference

Holds cryptographic session state for reentrant secure channel operations.
`#include <CW_Defs.h>`

© 2026 Cryptnox SA

Public Member Functions

- CW_SecureSession ()=default

  Zero-initialise all session keys, IV and counter.
- void clear ()

  Securely clear all session keys and IV.

Public Attributes

- uint8_t aesKey [CW_AESKEY_SIZE] {}
- uint8_t macKey [CW_MACKEY_SIZE] {}
- uint8_t iv [CW_IV_SIZE] {}
- uint32_t macCounter {}

4.9.1 Detailed Description

Holds cryptographic session state for reentrant secure channel operations.
Encapsulates all session-specific cryptographic material (Kenc, Kmac, rolling IV), allowing functions to be reentrant by passing session state as a parameter.
Definition at line 200 of file CW_Defs.h.

4.9.2 Constructor & Destructor Documentation

4.9.2.1 CW_SecureSession()

```
CW_SecureSession::CW_SecureSession () [default]
```

Zero-initialise all session keys, IV and counter.

4.9.3 Member Function Documentation

4.9.3.1 clear()

```
void CW_SecureSession::clear () [inline]
```

Securely clear all session keys and IV.
Definition at line 210 of file CW_Defs.h.
References aesKey, iv, macCounter, macKey, and CW_Utils::secure_wipe().
Referenced by CW_SecureChannel::aesCbcEncrypt(), CryptnoxWallet::connect(), CryptnoxWallet::disconnect(), CW_SecureChannel::mutuallyAuthenticate(), and openSession().

4.9.4 Member Data Documentation

4.9.4.1 aesKey

```
uint8_t CW_SecureSession::aesKey[CW_AESKEY_SIZE] {}
```

AES-256 session encryption key (Kenc)
Definition at line 201 of file CW_Defs.h.
Referenced by CW_SecureChannel::aesCbcDecrypt(), CW_SecureChannel::aesCbcEncrypt(), clear(), isCleared(), CryptnoxWallet::isSecureChannelOpen(), CW_SecureChannel::mutuallyAuthenticate(), and openSession().

4.9.4.2 iv

```
uint8_t CW_SecureSession::iv[CW_IV_SIZE] {}
```

Current AES-CBC IV (rolling IV)
Definition at line 203 of file CW_Defs.h.
Referenced by CW_SecureChannel::aesCbcEncrypt(), clear(), CW_SecureChannel::mutuallyAuthenticate(), and openSession().

© 2026 Cryptnox SA

4.9.4.3 macCounter

`uint32_t CW_SecureSession::macCounter {}`
GAP-01 anti-replay counter (applet 2.0+): +1 per wrapped command, folded into both MACs, never sent
Definition at line 204 of file CW_Defs.h.
Referenced by CW_SecureChannel::aesCbcEncrypt(), check_session_clearing(), clear(), isCleared(), CW_SecureChannel::macIvForCounter(), and CW_SecureChannel::mutuallyAuthenticate().

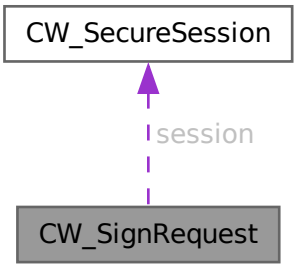
4.9.4.4 macKey

`uint8_t CW_SecureSession::macKey[CW_MACKEY_SIZE] {}`
AES-256 session MAC key (Kmac)
Definition at line 202 of file CW_Defs.h.
Referenced by CW_SecureChannel::aesCbcDecrypt(), CW_SecureChannel::aesCbcEncrypt(), clear(), isCleared(), CW_SecureChannel::macIvForCounter(), CW_SecureChannel::mutuallyAuthenticate(), and openSession().
The documentation for this struct was generated from the following file:

- CW_Defs.h

4.10 CW_SignRequest Struct Reference

Request parameters for CryptnoxWallet::sign.
`#include <CryptnoxWallet.h>`
Collaboration diagram for CW_SignRequest:



Public Member Functions

- CW_SignRequest (CW_SecureSession &sess, uint8_t kType=CW_SIGN_CURR_K1, uint8_t sigType=CW_SIGN_SIG_ECDSA_LOW_S, bool pinless=CW_SIGN_WITH_PIN)
  Construct a sign request with sensible defaults.
- ~CW_SignRequest ()
  Securely wipes the PIN buffer.

Public Attributes

- CW_SecureSession & session
- uint8_t keyType {}
- uint8_t signatureType {}

© 2026 Cryptnox SA

- uint8_t pin [CW_MAX_PIN_LENGTH] {}
- bool pinLessMode {}
- const uint8_t * hash {}
- uint8_t hashLength {}
- const uint8_t * derivePath {}
- uint8_t derivePathLength {}

4.10.1 Detailed Description

Request parameters for CryptnoxWallet::sign.
Owns the PIN buffer for the lifetime of the request — the destructor securely wipes it (CW_Utils::secure_wipe), so allocating the request on the stack inside a tight scope is the recommended pattern.
Definition at line 71 of file CryptnoxWallet.h.

4.10.2 Constructor & Destructor Documentation

4.10.2.1 CW_SignRequest()

```
CW_SignRequest::CW_SignRequest (
            CW_SecureSession & sess,
            uint8_t kType = CW_SIGN_CURR_K1,
            uint8_t sigType = CW_SIGN_SIG_ECDSA_LOW_S,
            bool pinless = CW_SIGN_WITH_PIN)  [inline], [explicit]
```

Construct a sign request with sensible defaults.

Parameters

in	*sess*	Open secure session.
in	*kType*	Key type. Defaults to CW_SIGN_CURR_K1.
in	*sigType*	Signature type. Defaults to CW_SIGN_SIG_ECDSA_LOW_S.
in	*pinless*	PIN mode. Defaults to PIN required (CW_SIGN_WITH_PIN).

Definition at line 89 of file CryptnoxWallet.h.
References CW_SIGN_CURR_K1, CW_SIGN_SIG_ECDSA_LOW_S, CW_SIGN_WITH_PIN, derivePath, derivePathLength, hash, hashLength, keyType, pinLessMode, session, and signatureType.

4.10.2.2 ~CW_SignRequest()

```
CW_SignRequest::~CW_SignRequest ()  [inline]
```

Securely wipes the PIN buffer.
Definition at line 99 of file CryptnoxWallet.h.
References pin, and CW_Utils::secure_wipe().

4.10.3 Member Data Documentation

4.10.3.1 derivePath

```
const uint8_t* CW_SignRequest::derivePath {}
```

BIP32 path bytes for DERIVE modes; NULL for CURR / PINLESS modes.
Definition at line 79 of file CryptnoxWallet.h.
Referenced by CryptnoxWallet::buildSignPayload(), CW_SignRequest(), cwMaxEd25519Message(), and CryptnoxWallet::validateSignRequest().

4.10.3.2 derivePathLength

```
uint8_t CW_SignRequest::derivePathLength {}
```

Length of derivePath in bytes (must be a multiple of 4).

© 2026 Cryptnox SA

Definition at line 80 of file CryptnoxWallet.h.
Referenced by CryptnoxWallet::buildSignPayload(), CW_SignRequest(), cwMaxEd25519Message(), and CryptnoxWallet::validateSignRequest().

4.10.3.3 hash

```
const uint8_t* CW_SignRequest::hash {}
```

Data to sign. For ECDSA: a digest (typically 32 bytes). For Ed25519 (`CW_SIGN_*_ED25519`): the RAW message — the card hashes it internally.
Definition at line 77 of file CryptnoxWallet.h.
Referenced by CryptnoxWallet::buildSignPayload(), CW_SignRequest(), signWith(), and CryptnoxWallet::validateSignRequest().

4.10.3.4 hashLength

```
uint8_t CW_SignRequest::hashLength {}
```

Length of `hash`. ECDSA: $\leq$ CW_HASH_SIZE. Ed25519: whatever the page leaves after the length prefix, the path actually sent and the PIN — 197 B with the current key, 181 B with a 4-level path; CW_MAX_ED25519_MESSAGE_LENGTH (177) is the worst case.
Definition at line 78 of file CryptnoxWallet.h.
Referenced by CryptnoxWallet::buildSignPayload(), CW_SignRequest(), signWith(), and CryptnoxWallet::validateSignRequest().

4.10.3.5 keyType

```
uint8_t CW_SignRequest::keyType {}
```

Key / path type — one of the `CW_SIGN_CURR_*`, `CW_SIGN_DERIVE_*`, `CW_SIGN_PINLESS_K1`, `CW_SIGN_*_ED25519` constants.
Definition at line 73 of file CryptnoxWallet.h.
Referenced by CryptnoxWallet::buildSignPayload(), CW_SignRequest(), cwMaxEd25519Message(), CryptnoxWallet::sendSignApdu(), CryptnoxWallet::sign(), and CryptnoxWallet::validateSignRequest().

4.10.3.6 pin

```
uint8_t CW_SignRequest::pin[CW_MAX_PIN_LENGTH] {}
```

PIN bytes (4–9 ASCII digits). Zero-padded; cleared in the destructor.
Definition at line 75 of file CryptnoxWallet.h.
Referenced by CryptnoxWallet::buildSignPayload(), signWith(), CryptnoxWallet::validateSignRequest(), and ~CW_SignRequest().

4.10.3.7 pinLessMode

```
bool CW_SignRequest::pinLessMode {}
```

false = PIN path, true = PIN-less path (requires `keyType` == CW_SIGN_PINLESS_K1).
Definition at line 76 of file CryptnoxWallet.h.
Referenced by CryptnoxWallet::buildSignPayload(), CW_SignRequest(), cwMaxEd25519Message(), and CryptnoxWallet::validateSignRequest().

4.10.3.8 session

```
CW_SecureSession& CW_SignRequest::session
```

Reference to an open secure session.
Definition at line 72 of file CryptnoxWallet.h.
Referenced by CW_SignRequest(), CryptnoxWallet::sendSignApdu(), and CryptnoxWallet::validateSignRequest().

© 2026 Cryptnox SA

4.10.3.9 signatureType

`uint8_t CW_SignRequest::signatureType {}`
Signature format — `CW_SIGN_SIG_ECDSA_LOW_S` / EOSIO / SCHNORR_BIP340. Ignored for Ed25519 key types (EdDSA is forced).
Definition at line 74 of file CryptnoxWallet.h.
Referenced by CW_SignRequest(), CryptnoxWallet::sendSignApdu(), and CryptnoxWallet::sign().
The documentation for this struct was generated from the following file:

- CryptnoxWallet.h

4.11 CW_SignResult Struct Reference

Result of CryptnoxWallet::sign.
`#include <CryptnoxWallet.h>`

Public Member Functions

- CW_SignResult ()
  Construct a default-failure result.

Public Attributes

- uint8_t signature [CW_RAW_SIGNATURE_SIZE] {}
- uint8_t errorCode {}

4.11.1 Detailed Description

Result of CryptnoxWallet::sign.
The error code is checked first: when it is CW_OK the signature is valid raw (r || s) on secp256k1 / secp256r1 (depending on the `keyType` used). On any other code `signature` is left zero.
Definition at line 112 of file CryptnoxWallet.h.

4.11.2 Constructor & Destructor Documentation

4.11.2.1 CW_SignResult()

`CW_SignResult::CW_SignResult () [inline]`
Construct a default-failure result.
Definition at line 117 of file CryptnoxWallet.h.
References CW_NOK, and errorCode.

4.11.3 Member Data Documentation

4.11.3.1 errorCode

`uint8_t CW_SignResult::errorCode {}`
CW_OK on success, otherwise a `CW_SIGN_*` / `CW_INVALID_SESSION` code.
Definition at line 114 of file CryptnoxWallet.h.
Referenced by CW_SignResult(), CryptnoxWallet::extractRawSignature(), CryptnoxWallet::sendSignApdu(), CryptnoxWallet::sign(), and CryptnoxWallet::validateSignRequest().

4.11.3.2 signature

`uint8_t CW_SignResult::signature[CW_RAW_SIGNATURE_SIZE] {}`
Raw 64-byte signature: ECDSA r[32]||s[32], or Ed25519 R[32]||S[32]. Zero on failure.
Definition at line 113 of file CryptnoxWallet.h.
Referenced by CryptnoxWallet::extractRawSignature(), and CryptnoxWallet::sign().

© 2026 Cryptnox SA

The documentation for this struct was generated from the following file:

- CryptnoxWallet.h

4.12 CW_Tron Class Reference

TRON address derivation and Base58Check encoding.
`#include <CW_Tron.h>`

Static Public Member Functions

- static bool addressBytesFromPublicKey (const uint8_t *publicKey64, uint8_t *out)

  Derive the raw 21-byte TRON address from an uncompressed public key.
- static bool addressFromPublicKey (const uint8_t *publicKey64, CW_CryptoProvider &crypto, char *out, size_t outSize)

  Derive the Base58Check TRON address from an uncompressed public key.
- static bool encodeAddress (const uint8_t *addressBytes, CW_CryptoProvider &crypto, char *out, size_t outSize)

  Base58Check-encode a raw 21-byte TRON address.
- static bool decodeAddress (const char *address, CW_CryptoProvider &crypto, uint8_t *out)

  Decode and validate a Base58Check TRON address.
- static bool keccak256 (const uint8_t *data, size_t len, uint8_t *out)

  Compute Keccak-256 (the pre-standard padding, as used by Ethereum and TRON — not SHA3-256).

4.12.1 Detailed Description

TRON address derivation and Base58Check encoding.
Stateless helpers. The Base58Check checksum is a double SHA-256, so the encode / decode entry points take the host's CW_CryptoProvider rather than bundling a second hash implementation. Keccak-256 is implemented internally — no adapter method is needed for it.
Definition at line 80 of file CW_Tron.h.

4.12.2 Member Function Documentation

4.12.2.1 addressBytesFromPublicKey()

```
bool CW_Tron::addressBytesFromPublicKey (
            const uint8_t * publicKey64,
            uint8_t * out)  [static]
```

Derive the raw 21-byte TRON address from an uncompressed public key.
Computes `0x41` || keccak256(publicKey64)[12..32].

Parameters

`in`	*publicKey64*	64-byte public key (X||Y, no `0x04` prefix).
`out`	*out*	21-byte output buffer.

Returns

true on success, false if either pointer is `NULL`.

Definition at line 281 of file CW_Tron.cpp.
References CW_TRON_ADDRESS_PREFIX, ETH_ADDRESS_BYTES, keccak256(), KECCAK_DIGEST_BYTES, and PUBLIC_KEY_BYTES.
Referenced by addressFromPublicKey().

© 2026 Cryptnox SA

4.12.2.2 addressFromPublicKey()

```
bool CW_Tron::addressFromPublicKey (
            const uint8_t * publicKey64,
            CW_CryptoProvider & crypto,
            char * out,
            size_t outSize)  [static]
```

Derive the Base58Check TRON address from an uncompressed public key.
Convenience wrapper: addressBytesFromPublicKey followed by encodeAddress.

Parameters

in	*publicKey64*	64-byte public key (X||Y, no 0x04 prefix).
in	*crypto*	Crypto provider (used for the SHA-256 checksum).
out	*out*	Output buffer for the NUL-terminated address.
in	*outSize*	Capacity of out; must be at least CW_TRON_ADDRESS_STR_SIZE.

Returns

true on success, false on bad arguments or hash failure.

Definition at line 316 of file CW_Tron.cpp.
References addressBytesFromPublicKey(), CW_TRON_ADDRESS_BYTES, and encodeAddress().

4.12.2.3 decodeAddress()

```
bool CW_Tron::decodeAddress (
            const char * address,
            CW_CryptoProvider & crypto,
            uint8_t * out)  [static]
```

Decode and validate a Base58Check TRON address.
Rejects anything that is not a well-formed mainnet address: wrong length, characters outside the Base58 alphabet, a prefix other than 0x41, or a checksum mismatch. Use this on every address that arrives from outside the firmware (config, QR code, RPC response) before it is used to build or verify a transaction.

Parameters

in	*address*	34-character NUL-terminated Base58Check address.
in	*crypto*	Crypto provider (used for the SHA-256 checksum).
out	*out*	21-byte output buffer for the raw address.

Returns

true if the address is valid and out was written, false otherwise (out is then left untouched).

Definition at line 325 of file CW_Tron.cpp.
References base58_decode_payload(), CW_TRON_ADDRESS_BYTES, CW_TRON_ADDRESS_PREFIX, CW_TRON_ADDRESS_STR_LEN, CW_Utils::secure_compare(), tron_checksum(), TRON_CHECKSUM_BYTES, and TRON_PAYLOAD_BYTES.

4.12.2.4 encodeAddress()

```
bool CW_Tron::encodeAddress (
            const uint8_t * addressBytes,
            CW_CryptoProvider & crypto,
            char * out,
```

© 2026 Cryptnox SA

```
            size_t outSize)  [static]
```

Base58Check-encode a raw 21-byte TRON address.
Appends the first 4 bytes of `sha256(sha256(addressBytes))` as the checksum and Base58-encodes the 25-byte result.

Parameters

`in`	*addressBytes*	21-byte address (must start with `0x41`).
`in`	*crypto*	Crypto provider (used for the SHA-256 checksum).
`out`	*out*	Output buffer for the NUL-terminated address.
`in`	*outSize*	Capacity of `out`; must be at least CW_TRON_ADDRESS_STR_SIZE.

Returns

true on success, false on bad arguments, wrong prefix, or hash failure.

Definition at line 297 of file CW_Tron.cpp.
References base58_encode_payload(), CW_TRON_ADDRESS_BYTES, CW_TRON_ADDRESS_PREFIX, CW_TRON_ADDRESS_STR_SIZE, tron_checksum(), and TRON_PAYLOAD_BYTES.
Referenced by addressFromPublicKey().

4.12.2.5 keccak256()

```
bool CW_Tron::keccak256 (
            const uint8_t * data,
            size_t len,
            uint8_t * out)  [static]
```

Compute Keccak-256 (the pre-standard padding, as used by Ethereum and TRON — not SHA3-256).
Exposed because TRON address derivation needs it and no CW_CryptoProvider method covers it.

Parameters

`in`	*data*	Input buffer.
`in`	*len*	Number of bytes to hash.
`out`	*out*	32-byte digest output.

Returns

true on success, false if a pointer is `NULL`.

Definition at line 157 of file CW_Tron.cpp.
References keccak_absorb(), KECCAK_DIGEST_BYTES, and KECCAK_STATE_LANES.
Referenced by addressBytesFromPublicKey().
The documentation for this class was generated from the following files:

- CW_Tron.h
- CW_Tron.cpp

4.13 CW_Utils Class Reference

Portable utility functions for cryptographic and security operations.
`#include <CW_Utils.h>`

© 2026 Cryptnox SA

Static Public Member Functions

- static bool secure_compare (const uint8_t *a, const uint8_t *b, size_t len)

  Constant-time buffer comparison, resistant to timing side-channel attacks.
- static void secure_wipe (uint8_t *buf, size_t len)

  Securely zero a buffer, guaranteed not to be optimised away.
- static bool safe_memcpy (uint8_t *dst, size_t dstSize, const uint8_t *src, size_t count)

  Safe memcpy — validates pointers, sizes, and checks for overlap.
- static bool fill_secure_random (uint8_t *dest, size_t len)

  Fill `len` bytes at `dest` with cryptographically random data.

4.13.1 Detailed Description

Portable utility functions for cryptographic and security operations.
All methods here are platform-independent pure C++ with no dependency on Arduino or any hardware-specific library.
Hardware-specific helpers (e.g. TRNG byte generation) live in the concrete crypto provider implementation (ArduinoCryptoProvider).
Definition at line 45 of file CW_Utils.h.

4.13.2 Member Function Documentation

4.13.2.1 fill_secure_random()

```
bool CW_Utils::fill_secure_random (
            uint8_t * dest,
            size_t len)  [static]
```

Fill `len` bytes at `dest` with cryptographically random data.
The implementation is platform-specific. On ESP32 it calls esp_fill_random() after verifying that Wi-Fi or Bluetooth is active, ensuring the hardware TRNG is properly seeded. Returns false (hard failure) if neither radio is on (SEC-001).

Parameters

dest	Destination buffer.
len	Number of random bytes to generate.

Returns

true on success, false if dest is NULL, len is zero, or no radio is active.

4.13.2.2 safe_memcpy()

```
bool CW_Utils::safe_memcpy (
            uint8_t * dst,
            size_t dstSize,
            const uint8_t * src,
            size_t count)  [static]
```

Safe memcpy — validates pointers, sizes, and checks for overlap.
Safe memcpy — checks src, dst and size before copying.

Parameters

dst	Destination buffer.
dstSize	Capacity of the destination buffer.

© 2026 Cryptnox SA

src	Source buffer.
count	Number of bytes to copy.

Returns

true if copy succeeded, false if parameters are invalid.

true if copy succeeded, false otherwise.

Definition at line 50 of file CW_Utils.cpp.
Referenced by CW_SecureChannel::aesCbcDecrypt(), CW_SecureChannel::aesCbcEncrypt(), CryptnoxWallet::buildSignPayload(), CryptnoxWallet::extractRawSignature(), CW_SecureChannel::getCardCertificate(), CryptnoxWallet::getCardInfo(), CW_SecureChannel::getManufacturerCertificate(), CryptnoxWallet::getPublicKey(), CryptnoxWallet::getPublicKey(), CW_SecureChannel::mutuallyAuthenticate(), CW_SecureChannel::openSecureChanne CryptnoxWallet::parseDerSignature(), CW_SecureChannel::parseDerSigToRaw(), CryptnoxWallet::sign(), and CryptnoxWallet::verifyPin().

4.13.2.3 secure_compare()

```
bool CW_Utils::secure_compare (
            const uint8_t * a,
            const uint8_t * b,
            size_t len)  [static]
```

Constant-time buffer comparison, resistant to timing side-channel attacks.
Always iterates over the full length regardless of where the first difference occurs, preventing an attacker from inferring the correct value byte-by-byte via timing measurements.

Parameters

a	Pointer to the first buffer.
b	Pointer to the second buffer.
len	Number of bytes to compare.

Returns

true if the buffers are identical, false otherwise.

Definition at line 22 of file CW_Utils.cpp.
Referenced by CW_SecureChannel::aesCbcDecrypt(), CW_Tron::decodeAddress(), CW_SecureChannel::mutuallyAuth and CW_SecureChannel::verifyCertificateChain().

4.13.2.4 secure_wipe()

```
void CW_Utils::secure_wipe (
            uint8_t * buf,
            size_t len)  [static]
```

Securely zero a buffer, guaranteed not to be optimised away.
Uses a volatile pointer so the compiler cannot elide the writes, ensuring sensitive material is actually erased from memory.

Parameters

buf	Pointer to the buffer to wipe.
len	Number of bytes to zero.

Definition at line 37 of file CW_Utils.cpp.

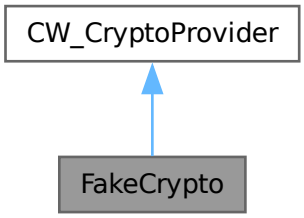
© 2026 Cryptnox SA

Referenced by CW_SecureChannel::aesCbcDecrypt(), CW_SecureChannel::aesCbcEncrypt(), CW_SecureSession::clear(), CryptnoxWallet::establishSecureChannel(), CryptnoxWallet::extractRawSignature(), CryptnoxWallet::getCardInfo(), CryptnoxWallet::getPublicKey(), CryptnoxWallet::getPublicKey(), CW_SecureChannel::mutuallyAuthenticate(), CryptnoxWallet::sign(), CW_SecureChannel::verifyCertificateChain(), CryptnoxWallet::verifyPin(), and CW_SignRequest::~CW_SignRequest().
The documentation for this class was generated from the following files:

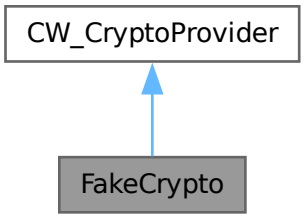
- CW_Utils.h
- CW_Utils.cpp

4.14 FakeCrypto Class Reference

Inheritance diagram for FakeCrypto:



Collaboration diagram for FakeCrypto:



Public Member Functions

- bool sha256 (const uint8_t *, size_t, uint8_t *out) override
  Compute SHA-256 over a contiguous data buffer.
- bool sha512 (const uint8_t *, size_t, uint8_t *out) override
  Compute SHA-512 over a contiguous data buffer.
- bool ecdh (const uint8_t *, const uint8_t *, uint8_t *s, CW_Curve) override
  ECDH shared secret computation.
- bool makeKey (uint8_t *pub, uint8_t *priv, CW_Curve) override
  Generate a new EC key pair.

© 2026 Cryptnox SA

- bool random (uint8_t *d, unsigned n) override

  Fill a buffer with cryptographically random bytes.
- bool ecdsaVerify (const uint8_t *, const uint8_t *, size_t, const uint8_t *, CW_Curve) override

  Verify an ECDSA signature (raw r||s, 64 bytes) against a message hash.
- uint16_t aesCbcEncrypt (const uint8_t *in, uint16_t len, uint8_t *out, const uint8_t *key, uint8_t, uint8_t *iv, bool bitPadding) override

  AES-CBC encrypt.
- uint16_t aesCbcDecrypt (uint8_t *in, uint16_t len, uint8_t *out, const uint8_t *key, uint8_t, uint8_t *iv, bool bitPadding) override

  AES-CBC decrypt.

Public Member Functions inherited from CW_CryptoProvider

- virtual ~CW_CryptoProvider ()

Public Attributes

- bool failMacIv = false
- bool failMac = false

4.14.1 Detailed Description

Definition at line 58 of file test_pr28.cpp.

4.14.2 Member Function Documentation

4.14.2.1 aesCbcDecrypt()

```
uint16_t FakeCrypto::aesCbcDecrypt (
            uint8_t * in,
            uint16_t len,
            uint8_t * out,
            const uint8_t * key,
            uint8_t keyLen,
            uint8_t * iv,
            bool bitPadding)  [inline], [override], [virtual]
```

AES-CBC decrypt.

Parameters

in	*in*	Ciphertext input buffer (non-const; may be modified internally).
in	*len*	Length of the ciphertext.
out	*out*	Plaintext output buffer.
in	*key*	AES key bytes.
in	*keyLen*	AES key length in bytes.
in,out	*iv*	16-byte IV used as decrypt IV.
in	*bitPadding*	true = Bit padding removal; false = Null padding (no removal).

Returns

Length of the plaintext written to out.

Implements CW_CryptoProvider.
Definition at line 88 of file test_pr28.cpp.

© 2026 Cryptnox SA

4.14.2.2 aesCbcEncrypt()

```
uint16_t FakeCrypto::aesCbcEncrypt (
            const uint8_t * in,
            uint16_t len,
            uint8_t * out,
            const uint8_t * key,
            uint8_t keyLen,
            uint8_t * iv,
            bool bitPadding)  [inline], [override], [virtual]
```

AES-CBC encrypt.

Parameters

in	*in*	Plaintext input buffer.
in	*len*	Length of the plaintext.
out	*out*	Ciphertext output buffer (must be large enough for padding).
in	*key*	AES key bytes.
in	*keyLen*	AES key length in bytes (16, 24, or 32).
in,out	*iv*	16-byte IV; updated to last cipher block on return.
in	*bitPadding*	true = ISO/IEC 9797-1 Method 2 (Bit) padding; false = Null padding (no padding added).

Returns

Length of the ciphertext written to `out`.

Implements CW_CryptoProvider.
Definition at line 70 of file test_pr28.cpp.
References failMac, and failMacIv.

4.14.2.3 ecdh()

```
bool FakeCrypto::ecdh (
            const uint8_t * pubKey,
            const uint8_t * privKey,
            uint8_t * secret,
            CW_Curve curve)  [inline], [override], [virtual]
```

ECDH shared secret computation.

Parameters

in	*pubKey*	Remote public key (64 bytes, X||Y, no 0x04 prefix).
in	*privKey*	Local private key (32 bytes).
out	*secret*	32-byte shared secret output.
in	*curve*	Curve identifier (CW_CURVE_SECP256R1 or CW_CURVE_SECP256K1).

Returns

true on success, false otherwise.

Implements CW_CryptoProvider.
Definition at line 65 of file test_pr28.cpp.

© 2026 Cryptnox SA

4.14.2.4 ecdsaVerify()

```
bool FakeCrypto::ecdsaVerify (
            const uint8_t * pubKey64,
            const uint8_t * hash,
            size_t hashLen,
            const uint8_t * sig,
            CW_Curve curve)  [inline], [override], [virtual]
```

Verify an ECDSA signature (raw r||s, 64 bytes) against a message hash.

Parameters

in	*pubKey64*	64-byte public key (X||Y, no 0x04 prefix).
in	*hash*	Message hash buffer.
in	*hashLen*	Length of the hash in bytes.
in	*sig*	64-byte raw signature (r[32]||s[32]).
in	*curve*	Curve identifier for the verification operation.

Returns

true if the signature is valid, false otherwise.

Implements CW_CryptoProvider.
Definition at line 68 of file test_pr28.cpp.

4.14.2.5 makeKey()

```
bool FakeCrypto::makeKey (
            uint8_t * pubKey,
            uint8_t * privKey,
            CW_Curve curve)  [inline], [override], [virtual]
```

Generate a new EC key pair.

Parameters

out	*pubKey*	64-byte public key output (X||Y, no prefix).
out	*privKey*	32-byte private key output.
in	*curve*	Curve identifier (CW_CURVE_SECP256R1 or CW_CURVE_SECP256K1).

Returns

true on success, false otherwise.

Implements CW_CryptoProvider.
Definition at line 66 of file test_pr28.cpp.

4.14.2.6 random()

```
bool FakeCrypto::random (
            uint8_t * dest,
            unsigned size)  [inline], [override], [virtual]
```

Fill a buffer with cryptographically random bytes.

Parameters

© 2026 Cryptnox SA

out	*dest*	Buffer to fill.
in	*size*	Number of bytes to generate.

Returns

true on success, false otherwise.

Implements CW_CryptoProvider.
Definition at line 67 of file test_pr28.cpp.

4.14.2.7 sha256()

```
bool FakeCrypto::sha256 (
            const uint8_t * data,
            size_t len,
            uint8_t * out)  [inline], [override], [virtual]
```

Compute SHA-256 over a contiguous data buffer.

Parameters

in	*data*	Input buffer.
in	*len*	Number of bytes to hash.
out	*out*	32-byte output buffer.

Returns

true on success, false if the underlying hash accelerator faults.

Implements CW_CryptoProvider.
Definition at line 63 of file test_pr28.cpp.

4.14.2.8 sha512()

```
bool FakeCrypto::sha512 (
            const uint8_t * data,
            size_t len,
            uint8_t * out)  [inline], [override], [virtual]
```

Compute SHA-512 over a contiguous data buffer.

Parameters

in	*data*	Input buffer.
in	*len*	Number of bytes to hash.
out	*out*	64-byte output buffer.

Returns

true on success, false if the underlying hash accelerator faults.

Implements CW_CryptoProvider.
Definition at line 64 of file test_pr28.cpp.

4.14.3 Member Data Documentation

© 2026 Cryptnox SA

4.14.3.1 failMac

```
bool FakeCrypto::failMac = false
```
Definition at line 61 of file test_pr28.cpp.
Referenced by aesCbcEncrypt(), and check_session_clearing().

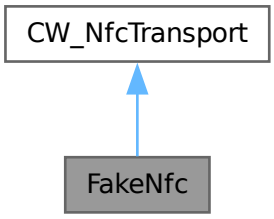
4.14.3.2 failMacIv

```
bool FakeCrypto::failMacIv = false
```
Definition at line 60 of file test_pr28.cpp.
Referenced by aesCbcEncrypt(), and check_session_clearing().
The documentation for this class was generated from the following file:

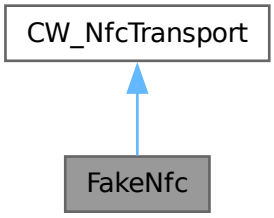
- tests/test_pr28.cpp

4.15 FakeNfc Class Reference

Inheritance diagram for FakeNfc:



Collaboration diagram for FakeNfc:



Public Member Functions

- bool begin () override

  Initialize the NFC transport hardware.
- bool inListPassiveTarget () override

  Detect the presence of a passive ISO-DEP NFC target.

© 2026 Cryptnox SA

- void resetReader () override

  Reset the NFC reader/field for the next card detection cycle.
- bool printFirmwareVersion () override

  Print NFC module firmware version information to the logger.
- bool sendAPDU (const uint8_t *apdu, uint8_t apduLen, uint8_t *response, uint8_t &responseLen) override

  Send an APDU command to the card and receive the response.

Public Member Functions inherited from CW_NfcTransport

- virtual bool sendAPDULarge (const uint8_t *apdu, uint8_t apduLen, uint8_t *response, uint16_t &responseLen)

  Send an APDU and receive a response that may exceed 255 bytes.
- virtual ~CW_NfcTransport ()

Public Attributes

- uint8_t appletMajor = 2U
- bool sendOk = false
- uint8_t resp [255] = { 0U }
- uint8_t respLen = 0U
- unsigned sent = 0U
- uint8_t last [255] = { 0U }

4.15.1 Detailed Description

Definition at line 28 of file test_pr28.cpp.

4.15.2 Member Function Documentation

4.15.2.1 begin()

```
bool FakeNfc::begin ()  [inline], [override], [virtual]
```

Initialize the NFC transport hardware.

Returns

true if initialization succeeded, false otherwise.

Implements CW_NfcTransport.
Definition at line 37 of file test_pr28.cpp.

4.15.2.2 inListPassiveTarget()

```
bool FakeNfc::inListPassiveTarget ()  [inline], [override], [virtual]
```

Detect the presence of a passive ISO-DEP NFC target.

Returns

true if a card is detected, false otherwise.

Implements CW_NfcTransport.
Definition at line 38 of file test_pr28.cpp.

© 2026 Cryptnox SA

4.15.2.3 printFirmwareVersion()

```
bool FakeNfc::printFirmwareVersion ()  [inline], [override], [virtual]
```

Print NFC module firmware version information to the logger.

Returns

true if firmware info was retrieved successfully, false otherwise.

Implements CW_NfcTransport.
Definition at line 40 of file test_pr28.cpp.

4.15.2.4 resetReader()

```
void FakeNfc::resetReader ()  [inline], [override], [virtual]
```

Reset the NFC reader/field for the next card detection cycle.
Implements CW_NfcTransport.
Definition at line 39 of file test_pr28.cpp.

4.15.2.5 sendAPDU()

```
bool FakeNfc::sendAPDU (
            const uint8_t * apdu,
            uint8_t apduLen,
            uint8_t * response,
            uint8_t & responseLen)  [inline], [override], [virtual]
```

Send an APDU command to the card and receive the response.

Parameters

in	*apdu*	APDU command bytes.
in	*apduLen*	Length of the APDU command.
out	*response*	Buffer to receive the card response.
out	*responseLen*	Actual number of bytes written to response.

Returns

true if the exchange succeeded, false otherwise.

Implements CW_NfcTransport.
Definition at line 42 of file test_pr28.cpp.
References appletMajor, last, resp, respLen, sendOk, and sent.

4.15.3 Member Data Documentation

4.15.3.1 appletMajor

```
uint8_t FakeNfc::appletMajor = 2U
```

Definition at line 30 of file test_pr28.cpp.
Referenced by check_ed25519_applet_gate(), and sendAPDU().

4.15.3.2 last

```
uint8_t FakeNfc::last[255] = { 0U }
```

Definition at line 35 of file test_pr28.cpp.
Referenced by check_ed25519_applet_gate(), check_pubkey_allow_list(), and sendAPDU().

© 2026 Cryptnox SA

4.15.3.3 resp

```
uint8_t FakeNfc::resp[255] = { 0U }
```
Definition at line 32 of file test_pr28.cpp.
Referenced by check_session_clearing(), and sendAPDU().

4.15.3.4 respLen

```
uint8_t FakeNfc::respLen = 0U
```
Definition at line 33 of file test_pr28.cpp.
Referenced by check_session_clearing(), and sendAPDU().

4.15.3.5 sendOk

```
bool FakeNfc::sendOk = false
```
Definition at line 31 of file test_pr28.cpp.
Referenced by check_session_clearing(), and sendAPDU().

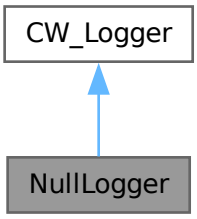
4.15.3.6 sent

```
unsigned FakeNfc::sent = 0U
```
Definition at line 34 of file test_pr28.cpp.
Referenced by check_ed25519_applet_gate(), check_pubkey_allow_list(), check_session_clearing(), selectOnly(), and sendAPDU().
The documentation for this class was generated from the following file:

- tests/test_pr28.cpp

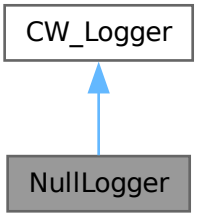
4.16 NullLogger Class Reference

Inheritance diagram for NullLogger:



© 2026 Cryptnox SA

Collaboration diagram for NullLogger:



Public Member Functions

- bool begin (unsigned long) override
  Initialize the logging interface.
- void print (const __FlashStringHelper *) override
- void print (const char *) override
- void print (char) override
- void print (uint8_t, int) override
- void print (uint16_t, int) override
- void print (uint32_t, int) override
- void print (int, int) override
- void println () override
- void println (const __FlashStringHelper *) override
- void println (const char *) override
- void println (char) override
- void println (uint8_t, int) override
- void println (uint16_t, int) override
- void println (uint32_t, int) override
- void println (int, int) override

Public Member Functions inherited from CW_Logger

- virtual ~CW_Logger ()

4.16.1 Detailed Description

Definition at line 105 of file test_pr28.cpp.

4.16.2 Member Function Documentation

4.16.2.1 begin()

```
bool NullLogger::begin (
            unsigned long baudRate)  [inline], [override], [virtual]
```

Initialize the logging interface.

Parameters

© 2026 Cryptnox SA

baudRate	Baud rate (relevant for UART implementations).

Returns

true if initialization succeeded, false otherwise.

Implements CW_Logger.
Definition at line 107 of file test_pr28.cpp.

4.16.2.2 print() [1/7]

```
void NullLogger::print (
            char )  [inline], [override], [virtual]
```

Implements CW_Logger.
Definition at line 110 of file test_pr28.cpp.

4.16.2.3 print() [2/7]

```
void NullLogger::print (
            const __FlashStringHelper * )  [inline], [override], [virtual]
```

Implements CW_Logger.
Definition at line 108 of file test_pr28.cpp.

4.16.2.4 print() [3/7]

```
void NullLogger::print (
            const char * )  [inline], [override], [virtual]
```

Implements CW_Logger.
Definition at line 109 of file test_pr28.cpp.

4.16.2.5 print() [4/7]

```
void NullLogger::print (
            int ,
            int )  [inline], [override], [virtual]
```

Implements CW_Logger.
Definition at line 114 of file test_pr28.cpp.

4.16.2.6 print() [5/7]

```
void NullLogger::print (
            uint16_t ,
            int )  [inline], [override], [virtual]
```

Implements CW_Logger.
Definition at line 112 of file test_pr28.cpp.

4.16.2.7 print() [6/7]

```
void NullLogger::print (
            uint32_t ,
            int )  [inline], [override], [virtual]
```

Implements CW_Logger.
Definition at line 113 of file test_pr28.cpp.

4.16.2.8 print() [7/7]

```
void NullLogger::print (
            uint8_t ,
            int )  [inline], [override], [virtual]
```

Implements CW_Logger.
Definition at line 111 of file test_pr28.cpp.

© 2026 Cryptnox SA

4.16.2.9 println() [1/8]

```
void NullLogger::println ()  [inline], [override], [virtual]
```

Implements CW_Logger.
Definition at line 115 of file test_pr28.cpp.

4.16.2.10 println() [2/8]

```
void NullLogger::println (
            char )  [inline], [override], [virtual]
```

Implements CW_Logger.
Definition at line 118 of file test_pr28.cpp.

4.16.2.11 println() [3/8]

```
void NullLogger::println (
            const __FlashStringHelper * )  [inline], [override], [virtual]
```

Implements CW_Logger.
Definition at line 116 of file test_pr28.cpp.

4.16.2.12 println() [4/8]

```
void NullLogger::println (
            const char * )  [inline], [override], [virtual]
```

Implements CW_Logger.
Definition at line 117 of file test_pr28.cpp.

4.16.2.13 println() [5/8]

```
void NullLogger::println (
            int ,
            int )  [inline], [override], [virtual]
```

Implements CW_Logger.
Definition at line 122 of file test_pr28.cpp.

4.16.2.14 println() [6/8]

```
void NullLogger::println (
            uint16_t ,
            int )  [inline], [override], [virtual]
```

Implements CW_Logger.
Definition at line 120 of file test_pr28.cpp.

4.16.2.15 println() [7/8]

```
void NullLogger::println (
            uint32_t ,
            int )  [inline], [override], [virtual]
```

Implements CW_Logger.
Definition at line 121 of file test_pr28.cpp.

4.16.2.16 println() [8/8]

```
void NullLogger::println (
            uint8_t ,
            int )  [inline], [override], [virtual]
```

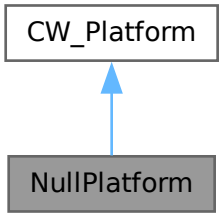
Implements CW_Logger.
Definition at line 119 of file test_pr28.cpp.
The documentation for this class was generated from the following file:

- tests/test_pr28.cpp

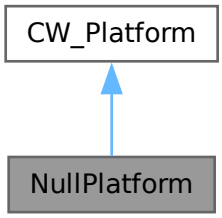
© 2026 Cryptnox SA

4.17 NullPlatform Class Reference

Inheritance diagram for NullPlatform:



Collaboration diagram for NullPlatform:



Public Member Functions

- void sleep_ms (uint32_t) override
  Block for at least `ms` milliseconds.

Public Member Functions inherited from CW_Platform

- virtual ~CW_Platform ()

4.17.1 Detailed Description

Definition at line 125 of file test_pr28.cpp.

4.17.2 Member Function Documentation

4.17.2.1 sleep_ms()

```
void NullPlatform::sleep_ms (
            uint32_t ms)  [inline], [override], [virtual]
```

Block for at least `ms` milliseconds.

© 2026 Cryptnox SA

Parameters

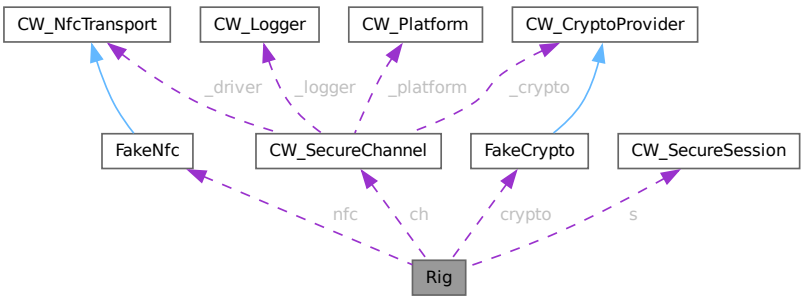
in	*ms*	Duration to sleep in milliseconds.

Implements CW_Platform.
Definition at line 127 of file test_pr28.cpp.
The documentation for this class was generated from the following file:

- tests/test_pr28.cpp

4.18 Rig Struct Reference

Collaboration diagram for Rig:



Public Member Functions

- Rig ()
- bool send (const uint8_t *data, uint16_t len)

Public Attributes

- FakeNfc nfc
- FakeCrypto crypto
- CW_SecureChannel ch
- CW_SecureSession s

4.18.1 Detailed Description

Definition at line 155 of file test_pr28.cpp.

4.18.2 Constructor & Destructor Documentation

4.18.2.1 Rig()

```
Rig::Rig () [inline]
```

Definition at line 160 of file test_pr28.cpp.
References ch, CHECK, crypto, logger, nfc, openSession(), platform, and s.

© 2026 Cryptnox SA

4.18.3 Member Function Documentation

4.18.3.1 send()

```
bool Rig::send (
            const uint8_t * data,
            uint16_t len)  [inline]
```

Definition at line 164 of file test_pr28.cpp.
References ch, HDR, and s.
Referenced by check_session_clearing().

4.18.4 Member Data Documentation

4.18.4.1 ch

```
CW_SecureChannel Rig::ch
```

Definition at line 158 of file test_pr28.cpp.
Referenced by Rig(), and send().

4.18.4.2 crypto

```
FakeCrypto Rig::crypto
```

Definition at line 157 of file test_pr28.cpp.
Referenced by check_session_clearing(), and Rig().

4.18.4.3 nfc

```
FakeNfc Rig::nfc
```

Definition at line 156 of file test_pr28.cpp.
Referenced by check_session_clearing(), and Rig().

4.18.4.4 s

```
CW_SecureSession Rig::s
```

Definition at line 159 of file test_pr28.cpp.
Referenced by check_session_clearing(), Rig(), and send().
The documentation for this struct was generated from the following file:

- tests/test_pr28.cpp

© 2026 Cryptnox SA

Chapter 5

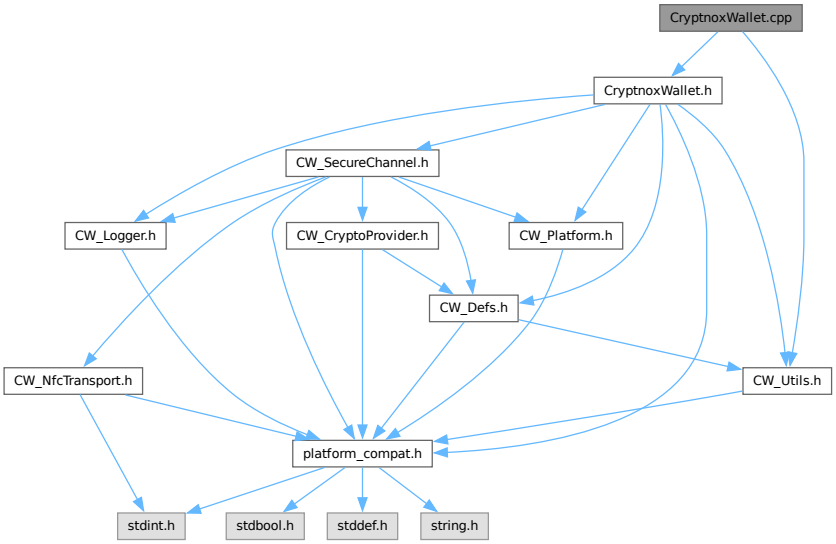
File Documentation

5.1 CryptnoxWallet.cpp File Reference

Implementation of the high-level CryptnoxWallet API.

```
#include "CryptnoxWallet.h"
#include "CW_Utils.h"
```

Include dependency graph for CryptnoxWallet.cpp:



Functions

- static bool cwIsEd25519 (uint8_t keyType)
- static bool cwSignNeedsPath (uint8_t keyType)
- static bool cwPubKeyTypeAllowed (uint8_t keyType)
- static uint16_t cwMaxEd25519Message (const CW_SignRequest &request)

5.1.1 Detailed Description

Implementation of the high-level CryptnoxWallet API.
Coordinates the secure channel layer (CW_SecureChannel) with the higher-level wallet operations declared in CryptnoxWallet.h. Handles connection retries, sensitive buffer wiping on every exit path, PIN/sign payload assembly, and DER signature parsing.
Definition in file CryptnoxWallet.cpp.

© 2026 Cryptnox SA

5.1.2 Function Documentation

5.1.2.1 cwIsEd25519()

```
bool cwIsEd25519 (
            uint8_t keyType)  [static]
```

Definition at line 23 of file CryptnoxWallet.cpp.
Referenced by CryptnoxWallet::buildSignPayload(), CryptnoxWallet::getPublicKey(), CryptnoxWallet::sendSignApdu(), CryptnoxWallet::sign(), and CryptnoxWallet::validateSignRequest().

5.1.2.2 cwMaxEd25519Message()

```
uint16_t cwMaxEd25519Message (
            const CW_SignRequest & request)  [static]
```

Definition at line 59 of file CryptnoxWallet.cpp.
References CW_MAX_PIN_LENGTH, CW_USER_DATA_PAGE_SIZE, cwSignNeedsPath(), CW_SignRequest::derivePath, CW_SignRequest::derivePathLength, CW_SignRequest::keyType, and CW_SignRequest::pinLessMode.
Referenced by CryptnoxWallet::validateSignRequest().

5.1.2.3 cwPubKeyTypeAllowed()

```
bool cwPubKeyTypeAllowed (
            uint8_t keyType)  [static]
```

Definition at line 38 of file CryptnoxWallet.cpp.
References CW_SIGN_CURR_ED25519, CW_SIGN_CURR_K1, CW_SIGN_CURR_R1, CW_SIGN_DERIVE_ED25519, CW_SIGN_DERIVE_K1, and CW_SIGN_DERIVE_R1.
Referenced by CryptnoxWallet::getPublicKey().

5.1.2.4 cwSignNeedsPath()

```
bool cwSignNeedsPath (
            uint8_t keyType)  [static]
```

Definition at line 29 of file CryptnoxWallet.cpp.
Referenced by CryptnoxWallet::buildSignPayload(), cwMaxEd25519Message(), CryptnoxWallet::getPublicKey(), and CryptnoxWallet::validateSignRequest().

5.2 CryptnoxWallet.cpp

Go to the documentation of this file.

```
00001 /*
00002  * SPDX-License-Identifier: LGPL-3.0-or-later
00003  * Copyright (c) 2026 Cryptnox SA
00004  */
00005
00015
00016 /* NOTE: Do NOT include <Arduino.h> here -- this is a platform-independent file.
00017  * Arduino compatibility shims (F(), HEX, delay) are provided via
00018  * platform_compat.h which is pulled in transitively through CryptnoxWallet.h. */
00019 #include "CryptnoxWallet.h"
00020 #include "CW_Utils.h"
00021
00022 /* True for the Ed25519 key types (high nibble 0x2 = ed25519 curve). */
00023 static bool cwIsEd25519(uint8_t keyType) {
00024     return (keyType & 0xF0U) == 0x20U;
00025 }
00026
00027 /* True when the SIGN key type carries a BIP32 derivation path
00028  * (low nibble 1 = derive, 2 = derive-and-make-current). */
00029 static bool cwSignNeedsPath(uint8_t keyType) {
00030     uint8_t derivation = keyType & 0x0FU;
00031     return (derivation == 1U) || (derivation == 2U);
00032 }
00033
00034 /* GET PUBLIC KEY is a read: only current / derive on k1, r1, ed25519.
00035  * Excluded:
00036  *  - derive-and-make-current (low nibble 2): would change the card's current key;
00037  *  - PIN-less path (low nibble 3, CW_SIGN_PINLESS_K1): a SIGN-only mode, not a key to read. */
```

© 2026 Cryptnox SA

```
00038 static bool cwPubKeyTypeAllowed(uint8_t keyType) {
00039     return (keyType == CW_SIGN_CURR_K1)       || (keyType == CW_SIGN_DERIVE_K1) ||
00040            (keyType == CW_SIGN_CURR_R1)       || (keyType == CW_SIGN_DERIVE_R1) ||
00041            (keyType == CW_SIGN_CURR_ED25519) || (keyType == CW_SIGN_DERIVE_ED25519);
00042 }
00043
00044 /* Largest Ed25519 message that still fits one secure-channel page for THIS
00045  * request. buildSignPayload() lays the data field out as
00046  * [len(2)|msg|path?|pin(9)], so the room left for the message depends on the
00047  * path the request actually carries.
00048  *
00049  * CW_MAX_ED25519_MESSAGE_LENGTH assumes the worst case,
00050  * CW_MAX_DERIVE_PATH_LENGTH (20), and so rejects messages that do fit: a 4-level
00051  * Solana path is 16 bytes, which leaves room for 181 rather than 177. That
00052  * difference is what decides whether a ~180-byte SPL transfer can be signed with
00053  * the path inside the SIGN command (P1 0x21) instead of requiring the key to be
00054  * made current beforehand.
00055  *
00056  * The card itself is not the limit here: the v2.0 SIGN spec allows up to 1200
00057  * bytes of raw data for P1 0x20/0x21. The 208-byte ceiling is this SDK's
00058  * single-page secure-channel buffer. */
00059 static uint16_t cwMaxEd25519Message(const CW_SignRequest& request) {
00060     /* Callers must have rejected an over-long path first (validateSignRequest
00061      * does), otherwise the subtraction below would wrap. */
00062     uint16_t overhead = 2U;  /* big-endian message length prefix */
00063
00064     if (cwSignNeedsPath(request.keyType) && (request.derivePath != NULL)) {
00065         overhead = (uint16_t)(overhead + request.derivePathLength);
00066     }
00067     if (!request.pinLessMode) {
00068         overhead = (uint16_t)(overhead + CW_MAX_PIN_LENGTH);
00069     }
00070
00071     return (uint16_t)(CW_USER_DATA_PAGE_SIZE - overhead);
00072 }
00073
00074 /**************************************************************************
00075  * Constructor
00076  **************************************************************************/
00077
00078 // cppcheck-suppress misra-c2012-12.3 -- C++: member initializer-list commas are not the comma
      operator
00079 CryptnoxWallet::CryptnoxWallet(CW_NfcTransport& driver, CW_Logger& logger,
00080                                CW_CryptoProvider& crypto, CW_Platform& platform)
00081     : _logger(logger), _platform(platform), _secure(driver, logger, crypto, platform) {
00082 }
00083
00084 /**************************************************************************
00085  * Public methods
00086  **************************************************************************/
00087
00088 bool CryptnoxWallet::begin() {
00089     bool ret = _secure.begin();
00090     if (ret) {
00091         printPN532FirmwareVersion();
00092     }
00093     return ret;
00094 }
00095
00096 bool CryptnoxWallet::connect(CW_SecureSession& session) {
00097     bool ret = false;
00098     session.clear(); /* CRIT-04: clear any stale keys from a previous or partial session */
00099
00100     for (uint8_t attempt = 0U; (attempt < CW_CONNECT_MAX_ATTEMPTS) && (ret == false); attempt++) {
00101         if (attempt > 0U) {
00102             session.clear(); /* CRIT-04: clear partial keys left by a failed attempt before retrying
      */
00103 #if CW_DEBUG_LOGGING
00104             _logger.print(F("Retrying card connection (attempt "));
00105             _logger.print((uint8_t)(attempt + 1U));
00106             _logger.println(F(")..."));
00107 #endif
00108             _secure.resetReader();
00109             _platform.sleep_ms(200U);
00110         }
00111
00112         if (_secure.inListPassiveTarget()) {
00113             _platform.sleep_ms(200U);
00114             if (establishSecureChannel(session)) {
00115                 ret = true;
00116             }
00117         }
00118     }
00119
00120     if (!ret) {
00121         session.clear(); /* CRIT-04: clear any partial keys from the final failed attempt */
00122     }
```

© 2026 Cryptnox SA

```
00123
00124     return ret;
00125 }
00126
00127 bool CryptnoxWallet::establishSecureChannel(CW_SecureSession& session) {
00128     bool ret = false;
00129
00130     /* Declare all sensitive stack buffers at function entry so they can be
00131      * wiped on every exit path (H-01, M-02). */
00132     uint8_t cardCertificate[146U]      = { 0U };
00133     uint8_t cardCertificateLength      = 0U;
00134     uint8_t cardEphemeralPubKey[64U]   = { 0U };
00135     uint8_t openSecureChannelSalt[32U] = { 0U };
00136     uint8_t clientPrivateKey[32U]      = { 0U };
00137     uint8_t clientPublicKey[64U]       = { 0U };
00138     CW_Curve sessionCurve              = CW_CURVE_SECP256R1;
00139
00140     if (_secure.selectApdu()) {
00141         /* Fetch the manufacturer certificate BEFORE getCardCertificate().
00142          * The Cryptnox card state machine advances after GET_CARD_CERTIFICATE
00143          * (INS=F8) and will not respond to GET_MANUFACTURER_CERTIFICATE (INS=F7)
00144          * after that point.  Pre-fetching here caches the cert inside
00145          * CW_SecureChannel so that verifyCertificateChain() can use it without
00146          * issuing another APDU. */
00147         if (!_secure.preFetchManufacturerCert()) {
00148 #if CW_DEBUG_LOGGING
00149             _logger.println(F("Failed to pre-fetch manufacturer certificate"));
00150 #endif
00151         } else {
00152             if (_secure.getCardCertificate(cardCertificate, cardCertificateLength)) {
00153 #if CW_VERIFY_CERT
00154                 uint8_t certResult = _secure.verifyCertificateChain(cardCertificate,
00155                                                                     cardCertificateLength);
00156 #else
00157                 /* CW_VERIFY_CERT=0: the chain is not checked at all, so any card
00158                  * is accepted and will sign whatever it is given. Logged outside
00159                  * CW_DEBUG_LOGGING so the build can never do this silently. */
00160                 uint8_t certResult = CW_CERT_OK;
00161                 _logger.println(F("*** CW_VERIFY_CERT=0: card authenticity NOT verified."));
00162 #endif
00163                 if (certResult != CW_CERT_OK) {
00164 #if CW_DEBUG_LOGGING
00165                     _logger.print(F("Card authenticity check failed (code 0x"));
00166                     _logger.print(certResult, HEX);
00167                     _logger.println(F("). Aborting."));
00168 #endif
00169                 } else {
00170                     if (_secure.extractCardEphemeralKey(cardCertificate, cardEphemeralPubKey)) {
00171                         if (_secure.openSecureChannel(openSecureChannelSalt, clientPublicKey,
00172                                                       clientPrivateKey, sessionCurve)) {
00173                             if (_secure.mutuallyAuthenticate(session, openSecureChannelSalt,
00174                                                              clientPublicKey, clientPrivateKey,
00175                                                              sessionCurve, cardEphemeralPubKey)) {
00176 #if CW_DEBUG_LOGGING
00177                                 _logger.println(F("Secure channel established"));
00178 #endif
00179                                 ret = true;
00180                             } else {
00181 #if CW_DEBUG_LOGGING
00182                                 _logger.println(F("Mutual authentication failed"));
00183 #endif
00184                             }
00185                         } else {
00186 #if CW_DEBUG_LOGGING
00187                             _logger.println(F("Failed to open secure channel"));
00188 #endif
00189                         }
00190                     } else {
00191 #if CW_DEBUG_LOGGING
00192                         _logger.println(F("Failed to extract card ephemeral key"));
00193 #endif
00194                     }
00195                 }
00196             } else {
00197 #if CW_DEBUG_LOGGING
00198                 _logger.println(F("Failed to get card certificate"));
00199 #endif
00200             }
00201         } /* end preFetchManufacturerCert else */
00202     } else {
00203 #if CW_DEBUG_LOGGING
00204         _logger.println(F("Failed to select Cryptnox application"));
00205 #endif
00206     }
00207
00208     /* Wipe all sensitive ephemeral key material on every exit path (H-01, M-02). */
00209     CW_Utils::secure_wipe(clientPrivateKey,      sizeof(clientPrivateKey));
```

© 2026 Cryptnox SA

```
00210     CW_Utils::secure_wipe(openSecureChannelSalt, sizeof(openSecureChannelSalt));
00211     CW_Utils::secure_wipe(clientPublicKey,       sizeof(clientPublicKey));
00212     CW_Utils::secure_wipe(cardEphemeralPubKey,   sizeof(cardEphemeralPubKey));
00213     CW_Utils::secure_wipe(cardCertificate,       sizeof(cardCertificate));
00214
00215     return ret;
00216 }
00217
00218 void CryptnoxWallet::disconnect(CW_SecureSession& session) {
00219     if (isSecureChannelOpen(session)) {
00220         session.clear();
00221     }
00222     _secure.resetReader();
00223 }
00224
00225 bool CryptnoxWallet::getCardInfo(CW_SecureSession& session, CW_CardInfo* info) {
00226     bool ret = false;
00227     if (!isSecureChannelOpen(session)) {
00228 #if CW_DEBUG_LOGGING
00229         _logger.println(F("Error: Secure channel not open. Cannot get card info."));
00230 #endif
00231         return false;
00232     }
00233     uint8_t data[] = { 0x00U };
00234     uint8_t apdu[] = { 0x80U, 0xFAU, 0x00U, 0x00U };
00235
00236     uint8_t  decrypted[255U] = { 0U };
00237     uint16_t decryptedLen    = 0U;
00238
00239     ret = _secure.aesCbcEncrypt(session, apdu, sizeof(apdu),
00240                                 data, sizeof(data),
00241                                 decrypted, sizeof(decrypted), &decryptedLen);
00242
00243     if (ret && (info != NULL)) {
00244         /* Response layout (Cryptnox basic_g1 spec):
00245          *   [byte0] [name_len(1)] [name(name_len)]
00246          *           [email_len(1)] [email(email_len)] [... more fields ...]
00247          * byte0 = unused/flags. */
00248         ret = false;
00249         if (decryptedLen >= 4U) {
00250             uint16_t pos     = 1U;
00251             uint8_t  nameLen = decrypted[pos];
00252             pos += 1U;
00253             if ((nameLen <= CW_CARD_NAME_MAX_LEN) &&
00254                 ((uint16_t)(pos + nameLen + 1U) <= decryptedLen)) {
00255                 (void)CW_Utils::safe_memcpy(reinterpret_cast<uint8_t*>(info->name),
00256                                             sizeof(info->name),
00257                                             decrypted + pos, nameLen);
00258                 info->name[nameLen] = '\0';
00259                 pos += nameLen;
00260
00261                 uint8_t emailLen = decrypted[pos];
00262                 pos += 1U;
00263                 if ((emailLen <= CW_CARD_EMAIL_MAX_LEN) &&
00264                     ((uint16_t)(pos + emailLen) <= decryptedLen)) {
00265                     (void)CW_Utils::safe_memcpy(reinterpret_cast<uint8_t*>(info->email),
00266                                                 sizeof(info->email),
00267                                                 decrypted + pos, emailLen);
00268                     info->email[emailLen] = '\0';
00269                     ret = true;
00270                 }
00271             }
00272         }
00273     }
00274
00275     CW_Utils::secure_wipe(decrypted, sizeof(decrypted));
00276     return ret;
00277 }
00278
00279 bool CryptnoxWallet::verifyPin(CW_SecureSession& session, const uint8_t* pin, uint8_t pinLength) {
00280     bool ret = false;
00281     if (!isSecureChannelOpen(session)) {
00282 #if CW_DEBUG_LOGGING
00283         _logger.println(F("Error: Secure channel not open. Cannot verify PIN."));
00284 #endif
00285     }
00286     else if ((pin == NULL) || (pinLength < CW_MIN_PIN_LENGTH) || (pinLength > CW_MAX_PIN_LENGTH)) {
00287 #if CW_DEBUG_LOGGING
00288         _logger.println(F("Error: Invalid PIN (must be 4-9 digits)."));
00289 #endif
00290     }
00291     else {
00292         uint8_t paddedPin[CW_MAX_PIN_LENGTH] = { 0U };
00293         (void)CW_Utils::safe_memcpy(paddedPin, sizeof(paddedPin), pin, pinLength);
00294         uint8_t apdu[] = { 0x80U, 0x20U, 0x00U, 0x00U };
00295         ret = _secure.aesCbcEncrypt(session, apdu, sizeof(apdu), paddedPin, CW_MAX_PIN_LENGTH);
00296         CW_Utils::secure_wipe(paddedPin, sizeof(paddedPin));
```

© 2026 Cryptnox SA

```
00297     }
00298     return ret;
00299 }
00300
00301
00302 bool CryptnoxWallet::writeUserData(CW_SecureSession& session,
00303                                    const uint8_t* data, uint16_t dataLength) {
00304     bool ret = false;
00305
00306     if (!isSecureChannelOpen(session)) {
00307 #if CW_DEBUG_LOGGING
00308         _logger.println(F("Error: Secure channel not open. Cannot write user data."));
00309 #endif
00310     }
00311     else if ((data == NULL) || (dataLength == 0U)) {
00312 #if CW_DEBUG_LOGGING
00313         _logger.println(F("Error: Invalid data for write user data."));
00314 #endif
00315     }
00316     else if (dataLength > CW_USER_DATA_PAGE_SIZE) {
00317         /* The applet's WRITE (0xFC) P2 selects a 1200-byte logical page written
00318          * at offset 1200*P2, not a chunk counter -- so multiple single-APDU
00319          * chunks cannot be concatenated over this transport. Reject anything
00320          * that won't fit one page instead of silently corrupting the write. */
00321 #if CW_DEBUG_LOGGING
00322         _logger.print(F("Error: user data exceeds one page ("));
00323         _logger.print((uint8_t)CW_USER_DATA_PAGE_SIZE);
00324         _logger.println(F(" B max for this transport)."));
00325 #endif
00326     }
00327     else {
00328         /* P1 = 0 (write user data slot; P1 is the applet's operation selector,
00329          * not a slot index), P2 = 0 (logical page 0). */
00330         uint8_t apdu[] = { 0x80U, 0xFCU, 0x00U, 0x00U };
00331         ret = _secure.aesCbcEncrypt(session, apdu, sizeof(apdu), data, dataLength);
00332 #if CW_DEBUG_LOGGING
00333         if (!ret) {
00334             _logger.println(F("Error: Write user data failed."));
00335         }
00336 #endif
00337     }
00338
00339     return ret;
00340 }
00341
00342 bool CryptnoxWallet::getPublicKey(CW_SecureSession& session,
00343                                   const uint8_t* derivePath,
00344                                   uint8_t derivePathLength,
00345                                   uint8_t* publicKey64) {
00346     bool ret = false;
00347     /* SEC1 uncompressed point: 0x04 || X || Y. Some applet versions answer
00348      * with the bare coordinates, so accept both. */
00349     uint8_t  key[CW_SEC1_PUBKEY_SIZE] = { 0U };
00350     uint16_t keyLength = 0U;
00351     const uint8_t keyType = (derivePath != NULL) ? CW_SIGN_DERIVE_K1 : CW_SIGN_CURR_K1;
00352
00353     if ((publicKey64 == NULL) ||
00354         ((derivePath == NULL) != (derivePathLength == 0U))) {
00355 #if CW_DEBUG_LOGGING
00356         _logger.println(F("Error: Invalid arguments for get public key."));
00357 #endif
00358     }
00359     else if (getPublicKey(session, keyType, derivePath, derivePathLength,
00360                           key, sizeof(key), keyLength)) {
00361         if ((keyLength == CW_SEC1_PUBKEY_SIZE) && (key[0] == CW_SEC1_UNCOMPRESSED_TAG)) {
00362             ret = CW_Utils::safe_memcpy(publicKey64, CW_RAW_PUBKEY_SIZE, &key[1], CW_RAW_PUBKEY_SIZE);
00363         }
00364         else if (keyLength == CW_RAW_PUBKEY_SIZE) {
00365             ret = CW_Utils::safe_memcpy(publicKey64, CW_RAW_PUBKEY_SIZE, key, CW_RAW_PUBKEY_SIZE);
00366         }
00367         else {
00368 #if CW_DEBUG_LOGGING
00369             _logger.println(F("Get public key: unexpected response length."));
00370 #endif
00371         }
00372     }
00373     else {
00374         /* getPublicKey() above already logged the cause. */
00375     }
00376
00377     CW_Utils::secure_wipe(key, sizeof(key));
00378     return ret;
00379 }
00380
00381 CW_SignResult CryptnoxWallet::sign(CW_SignRequest& request) {
00382     CW_SignResult result{};
00383
```

© 2026 Cryptnox SA

```
00384     if (validateSignRequest(request, result)) {
00385         /* One secure-channel page holds the largest framed payload for either
00386          * scheme: ECDSA [hash|path|pin] or Ed25519 [len(2)|msg|path|pin]. */
00387         uint8_t data[CW_USER_DATA_PAGE_SIZE] = { 0U };
00388         uint16_t dataLength = 0U;
00389 
00390         buildSignPayload(request, data, dataLength);
00391 
00392         uint8_t derResponse[255U] = { 0U };
00393         uint16_t derLength = 0U;
00394 
00395         if (sendSignApdu(request, data, dataLength, derResponse, sizeof(derResponse),
00396                          derLength, result)) {
00397             /* Ed25519 (EdDSA) and Schnorr BIP340 both return a raw 64-byte
00398              * R||S signature with no DER wrapper; ECDSA (low-S / EOSIO) returns
00399              * a DER SEQUENCE that extractRawSignature() unpacks. */
00400             const bool rawSig = cwIsEd25519(request.keyType) ||
00401                                 (request.signatureType == CW_SIGN_SIG_SCHNORR_BIP340);
00402             if (rawSig) {
00403                 if (derLength == CW_RAW_SIGNATURE_SIZE) {
00404                     (void)CW_Utils::safe_memcpy(result.signature, sizeof(result.signature),
00405                                                 derResponse, CW_RAW_SIGNATURE_SIZE);
00406                     debugPrintSignature(result.signature);
00407                     result.errorCode = CW_OK;
00408                 } else {
00409 #if CW_DEBUG_LOGGING
00410                     _logger.println(F("Error: raw signature not 64 bytes."));
00411 #endif
00412                     result.errorCode = CW_NOK;
00413                 }
00414             } else if (extractRawSignature(derResponse, derLength, result)) {
00415                 debugPrintSignature(result.signature);
00416                 result.errorCode = CW_OK;
00417             }
00418         }
00419         CW_Utils::secure_wipe(data, sizeof(data));
00420         CW_Utils::secure_wipe(derResponse, sizeof(derResponse));
00421     }
00422 
00423     return result;
00424 }
00425 
00426 bool CryptnoxWallet::getPublicKey(CW_SecureSession& session, uint8_t keyType,
00427                                   const uint8_t* derivePath, uint8_t derivePathLength,
00428                                   uint8_t* pubKey, uint16_t pubKeyBufSize,
00429                                   uint16_t& pubKeyLength) {
00430     bool ret = false;
00431     pubKeyLength = 0U;
00432 
00433     if (!isSecureChannelOpen(session)) {
00434 #if CW_DEBUG_LOGGING
00435         _logger.println(F("Error: Secure channel not open. Cannot get public key."));
00436 #endif
00437     }
00438     else if ((pubKey == NULL) || (pubKeyBufSize == 0U)) {
00439 #if CW_DEBUG_LOGGING
00440         _logger.println(F("Error: Invalid output buffer for get public key."));
00441 #endif
00442     }
00443     else if (!cwPubKeyTypeAllowed(keyType)) {
00444 #if CW_DEBUG_LOGGING
00445         _logger.println(F("Error: Unsupported key type for get public key."));
00446 #endif
00447     }
00448     else if (cwIsEd25519(keyType) && (_secure.appletMajor() < 2U)) {
00449 #if CW_DEBUG_LOGGING
00450         _logger.println(F("Error: Ed25519 requires applet 2.0+."));
00451 #endif
00452     }
00453     else if (cwSignNeedsPath(keyType) &&
00454              ((derivePath == NULL) || (derivePathLength == 0U) ||
00455               (derivePathLength > CW_MAX_DERIVE_PATH_LENGTH) ||
00456               ((derivePathLength % 4U) != 0U))) {
00457 #if CW_DEBUG_LOGGING
00458         _logger.println(F("Error: Invalid derivation path for get public key."));
00459 #endif
00460     }
00461     else {
00462         /* GET PUBLIC KEY (APDU 0x80C2): P1 = derivation|curve, P2 = 0x01.
00463          * Data is the BIP32 path for DERIVE key types, empty for current-key. */
00464         uint8_t apdu[] = { 0x80U, 0xC2U, keyType, 0x01U };
00465         const uint8_t* data = cwSignNeedsPath(keyType) ? derivePath : NULL;
00466         uint16_t dataLen = cwSignNeedsPath(keyType) ? derivePathLength : 0U;
00467 
00468         uint8_t  decrypted[255U] = { 0U };
00469         uint16_t decryptedLen    = 0U;
00470 
```

© 2026 Cryptnox SA

```
00471         if (_secure.aesCbcEncrypt(session, apdu, sizeof(apdu), data, dataLen,
00472                                   decrypted, sizeof(decrypted), &decryptedLen)) {
00473             if ((decryptedLen > 0U) && (decryptedLen <= pubKeyBufSize)) {
00474                 (void)CW_Utils::safe_memcpy(pubKey, pubKeyBufSize, decrypted, decryptedLen);
00475                 pubKeyLength = decryptedLen;
00476                 ret = true;
00477             }
00478 #if CW_DEBUG_LOGGING
00479             else {
00480                 _logger.println(F("Error: public key buffer too small or empty response."));
00481             }
00482 #endif
00483         }
00484         CW_Utils::secure_wipe(decrypted, sizeof(decrypted));
00485     }
00486 
00487     return ret;
00488 }
00489 
00490 /**************************************************************************
00491  * Static public methods
00492  **************************************************************************/
00493 
00494 bool CryptnoxWallet::parseDerSignature(const uint8_t* der, uint8_t derLength,
00495                                        uint8_t* r, uint8_t& rLength,
00496                                        uint8_t* s, uint8_t& sLength) {
00497     bool ret = false;
00498 
00499     if ((der == NULL) || (derLength < 6U) || (r == NULL) || (s == NULL)) {
00500     }
00501     else if (der[0] != CW_DER_TAG_SEQUENCE) {
00502     }
00503     else {
00504         uint8_t pos = 2U;
00505 
00506         if (der[pos] != CW_DER_TAG_INTEGER) {
00507         }
00508         else {
00509             pos++;
00510             rLength = der[pos];
00511             pos++;
00512             if ((rLength > 33U) || ((pos + rLength) > derLength)) {
00513             }
00514             else {
00515                 (void)CW_Utils::safe_memcpy(r, 33U, der + pos, rLength);
00516                 pos += rLength;
00517 
00518                 if ((pos >= derLength) || (der[pos] != CW_DER_TAG_INTEGER)) {
00519                 }
00520                 else {
00521                     pos++;
00522                     sLength = der[pos];
00523                     pos++;
00524                     if ((sLength > 33U) || ((pos + sLength) > derLength)) {
00525                     }
00526                     else {
00527                         (void)CW_Utils::safe_memcpy(s, 33U, der + pos, sLength);
00528                         ret = true;
00529                     }
00530                 }
00531             }
00532         }
00533     }
00534 
00535     return ret;
00536 }
00537 
00538 /**************************************************************************
00539  * Private methods
00540  **************************************************************************/
00541 
00542 bool CryptnoxWallet::isSecureChannelOpen(const CW_SecureSession& session) {
00543     uint8_t acc = 0U;
00544     for (uint8_t i = 0U; i < CW_AESKEY_SIZE; i++) {
00545         acc |= session.aesKey[i];
00546     }
00547     return (acc != 0U);
00548 }
00549 
00550 bool CryptnoxWallet::printPN532FirmwareVersion() {
00551     return _secure.printFirmwareVersion();
00552 }
00553 
00554 bool CryptnoxWallet::validateSignRequest(const CW_SignRequest& request, CW_SignResult& result) {
00555     bool ret = false;
00556 
00557     if (!isSecureChannelOpen(request.session)) {
```

© 2026 Cryptnox SA

```
00558 #if CW_DEBUG_LOGGING
00559         _logger.println(F("Error: Secure channel not open. Cannot sign."));
00560 #endif
00561         result.errorCode = CW_INVALID_SESSION;
00562     }
00563     else if ((request.hash == NULL) || (request.hashLength == 0U)) {
00564 #if CW_DEBUG_LOGGING
00565         _logger.println(F("Error: Invalid parameters for sign."));
00566 #endif
00567         result.errorCode = CW_SIGN_KEY_TOO_SHORT;
00568     }
00569     else if (cwIsEd25519(request.keyType) && (_secure.appletMajor() < 2U)) {
00570 #if CW_DEBUG_LOGGING
00571         _logger.println(F("Error: Ed25519 requires applet 2.0+."));
00572 #endif
00573         result.errorCode = CW_SIGN_UNSUPPORTED_KEY_TYPE;
00574     }
00575     /* Validate the path BEFORE sizing the message, which subtracts its length
00576      * from the page: an out-of-range path would make that subtraction wrap and
00577      * report a nonsense cap. getPublicKey() already applies these same rules. */
00578     else if (cwSignNeedsPath(request.keyType) &&
00579              ((request.derivePath == NULL) || (request.derivePathLength == 0U) ||
00580               (request.derivePathLength > CW_MAX_DERIVE_PATH_LENGTH) ||
00581               ((request.derivePathLength % 4U) != 0U))) {
00582 #if CW_DEBUG_LOGGING
00583         _logger.println(F("Error: Invalid derivation path for sign."));
00584 #endif
00585         result.errorCode = CW_SIGN_INVALID_PATH;
00586     }
00587     else if (request.hashLength > (cwIsEd25519(request.keyType)
00588                                    ? cwMaxEd25519Message(request)
00589                                    : CW_HASH_SIZE)) {
00590 #if CW_DEBUG_LOGGING
00591         _logger.println(F("Error: Message/hash too large to sign."));
00592 #endif
00593         result.errorCode = CW_SIGN_MESSAGE_TOO_LONG;
00594     }
00595     else if ((request.pinLessMode) && (request.keyType != CW_SIGN_PINLESS_K1)) {
00596 #if CW_DEBUG_LOGGING
00597         _logger.println(F("Error: PIN-less mode requires CW_SIGN_PINLESS_K1 key type."));
00598 #endif
00599         result.errorCode = CW_SIGN_KEY_TOO_SHORT_WITH_PINLESS_MODE;
00600     }
00601     else {
00602         ret = true;
00603
00604         if (!request.pinLessMode) {
00605             uint8_t pinLength = 0U;
00606             for (uint8_t i = 0U; i < CW_MAX_PIN_LENGTH; i++) {
00607                 if (request.pin[i] == 0U) { break; }
00608                 pinLength++;
00609             }
00610             if ((pinLength > 0U) && (pinLength < CW_MIN_PIN_LENGTH)) {
00611 #if CW_DEBUG_LOGGING
00612                 _logger.println(F("Error: PIN too short (must be 4-9 digits)."));
00613 #endif
00614                 result.errorCode = CW_SIGN_PIN_INCORRECT;
00615                 ret = false;
00616             }
00617         }
00618     }
00619
00620     return ret;
00621 }
00622
00623 void CryptnoxWallet::buildSignPayload(const CW_SignRequest& request,
00624                                       uint8_t* data, uint16_t& dataLength) {
00625     const size_t kDataBufSize = static_cast<size_t>(CW_USER_DATA_PAGE_SIZE);
00626     dataLength = 0U;
00627
00628     if (cwIsEd25519(request.keyType)) {
00629         /* EdDSA data field (docs.cryptnox.com v2.0 SIGN command):
00630          *   [2-byte big-endian message length][raw message][path?][PIN(9B, 0x00-padded)]
00631          * The card hashes the raw message internally, so the length prefix is
00632          * what delimits the variable-length message from the trailing fields. */
00633         data[0] = (uint8_t)((request.hashLength >> 8) & 0xFFU);
00634         data[1] = (uint8_t)(request.hashLength & 0xFFU);
00635         dataLength = 2U;
00636     }
00637
00638     (void)CW_Utils::safe_memcpy(data + dataLength, kDataBufSize - static_cast<size_t>(dataLength),
00639                                 request.hash, request.hashLength);
00640     dataLength += request.hashLength;
00641
00642     if (cwSignNeedsPath(request.keyType) &&
00643         (request.derivePath != NULL) && (request.derivePathLength > 0U)) {
00644         (void)CW_Utils::safe_memcpy(data + dataLength, kDataBufSize - static_cast<size_t>(dataLength),
```

© 2026 Cryptnox SA

```
00645                                   request.derivePath, request.derivePathLength);
00646         dataLength += request.derivePathLength;
00647     }
00648
00649     if (!request.pinLessMode) {
00650         uint8_t pinLength = 0U;
00651         for (uint8_t i = 0U; i < CW_MAX_PIN_LENGTH; i++) {
00652             if (request.pin[i] == 0U) { break; }
00653             pinLength++;
00654         }
00655         if (pinLength > 0U) {
00656             /* PIN is a fixed 9-byte field, right-padded with 0x00. */
00657             (void)CW_Utils::safe_memcpy(data + dataLength, kDataBufSize -
      static_cast<size_t>(dataLength),
00658                                         request.pin, CW_MAX_PIN_LENGTH);
00659             dataLength += CW_MAX_PIN_LENGTH;
00660         }
00661     }
00662 }
00663
00664 bool CryptnoxWallet::sendSignApdu(CW_SignRequest& request, const uint8_t* data,
00665                                   uint16_t dataLength, uint8_t* derResponse,
00666                                   uint16_t derResponseSize,
00667                                   uint16_t& derLength, CW_SignResult& result) {
00668     bool ret = false;
00669     /* Ed25519 forces P2 = EdDSA; the caller-supplied signatureType is ignored. */
00670     uint8_t p2 = cwIsEd25519(request.keyType) ? CW_SIGN_SIG_EDDSA : request.signatureType;
00671     uint8_t apdu[] = { 0x80U, 0xC0U, request.keyType, p2 };
00672
00673 #if CW_DEBUG_LOGGING
00674     _logger.println(F("Sending SIGN APDU..."));
00675 #endif
00676
00677     if (_secure.aesCbcEncrypt(request.session, apdu, sizeof(apdu), data, dataLength,
00678                               derResponse, derResponseSize, &derLength)) {
00679         ret = true;
00680     }
00681     else {
00682 #if CW_DEBUG_LOGGING
00683         _logger.println(F("Sign APDU failed."));
00684 #endif
00685         result.errorCode = CW_SIGN_NO_KEY_LOADED;
00686     }
00687
00688     return ret;
00689 }
00690
00691 bool CryptnoxWallet::extractRawSignature(const uint8_t* derResponse, uint16_t derLength,
00692                                          CW_SignResult& result) {
00693     bool ret = false;
00694
00695     if ((derLength < 2U) || (derResponse[0] != CW_DER_TAG_SEQUENCE)) {
00696 #if CW_DEBUG_LOGGING
00697         _logger.println(F("Error: Invalid signature data (missing DER SEQUENCE tag)."));
00698 #endif
00699         result.errorCode = CW_NOK;
00700     }
00701     else {
00702         uint8_t derContentLength = derResponse[1];
00703         uint8_t derTotalLength   = 2U + derContentLength;
00704
00705         if (derTotalLength > derLength) {
00706 #if CW_DEBUG_LOGGING
00707             _logger.println(F("Error: DER signature length exceeds response."));
00708 #endif
00709             result.errorCode = CW_NOK;
00710         }
00711         else {
00712             uint8_t r[33U] = { 0U };
00713             uint8_t s[33U] = { 0U };
00714             uint8_t rLen = 0U;
00715             uint8_t sLen = 0U;
00716
00717             if (!parseDerSignature(derResponse, derTotalLength, r, rLen, s, sLen)) {
00718 #if CW_DEBUG_LOGGING
00719                 _logger.println(F("Error: Failed to parse DER signature."));
00720 #endif
00721                 result.errorCode = CW_NOK;
00722             }
00723             else {
00724                 memset(result.signature, 0U, CW_RAW_SIGNATURE_SIZE);
00725
00726                 if (rLen > 0U) {
00727                     uint8_t rSrc = 0U;
00728                     uint8_t rDstLen = 32U;
00729                     if ((rLen == 33U) && (r[0] == 0x00U)) { rSrc = 1U; rLen = 32U; }
00730                     if (rLen <= rDstLen) {
```

© 2026 Cryptnox SA

```
00731                         (void)CW_Utils::safe_memcpy(result.signature + (rDstLen - rLen),
      static_cast<size_t>(rDstLen + rLen), r + rSrc, rLen);
00732                     }
00733                 }
00734
00735                 if (sLen > 0U) {
00736                     uint8_t sSrc = 0U;
00737                     uint8_t sDstLen = 32U;
00738                     if ((sLen == 33U) && (s[0] == 0x00U)) { sSrc = 1U; sLen = 32U; }
00739                     if (sLen <= sDstLen) {
00740                         (void)CW_Utils::safe_memcpy(result.signature + 32U + (sDstLen - sLen),
      static_cast<size_t>(sLen), s + sSrc, sLen);
00741                     }
00742                 }
00743
00744                 ret = true;
00745             }
00746             CW_Utils::secure_wipe(r, sizeof(r));
00747             CW_Utils::secure_wipe(s, sizeof(s));
00748         }
00749     }
00750
00751     return ret;
00752 }
00753
00754 void CryptnoxWallet::debugPrintSignature(const uint8_t* signature) {
00755 #if CW_DEBUG_LOGGING
00756     _logger.print(F("Signature ("));
00757     _logger.print((uint8_t)CW_RAW_SIGNATURE_SIZE);
00758     _logger.println(F(" bytes):"));
00759     for (uint8_t i = 0U; i < CW_RAW_SIGNATURE_SIZE; i++) {
00760         _logger.print(F("0x"));
00761         if (signature[i] < 0x10U) { _logger.print(F("0")); }
00762         _logger.print(signature[i], HEX);
00763         _logger.print(F(" "));
00764         if (((i + 1U) % 16U == 0U) && ((i + 1U) != CW_RAW_SIGNATURE_SIZE)) { _logger.println(); }
00765     }
00766     _logger.println();
00767 #else
00768     (void)signature;
00769 #endif
00770 }
```

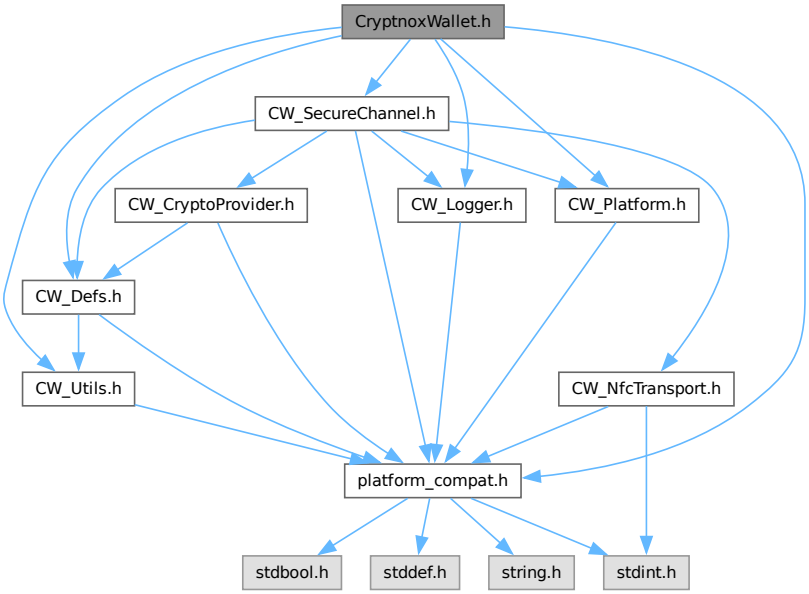
5.3 CryptnoxWallet.h File Reference

High-level API for interacting with a Cryptnox Hardware Wallet over NFC.

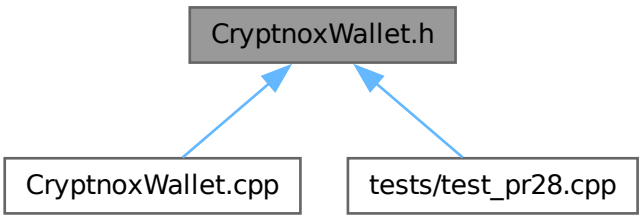
```
#include "platform_compat.h"
#include "CW_Defs.h"
#include "CW_Logger.h"
#include "CW_Platform.h"
#include "CW_SecureChannel.h"
#include "CW_Utils.h"
```

© 2026 Cryptnox SA

Include dependency graph for CryptnoxWallet.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct CW_CardInfo
  Subset of the Cryptnox card info returned by APDU 0x80FA0000.
- struct CW_SignRequest
  Request parameters for CryptnoxWallet::sign.
- struct CW_SignResult
  Result of CryptnoxWallet::sign.
- class CryptnoxWallet
  High-level interface for interacting with a Cryptnox Hardware Wallet over NFC.

Macros

- #define CW_CARD_NAME_MAX_LEN (20U)

© 2026 Cryptnox SA

Max name length stored on a Cryptnox card (per card spec).

- #define CW_CARD_EMAIL_MAX_LEN (60U)

  Max email length stored on a Cryptnox card (per card spec).

5.3.1 Detailed Description

High-level API for interacting with a Cryptnox Hardware Wallet over NFC.
Declares CryptnoxWallet, the main entry point for application code. The class wires together the four abstract adapters supplied by the host integration (NFC transport, crypto provider, logger, platform) and exposes the wallet operations: card connection, secure channel establishment, card info retrieval, PIN verification, transaction signing, and user-data writing.

See also

CW_NfcTransport

CW_CryptoProvider

CW_Logger

CW_Platform

Definition in file CryptnoxWallet.h.

5.3.2 Macro Definition Documentation

5.3.2.1 CW_CARD_EMAIL_MAX_LEN

```
#define CW_CARD_EMAIL_MAX_LEN (60U)
```

Max email length stored on a Cryptnox card (per card spec).
Definition at line 45 of file CryptnoxWallet.h.
Referenced by CryptnoxWallet::getCardInfo().

5.3.2.2 CW_CARD_NAME_MAX_LEN

```
#define CW_CARD_NAME_MAX_LEN (20U)
```

Max name length stored on a Cryptnox card (per card spec).
Definition at line 42 of file CryptnoxWallet.h.
Referenced by CryptnoxWallet::getCardInfo().

5.4 CryptnoxWallet.h

Go to the documentation of this file.

```
00001 /*
00002  * SPDX-License-Identifier: LGPL-3.0-or-later
00003  * Copyright (c) 2026 Cryptnox SA
00004  */
00005
00022
00023 #ifndef CRYPTNOXWALLET_H
00024 #define CRYPTNOXWALLET_H
00025
00026 /**********************************************************************
00027  * 1. Included files
00028  **********************************************************************/
00029
00030 #include "platform_compat.h"
00031 #include "CW_Defs.h"
00032 #include "CW_Logger.h"
00033 #include "CW_Platform.h"
00034 #include "CW_SecureChannel.h"
00035 #include "CW_Utils.h"
00036
00037 /**********************************************************************
00038  * 2. Typedefs / structs (sign API)
00039  **********************************************************************/
00040
00042 #define CW_CARD_NAME_MAX_LEN  (20U)
```

© 2026 Cryptnox SA

```
00043
00045 #define CW_CARD_EMAIL_MAX_LEN (60U)
00046
00055 struct CW_CardInfo {
00056     char name[CW_CARD_NAME_MAX_LEN + 1U] {};
00057     char email[CW_CARD_EMAIL_MAX_LEN + 1U] {};
00058
00059     CW_CardInfo() = default;
00060 };
00061
00071 struct CW_SignRequest {
00072     CW_SecureSession& session;
00073     uint8_t keyType {};
00074     uint8_t signatureType {};
00075     uint8_t pin[CW_MAX_PIN_LENGTH] {};
00076     bool pinLessMode {};
00077     const uint8_t* hash {};
00078     uint8_t hashLength {};
00079     const uint8_t* derivePath {};
00080     uint8_t derivePathLength {};
00081
00089     explicit CW_SignRequest(CW_SecureSession& sess,
00090                             uint8_t kType   = CW_SIGN_CURR_K1,
00091                             uint8_t sigType = CW_SIGN_SIG_ECDSA_LOW_S,
00092                             bool pinless    = CW_SIGN_WITH_PIN)
00093         : session(sess), keyType(kType), signatureType(sigType),
00094           pinLessMode(pinless), hash(NULL), hashLength(0U),
00095           derivePath(NULL), derivePathLength(0U) {
00096     }
00097
00099     ~CW_SignRequest() {
00100         CW_Utils::secure_wipe(pin, sizeof(pin));
00101     }
00102 };
00103
00112 struct CW_SignResult {
00113     uint8_t signature[CW_RAW_SIGNATURE_SIZE] {};
00114     uint8_t errorCode {};
00115
00117     CW_SignResult() : errorCode(CW_NOK) {
00118     }
00119 };
00120
00121 /**********************************************************************
00122  * 3. CryptnoxWallet class
00123  **********************************************************************/
00124
00155 class CryptnoxWallet {
00156 public:
00165     CryptnoxWallet(CW_NfcTransport& driver, CW_Logger& logger,
00166                    CW_CryptoProvider& crypto, CW_Platform& platform);
00167
00168     CryptnoxWallet(const CryptnoxWallet&) = delete;
00169     CryptnoxWallet& operator=(const CryptnoxWallet&) = delete;
00170
00175     bool begin();
00176
00195     bool connect(CW_SecureSession& session);
00196
00212     bool establishSecureChannel(CW_SecureSession& session);
00213
00227     void disconnect(CW_SecureSession& session);
00228
00240     bool getCardInfo(CW_SecureSession& session, CW_CardInfo* info = NULL);
00241
00263     bool verifyPin(CW_SecureSession& session, const uint8_t* pin, uint8_t pinLength);
00264
00294     bool getPublicKey(CW_SecureSession& session,
00295                       const uint8_t* derivePath, uint8_t derivePathLength,
00296                       uint8_t* publicKey64);
00297
00342     CW_SignResult sign(CW_SignRequest& request);
00343
00365     bool getPublicKey(CW_SecureSession& session, uint8_t keyType,
00366                       const uint8_t* derivePath, uint8_t derivePathLength,
00367                       uint8_t* pubKey, uint16_t pubKeyBufSize, uint16_t& pubKeyLength);
00368
00393     bool writeUserData(CW_SecureSession& session,
00394                        const uint8_t* data, uint16_t dataLength);
00395
00407     static bool parseDerSignature(const uint8_t* der, uint8_t derLength,
00408                                   uint8_t* r, uint8_t& rLength,
00409                                   uint8_t* s, uint8_t& sLength);
00410
00411 private:
00412     CW_Logger&       _logger;
00413     CW_Platform&     _platform;
```

© 2026 Cryptnox SA

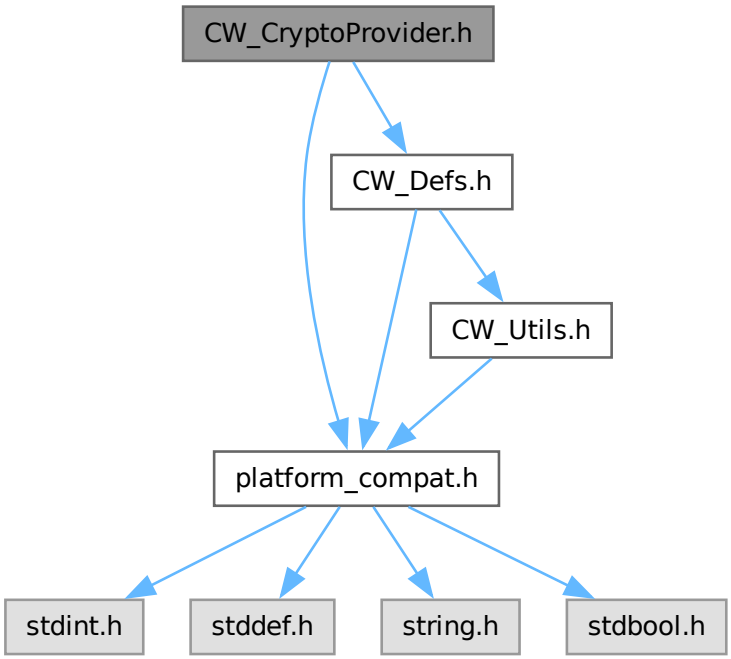
```
00414     CW_SecureChannel _secure;
00415
00416     static bool isSecureChannelOpen(const CW_SecureSession& session);
00417     bool printPN532FirmwareVersion();
00418
00419     /* Sign helper methods */
00420     bool validateSignRequest(const CW_SignRequest& request, CW_SignResult& result);
00421     static void buildSignPayload(const CW_SignRequest& request, uint8_t* data, uint16_t& dataLength);
00422     bool sendSignApdu(CW_SignRequest& request, const uint8_t* data, uint16_t dataLength,
00423                       uint8_t* derResponse, uint16_t derResponseSize,
00424                       uint16_t& derLength, CW_SignResult& result);
00425     bool extractRawSignature(const uint8_t* derResponse, uint16_t derLength, CW_SignResult& result);
00426     void debugPrintSignature(const uint8_t* signature);
00427 };
00428
00429 #endif // CRYPTNOXWALLET_H
```

5.5 CW_CryptoProvider.h File Reference

Abstract cryptographic primitives interface.

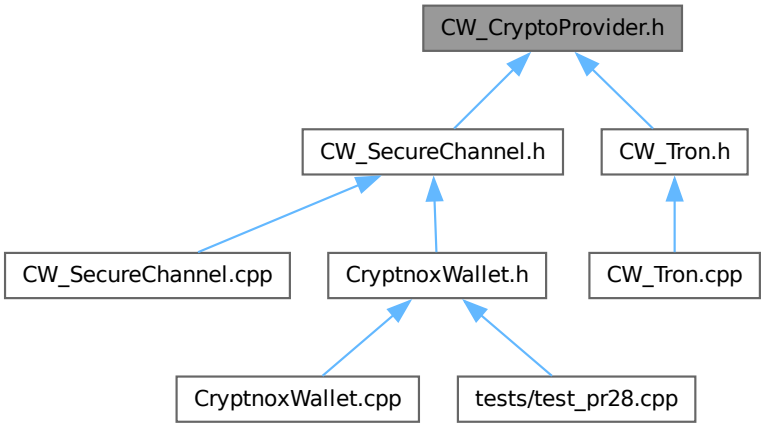
```
#include "platform_compat.h"
#include "CW_Defs.h"
```

Include dependency graph for CW_CryptoProvider.h:



© 2026 Cryptnox SA

This graph shows which files directly or indirectly include this file:



Classes

- class CW_CryptoProvider

  Abstract interface for cryptographic operations used by CW_SecureChannel.

5.5.1 Detailed Description

Abstract cryptographic primitives interface.
Declares CW_CryptoProvider, the contract that any concrete cryptographic backend (mbedTLS, BearSSL, OpenSSL, ESP32 hardware crypto, micro-ecc + AESLib + SHA512, ...) must implement so the secure channel remains decoupled from any specific crypto library.
Provides: SHA-256/512, AES-CBC encrypt/decrypt, ECDH shared secret, EC key pair generation, and cryptographically secure RNG.
One of the three adapter interfaces a host integration must provide.
Definition in file CW_CryptoProvider.h.

5.6 CW_CryptoProvider.h

Go to the documentation of this file.

```
00001 /*
00002  * SPDX-License-Identifier: LGPL-3.0-or-later
00003  * Copyright (c) 2026 Cryptnox SA
00004  */
00005
00020
00021 #ifndef CW_CRYPTOPROVIDER_H
00022 #define CW_CRYPTOPROVIDER_H
00023
00024 /**********************************************************************
00025  * 1. Included files
00026  **********************************************************************/
00027
00028 #include "platform_compat.h"
00029 #include "CW_Defs.h"
00030
00031 /**********************************************************************
00032  * 2. Class declaration
00033  **********************************************************************/
00034
00045 class CW_CryptoProvider {
00046 public:
00055     virtual bool sha256(const uint8_t* data, size_t len, uint8_t* out) = 0;
```

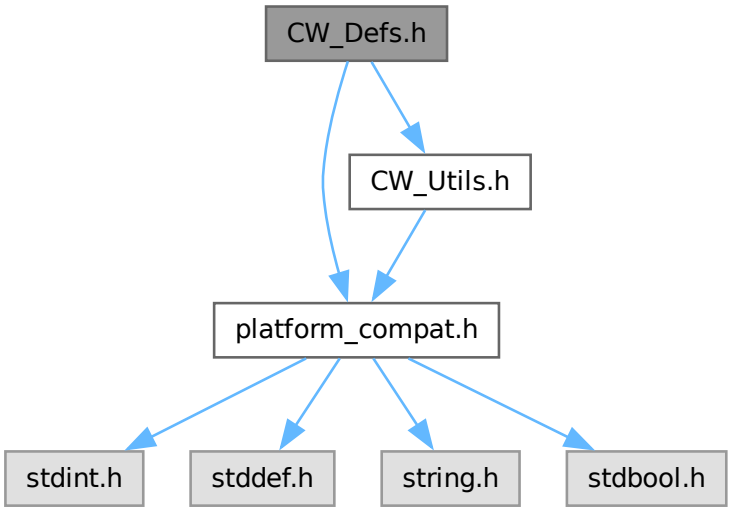
```
00056
00065     virtual bool sha512(const uint8_t* data, size_t len, uint8_t* out) = 0;
00066
00080     virtual uint16_t aesCbcEncrypt(const uint8_t* in, uint16_t len, uint8_t* out,
00081                                    const uint8_t* key, uint8_t keyLen,
00082                                    uint8_t* iv, bool bitPadding) = 0;
00083
00096     virtual uint16_t aesCbcDecrypt(uint8_t* in, uint16_t len, uint8_t* out,
00097                                    const uint8_t* key, uint8_t keyLen,
00098                                    uint8_t* iv, bool bitPadding) = 0;
00099
00109     virtual bool ecdh(const uint8_t* pubKey, const uint8_t* privKey,
00110                       uint8_t* secret, CW_Curve curve) = 0;
00111
00120     virtual bool makeKey(uint8_t* pubKey, uint8_t* privKey,
00121                          CW_Curve curve) = 0;
00122
00130     virtual bool random(uint8_t* dest, unsigned size) = 0;
00131
00142     virtual bool ecdsaVerify(const uint8_t* pubKey64,
00143                              const uint8_t* hash, size_t hashLen,
00144                              const uint8_t* sig, CW_Curve curve) = 0;
00145
00146     virtual ~CW_CryptoProvider() {}
00147 };
00148
00149 #endif // CW_CRYPTOPROVIDER_H
```

5.7 CW_Defs.h File Reference

Shared constants, error codes, and session state for the SDK.

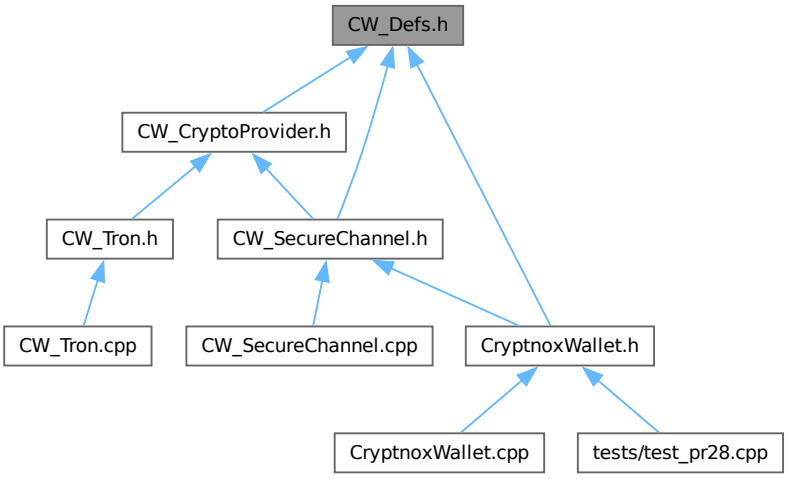
```
#include "platform_compat.h"
#include "CW_Utils.h"
```

Include dependency graph for CW_Defs.h:



© 2026 Cryptnox SA

This graph shows which files directly or indirectly include this file:



Classes

- struct CW_SecureSession

  Holds cryptographic session state for reentrant secure channel operations.

Macros

- #define CW_AESKEY_SIZE (32U)
- #define CW_MACKEY_SIZE (32U)
- #define CW_IV_SIZE (16U)
- #define CW_OK (0x00U)
- #define CW_NOK (0x01U)
- #define CW_INVALID_SESSION (0x02U)
- #define CW_SIGN_CURR_K1 (0x00U)
- #define CW_SIGN_CURR_R1 (0x10U)
- #define CW_SIGN_DERIVE_K1 (0x01U)
- #define CW_SIGN_DERIVE_R1 (0x11U)
- #define CW_SIGN_PINLESS_K1 (0x03U)
- #define CW_SIGN_CURR_ED25519 (0x20U)
- #define CW_SIGN_DERIVE_ED25519 (0x21U)
- #define CW_SIGN_WITH_PIN (false)
- #define CW_SIGN_PINLESS (true)
- #define CW_SIGN_SIG_ECDSA_LOW_S (0x00U)
- #define CW_SIGN_SIG_ECDSA_EOSIO (0x01U)
- #define CW_SIGN_SIG_SCHNORR_BIP340 (0x02U)
- #define CW_SIGN_SIG_EDDSA (0x03U)
- #define CW_SIGN_KEY_TOO_SHORT (0x80U)
- #define CW_SIGN_NO_KEY_LOADED (0x81U)
- #define CW_SIGN_PIN_INCORRECT (0x82U)
- #define CW_SIGN_KEY_TOO_SHORT_WITH_PINLESS_MODE (0x83U)
- #define CW_SIGN_INVALID_PATH (0x84U)
- #define CW_SIGN_MESSAGE_TOO_LONG (0x85U)
- #define CW_SIGN_UNSUPPORTED_KEY_TYPE (0x86U)

© 2026 Cryptnox SA

- #define CW_RAW_SIGNATURE_SIZE (64U)
- #define CW_HASH_SIZE (32U)
- #define CW_ED25519_PUBKEY_SIZE (32U)
- #define CW_RAW_PUBKEY_SIZE (64U)
- #define CW_SEC1_PUBKEY_SIZE (65U)
- #define CW_SEC1_UNCOMPRESSED_TAG (0x04U)
- #define CW_MAX_ED25519_MESSAGE_LENGTH (CW_USER_DATA_PAGE_SIZE - 2U - CW_MAX_DERIVE_PATH_LENGTH - CW_MAX_PIN_LENGTH) /* 177 */
- #define CW_MAX_ED25519_MESSAGE_LENGTH_CURR (CW_USER_DATA_PAGE_SIZE - 2U - CW_MAX_PIN_LENGTH) /* 197 */
- #define CW_MAX_DERIVE_PATH_LENGTH (20U)
- #define CW_MIN_PIN_LENGTH (4U)
- #define CW_MAX_PIN_LENGTH (9U)
- #define CW_USER_DATA_PAGE_SIZE (208U)
- #define CW_SECURE_PAYLOAD_MAX (CW_USER_DATA_PAGE_SIZE + 16U)
- #define CW_CONNECT_MAX_ATTEMPTS (5U)
- #define CW_SIG_R_OFFSET (0U)
- #define CW_SIG_S_OFFSET (32U)
- #define CW_DER_TAG_SEQUENCE (0x30U)
- #define CW_DER_TAG_INTEGER (0x02U)
- #define CW_CERT_NONCE_SIZE (8U)
- #define CW_CERT_OK (0x00U)
- #define CW_CERT_FORMAT_ERROR (0x10U)
- #define CW_CERT_NONCE_MISMATCH (0x11U)
- #define CW_CERT_CARD_SIG_INVALID (0x12U)
- #define CW_CERT_MANUF_SIG_INVALID (0x13U)
- #define CW_CERT_KEY_NOT_FOUND (0x14U)
- #define CW_MANUF_CERT_MAX_BYTES (420U)
- #define CW_VERIFY_CERT 1
- #define CW_MIN_APPLET_MAJOR 0U
- #define CW_DEBUG_LOGGING 0

Enumerations

- enum CW_Curve { CW_CURVE_SECP256R1 = 0 , CW_CURVE_SECP256K1 = 1 }
  Portable curve identifier used throughout the SDK.

5.7.1 Detailed Description

Shared constants, error codes, and session state for the SDK.
Defines:

- AES session key / IV sizes
- Generic and SIGN-specific error codes (`CW_OK`, `CW_NOK`, `CW_SIGN_*`)
- SIGN APDU parameter values (key types, signature types, PIN/PIN-less)
- Buffer and protocol size limits
- Certificate verification result codes (`CW_CERT_*`)
- CW_Curve enum (portable curve identifier)
- CW_SecureSession (encryption + MAC keys + rolling IV)
- Compile-time security gates: `CW_VERIFY_CERT`, `CW_DEBUG_LOGGING`

Definition in file CW_Defs.h.

© 2026 Cryptnox SA

5.7.2 Macro Definition Documentation

5.7.2.1 CW_AESKEY_SIZE

```
#define CW_AESKEY_SIZE (32U)
```

AES-256 session encryption key size in bytes
Definition at line 75 of file CW_Defs.h.
Referenced by CryptnoxWallet::isSecureChannelOpen(), and CW_SecureChannel::mutuallyAuthenticate().

5.7.2.2 CW_CERT_CARD_SIG_INVALID

```
#define CW_CERT_CARD_SIG_INVALID (0x12U)
```

Card cert ECDSA sig failed
Definition at line 162 of file CW_Defs.h.
Referenced by CW_SecureChannel::verifyCertificateChain().

5.7.2.3 CW_CERT_FORMAT_ERROR

```
#define CW_CERT_FORMAT_ERROR (0x10U)
```

Malformed certificate data
Definition at line 160 of file CW_Defs.h.
Referenced by CW_SecureChannel::verifyCertificateChain().

5.7.2.4 CW_CERT_KEY_NOT_FOUND

```
#define CW_CERT_KEY_NOT_FOUND (0x14U)
```

Device public key OID not found
Definition at line 164 of file CW_Defs.h.
Referenced by CW_SecureChannel::verifyCertificateChain().

5.7.2.5 CW_CERT_MANUF_SIG_INVALID

```
#define CW_CERT_MANUF_SIG_INVALID (0x13U)
```

Manufacturer cert ECDSA sig failed
Definition at line 163 of file CW_Defs.h.
Referenced by CW_SecureChannel::verifyCertificateChain().

5.7.2.6 CW_CERT_NONCE_MISMATCH

```
#define CW_CERT_NONCE_MISMATCH (0x11U)
```

Challenge nonce not echoed
Definition at line 161 of file CW_Defs.h.
Referenced by CW_SecureChannel::verifyCertificateChain().

5.7.2.7 CW_CERT_NONCE_SIZE

```
#define CW_CERT_NONCE_SIZE (8U)
```

Challenge nonce length in bytes
Definition at line 156 of file CW_Defs.h.
Referenced by CW_SecureChannel::verifyCertificateChain().

5.7.2.8 CW_CERT_OK

```
#define CW_CERT_OK (0x00U)
```

Certificate chain verified
Definition at line 159 of file CW_Defs.h.
Referenced by CryptnoxWallet::establishSecureChannel(), and CW_SecureChannel::verifyCertificateChain().

© 2026 Cryptnox SA

5.7.2.9 CW_CONNECT_MAX_ATTEMPTS

```
#define CW_CONNECT_MAX_ATTEMPTS (5U)
```

Max NFC connection retry attempts
Definition at line 145 of file CW_Defs.h.
Referenced by CryptnoxWallet::connect().

5.7.2.10 CW_DEBUG_LOGGING

```
#define CW_DEBUG_LOGGING 0
```

Set to 1 to enable library-internal debug logging via CW_Logger.
Off by default. Enabling it kills flash optimisation — measured on Arduino UNO R4 (Renesas RA4↩ M1): +149 KB flash, +6 KB SRAM (31 % → 88 % of a 256 KB image on the Sign example). Same order of magnitude on every constrained MCU.
Also leaks session state over UART (SEC-012). Bring-up only, never in release builds.
Definition at line 279 of file CW_Defs.h.

5.7.2.11 CW_DER_TAG_INTEGER

```
#define CW_DER_TAG_INTEGER (0x02U)
```

Definition at line 153 of file CW_Defs.h.
Referenced by CryptnoxWallet::parseDerSignature().

5.7.2.12 CW_DER_TAG_SEQUENCE

```
#define CW_DER_TAG_SEQUENCE (0x30U)
```

Definition at line 152 of file CW_Defs.h.
Referenced by CryptnoxWallet::extractRawSignature(), and CryptnoxWallet::parseDerSignature().

5.7.2.13 CW_ED25519_PUBKEY_SIZE

```
#define CW_ED25519_PUBKEY_SIZE (32U)
```

Raw Ed25519 public key size (Solana address)
Definition at line 119 of file CW_Defs.h.

5.7.2.14 CW_HASH_SIZE

```
#define CW_HASH_SIZE (32U)
```

Standard hash size (ECDSA digest)
Definition at line 118 of file CW_Defs.h.
Referenced by CryptnoxWallet::validateSignRequest().

5.7.2.15 CW_INVALID_SESSION

```
#define CW_INVALID_SESSION (0x02U)
```

Invalid session
Definition at line 82 of file CW_Defs.h.
Referenced by CryptnoxWallet::validateSignRequest().

5.7.2.16 CW_IV_SIZE

```
#define CW_IV_SIZE (16U)
```

AES-CBC IV size in bytes
Definition at line 77 of file CW_Defs.h.
Referenced by CW_SecureChannel::aesCbcEncrypt(), and CW_SecureChannel::mutuallyAuthenticate().

© 2026 Cryptnox SA

5.7.2.17 CW_MACKEY_SIZE

```
#define CW_MACKEY_SIZE (32U)
```
AES-256 session MAC key size in bytes
Definition at line 76 of file CW_Defs.h.
Referenced by CW_SecureChannel::mutuallyAuthenticate().

5.7.2.18 CW_MANUF_CERT_MAX_BYTES

```
#define CW_MANUF_CERT_MAX_BYTES (420U)
```
Definition at line 168 of file CW_Defs.h.
Referenced by CW_SecureChannel::getManufacturerCertificate().

5.7.2.19 CW_MAX_DERIVE_PATH_LENGTH

```
#define CW_MAX_DERIVE_PATH_LENGTH (20U)
```
Max BIP32 path bytes
Definition at line 138 of file CW_Defs.h.
Referenced by CryptnoxWallet::getPublicKey(), and CryptnoxWallet::validateSignRequest().

5.7.2.20 CW_MAX_ED25519_MESSAGE_LENGTH

```
#define CW_MAX_ED25519_MESSAGE_LENGTH (CW_USER_DATA_PAGE_SIZE - 2U - CW_MAX_DERIVE_PATH_LENGTH
- CW_MAX_PIN_LENGTH) /* 177 */
```
Definition at line 136 of file CW_Defs.h.

5.7.2.21 CW_MAX_ED25519_MESSAGE_LENGTH_CURR

```
#define CW_MAX_ED25519_MESSAGE_LENGTH_CURR (CW_USER_DATA_PAGE_SIZE - 2U - CW_MAX_PIN_LENGTH)
/* 197 */
```
Definition at line 137 of file CW_Defs.h.

5.7.2.22 CW_MAX_PIN_LENGTH

```
#define CW_MAX_PIN_LENGTH (9U)
```
Maximum PIN length
Definition at line 140 of file CW_Defs.h.
Referenced by CryptnoxWallet::buildSignPayload(), cwMaxEd25519Message(), CryptnoxWallet::validateSignRequest(), and CryptnoxWallet::verifyPin().

5.7.2.23 CW_MIN_APPLET_MAJOR

```
#define CW_MIN_APPLET_MAJOR 0U
```
Minimum applet major version accepted at SELECT (PR #27 review #9).
The version comes from the unauthenticated SELECT response, so an NFC man-in-the-middle can report 1.x to a 2.0 card and force the legacy channel (no anti-replay counter, no mutual-auth proof). Against a genuine 2.0 card that only breaks the channel, but a fleet made only of 2.0+ cards can refuse legacy outright with -DCW_MIN_APPLET_MAJOR=2. An unreadable version counts as 0 and is refused too. Default 0 accepts every applet.
Definition at line 264 of file CW_Defs.h.
Referenced by CW_SecureChannel::selectApdu().

5.7.2.24 CW_MIN_PIN_LENGTH

```
#define CW_MIN_PIN_LENGTH (4U)
```
Minimum PIN length
Definition at line 139 of file CW_Defs.h.
Referenced by CryptnoxWallet::validateSignRequest(), and CryptnoxWallet::verifyPin().

© 2026 Cryptnox SA

5.7.2.25 CW_NOK

```
#define CW_NOK (0x01U)
```

NOK
Definition at line 81 of file CW_Defs.h.
Referenced by CW_SignResult::CW_SignResult(), CryptnoxWallet::extractRawSignature(), and CryptnoxWallet::sign().

5.7.2.26 CW_OK

```
#define CW_OK (0x00U)
```

OK
Definition at line 80 of file CW_Defs.h.
Referenced by CryptnoxWallet::sign().

5.7.2.27 CW_RAW_PUBKEY_SIZE

```
#define CW_RAW_PUBKEY_SIZE (64U)
```

Raw k1/r1 public key (X[32] || Y[32])
Definition at line 120 of file CW_Defs.h.
Referenced by CryptnoxWallet::getPublicKey().

5.7.2.28 CW_RAW_SIGNATURE_SIZE

```
#define CW_RAW_SIGNATURE_SIZE (64U)
```

Raw signature (r[32] + s[32], or Ed25519 R||S)
Definition at line 117 of file CW_Defs.h.
Referenced by CryptnoxWallet::debugPrintSignature(), CryptnoxWallet::extractRawSignature(), and CryptnoxWallet::sign().

5.7.2.29 CW_SEC1_PUBKEY_SIZE

```
#define CW_SEC1_PUBKEY_SIZE (65U)
```

SEC1 uncompressed public key (0x04 || X || Y)
Definition at line 121 of file CW_Defs.h.
Referenced by CryptnoxWallet::getPublicKey().

5.7.2.30 CW_SEC1_UNCOMPRESSED_TAG

```
#define CW_SEC1_UNCOMPRESSED_TAG (0x04U)
```

SEC1 uncompressed point prefix
Definition at line 122 of file CW_Defs.h.
Referenced by CryptnoxWallet::getPublicKey().

5.7.2.31 CW_SECURE_PAYLOAD_MAX

```
#define CW_SECURE_PAYLOAD_MAX (CW_USER_DATA_PAGE_SIZE + 16U)
```

224
Definition at line 144 of file CW_Defs.h.

5.7.2.32 CW_SIG_R_OFFSET

```
#define CW_SIG_R_OFFSET (0U)
```

Byte offset of the r component
Definition at line 148 of file CW_Defs.h.

© 2026 Cryptnox SA

5.7.2.33 CW_SIG_S_OFFSET

```
#define CW_SIG_S_OFFSET (32U)
```
Byte offset of the s component
Definition at line 149 of file CW_Defs.h.

5.7.2.34 CW_SIGN_CURR_ED25519

```
#define CW_SIGN_CURR_ED25519 (0x20U)
```
Current Ed25519 key (Solana; applet v2.0+)
Definition at line 93 of file CW_Defs.h.
Referenced by check_ed25519_applet_gate(), check_pubkey_allow_list(), and cwPubKeyTypeAllowed().

5.7.2.35 CW_SIGN_CURR_K1

```
#define CW_SIGN_CURR_K1 (0x00U)
```
Current key (k1)
Definition at line 88 of file CW_Defs.h.
Referenced by check_ed25519_applet_gate(), check_pubkey_allow_list(), CW_SignRequest::CW_SignRequest(), cwPubKeyTypeAllowed(), and CryptnoxWallet::getPublicKey().

5.7.2.36 CW_SIGN_CURR_R1

```
#define CW_SIGN_CURR_R1 (0x10U)
```
Current key (r1)
Definition at line 89 of file CW_Defs.h.
Referenced by check_pubkey_allow_list(), and cwPubKeyTypeAllowed().

5.7.2.37 CW_SIGN_DERIVE_ED25519

```
#define CW_SIGN_DERIVE_ED25519 (0x21U)
```
Derive Ed25519 key (Solana; applet v2.0+)
Definition at line 94 of file CW_Defs.h.
Referenced by check_pubkey_allow_list(), and cwPubKeyTypeAllowed().

5.7.2.38 CW_SIGN_DERIVE_K1

```
#define CW_SIGN_DERIVE_K1 (0x01U)
```
Derive with k1 curve
Definition at line 90 of file CW_Defs.h.
Referenced by check_pubkey_allow_list(), cwPubKeyTypeAllowed(), and CryptnoxWallet::getPublicKey().

5.7.2.39 CW_SIGN_DERIVE_R1

```
#define CW_SIGN_DERIVE_R1 (0x11U)
```
Derive with r1 curve
Definition at line 91 of file CW_Defs.h.
Referenced by check_pubkey_allow_list(), and cwPubKeyTypeAllowed().

5.7.2.40 CW_SIGN_INVALID_PATH

```
#define CW_SIGN_INVALID_PATH (0x84U)
```
Derivation path missing, too long or not a multiple of 4
Definition at line 112 of file CW_Defs.h.
Referenced by CryptnoxWallet::validateSignRequest().

© 2026 Cryptnox SA

5.7.2.41 CW_SIGN_KEY_TOO_SHORT

```
#define CW_SIGN_KEY_TOO_SHORT (0x80U)
```
Definition at line 108 of file CW_Defs.h.
Referenced by CryptnoxWallet::validateSignRequest().

5.7.2.42 CW_SIGN_KEY_TOO_SHORT_WITH_PINLESS_MODE

```
#define CW_SIGN_KEY_TOO_SHORT_WITH_PINLESS_MODE (0x83U)
```
Definition at line 111 of file CW_Defs.h.
Referenced by CryptnoxWallet::validateSignRequest().

5.7.2.43 CW_SIGN_MESSAGE_TOO_LONG

```
#define CW_SIGN_MESSAGE_TOO_LONG (0x85U)
```
Hash/message longer than the key type allows
Definition at line 113 of file CW_Defs.h.
Referenced by CryptnoxWallet::validateSignRequest().

5.7.2.44 CW_SIGN_NO_KEY_LOADED

```
#define CW_SIGN_NO_KEY_LOADED (0x81U)
```
Definition at line 109 of file CW_Defs.h.
Referenced by CryptnoxWallet::sendSignApdu().

5.7.2.45 CW_SIGN_PIN_INCORRECT

```
#define CW_SIGN_PIN_INCORRECT (0x82U)
```
Definition at line 110 of file CW_Defs.h.
Referenced by CryptnoxWallet::validateSignRequest().

5.7.2.46 CW_SIGN_PINLESS

```
#define CW_SIGN_PINLESS (true)
```
PIN-less path
Definition at line 99 of file CW_Defs.h.

5.7.2.47 CW_SIGN_PINLESS_K1

```
#define CW_SIGN_PINLESS_K1 (0x03U)
```
PIN-less path (k1 only)
Definition at line 92 of file CW_Defs.h.
Referenced by CryptnoxWallet::validateSignRequest().

5.7.2.48 CW_SIGN_SIG_ECDSA_EOSIO

```
#define CW_SIGN_SIG_ECDSA_EOSIO (0x01U)
```
ECDSA EOSIO format
Definition at line 103 of file CW_Defs.h.

5.7.2.49 CW_SIGN_SIG_ECDSA_LOW_S

```
#define CW_SIGN_SIG_ECDSA_LOW_S (0x00U)
```
ECDSA with canonical low S
Definition at line 102 of file CW_Defs.h.
Referenced by CW_SignRequest::CW_SignRequest(), and signWith().

© 2026 Cryptnox SA

5.7.2.50 CW_SIGN_SIG_EDDSA

```
#define CW_SIGN_SIG_EDDSA (0x03U)
```
EdDSA (Ed25519) — forced automatically for Ed25519 key types
Definition at line 105 of file CW_Defs.h.
Referenced by check_ed25519_applet_gate(), and CryptnoxWallet::sendSignApdu().

5.7.2.51 CW_SIGN_SIG_SCHNORR_BIP340

```
#define CW_SIGN_SIG_SCHNORR_BIP340 (0x02U)
```
Schnorr BIP340
Definition at line 104 of file CW_Defs.h.
Referenced by CryptnoxWallet::sign().

5.7.2.52 CW_SIGN_UNSUPPORTED_KEY_TYPE

```
#define CW_SIGN_UNSUPPORTED_KEY_TYPE (0x86U)
```
Key type needs a newer applet (Ed25519 on $<$ 2.0)
Definition at line 114 of file CW_Defs.h.
Referenced by check_ed25519_applet_gate(), and CryptnoxWallet::validateSignRequest().

5.7.2.53 CW_SIGN_WITH_PIN

```
#define CW_SIGN_WITH_PIN (false)
```
PIN path
Definition at line 98 of file CW_Defs.h.
Referenced by CW_SignRequest::CW_SignRequest(), and signWith().

5.7.2.54 CW_USER_DATA_PAGE_SIZE

```
#define CW_USER_DATA_PAGE_SIZE (208U)
```
Max plaintext bytes per write user data page
Definition at line 141 of file CW_Defs.h.
Referenced by CryptnoxWallet::buildSignPayload(), check_session_clearing(), cwMaxEd25519Message(), CryptnoxWallet::sign(), and CryptnoxWallet::writeUserData().

5.7.2.55 CW_VERIFY_CERT

```
#define CW_VERIFY_CERT 1
```
Card certificate chain verification (SEC-004 / H-07).
At 1, establishSecureChannel() checks the card's chain against CW_TRUSTED_CA_KEYS and refuses to open a session on failure. At 0 the check is compiled out entirely and ANY card is accepted, including a forged one, which will then sign whatever it is given. Release builds must ship 1.
0 is for bring-up against a card signed by a non-production CA, and is meant to be passed from the build (-DCW_VERIFY_CERT=0) rather than edited in here, so a bench setting cannot reach a commit. The better answer in that case is to add the dev CA public key to CW_TRUSTED_CA_KEYS in CW_TrustedKeys.h — the table holds several and tries each in turn, so verification stays enabled and nothing has to be weakened.
This flag gates the chain check only. Providers must keep sha256() working at 0: the applet 2.↩
0 mutual-auth proof (SHA256(Kenc || challenge)) relies on it.
0 alone is a build error, so a stray -DCW_VERIFY_CERT=0 cannot reach a release. Bring-up builds must also pass -DCW_ALLOW_UNVERIFIED_CARD=1 to acknowledge it; that turns the error into a warning.
Definition at line 244 of file CW_Defs.h.

5.8 CW_Defs.h

Go to the documentation of this file.

© 2026 Cryptnox SA

```
00001 /*
00002  * SPDX-License-Identifier: LGPL-3.0-or-later
00003  * Copyright (c) 2026 Cryptnox SA
00004  */
00005
00020
00021 #ifndef CW_DEFS_H
00022 #define CW_DEFS_H
00023
00024 /**********************************************************************
00025  * 0. Doxygen module groups (used by @ingroup throughout the SDK)
00026  **********************************************************************/
00027
00035
00044
00053
00062
00063 /**********************************************************************
00064  * 1. Included files
00065  **********************************************************************/
00066
00067 #include "platform_compat.h"
00068 #include "CW_Utils.h"
00069
00070 /**********************************************************************
00071  * 2. Constants / define declarations
00072  **********************************************************************/
00073
00074 /* Session key sizes */
00075 #define CW_AESKEY_SIZE    (32U)
00076 #define CW_MACKEY_SIZE    (32U)
00077 #define CW_IV_SIZE        (16U)
00078
00079 /* Generic error codes */
00080 #define CW_OK                         (0x00U)
00081 #define CW_NOK                        (0x01U)
00082 #define CW_INVALID_SESSION            (0x02U)
00083
00084 /* Key / path types for SIGN command (keyType).
00085  * Encoded as (derivation | curve): low nibble = derivation
00086  * (0 current, 1 derive, 2 derive-and-make-current, 3 pinless),
00087  * high nibble = curve (0 k1, 1 r1, 2 ed25519). */
00088 #define CW_SIGN_CURR_K1               (0x00U)
00089 #define CW_SIGN_CURR_R1               (0x10U)
00090 #define CW_SIGN_DERIVE_K1             (0x01U)
00091 #define CW_SIGN_DERIVE_R1             (0x11U)
00092 #define CW_SIGN_PINLESS_K1            (0x03U)
00093 #define CW_SIGN_CURR_ED25519          (0x20U)
00094 #define CW_SIGN_DERIVE_ED25519        (0x21U)
00095
00096
00097 /* PIN mode for SIGN command */
00098 #define CW_SIGN_WITH_PIN              (false)
00099 #define CW_SIGN_PINLESS               (true)
00100
00101 /* Signature types for SIGN command (APDU P2) */
00102 #define CW_SIGN_SIG_ECDSA_LOW_S       (0x00U)
00103 #define CW_SIGN_SIG_ECDSA_EOSIO       (0x01U)
00104 #define CW_SIGN_SIG_SCHNORR_BIP340    (0x02U)
00105 #define CW_SIGN_SIG_EDDSA             (0x03U)
00106
00107 /* SIGN-specific error codes */
00108 #define CW_SIGN_KEY_TOO_SHORT                  (0x80U)
00109 #define CW_SIGN_NO_KEY_LOADED                  (0x81U)
00110 #define CW_SIGN_PIN_INCORRECT                  (0x82U)
00111 #define CW_SIGN_KEY_TOO_SHORT_WITH_PINLESS_MODE (0x83U)
00112 #define CW_SIGN_INVALID_PATH                   (0x84U)
00113 #define CW_SIGN_MESSAGE_TOO_LONG               (0x85U)
00114 #define CW_SIGN_UNSUPPORTED_KEY_TYPE           (0x86U)
00115
00116 /* Size constants */
00117 #define CW_RAW_SIGNATURE_SIZE         (64U)
00118 #define CW_HASH_SIZE                  (32U)
00119 #define CW_ED25519_PUBKEY_SIZE        (32U)
00120 #define CW_RAW_PUBKEY_SIZE            (64U)
00121 #define CW_SEC1_PUBKEY_SIZE           (65U)
00122 #define CW_SEC1_UNCOMPRESSED_TAG      (0x04U)
00123 /* Ed25519 signs the raw message (the card hashes internally). Cap the message
00124  * so the framed payload [len(2)|msg|path|pin(9)] fits one secure-channel page.
00125  *
00126  * These two are WORST-CASE bounds, kept for reference and for callers that want a
00127  * compile-time constant. CW_MAX_ED25519_MESSAGE_LENGTH reserves the full 20-byte
00128  * path, so it under-reports the room available to a shorter path -- a 4-level
00129  * Solana path (16 B) leaves 181, not 177. validateSignRequest() therefore
00130  * computes the bound from the actual request instead of using these; sizing a
00131  * caller-side buffer with CW_MAX_ED25519_MESSAGE_LENGTH stays safe, just
00132  * conservative.
```

© 2026 Cryptnox SA

```
00133  *
00134  * Neither figure is a card limit: the v2.0 SIGN spec accepts up to 1200 bytes of
00135  * raw data. The ceiling is this SDK's single-page secure-channel buffer. */
00136 #define CW_MAX_ED25519_MESSAGE_LENGTH      (CW_USER_DATA_PAGE_SIZE - 2U - CW_MAX_DERIVE_PATH_LENGTH -
      CW_MAX_PIN_LENGTH) /* 177 */
00137 #define CW_MAX_ED25519_MESSAGE_LENGTH_CURR (CW_USER_DATA_PAGE_SIZE - 2U - CW_MAX_PIN_LENGTH)
      /* 197 */
00138 #define CW_MAX_DERIVE_PATH_LENGTH     (20U)
00139 #define CW_MIN_PIN_LENGTH              (4U)
00140 #define CW_MAX_PIN_LENGTH              (9U)
00141 #define CW_USER_DATA_PAGE_SIZE        (208U)
00142 /* Largest plaintext one secure-channel exchange can hand back: a full page plus
00143  * the block the bit padding may add. Bounds every decryptedOutput buffer. */
00144 #define CW_SECURE_PAYLOAD_MAX         (CW_USER_DATA_PAGE_SIZE + 16U)
00145 #define CW_CONNECT_MAX_ATTEMPTS        (5U)
00146
00147 /* Byte offsets within a raw 64-byte signature (r[32] || s[32]) */
00148 #define CW_SIG_R_OFFSET               (0U)
00149 #define CW_SIG_S_OFFSET               (32U)
00150
00151 /* DER encoding tags (ASN.1) */
00152 #define CW_DER_TAG_SEQUENCE           (0x30U)
00153 #define CW_DER_TAG_INTEGER            (0x02U)
00154
00155 /* Certificate verification constants */
00156 #define CW_CERT_NONCE_SIZE            (8U)
00157
00158 /* Certificate verification result codes */
00159 #define CW_CERT_OK                    (0x00U)
00160 #define CW_CERT_FORMAT_ERROR          (0x10U)
00161 #define CW_CERT_NONCE_MISMATCH        (0x11U)
00162 #define CW_CERT_CARD_SIG_INVALID      (0x12U)
00163 #define CW_CERT_MANUF_SIG_INVALID     (0x13U)
00164 #define CW_CERT_KEY_NOT_FOUND         (0x14U)
00165
00166 /* Manufacturer certificate maximum buffer size (bytes).
00167  * Actual Cryptnox Basic G1 manufacturer certificate is 411 bytes (0x019B). */
00168 #define CW_MANUF_CERT_MAX_BYTES       (420U)
00169
00170 /****************************************************************
00171  * 3. CW_Curve enum
00172  ****************************************************************/
00173
00183 enum CW_Curve {
00184     CW_CURVE_SECP256R1 = 0,
00185     CW_CURVE_SECP256K1 = 1
00186 };
00187
00188 /****************************************************************
00189  * 4. CW_SecureSession struct
00190  ****************************************************************/
00191
00200 struct CW_SecureSession {
00201     uint8_t aesKey[CW_AESKEY_SIZE] {};
00202     uint8_t macKey[CW_MACKEY_SIZE] {};
00203     uint8_t iv[CW_IV_SIZE] {};
00204     uint32_t macCounter {};
00205
00207     CW_SecureSession() = default;
00208
00210     void clear() {
00211         CW_Utils::secure_wipe(aesKey, sizeof(aesKey));
00212         CW_Utils::secure_wipe(macKey, sizeof(macKey));
00213         CW_Utils::secure_wipe(iv,     sizeof(iv));
00214         macCounter = 0U;
00215     }
00216 };
00217
00218 /****************************************************************
00219  * 5. Compile-time feature flags
00220  ****************************************************************/
00221
00243 #ifndef CW_VERIFY_CERT
00244 #define CW_VERIFY_CERT 1
00245 #endif
00246 #if CW_VERIFY_CERT == 0
00247 #  if defined(CW_ALLOW_UNVERIFIED_CARD) && (CW_ALLOW_UNVERIFIED_CARD == 1)
00248 #    warning "CW_VERIFY_CERT=0: card authenticity verification is compiled out (CRIT-02/H-07). Never
      ship this."
00249 #  else
00250 #    error "CW_VERIFY_CERT=0 disables card authenticity verification (CRIT-02/H-07). Add
      -DCW_ALLOW_UNVERIFIED_CARD=1 for bring-up builds only."
00251 #  endif
00252 #endif
00253
00263 #ifndef CW_MIN_APPLET_MAJOR
00264 #define CW_MIN_APPLET_MAJOR 0U
```

© 2026 Cryptnox SA

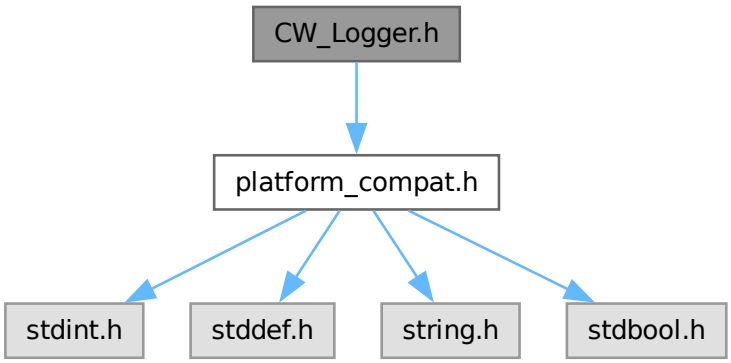
```
00265 #endif
00266
00278 #ifndef CW_DEBUG_LOGGING
00279 #  define CW_DEBUG_LOGGING 0
00280 #endif
00281
00282 #if CW_DEBUG_LOGGING && defined(NDEBUG)
00283 #  error "CW_DEBUG_LOGGING=1 is set but NDEBUG is defined (release/optimised build). " \
00284          "Debug logging must not ship in production firmware -- it leaks session state " \
00285          "over UART.  Remove -DCW_DEBUG_LOGGING=1 from your release build flags (SEC-012)."
00286 #endif
00287
00288 #endif // CW_DEFS_H
```

5.9 CW_Logger.h File Reference

Abstract logging interface.

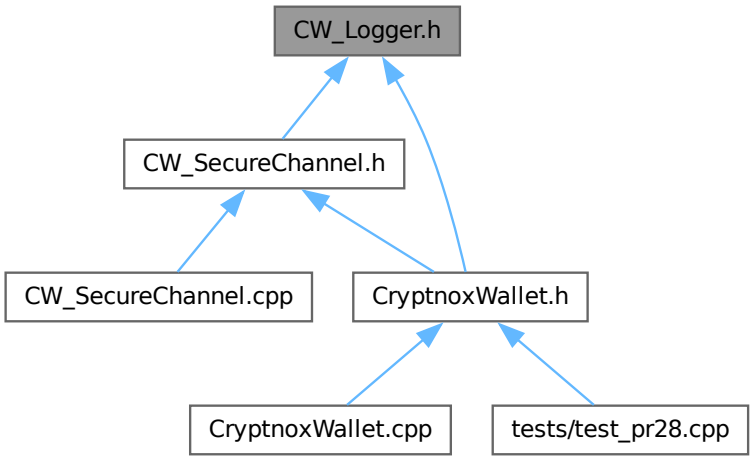
`#include "platform_compat.h"`

Include dependency graph for CW_Logger.h:



© 2026 Cryptnox SA

This graph shows which files directly or indirectly include this file:



Classes

- class CW_Logger

  Abstract interface for serial/debug output.

5.9.1 Detailed Description

Abstract logging interface.
Declares CW_Logger, the contract that any concrete output sink (UART, USB CDC, stdout, syslog, network, ...) must implement so the SDK remains independent of the host platform's logging facility.
The Arduino-style print/println overloads keep `F("...")`-quoted string literals in flash on Arduino targets; on non-Arduino targets `F()` is the identity macro (see platform_compat.h).
One of the three adapter interfaces a host integration must provide.
Definition in file CW_Logger.h.

5.10 CW_Logger.h

Go to the documentation of this file.

```
00001 /*
00002  * SPDX-License-Identifier: LGPL-3.0-or-later
00003  * Copyright (c) 2026 Cryptnox SA
00004  */
00005
00020
00021 #ifndef CW_LOGGER_H
00022 #define CW_LOGGER_H
00023
00024 /**************************************************************
00025  * 1. Included files
00026  **************************************************************/
00027
00028 #include "platform_compat.h"
00029
00030 /**************************************************************
00031  * 2. Class declaration
00032  **************************************************************/
00033
00048 class CW_Logger {
00049 public:
00055     virtual bool begin(unsigned long baudRate = 115200UL) = 0;
```

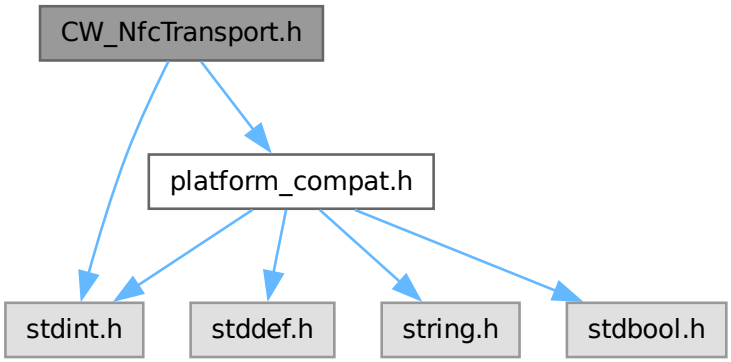
```
00056
00059     virtual void print(const __FlashStringHelper* str) = 0;
00060     virtual void print(const char* str) = 0;
00061     virtual void print(char c) = 0;
00062     virtual void print(uint8_t value, int base = DEC) = 0;
00063     virtual void print(uint16_t value, int base = DEC) = 0;
00064     virtual void print(uint32_t value, int base = DEC) = 0;
00065     virtual void print(int value, int base = DEC) = 0;
00067
00070     virtual void println() = 0;
00071     virtual void println(const __FlashStringHelper* str) = 0;
00072     virtual void println(const char* str) = 0;
00073     virtual void println(char c) = 0;
00074     virtual void println(uint8_t value, int base = DEC) = 0;
00075     virtual void println(uint16_t value, int base = DEC) = 0;
00076     virtual void println(uint32_t value, int base = DEC) = 0;
00077     virtual void println(int value, int base = DEC) = 0;
00079
00080     virtual ~CW_Logger() {}
00081 };
00082
00083 #endif // CW_LOGGER_H
```

5.11 CW_NfcTransport.h File Reference

Abstract NFC transport interface.

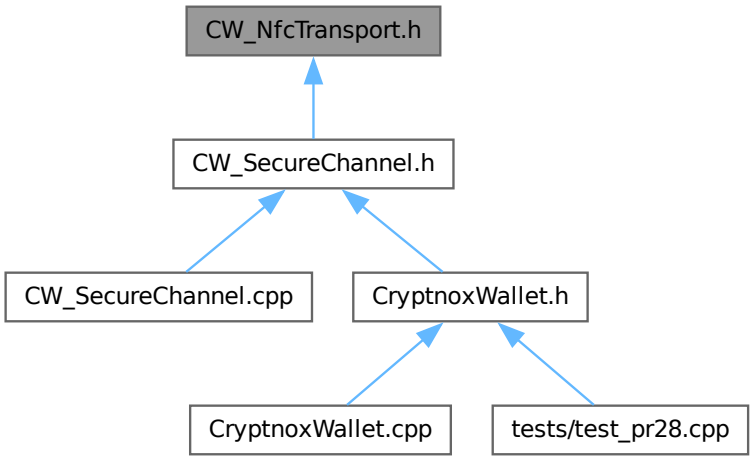
```
#include "platform_compat.h"
#include <stdint.h>
```

Include dependency graph for CW_NfcTransport.h:



© 2026 Cryptnox SA

This graph shows which files directly or indirectly include this file:



Classes

- class CW_NfcTransport

  Abstract interface for NFC transport operations.

5.11.1 Detailed Description

Abstract NFC transport interface.
Declares CW_NfcTransport, the contract that any concrete NFC reader driver (PN532, PN7150, PC/↩ SC, . . .) must implement so that CW_SecureChannel and CryptnoxWallet remain hardware-agnostic.
One of the three adapter interfaces a host integration must provide.
Definition in file CW_NfcTransport.h.

5.12 CW_NfcTransport.h

Go to the documentation of this file.

```
00001 /*
00002  * SPDX-License-Identifier: LGPL-3.0-or-later
00003  * Copyright (c) 2026 Cryptnox SA
00004  */
00005
00016
00017 #ifndef CW_NFCTRANSPORT_H
00018 #define CW_NFCTRANSPORT_H
00019
00020 /**************************************************************************
00021  * 1. Included files
00022  *************************************************************************/
00023
00024 #include "platform_compat.h"
00025 #include <stdint.h>
00026
00027 /**************************************************************************
00028  * 2. Class declaration
00029  *************************************************************************/
00030
00040 class CW_NfcTransport {
00041 public:
00046     virtual bool begin() = 0;
00047
00052     virtual bool inListPassiveTarget() = 0;
```

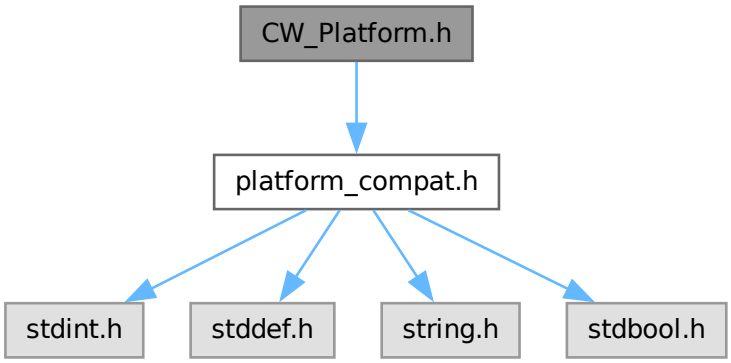
```
00053
00063     virtual bool sendAPDU(const uint8_t* apdu, uint8_t apduLen,
00064                           uint8_t* response, uint8_t& responseLen) = 0;
00065
00081     virtual bool sendAPDULarge(const uint8_t* apdu, uint8_t apduLen,
00082                                uint8_t* response, uint16_t& responseLen) {
00083         uint8_t smallLen = static_cast<uint8_t>(
00084             (responseLen > static_cast<uint16_t>(UINT8_MAX))
00085                 ? static_cast<uint16_t>(UINT8_MAX)
00086                 : responseLen);
00087         bool result = sendAPDU(apdu, apduLen, response, smallLen);
00088         responseLen = static_cast<uint16_t>(smallLen);
00089         return result;
00090     }
00091
00095     virtual void resetReader() = 0;
00096
00101     virtual bool printFirmwareVersion() = 0;
00102
00103     virtual ~CW_NfcTransport() {}
00104 };
00105
00106 #endif // CW_NFCTRANSPORT_H
```

5.13 CW_Platform.h File Reference

Abstract platform interface for timing primitives.

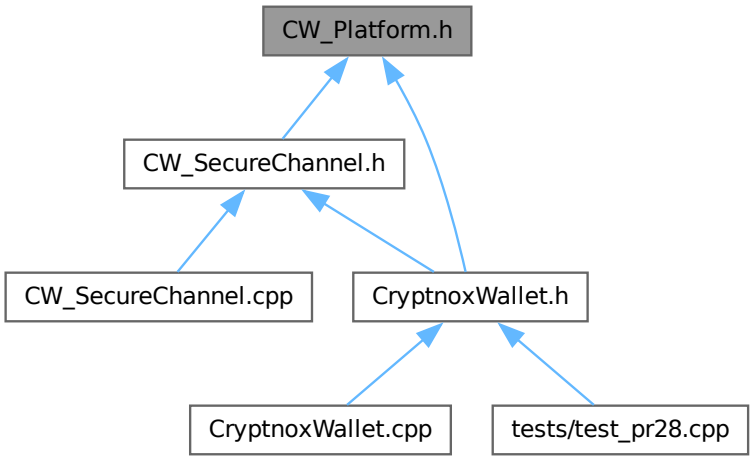
`#include "platform_compat.h"`

Include dependency graph for CW_Platform.h:



© 2026 Cryptnox SA

This graph shows which files directly or indirectly include this file:



Classes

- class CW_Platform

  Abstract interface for platform-specific operations used by the SDK.

5.13.1 Detailed Description

Abstract platform interface for timing primitives.
Declares CW_Platform, the contract that hosts implement so the SDK stays independent of any specific RTOS or bare-metal delay mechanism.
Currently exposes a single operation (CW_Platform::sleep_ms) used by CW_SecureChannel for inter-APDU spacing on slow NFC stacks.
Definition in file CW_Platform.h.

5.14 CW_Platform.h

Go to the documentation of this file.

```
00001 /*
00002  * SPDX-License-Identifier: LGPL-3.0-or-later
00003  * Copyright (c) 2026 Cryptnox SA
00004  */
00005
00016
00017 #ifndef CW_PLATFORM_H
00018 #define CW_PLATFORM_H
00019
00020 /**********************************************************************
00021  * 1. Included files
00022  **********************************************************************/
00023
00024 #include "platform_compat.h"
00025
00026 /**********************************************************************
00027  * 2. Class declaration
00028  **********************************************************************/
00029
00039 class CW_Platform {
00040 public:
00046     virtual void sleep_ms(uint32_t ms) = 0;
00047
```

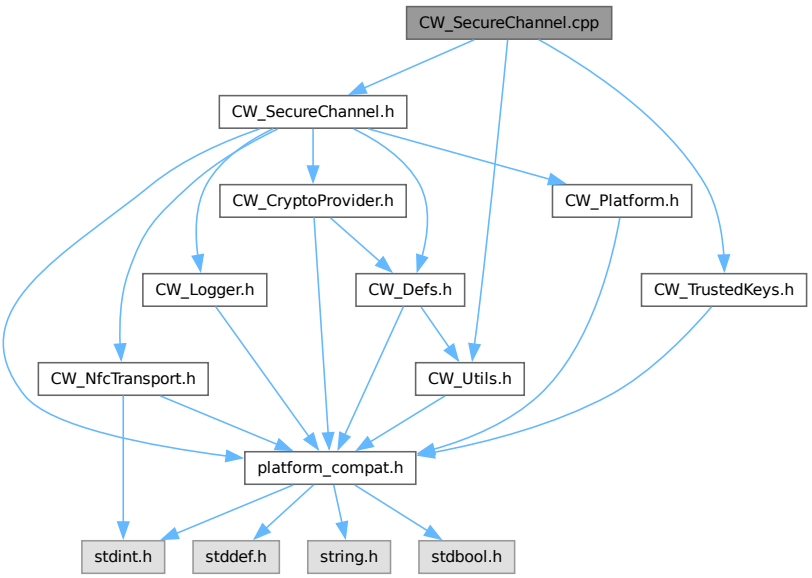
```
00048     virtual ~CW_Platform() {}
00049 };
00050
00051 #endif /* CW_PLATFORM_H */
```

5.15 CW_SecureChannel.cpp File Reference

Implementation of the Cryptnox secure channel protocol.

```
#include "CW_SecureChannel.h"
#include "CW_Utils.h"
#include "CW_TrustedKeys.h"
```

Include dependency graph for CW_SecureChannel.cpp:



Macros

- #define RESPONSE_GETCARDCERTIFICATE_IN_BYTES 148U
- #define RESPONSE_SELECT_IN_BYTES 40U
- #define RESPONSE_GETMANUFACTURERCERT_PAGE_IN_BYTES 420U
- #define RESPONSE_OPENSECURECHANNEL_IN_BYTES 34U
- #define MUTUALAUTH_CHALLENGE_IN_BYTES 32U
- #define RESPONSE_STATUS_WORDS_IN_BYTES 2U
- #define APPLET_MAJOR_V2 2U /∗ first applet with the MAC counter + mutual-auth proof ∗/
- #define OPENSECURECHANNEL_SALT_IN_BYTES (RESPONSE_OPENSECURECHANNEL_IN_BYTES - RESPONSE_STATUS_WORDS_IN_BYTES)
- #define GETCARDCERTIFICATE_IN_BYTES (RESPONSE_GETCARDCERTIFICATE_IN_BYTES - RESPONSE_STATUS_WORDS_IN_BYTES)
- #define RANDOM_BYTES 8U
- #define COMMON_PAIRING_DATA CW_PAIRING_DATA
- #define CLIENT_PRIVATE_KEY_SIZE 32U
- #define CLIENT_PUBLIC_KEY_SIZE 64U
- #define CARDEPHEMERALPUBKEY_SIZE 64U
- #define AES_BLOCK_SIZE 16U
- #define APDU_HEADER_LEN (4U)

© 2026 Cryptnox SA

- #define APDU_LC_LEN (1U)
- #define MAC_APDU_LEN (12U)
- #define INPUT_BUFFER_LIMIT (CW_USER_DATA_PAGE_SIZE)
- #define ENC_BUF_MAX_LEN (INPUT_BUFFER_LIMIT + AES_BLOCK_SIZE)
- #define MAX_MAC_DATA_LEN (APDU_HEADER_LEN + MAC_APDU_LEN + ENC_BUF_MAX_LEN)
- #define SEND_APDU_MAX_LEN (APDU_HEADER_LEN + APDU_LC_LEN + AES_BLOCK_SIZE + ENC_BUF_MAX_LEN)
- #define DER_TAG_SEQUENCE (0x30U) /* SEQUENCE (universal, constructed) */
- #define DER_TAG_BIT_STRING (0x03U) /* BIT STRING */
- #define DER_TAG_CTX0 (0xA0U) /* [0] EXPLICIT — version in v3 TBSCertificate */
- #define DER_LEN_LONG_FLAG (0x80U) /* set = long-form length */
- #define DER_LEN_LONG_1 (0x81U) /* long form, 1 following byte */
- #define DER_LEN_LONG_2 (0x82U) /* long form, 2 following bytes */
- #define DER_EC_UNCOMPRESSED (0x04U) /* uncompressed point prefix */
- #define DER_EC_POINT_BYTES (65U) /* 0x04 || X[32] || Y[32] */
- #define DER_BIT_UNUSED_ZERO (0x00U) /* BIT STRING unused-bits field must be 0 */

Functions

- static bool derReadLength (const uint8_t *buf, uint16_t bufLen, uint16_t &pos, uint16_t &fieldLen)
- static bool derSkipField (const uint8_t *buf, uint16_t bufLen, uint16_t &pos)
- static bool derWalkMfCert (const uint8_t *buf, uint16_t bufLen, uint16_t &tbsMsgStart, uint16_t &tbsMsgLen, const uint8_t *&pubKey65Ptr, const uint8_t *&sigPtr, uint8_t &sigLen)

Variables

- static uint8_t s_apduBuf [SEND_APDU_MAX_LEN] = { 0U }
- static uint8_t s_macBuf [MAX_MAC_DATA_LEN] = { 0U }
- static uint8_t s_dataBuf [ENC_BUF_MAX_LEN] = { 0U }
- static uint8_t s_mfCertBuf [CW_MANUF_CERT_MAX_BYTES] = { 0U }

5.15.1 Detailed Description

Implementation of the Cryptnox secure channel protocol.
Implements the methods declared in CW_SecureChannel.h: APDU framing, certificate chain verification against the trusted CA keys (CW_TrustedKeys.h), ECDH session key derivation, AES-CBC encrypted messaging with rolling IV, and MAC verification on every response.
Module-level static scratch buffers are reused across calls to keep the stack footprint small; secret material is wiped after use.
Definition in file CW_SecureChannel.cpp.

5.15.2 Macro Definition Documentation

5.15.2.1 AES_BLOCK_SIZE

```
#define AES_BLOCK_SIZE 16U
```

Definition at line 45 of file CW_SecureChannel.cpp.
Referenced by CW_SecureChannel::aesCbcDecrypt(), CW_SecureChannel::aesCbcEncrypt(), and CW_SecureChannel::macIvForCounter().

5.15.2.2 APDU_HEADER_LEN

```
#define APDU_HEADER_LEN (4U)
```

Definition at line 46 of file CW_SecureChannel.cpp.

© 2026 Cryptnox SA

5.15.2.3 APDU_LC_LEN

```
#define APDU_LC_LEN (1U)
```
Definition at line 47 of file CW_SecureChannel.cpp.
Referenced by CW_SecureChannel::aesCbcEncrypt().

5.15.2.4 APPLET_MAJOR_V2

```
#define APPLET_MAJOR_V2 2U /* first applet with the MAC counter + mutual-auth proof */
```
Definition at line 35 of file CW_SecureChannel.cpp.
Referenced by CW_SecureChannel::macIvForCounter(), and CW_SecureChannel::mutuallyAuthenticate().

5.15.2.5 CARDEPHEMERALPUBKEY_SIZE

```
#define CARDEPHEMERALPUBKEY_SIZE 64U
```
Definition at line 44 of file CW_SecureChannel.cpp.

5.15.2.6 CLIENT_PRIVATE_KEY_SIZE

```
#define CLIENT_PRIVATE_KEY_SIZE 32U
```
Definition at line 42 of file CW_SecureChannel.cpp.

5.15.2.7 CLIENT_PUBLIC_KEY_SIZE

```
#define CLIENT_PUBLIC_KEY_SIZE 64U
```
Definition at line 43 of file CW_SecureChannel.cpp.
Referenced by CW_SecureChannel::openSecureChannel().

5.15.2.8 COMMON_PAIRING_DATA

```
#define COMMON_PAIRING_DATA CW_PAIRING_DATA
```
Definition at line 41 of file CW_SecureChannel.cpp.
Referenced by CW_SecureChannel::mutuallyAuthenticate().

5.15.2.9 DER_BIT_UNUSED_ZERO

```
#define DER_BIT_UNUSED_ZERO (0x00U) /* BIT STRING unused-bits field must be 0 */
```
Definition at line 84 of file CW_SecureChannel.cpp.
Referenced by derWalkMfCert().

5.15.2.10 DER_EC_POINT_BYTES

```
#define DER_EC_POINT_BYTES (65U) /* 0x04 || X[32] || Y[32] */
```
Definition at line 83 of file CW_SecureChannel.cpp.
Referenced by derWalkMfCert().

5.15.2.11 DER_EC_UNCOMPRESSED

```
#define DER_EC_UNCOMPRESSED (0x04U) /* uncompressed point prefix */
```
Definition at line 82 of file CW_SecureChannel.cpp.
Referenced by derWalkMfCert().

5.15.2.12 DER_LEN_LONG_1

```
#define DER_LEN_LONG_1 (0x81U) /* long form, 1 following byte */
```
Definition at line 78 of file CW_SecureChannel.cpp.
Referenced by derReadLength().

© 2026 Cryptnox SA

5.15.2.13 DER_LEN_LONG_2

```
#define DER_LEN_LONG_2 (0x82U) /* long form, 2 following bytes */
```
Definition at line 79 of file CW_SecureChannel.cpp.
Referenced by derReadLength().

5.15.2.14 DER_LEN_LONG_FLAG

```
#define DER_LEN_LONG_FLAG (0x80U) /* set = long-form length */
```
Definition at line 77 of file CW_SecureChannel.cpp.
Referenced by derReadLength().

5.15.2.15 DER_TAG_BIT_STRING

```
#define DER_TAG_BIT_STRING (0x03U) /* BIT STRING */
```
Definition at line 73 of file CW_SecureChannel.cpp.
Referenced by derWalkMfCert().

5.15.2.16 DER_TAG_CTX0

```
#define DER_TAG_CTX0 (0xA0U) /* [0] EXPLICIT -- version in v3 TBSCertificate */
```
Definition at line 74 of file CW_SecureChannel.cpp.
Referenced by derWalkMfCert().

5.15.2.17 DER_TAG_SEQUENCE

```
#define DER_TAG_SEQUENCE (0x30U) /* SEQUENCE (universal, constructed) */
```
Definition at line 72 of file CW_SecureChannel.cpp.
Referenced by derWalkMfCert().

5.15.2.18 ENC_BUF_MAX_LEN

```
#define ENC_BUF_MAX_LEN (INPUT_BUFFER_LIMIT + AES_BLOCK_SIZE)
```
Definition at line 50 of file CW_SecureChannel.cpp.
Referenced by CW_SecureChannel::mutuallyAuthenticate().

5.15.2.19 GETCARDCERTIFICATE_IN_BYTES

```
#define GETCARDCERTIFICATE_IN_BYTES (RESPONSE_GETCARDCERTIFICATE_IN_BYTES - RESPONSE_STATUS_WORDS_IN_BYTES)
```
Definition at line 38 of file CW_SecureChannel.cpp.
Referenced by CW_SecureChannel::getCardCertificate().

5.15.2.20 INPUT_BUFFER_LIMIT

```
#define INPUT_BUFFER_LIMIT (CW_USER_DATA_PAGE_SIZE)
```
Definition at line 49 of file CW_SecureChannel.cpp.
Referenced by CW_SecureChannel::aesCbcEncrypt().

5.15.2.21 MAC_APDU_LEN

```
#define MAC_APDU_LEN (12U)
```
Definition at line 48 of file CW_SecureChannel.cpp.
Referenced by CW_SecureChannel::aesCbcEncrypt().

5.15.2.22 MAX_MAC_DATA_LEN

```
#define MAX_MAC_DATA_LEN (APDU_HEADER_LEN + MAC_APDU_LEN + ENC_BUF_MAX_LEN)
```
Definition at line 51 of file CW_SecureChannel.cpp.
Referenced by CW_SecureChannel::aesCbcEncrypt().

© 2026 Cryptnox SA

5.15.2.23 MUTUALAUTH_CHALLENGE_IN_BYTES

```
#define MUTUALAUTH_CHALLENGE_IN_BYTES 32U
```

Definition at line 33 of file CW_SecureChannel.cpp.
Referenced by CW_SecureChannel::mutuallyAuthenticate().

5.15.2.24 OPENSECURECHANNEL_SALT_IN_BYTES

```
#define OPENSECURECHANNEL_SALT_IN_BYTES (RESPONSE_OPENSECURECHANNEL_IN_BYTES - RESPONSE_STATUS_WORDS_IN_BYTES)
```

Definition at line 37 of file CW_SecureChannel.cpp.
Referenced by CW_SecureChannel::openSecureChannel().

5.15.2.25 RANDOM_BYTES

```
#define RANDOM_BYTES 8U
```

Definition at line 40 of file CW_SecureChannel.cpp.
Referenced by CW_SecureChannel::getCardCertificate().

5.15.2.26 RESPONSE_GETCARDCERTIFICATE_IN_BYTES

```
#define RESPONSE_GETCARDCERTIFICATE_IN_BYTES 148U
```

Definition at line 27 of file CW_SecureChannel.cpp.
Referenced by CW_SecureChannel::getCardCertificate().

5.15.2.27 RESPONSE_GETMANUFACTURERCERT_PAGE_IN_BYTES

```
#define RESPONSE_GETMANUFACTURERCERT_PAGE_IN_BYTES 420U
```

Definition at line 31 of file CW_SecureChannel.cpp.
Referenced by CW_SecureChannel::getManufacturerCertificate().

5.15.2.28 RESPONSE_OPENSECURECHANNEL_IN_BYTES

```
#define RESPONSE_OPENSECURECHANNEL_IN_BYTES 34U
```

Definition at line 32 of file CW_SecureChannel.cpp.
Referenced by CW_SecureChannel::openSecureChannel().

5.15.2.29 RESPONSE_SELECT_IN_BYTES

```
#define RESPONSE_SELECT_IN_BYTES 40U
```

Definition at line 29 of file CW_SecureChannel.cpp.
Referenced by CW_SecureChannel::selectApdu().

5.15.2.30 RESPONSE_STATUS_WORDS_IN_BYTES

```
#define RESPONSE_STATUS_WORDS_IN_BYTES 2U
```

Definition at line 34 of file CW_SecureChannel.cpp.
Referenced by CW_SecureChannel::getCardCertificate(), CW_SecureChannel::getManufacturerCertificate(), and CW_SecureChannel::selectApdu().

5.15.2.31 SEND_APDU_MAX_LEN

```
#define SEND_APDU_MAX_LEN (APDU_HEADER_LEN + APDU_LC_LEN + AES_BLOCK_SIZE + ENC_BUF_MAX_LEN)
```

Definition at line 52 of file CW_SecureChannel.cpp.
Referenced by CW_SecureChannel::aesCbcEncrypt().

© 2026 Cryptnox SA

5.15.3 Function Documentation

5.15.3.1 derReadLength()

```
bool derReadLength (
            const uint8_t * buf,
            uint16_t bufLen,
            uint16_t & pos,
            uint16_t & fieldLen)  [static]
```

Definition at line 798 of file CW_SecureChannel.cpp.
References DER_LEN_LONG_1, DER_LEN_LONG_2, and DER_LEN_LONG_FLAG.
Referenced by derSkipField(), and derWalkMfCert().

5.15.3.2 derSkipField()

```
bool derSkipField (
            const uint8_t * buf,
            uint16_t bufLen,
            uint16_t & pos)  [static]
```

Definition at line 833 of file CW_SecureChannel.cpp.
References derReadLength().
Referenced by derWalkMfCert().

5.15.3.3 derWalkMfCert()

```
bool derWalkMfCert (
            const uint8_t * buf,
            uint16_t bufLen,
            uint16_t & tbsMsgStart,
            uint16_t & tbsMsgLen,
            const uint8_t *& pubKey65Ptr,
            const uint8_t *& sigPtr,
            uint8_t & sigLen)  [static]
```

Definition at line 857 of file CW_SecureChannel.cpp.
References DER_BIT_UNUSED_ZERO, DER_EC_POINT_BYTES, DER_EC_UNCOMPRESSED, DER_TAG_BIT_STRING, DER_TAG_CTX0, DER_TAG_SEQUENCE, derReadLength(), and derSkipField().
Referenced by CW_SecureChannel::verifyCertificateChain().

5.15.4 Variable Documentation

5.15.4.1 s_apduBuf

```
uint8_t s_apduBuf[SEND_APDU_MAX_LEN] = { 0U }  [static]
```

Definition at line 64 of file CW_SecureChannel.cpp.
Referenced by CW_SecureChannel::aesCbcDecrypt(), and CW_SecureChannel::aesCbcEncrypt().

5.15.4.2 s_dataBuf

```
uint8_t s_dataBuf[ENC_BUF_MAX_LEN] = { 0U }  [static]
```

Definition at line 66 of file CW_SecureChannel.cpp.
Referenced by CW_SecureChannel::aesCbcDecrypt(), and CW_SecureChannel::aesCbcEncrypt().

5.15.4.3 s_macBuf

```
uint8_t s_macBuf[MAX_MAC_DATA_LEN] = { 0U }  [static]
```

Definition at line 65 of file CW_SecureChannel.cpp.
Referenced by CW_SecureChannel::aesCbcDecrypt(), and CW_SecureChannel::aesCbcEncrypt().

© 2026 Cryptnox SA

5.15.4.4 s_mfCertBuf

```
uint8_t s_mfCertBuf[CW_MANUF_CERT_MAX_BYTES] = { 0U }  [static]
```

Definition at line 69 of file CW_SecureChannel.cpp.
Referenced by CW_SecureChannel::preFetchManufacturerCert(), and CW_SecureChannel::verifyCertificateChain().

5.16 CW_SecureChannel.cpp

Go to the documentation of this file.

```
00001 /*
00002  * SPDX-License-Identifier: LGPL-3.0-or-later
00003  * Copyright (c) 2026 Cryptnox SA
00004  */
00005
00018
00019 #include "CW_SecureChannel.h"
00020 #include "CW_Utils.h"
00021 #include "CW_TrustedKeys.h"
00022
00023 /******************************************************************
00024  * Module-level constants
00025  ******************************************************************/
00026
00027 #define RESPONSE_GETCARDCERTIFICATE_IN_BYTES        148U
00028 /* SELECT AID: 1 (type) + 3 (ver) + 32 (status) + 2 (SW) = 38 bytes */
00029 #define RESPONSE_SELECT_IN_BYTES                     40U
00030 /* GET_MANUFACTURER_CERT: full DataOut up to certLen(2)+cert(411)+SW(2)=415 bytes; 420 for margin. */
00031 #define RESPONSE_GETMANUFACTURERCERT_PAGE_IN_BYTES  420U
00032 #define RESPONSE_OPENSECURECHANNEL_IN_BYTES      34U
00033 #define MUTUALAUTH_CHALLENGE_IN_BYTES            32U
00034 #define RESPONSE_STATUS_WORDS_IN_BYTES            2U
00035 #define APPLET_MAJOR_V2                           2U  /* first applet with the MAC counter +
      mutual-auth proof */
00036
00037 #define OPENSECURECHANNEL_SALT_IN_BYTES   (RESPONSE_OPENSECURECHANNEL_IN_BYTES -
      RESPONSE_STATUS_WORDS_IN_BYTES)
00038 #define GETCARDCERTIFICATE_IN_BYTES       (RESPONSE_GETCARDCERTIFICATE_IN_BYTES -
      RESPONSE_STATUS_WORDS_IN_BYTES)
00039
00040 #define RANDOM_BYTES              8U
00041 #define COMMON_PAIRING_DATA       CW_PAIRING_DATA
00042 #define CLIENT_PRIVATE_KEY_SIZE  32U
00043 #define CLIENT_PUBLIC_KEY_SIZE   64U
00044 #define CARDEPHEMERALPUBKEY_SIZE 64U
00045 #define AES_BLOCK_SIZE           16U
00046 #define APDU_HEADER_LEN           (4U)
00047 #define APDU_LC_LEN               (1U)
00048 #define MAC_APDU_LEN             (12U)
00049 #define INPUT_BUFFER_LIMIT        (CW_USER_DATA_PAGE_SIZE)
00050 #define ENC_BUF_MAX_LEN           (INPUT_BUFFER_LIMIT + AES_BLOCK_SIZE)
00051 #define MAX_MAC_DATA_LEN          (APDU_HEADER_LEN + MAC_APDU_LEN + ENC_BUF_MAX_LEN)
00052 #define SEND_APDU_MAX_LEN         (APDU_HEADER_LEN + APDU_LC_LEN + AES_BLOCK_SIZE + ENC_BUF_MAX_LEN)
00053
00054 /* The header's default output bound must track the internal buffer it stands for. */
00055 static_assert(ENC_BUF_MAX_LEN == CW_SECURE_PAYLOAD_MAX,
00056               "CW_SECURE_PAYLOAD_MAX out of step with ENC_BUF_MAX_LEN");
00057
00058 /* Enforce APDU fits within a single PN532 APDU (255 bytes max) */
00059 static_assert(APDU_HEADER_LEN + APDU_LC_LEN + AES_BLOCK_SIZE + ENC_BUF_MAX_LEN <= 255U,
00060               "CW_USER_DATA_PAGE_SIZE too large for PN532 single APDU transport");
00061
00062 /* Shared static crypto scratch buffers -- reuse is safe because decrypt is
00063  * always called from inside encrypt AFTER encrypt's large buffers are done. */
00064 static uint8_t s_apduBuf[SEND_APDU_MAX_LEN] = { 0U };  /* 245 bytes */
00065 static uint8_t s_macBuf [MAX_MAC_DATA_LEN] = { 0U };   /* 240 bytes */
00066 static uint8_t s_dataBuf[ENC_BUF_MAX_LEN] = { 0U };   /* 224 bytes */
00067
00068 /* Manufacturer certificate assembly buffer (used only during verifyCertificateChain). */
00069 static uint8_t s_mfCertBuf[CW_MANUF_CERT_MAX_BYTES] = { 0U };
00070
00071 /* DER TLV tag bytes */
00072 #define DER_TAG_SEQUENCE    (0x30U)  /* SEQUENCE (universal, constructed) */
00073 #define DER_TAG_BIT_STRING  (0x03U)  /* BIT STRING */
00074 #define DER_TAG_CTX0        (0xA0U)  /* [0] EXPLICIT -- version in v3 TBSCertificate */
00075
00076 /* DER length-field encoding */
00077 #define DER_LEN_LONG_FLAG   (0x80U)  /* set = long-form length */
00078 #define DER_LEN_LONG_1      (0x81U)  /* long form, 1 following byte */
00079 #define DER_LEN_LONG_2      (0x82U)  /* long form, 2 following bytes */
00080
00081 /* EC-point encoding */
00082 #define DER_EC_UNCOMPRESSED (0x04U)  /* uncompressed point prefix */
```

```
00083 #define DER_EC_POINT_BYTES  (65U)    /* 0x04 || X[32] || Y[32] */
00084 #define DER_BIT_UNUSED_ZERO (0x00U)  /* BIT STRING unused-bits field must be 0 */
00085
00086 /**************************************************************************
00087  * Constructor
00088  **************************************************************************/
00089
00090 // cppcheck-suppress misra-c2012-12.3 -- C++: member initializer-list commas are not the comma
      operator
00091 CW_SecureChannel::CW_SecureChannel(CW_NfcTransport& driver,
00092                                    CW_Logger& logger,
00093                                    CW_CryptoProvider& crypto,
00094                                    CW_Platform& platform)
00095     : _driver(driver), _logger(logger), _crypto(crypto), _platform(platform),
00096       _cachedMfCertLen(0U), _appletMajor(0U) {
00097 }
00098
00099 /**************************************************************************
00100  * Transport delegation methods
00101  **************************************************************************/
00102
00103 bool CW_SecureChannel::begin() {
00104     return _driver.begin();
00105 }
00106
00107 bool CW_SecureChannel::inListPassiveTarget() {
00108     return _driver.inListPassiveTarget();
00109 }
00110
00111 void CW_SecureChannel::resetReader() {
00112     _driver.resetReader();
00113 }
00114
00115 bool CW_SecureChannel::printFirmwareVersion() {
00116     return _driver.printFirmwareVersion();
00117 }
00118
00119 /**************************************************************************
00120  * Private helpers
00121  **************************************************************************/
00122
00123 bool CW_SecureChannel::checkStatusWord(const uint8_t* response, uint16_t responseLength,
00124                                        uint8_t sw1Expected, uint8_t sw2Expected) {
00125     bool ret = false;
00126
00127     if ((response == NULL) || (responseLength < 2U)) {
00128 #if CW_DEBUG_LOGGING
00129         _logger.println(F("checkStatusWord: response too short."));
00130 #endif
00131     }
00132     else {
00133         uint8_t sw1 = response[responseLength - 2U];
00134         uint8_t sw2 = response[responseLength - 1U];
00135
00136         if ((sw1 == sw1Expected) && (sw2 == sw2Expected)) {
00137             ret = true;
00138         }
00139         else {
00140 #if CW_DEBUG_LOGGING
00141             _logger.print(F("SW: 0x"));
00142             if (sw1 < 16U) { _logger.print(F("0")); }
00143             _logger.print(sw1, HEX);
00144             _logger.print(F(" 0x"));
00145             if (sw2 < 16U) { _logger.print(F("0")); }
00146             _logger.println(sw2, HEX);
00147 #endif
00148         }
00149     }
00150
00151     return ret;
00152 }
00153
00154 /**************************************************************************
00155  * Public methods
00156  **************************************************************************/
00157
00158 bool CW_SecureChannel::selectApdu() {
00159     bool ret = false;
00160
00161     uint8_t selectApduCmd[] = {
00162         0x00, 0xA4, 0x04, 0x00,
00163         0x07,
00164         0xA0, 0x00, 0x00, 0x10, 0x00, 0x01, 0x12
00165     };
00166
00167     uint8_t response[RESPONSE_SELECT_IN_BYTES] = { 0U };
00168     uint8_t responseLength = static_cast<uint8_t>(sizeof(response));
```

© 2026 Cryptnox SA

```
00169     _appletMajor = 0U;
00170
00171     if (_driver.sendAPDU(selectApduCmd, sizeof(selectApduCmd), response, responseLength)) {
00172         if (checkStatusWord(response, responseLength, 0x90U, 0x00U)) {
00173             /* Layout: type(1) | version major, minor, patch (3) | status ... | SW */
00174             if (responseLength >= (4U + RESPONSE_STATUS_WORDS_IN_BYTES)) {
00175                 _appletMajor = response[1];
00176             }
00177             ret = true;
00178 #if CW_MIN_APPLET_MAJOR > 0
00179             if (_appletMajor < (uint8_t)CW_MIN_APPLET_MAJOR) {
00180 #if CW_DEBUG_LOGGING
00181                 _logger.println(F("Applet older than CW_MIN_APPLET_MAJOR, refused."));
00182 #endif
00183                 ret = false;
00184             }
00185 #endif
00186         } else {
00187 #if CW_DEBUG_LOGGING
00188             _logger.println(F("Select APDU failed."));
00189 #endif
00190         }
00191     } else {
00192 #if CW_DEBUG_LOGGING
00193         _logger.println(F("APDU select failed."));
00194 #endif
00195     }
00196
00197     return ret;
00198 }
00199
00200 bool CW_SecureChannel::getCardCertificate(uint8_t* cardCertificate, uint8_t& cardCertificateLength) {
00201     bool ret = false;
00202     uint8_t getCardCertificateResponse[RESPONSE_GETCARDCERTIFICATE_IN_BYTES] = { 0U };
00203     uint8_t getCardCertificateResponseLength =
    static_cast<uint8_t>(sizeof(getCardCertificateResponse));
00204
00205     if (cardCertificate != NULL) {
00206         uint8_t randomBytes[RANDOM_BYTES] = { 0U };
00207         if (!_crypto.random(randomBytes, RANDOM_BYTES)) {
00208 #if CW_DEBUG_LOGGING
00209             _logger.println(F("getCardCertificate: RNG failed."));
00210 #endif
00211             return false;
00212         }
00213
00214         /* Store nonce for replay check in verifyCertificateChain(). */
00215         (void)CW_Utils::safe_memcpy(_lastNonce, RANDOM_BYTES, randomBytes, RANDOM_BYTES);
00216
00217         uint8_t getCardCertificateApdu[] = {
00218             0x80, 0xF8, 0x00, 0x00, 0x08
00219         };
00220
00221         uint8_t fullApdu[sizeof(getCardCertificateApdu) + RANDOM_BYTES] = { 0U };
00222         (void)CW_Utils::safe_memcpy(fullApdu, sizeof(fullApdu), getCardCertificateApdu,
    sizeof(getCardCertificateApdu));
00223         (void)CW_Utils::safe_memcpy(fullApdu + sizeof(getCardCertificateApdu), RANDOM_BYTES,
    randomBytes, RANDOM_BYTES);
00224
00225         if (_driver.sendAPDU(fullApdu, sizeof(fullApdu),
00226                              getCardCertificateResponse, getCardCertificateResponseLength)) {
00227             if (checkStatusWord(getCardCertificateResponse, getCardCertificateResponseLength,
00228                                 0x90U, 0x00U)) {
00229                 cardCertificateLength = static_cast<uint8_t>(
00230                     static_cast<uint8_t>(getCardCertificateResponseLength -
    RESPONSE_STATUS_WORDS_IN_BYTES));
00231                 (void)CW_Utils::safe_memcpy(cardCertificate, GETCARDCERTIFICATE_IN_BYTES,
    getCardCertificateResponse, cardCertificateLength);
00232                 ret = true;
00233             } else {
00234 #if CW_DEBUG_LOGGING
00235                 _logger.println(F("getCardCertificate: bad SW."));
00236 #endif
00237             }
00238         } else {
00239 #if CW_DEBUG_LOGGING
00240             _logger.println(F("getCardCertificate APDU failed."));
00241 #endif
00242         }
00243     }
00244
00245     return ret;
00246 }
00247
00248 bool CW_SecureChannel::extractCardEphemeralKey(const uint8_t* cardCertificate,
00249                                                uint8_t* cardEphemeralPubKey,
00250                                                uint8_t* fullEphemeralPubKey65) {
```

© 2026 Cryptnox SA

```
00251     bool ret = false;
00252
00253     if ((cardCertificate == NULL) || (cardEphemeralPubKey == NULL)) {
00254         ret = false;
00255     }
00256     else {
00257         const uint8_t keyStart = 1U + 8U; /* skip 'C' and nonce */
00258         const uint8_t fullKeyLength = 65U;
00259
00260         /* Reject any key that is not an uncompressed point (0x04 prefix). */
00261         if (cardCertificate[keyStart] != 0x04U) {
00262             ret = false;
00263         }
00264         else {
00265             for (uint8_t i = 0U; i < fullKeyLength; i++) {
00266                 uint8_t b = cardCertificate[keyStart + i];
00267                 if (fullEphemeralPubKey65 != NULL) {
00268                     fullEphemeralPubKey65[i] = b;
00269                 }
00270                 if (i > 0U) {
00271                     cardEphemeralPubKey[i - 1U] = b;
00272                 }
00273             }
00274             ret = true;
00275         }
00276     }
00277
00278     return ret;
00279 }
00280
00281 bool CW_SecureChannel::openSecureChannel(uint8_t* salt,
00282                                          uint8_t* sessionPublicKey,
00283                                          uint8_t* sessionPrivateKey,
00284                                          CW_Curve sessionCurve) {
00285     bool ret = false;
00286
00287     if (!_crypto.makeKey(sessionPublicKey, sessionPrivateKey, sessionCurve)) {
00288 #if CW_DEBUG_LOGGING
00289         _logger.println(F("ECC key generation failed."));
00290 #endif
00291     }
00292     else {
00293         uint8_t opcApduHeader[] = {
00294             0x80, 0x10, 0x00, 0x00, 0x41, 0x04
00295         };
00296
00297         uint8_t fullApdu[sizeof(opcApduHeader) + CLIENT_PUBLIC_KEY_SIZE] = { 0U };
00298         (void)CW_Utils::safe_memcpy(fullApdu, sizeof(fullApdu), opcApduHeader, sizeof(opcApduHeader));
00299         (void)CW_Utils::safe_memcpy(fullApdu + sizeof(opcApduHeader), CLIENT_PUBLIC_KEY_SIZE,
      sessionPublicKey, CLIENT_PUBLIC_KEY_SIZE);
00300
00301         uint8_t response[RESPONSE_OPENSECURECHANNEL_IN_BYTES] = { 0U };
00302         uint8_t responseLength = static_cast<uint8_t>(sizeof(response));
00303
00304         if (_driver.sendAPDU(fullApdu, sizeof(fullApdu), response, responseLength)) {
00305             if (checkStatusWord(response, responseLength, 0x90U, 0x00U)) {
00306                 if (responseLength == static_cast<uint8_t>(RESPONSE_OPENSECURECHANNEL_IN_BYTES)) {
00307                     (void)CW_Utils::safe_memcpy(salt, OPENSECURECHANNEL_SALT_IN_BYTES, response,
      OPENSECURECHANNEL_SALT_IN_BYTES);
00308                     ret = true;
00309                 } else {
00310 #if CW_DEBUG_LOGGING
00311                     _logger.println(F("OpenSecureChannel: unexpected response size."));
00312 #endif
00313                 }
00314             } else {
00315 #if CW_DEBUG_LOGGING
00316                 _logger.println(F("OpenSecureChannel: bad SW."));
00317 #endif
00318             }
00319         } else {
00320 #if CW_DEBUG_LOGGING
00321             _logger.println(F("OpenSecureChannel APDU failed."));
00322 #endif
00323         }
00324     }
00325
00326     return ret;
00327 }
00328
00348 bool CW_SecureChannel::mutuallyAuthenticate(CW_SecureSession& session,
00349                                             const uint8_t* salt,
00350                                             uint8_t* clientPublicKey,
00351                                             const uint8_t* clientPrivateKey,
00352                                             CW_Curve sessionCurve,
00353                                             const uint8_t* cardEphemeralPubKey) {
00354     bool ret = false;
```

© 2026 Cryptnox SA

```
00355     uint8_t sharedSecret[32U] = { 0U };
00356     (void)clientPublicKey;
00357
00358     if (!_crypto.ecdh(cardEphemeralPubKey, clientPrivateKey, sharedSecret, sessionCurve)) {
00359 #if CW_DEBUG_LOGGING
00360         _logger.println(F("ECDH failed."));
00361 #endif
00362     }
00363     else {
00364         uint8_t concat[32U + sizeof(COMMON_PAIRING_DATA) - 1U + 32U] = { 0U };
00365         uint8_t sha512Output[64U] = { 0U };
00366         const size_t pairingKeyLen = sizeof(COMMON_PAIRING_DATA) - 1U;
00367         const size_t concatLen     = 32U + pairingKeyLen + 32U;
00368
00369         (void)CW_Utils::safe_memcpy(concat, sizeof(concat), sharedSecret, 32U);
00370         (void)CW_Utils::safe_memcpy(concat + 32U, sizeof(concat) - 32U, reinterpret_cast<const
      uint8_t*>(COMMON_PAIRING_DATA), pairingKeyLen);
00371         (void)CW_Utils::safe_memcpy(concat + 32U + pairingKeyLen, 32U, salt, 32U);
00372
00373         /* proof = Kenc || challenge, then SHA256 of it; one buffer for both. */
00374         uint8_t proofInput[CW_AESKEY_SIZE + MUTUALAUTH_CHALLENGE_IN_BYTES] = { 0U };
00375         uint8_t expected[32U] = { 0U };
00376         uint8_t response[ENC_BUF_MAX_LEN] = { 0U };  /* aesCbcEncrypt may write up to ENC_BUF_MAX_LEN
      */
00377         uint16_t responseLength = 0U;
00378         uint8_t* challenge = proofInput + CW_AESKEY_SIZE;
00379
00380         if (!_crypto.sha512(concat, concatLen, sha512Output)) {
00381 #if CW_DEBUG_LOGGING
00382             _logger.println(F("MutualAuth: SHA512 failed."));
00383 #endif
00384         } else if (!_crypto.random(challenge, MUTUALAUTH_CHALLENGE_IN_BYTES)) {
00385 #if CW_DEBUG_LOGGING
00386             _logger.println(F("MutualAuth: RNG failed (radio off? TRNG not seeded)."));
00387 #endif
00388         } else {
00389             (void)CW_Utils::safe_memcpy(session.aesKey, CW_AESKEY_SIZE, sha512Output, CW_AESKEY_SIZE);
00390             (void)CW_Utils::safe_memcpy(session.macKey, CW_MACKEY_SIZE, sha512Output + CW_AESKEY_SIZE,
      CW_MACKEY_SIZE);
00391             /* 2.0+: IV = Kenc[:16]. 1.6.x: fixed 0x01 IV, as the Python SDK does. */
00392             if (_appletMajor >= APPLET_MAJOR_V2) {
00393                 (void)CW_Utils::safe_memcpy(session.iv, CW_IV_SIZE, session.aesKey, CW_IV_SIZE);
00394             } else {
00395                 memset(session.iv, 0x01U, CW_IV_SIZE);
00396             }
00397             session.macCounter = 0U;
00398             (void)CW_Utils::safe_memcpy(proofInput, sizeof(proofInput), session.aesKey,
      CW_AESKEY_SIZE);
00399
00400             const uint8_t apdu[] = { 0x80U, 0x11U, 0x00U, 0x00U };
00401             if (aesCbcEncrypt(session, apdu, sizeof(apdu),
00402                               challenge, MUTUALAUTH_CHALLENGE_IN_BYTES,
00403                               response, sizeof(response), &responseLength) &&
00404                 (responseLength == MUTUALAUTH_CHALLENGE_IN_BYTES)) {
00405                 if (_appletMajor < APPLET_MAJOR_V2) {
00406                     ret = true;  /* 1.6.x: response is random, nothing to verify */
00407                 } else if (_crypto.sha256(proofInput, sizeof(proofInput), expected) &&
00408                            CW_Utils::secure_compare(expected, response, sizeof(expected))) {
00409                     ret = true;
00410                 } else {
00411 #if CW_DEBUG_LOGGING
00412                     _logger.println(F("MutualAuth: bad SHA256(Kenc || challenge) proof."));
00413 #endif
00414                 }
00415             } else {
00416 #if CW_DEBUG_LOGGING
00417                 _logger.println(F("MutualAuth APDU failed."));
00418 #endif
00419             }
00420         }
00421
00422         /* Secure cleanup */
00423         CW_Utils::secure_wipe(sharedSecret, sizeof(sharedSecret));
00424         CW_Utils::secure_wipe(sha512Output, sizeof(sha512Output));
00425         CW_Utils::secure_wipe(concat, sizeof(concat));
00426         CW_Utils::secure_wipe(proofInput, sizeof(proofInput));
00427         CW_Utils::secure_wipe(expected, sizeof(expected));
00428         CW_Utils::secure_wipe(response, sizeof(response));
00429
00430         /* Never leave a half-initialised session behind (CRIT-04). */
00431         if (!ret) {
00432             session.clear();
00433         }
00434     }
00435
00436     return ret;
00437 }
```

© 2026 Cryptnox SA

```
00438
00439 /* GAP-01 anti-replay (applet 2.0+). The card MACs (counter block || MAC data)
00440  * with a zero IV, the counter being a 16-byte big-endian block. In CBC that is
00441  * exactly the MAC of the MAC data alone with IV = AES_Kmac(counter block), so we
00442  * fold the counter into the IV instead of prepending it: the scratch buffers and
00443  * the maximum APDU size stay unchanged. 1.6.x: zero IV, legacy MAC. */
00444 bool CW_SecureChannel::macIvForCounter(const CW_SecureSession& session, uint8_t* macIv) {
00445     bool ret = true;
00446     memset(macIv, 0U, AES_BLOCK_SIZE);
00447     if (_appletMajor >= APPLET_MAJOR_V2) {
00448         uint8_t ctrBlock[AES_BLOCK_SIZE] = { 0U };
00449         uint8_t zeroIv[AES_BLOCK_SIZE] = { 0U };
00450         ctrBlock[12] = (uint8_t)(session.macCounter » 24U);
00451         ctrBlock[13] = (uint8_t)(session.macCounter » 16U);
00452         ctrBlock[14] = (uint8_t)(session.macCounter » 8U);
00453         ctrBlock[15] = (uint8_t)(session.macCounter);
00454         ret = (_crypto.aesCbcEncrypt(ctrBlock, AES_BLOCK_SIZE, macIv,
00455                                      session.macKey, sizeof(session.macKey),
00456                                      zeroIv, false) == AES_BLOCK_SIZE);
00457     }
00458     return ret;
00459 }
00460
00483 bool CW_SecureChannel::aesCbcEncrypt(CW_SecureSession& session,
00484                                      const uint8_t apdu[], uint16_t apduLength,
00485                                      const uint8_t data[], uint16_t dataLength) {
00486     return aesCbcEncrypt(session, apdu, apduLength, data, dataLength, NULL, 0U, NULL);
00487 }
00488
00489 bool CW_SecureChannel::aesCbcEncrypt(CW_SecureSession& session,
00490                                      const uint8_t apdu[], uint16_t apduLength,
00491                                      const uint8_t data[], uint16_t dataLength,
00492                                      uint8_t* decryptedOutput, uint16_t decryptedOutputSize,
00493                                      uint16_t* decryptedOutputLength) {
00494     bool ret = false;
00495
00496     /* Reject payloads that would overflow s_dataBuf (MED-01). */
00497     if (dataLength > INPUT_BUFFER_LIMIT) {
00498 #if CW_DEBUG_LOGGING
00499         _logger.println(F("Error: data too large for encryption buffer."));
00500 #endif
00501         return false;
00502     }
00503
00504     /* 1. Encrypt data with Kenc (Bit padding) */
00505     uint16_t encryptedLength = _crypto.aesCbcEncrypt(data, dataLength, s_dataBuf,
00506                                                      session.aesKey, sizeof(session.aesKey),
00507                                                      session.iv, true);
00508
00509     uint16_t lcValue = encryptedLength + (uint16_t)AES_BLOCK_SIZE;
00510     /* lcValue must fit in uint8_t: with INPUT_BUFFER_LIMIT=208, max encryptedLength=224,
00511      * so max lcValue=240. The static_assert above also caps APDU at 255 bytes (MED-03). */
00512     if (lcValue > 0xFFU) {
00513 #if CW_DEBUG_LOGGING
00514         _logger.println(F("Error: lcValue overflow -- payload too large."));
00515 #endif
00516         CW_Utils::secure_wipe(s_dataBuf, sizeof(s_dataBuf));
00517         return false;
00518     }
00519     uint8_t macApdu[MAC_APDU_LEN] = { 0U };
00520     /* MED-03: Single-byte length encoding and no direction byte are intentional --
00521      * this CBC-MAC construction matches the Cryptnox SCCP card firmware spec.
00522      * Changing to AES-CMAC, wider encoding, or adding a direction byte would
00523      * break the card protocol. lcValue overflow is prevented by the
00524      * INPUT_BUFFER_LIMIT precondition above (dataLength <= 208 → lcValue <= 240). */
00525     macApdu[0U] = (uint8_t)lcValue;
00526
00527     uint16_t macDataLength = apduLength + sizeof(macApdu) + encryptedLength;
00528     if (macDataLength > MAX_MAC_DATA_LEN) {
00529 #if CW_DEBUG_LOGGING
00530         _logger.println(F("Error: MAC data length exceeds buffer."));
00531 #endif
00532         CW_Utils::secure_wipe(s_dataBuf, sizeof(s_dataBuf));
00533         return false;
00534     }
00535
00536     /* 2. Build MAC input: APDU header || LC block || ciphertext */
00537     uint16_t offset = 0U;
00538     (void)CW_Utils::safe_memcpy(s_macBuf, sizeof(s_macBuf), apdu, apduLength);
00539     offset += apduLength;
00540     (void)CW_Utils::safe_memcpy(s_macBuf + offset, sizeof(s_macBuf) - static_cast<size_t>(offset),
    macApdu, sizeof(macApdu));
00541     offset += sizeof(macApdu);
00542     (void)CW_Utils::safe_memcpy(s_macBuf + offset, sizeof(s_macBuf) - static_cast<size_t>(offset),
    s_dataBuf, encryptedLength);
00543
00544     /* GAP-01: +1 once per command; the same value authenticates its response.
```

© 2026 Cryptnox SA

```
00545      * The card advances even when it answers a bare error SW, so stay in step.
00546      * (Kmac, counter) must never repeat: a wrap would let a recorded response
00547      * verify again. The card's counter is 16 bytes, so 32 bits here is the bound;
00548      * clearing the session forces a new ECDH, hence a fresh Kmac. */
00549     if (session.macCounter == UINT32_MAX) {
00550 #if CW_DEBUG_LOGGING
00551         _logger.println(F("Error: MAC counter exhausted -- renegotiate the channel."));
00552 #endif
00553         CW_Utils::secure_wipe(s_dataBuf, sizeof(s_dataBuf));
00554         CW_Utils::secure_wipe(s_macBuf,  sizeof(s_macBuf));
00555         session.clear();
00556         return false;
00557     }
00558     session.macCounter++;
00559     uint8_t macIv[AES_BLOCK_SIZE] = { 0U };
00560     if (!macIvForCounter(session, macIv)) {
00561 #if CW_DEBUG_LOGGING
00562         _logger.println(F("Error: MAC IV derivation failed."));
00563 #endif
00564         CW_Utils::secure_wipe(s_dataBuf, sizeof(s_dataBuf));
00565         CW_Utils::secure_wipe(s_macBuf,  sizeof(s_macBuf));
00566         session.clear(); /* counter already advanced: out of step with the card */
00567         return false;
00568     }
00569     uint16_t macEncryptedLength = _crypto.aesCbcEncrypt(s_macBuf, macDataLength, s_apduBuf,
00570                                                         session.macKey, sizeof(session.macKey),
00571                                                         macIv, false);
00572
00573     /* A provider failure returns 0: without this the offset below wraps. */
00574     if (macEncryptedLength < AES_BLOCK_SIZE) {
00575 #if CW_DEBUG_LOGGING
00576         _logger.println(F("Error: MAC computation failed."));
00577 #endif
00578         CW_Utils::secure_wipe(s_dataBuf, sizeof(s_dataBuf));
00579         CW_Utils::secure_wipe(s_macBuf, sizeof(s_macBuf));
00580         session.clear();
00581         return false;
00582     }
00583
00584     uint8_t macValue[AES_BLOCK_SIZE] = { 0U };
00585     uint16_t macOffset = macEncryptedLength - AES_BLOCK_SIZE;
00586     (void)CW_Utils::safe_memcpy(macValue, sizeof(macValue), s_apduBuf + macOffset, AES_BLOCK_SIZE);
00587
00588     /* 3. Build send APDU: header || Lc || MAC || ciphertext */
00589     const uint8_t lc = (uint8_t)lcValue;
00590     uint8_t sendApduLength = (uint8_t)(apduLength + APDU_LC_LEN + sizeof(macValue) + encryptedLength);
00591     if (sendApduLength > SEND_APDU_MAX_LEN) {
00592 #if CW_DEBUG_LOGGING
00593         _logger.println(F("Error: Send APDU length exceeds buffer."));
00594 #endif
00595         CW_Utils::secure_wipe(s_dataBuf, sizeof(s_dataBuf));
00596         CW_Utils::secure_wipe(s_macBuf,  sizeof(s_macBuf));
00597         session.clear();
00598         return false;
00599     }
00600
00601     offset = 0U;
00602     (void)CW_Utils::safe_memcpy(s_apduBuf, sizeof(s_apduBuf), apdu, apduLength);
00603     offset += apduLength;
00604     s_apduBuf[offset] = lc;
00605     offset += APDU_LC_LEN;
00606     (void)CW_Utils::safe_memcpy(s_apduBuf + offset, sizeof(s_apduBuf) - static_cast<size_t>(offset),
      macValue, sizeof(macValue));
00607     offset += sizeof(macValue);
00608     (void)CW_Utils::safe_memcpy(s_apduBuf + offset, sizeof(s_apduBuf) - static_cast<size_t>(offset),
      s_dataBuf, encryptedLength);
00609
00610     /* 4. Send APDU */
00611     uint8_t response[255U] = { 0U };
00612     uint8_t responseLength = static_cast<uint8_t>(sizeof(response));
00613
00614     if (_driver.sendAPDU(s_apduBuf, sendApduLength, response, responseLength)) {
00615         if (checkStatusWord(response, responseLength, 0x90U, 0x00U)) {
00616             /* Update session.iv ONLY after the response MAC is verified (HIGH-01).
00617              * Moving it before aesCbcDecrypt would let an attacker-chosen IV
00618              * desynchronise the rolling-IV state even on MAC failure. */
00619             ret = aesCbcDecrypt(session, response, static_cast<size_t>(responseLength), macValue,
00620                                 decryptedOutput, decryptedOutputSize, decryptedOutputLength);
00621             if (ret) {
00622                 (void)CW_Utils::safe_memcpy(session.iv, CW_IV_SIZE, response, CW_IV_SIZE);
00623             } else {
00624                 session.clear();
00625             }
00626         } else {
00627 #if CW_DEBUG_LOGGING
00628             /* Card-level error: surface the SW so the caller can diagnose
00629              * common cases (e.g. 0x63Cn = wrong PIN with n retries left). */
```

© 2026 Cryptnox SA

```
00630             if (responseLength >= 2U) {
00631                 _logger.print(F("Secured APDU: bad SW 0x"));
00632                 if (response[responseLength - 2U] < 0x10U) { _logger.print(F("0")); }
00633                 _logger.print(response[responseLength - 2U], HEX);
00634                 if (response[responseLength - 1U] < 0x10U) { _logger.print(F("0")); }
00635                 _logger.println(response[responseLength - 1U], HEX);
00636             }
00637 #endif
00638             /* A bare SW carries no MAC, so it may be forged in the NFC path: the
00639              * card may have run the command and moved its IV to the response MAC
00640              * we never saw. Keeping our old IV would make the card decrypt the
00641              * next command's first block wrongly, and silently (the MAC does not
00642              * cover the IV). Clear, so the caller renegotiates. */
00643             session.clear();
00644         }
00645     } else {
00646 #if CW_DEBUG_LOGGING
00647         _logger.println(F("Secured APDU failed."));
00648 #endif
00649         /* Transport failure: we cannot know whether the card advanced its counter. */
00650         session.clear();
00651     }
00652
00653     /* Wipe plaintext scratch buffers so they do not persist in .bss (MED-04). */
00654     CW_Utils::secure_wipe(s_dataBuf, sizeof(s_dataBuf));
00655     CW_Utils::secure_wipe(s_macBuf,  sizeof(s_macBuf));
00656
00657     return ret;
00658 }
00659
00660 bool CW_SecureChannel::aesCbcDecrypt(const CW_SecureSession& session,
00661                                      uint8_t* response, size_t response_len,
00662                                      uint8_t* mac_value,
00663                                      uint8_t* decryptedOutput, uint16_t decryptedOutputSize,
00664                                      uint16_t* decryptedOutputLength) {
00665     /* Precondition: response must hold at least MAC(16) + 1 ciphertext byte + SW(2) (HIGH-02).
00666      * Without this check, response_len < 18 causes size_t underflow in the subtractions below. */
00667     if ((response == NULL) || (response_len < (size_t)(AES_BLOCK_SIZE + 2U + 1U))) {
00668         return false;
00669     }
00670
00671     /* Response layout: MAC(16) || cipherText(N) || SW1(1) || SW2(1) */
00672     uint8_t rep_mac[AES_BLOCK_SIZE] = { 0U };
00673     (void)CW_Utils::safe_memcpy(rep_mac, sizeof(rep_mac), response, AES_BLOCK_SIZE);
00674     uint8_t* rep_data  = response + AES_BLOCK_SIZE;
00675     size_t totalDataLen = response_len - 2U;
00676     size_t cipherLen    = totalDataLen - AES_BLOCK_SIZE;
00677
00678     if (mac_value == NULL) {
00679         return false;
00680     }
00681
00682     /* Verify MAC: AES-CBC-MAC over [length_header(16)] || [all_ciphertext] */
00683     size_t macInputLen = AES_BLOCK_SIZE + cipherLen;
00684     if (macInputLen > sizeof(s_macBuf)) {
00685 #if CW_DEBUG_LOGGING
00686         _logger.println(F("Error: Response too large for MAC verification."));
00687 #endif
00688         return false;
00689     }
00690
00691     memset(s_macBuf, 0U, AES_BLOCK_SIZE);
00692     s_macBuf[0] = (uint8_t)totalDataLen;
00693     (void)CW_Utils::safe_memcpy(s_macBuf + AES_BLOCK_SIZE, sizeof(s_macBuf) - AES_BLOCK_SIZE,
    rep_data, cipherLen);
00694
00695     uint8_t mac_iv[AES_BLOCK_SIZE] = { 0U };
00696     if (!macIvForCounter(session, mac_iv)) {  /* same counter as the command (GAP-01) */
00697 #if CW_DEBUG_LOGGING
00698         _logger.println(F("Error: MAC IV derivation failed."));
00699 #endif
00700         CW_Utils::secure_wipe(s_macBuf, sizeof(s_macBuf));
00701         return false;
00702     }
00703     uint16_t macEncryptedLength = _crypto.aesCbcEncrypt(s_macBuf, (uint16_t)macInputLen, s_apduBuf,
00704                                                         session.macKey, sizeof(session.macKey),
00705                                                         mac_iv, false);
00706
00707     /* A provider failure returns 0: without this the offset below wraps. */
00708     if (macEncryptedLength < AES_BLOCK_SIZE) {
00709 #if CW_DEBUG_LOGGING
00710         _logger.println(F("Error: MAC computation failed."));
00711 #endif
00712         CW_Utils::secure_wipe(s_macBuf, sizeof(s_macBuf));
00713         return false;
00714     }
00715
```

© 2026 Cryptnox SA

```
00716     uint8_t recomputedMacValue[AES_BLOCK_SIZE] = { 0U };
00717     uint16_t macOffset = macEncryptedLength - AES_BLOCK_SIZE;
00718     (void)CW_Utils::safe_memcpy(recomputedMacValue, sizeof(recomputedMacValue), s_apduBuf + macOffset,
      AES_BLOCK_SIZE);
00719
00720     if (!CW_Utils::secure_compare(rep_mac, recomputedMacValue, AES_BLOCK_SIZE)) {
00721 #if CW_DEBUG_LOGGING
00722         _logger.println(F("MAC mismatch."));
00723 #endif
00724         CW_Utils::secure_wipe(s_macBuf, sizeof(s_macBuf));
00725         return false;
00726     }
00727
00728     /* Decrypt ciphertext using mac_value as IV (Bit padding removal) */
00729     uint16_t decryptedDataLength = _crypto.aesCbcDecrypt(rep_data, (uint16_t)cipherLen, s_dataBuf,
00730                                                          session.aesKey, sizeof(session.aesKey),
00731                                                          mac_value, true);
00732
00733     bool ret = false;
00734
00735     if (decryptedDataLength < 2U) {
00736 #if CW_DEBUG_LOGGING
00737         _logger.println(F("Error: Decoded data too short."));
00738 #endif
00739     }
00740     else if (decryptedDataLength > sizeof(s_dataBuf)) {
00741 #if CW_DEBUG_LOGGING
00742         _logger.println(F("Error: Decoded data length exceeds buffer."));
00743 #endif
00744     }
00745     else {
00746         uint8_t innerSW1 = 0U;
00747         uint8_t innerSW2 = 0U;
00748         uint16_t payloadLength = 0U;
00749
00750         innerSW1 = s_dataBuf[decryptedDataLength - 2U];
00751         innerSW2 = s_dataBuf[decryptedDataLength - 1U];
00752         payloadLength = (uint16_t)(decryptedDataLength - 2U);
00753
00754         if ((innerSW1 != 0x90U) || (innerSW2 != 0x00U)) {
00755 #if CW_DEBUG_LOGGING
00756             _logger.print(F("Card error SW: 0x"));
00757             if (innerSW1 < 0x10U) { _logger.print(F("0")); }
00758             _logger.print(innerSW1, HEX);
00759             _logger.print(F(" 0x"));
00760             if (innerSW2 < 0x10U) { _logger.print(F("0")); }
00761             _logger.println(innerSW2, HEX);
00762 #endif
00763         }
00764         else {
00765             ret = true;
00766         }
00767
00768         if ((decryptedOutput != NULL) && (decryptedOutputLength != NULL)) {
00769             /* The caller sized its buffer blind -- only the card knows how long the
00770              * answer is. Refuse rather than overrun: sizeof(s_dataBuf) here compared
00771              * the source against itself, so the check could never fail. */
00772             if (payloadLength > decryptedOutputSize) {
00773 #if CW_DEBUG_LOGGING
00774                 _logger.println(F("Error: payload larger than the caller's buffer."));
00775 #endif
00776                 ret = false;
00777             } else {
00778                 (void)CW_Utils::safe_memcpy(decryptedOutput, decryptedOutputSize, s_dataBuf,
      payloadLength);
00779                 *decryptedOutputLength = payloadLength;
00780             }
00781         }
00782     }
00783
00784     /* Wipe plaintext scratch buffers (MED-04). */
00785     CW_Utils::secure_wipe(s_dataBuf, sizeof(s_dataBuf));
00786     CW_Utils::secure_wipe(s_macBuf,  sizeof(s_macBuf));
00787
00788     return ret;
00789 }
00790
00791 /*****************************************************************
00792  * Certificate verification -- static helpers
00793  *****************************************************************/
00794
00795
00796 /* Read the DER length at buf[*pos]; advance *pos past the length bytes.
00797  * Supports short form and long form with 1 or 2 extra bytes only. */
00798 static bool derReadLength(const uint8_t* buf, uint16_t bufLen,
00799                           uint16_t& pos, uint16_t& fieldLen) {
00800     bool    ok   = false;
```

© 2026 Cryptnox SA

```
00801     fieldLen     = 0U;
00802
00803     if ((buf != NULL) && (pos < bufLen)) {
00804         uint8_t b = buf[pos];
00805         pos += 1U;
00806
00807         if ((b & DER_LEN_LONG_FLAG) == 0U) {
00808             fieldLen = static_cast<uint16_t>(b);
00809             ok = true;
00810         } else if (b == DER_LEN_LONG_1) {
00811             if (pos < bufLen) {
00812                 fieldLen = static_cast<uint16_t>(buf[pos]);
00813                 pos += 1U;
00814                 ok = true;
00815             }
00816         } else if (b == DER_LEN_LONG_2) {
00817             if ((pos + 1U) < bufLen) {
00818                 fieldLen  = static_cast<uint16_t>(
00819                                 static_cast<uint16_t>(buf[pos]) « 8U);
00820                 fieldLen |= static_cast<uint16_t>(buf[pos + 1U]);
00821                 pos += 2U;
00822                 ok = true;
00823             }
00824         } else {
00825             ok = false; /* indefinite form or > 2 extra bytes -- unsupported */
00826         }
00827     }
00828
00829     return ok;
00830 }
00831
00832 /* Skip one complete DER TLV (tag byte + length bytes + value) at buf[*pos]. */
00833 static bool derSkipField(const uint8_t* buf, uint16_t bufLen, uint16_t& pos) {
00834     bool ok = false;
00835
00836     if ((buf != NULL) && (pos < bufLen)) {
00837         uint16_t contentLen = 0U;
00838         pos += 1U; /* skip tag byte */
00839         if (derReadLength(buf, bufLen, pos, contentLen)) {
00840             if ((pos + contentLen) <= bufLen) {
00841                 pos += contentLen;
00842                 ok   = true;
00843             }
00844         }
00845     }
00846
00847     return ok;
00848 }
00849
00850 /* Walk a DER X.509 Certificate to extract -- without any byte-pattern search:
00851  *   tbsMsgStart  offset of TBSCertificate SEQUENCE tag inside buf
00852  *   tbsMsgLen    total byte count of TBSCertificate (tag + length + content)
00853  *   pubKey65Ptr  pointer to 65-byte uncompressed EC point (0x04 || X || Y)
00854  *   sigPtr       pointer to the DER ECDSA signature bytes
00855  *   sigLen       byte count of the DER ECDSA signature
00856  * Returns false on any format or bounds error. */
00857 static bool derWalkMfCert(const uint8_t* buf, uint16_t bufLen,
00858                           uint16_t& tbsMsgStart, uint16_t& tbsMsgLen,
00859                           const uint8_t*& pubKey65Ptr,
00860                           const uint8_t*& sigPtr, uint8_t& sigLen) {
00861     bool     ok             = true;
00862     uint16_t pos            = 0U;
00863     uint16_t certContentLen = 0U;
00864     uint16_t tbsContentLen  = 0U;
00865     uint16_t tbsEnd         = 0U;
00866     uint16_t spkiContentLen = 0U;
00867     uint16_t bsLen          = 0U;
00868
00869     tbsMsgStart = 0U;
00870     tbsMsgLen   = 0U;
00871     pubKey65Ptr = NULL;
00872     sigPtr      = NULL;
00873     sigLen      = 0U;
00874
00875     if ((buf == NULL) || (bufLen == 0U)) {
00876         ok = false;
00877     }
00878
00879     /* - outer Certificate SEQUENCE - */
00880     if (ok) {
00881         if (buf[pos] != DER_TAG_SEQUENCE) {
00882             ok = false;
00883         }
00884     }
00885     if (ok) {
00886         pos += 1U;
00887         if (!derReadLength(buf, bufLen, pos, certContentLen)) {
```

© 2026 Cryptnox SA

```
00888             ok = false;
00889         }
00890     }
00891     if (ok) {
00892         if ((pos + certContentLen) > bufLen) {
00893             ok = false;
00894         }
00895     }
00896
00897     /* - TBSCertificate SEQUENCE (first child) - */
00898     if (ok) {
00899         if (buf[pos] != DER_TAG_SEQUENCE) {
00900             ok = false;
00901         }
00902     }
00903     if (ok) {
00904         tbsMsgStart = pos;
00905         pos += 1U;
00906         if (!derReadLength(buf, bufLen, pos, tbsContentLen)) {
00907             ok = false;
00908         }
00909     }
00910     if (ok) {
00911         tbsMsgLen = (pos - tbsMsgStart) + tbsContentLen;
00912         tbsEnd    = pos + tbsContentLen;
00913         if (tbsEnd > bufLen) {
00914             ok = false;
00915         }
00916     }
00917
00918     /* - Walk TBSCertificate fields in order - */
00919
00920     /* [0] EXPLICIT version -- present in X.509 v3 */
00921     if (ok && (pos < tbsEnd)) {
00922         if (buf[pos] == DER_TAG_CTX0) {
00923             if (!derSkipField(buf, tbsEnd, pos)) {
00924                 ok = false;
00925             }
00926         }
00927     }
00928
00929     /* serialNumber INTEGER */
00930     if (ok) {
00931         if (!derSkipField(buf, tbsEnd, pos)) {
00932             ok = false;
00933         }
00934     }
00935
00936     /* signature AlgorithmIdentifier SEQUENCE */
00937     if (ok) {
00938         if (!derSkipField(buf, tbsEnd, pos)) {
00939             ok = false;
00940         }
00941     }
00942
00943     /* issuer Name SEQUENCE */
00944     if (ok) {
00945         if (!derSkipField(buf, tbsEnd, pos)) {
00946             ok = false;
00947         }
00948     }
00949
00950     /* validity SEQUENCE */
00951     if (ok) {
00952         if (!derSkipField(buf, tbsEnd, pos)) {
00953             ok = false;
00954         }
00955     }
00956
00957     /* subject Name SEQUENCE */
00958     if (ok) {
00959         if (!derSkipField(buf, tbsEnd, pos)) {
00960             ok = false;
00961         }
00962     }
00963
00964     /* - SubjectPublicKeyInfo SEQUENCE - */
00965     if (ok) {
00966         if (pos >= tbsEnd) {
00967             ok = false;
00968         }
00969     }
00970     if (ok) {
00971         if (buf[pos] != DER_TAG_SEQUENCE) {
00972             ok = false;
00973         }
00974     }
```

© 2026 Cryptnox SA

```
00975     if (ok) {
00976         pos += 1U;
00977         if (!derReadLength(buf, bufLen, pos, spkiContentLen)) {
00978             ok = false;
00979         }
00980     }
00981     if (ok) {
00982         if ((pos + spkiContentLen) > bufLen) {
00983             ok = false;
00984         }
00985     }
00986
00987     /* Skip AlgorithmIdentifier SEQUENCE inside SubjectPublicKeyInfo */
00988     if (ok) {
00989         if (!derSkipField(buf, bufLen, pos)) {
00990             ok = false;
00991         }
00992     }
00993
00994     /* subjectPublicKey BIT STRING */
00995     if (ok) {
00996         if (pos >= bufLen) {
00997             ok = false;
00998         }
00999     }
01000     if (ok) {
01001         if (buf[pos] != DER_TAG_BIT_STRING) {
01002             ok = false;
01003         }
01004     }
01005     if (ok) {
01006         pos += 1U;
01007         if (!derReadLength(buf, bufLen, pos, bsLen)) {
01008             ok = false;
01009         }
01010     }
01011     if (ok) {
01012         if ((pos + bsLen) > bufLen) {
01013             ok = false;
01014         }
01015     }
01016     if (ok) {
01017         if (bsLen < (1U + static_cast<uint16_t>(DER_EC_POINT_BYTES))) {
01018             ok = false; /* too short: unused-bits byte + 65-byte EC point */
01019         }
01020     }
01021     if (ok) {
01022         if (buf[pos] != DER_BIT_UNUSED_ZERO) {
01023             ok = false; /* unused bits must be 0 */
01024         } else if (buf[pos + 1U] != DER_EC_UNCOMPRESSED) {
01025             ok = false; /* must be an uncompressed EC point */
01026         } else {
01027             pubKey65Ptr = buf + pos + 1U; /* points to: 0x04 || X[32] || Y[32] */
01028         }
01029     }
01030
01031     /* Jump to end of TBSCertificate, then skip signatureAlgorithm SEQUENCE */
01032     if (ok) {
01033         pos = tbsEnd;
01034         if (!derSkipField(buf, bufLen, pos)) {
01035             ok = false;
01036         }
01037     }
01038
01039     /* - signatureValue BIT STRING (third child of Certificate) - */
01040     if (ok) {
01041         if (pos >= bufLen) {
01042             ok = false;
01043         }
01044     }
01045     if (ok) {
01046         if (buf[pos] != DER_TAG_BIT_STRING) {
01047             ok = false;
01048         }
01049     }
01050     if (ok) {
01051         pos += 1U;
01052         if (!derReadLength(buf, bufLen, pos, bsLen)) {
01053             ok = false;
01054         }
01055     }
01056     if (ok) {
01057         if ((pos + bsLen) > bufLen) {
01058             ok = false;
01059         }
01060     }
01061     if (ok) {
```

© 2026 Cryptnox SA

```
01062         if (bsLen < 2U) {
01063             ok = false; /* need unused-bits byte + at least 1 signature byte */
01064         }
01065     }
01066     if (ok) {
01067         if (buf[pos] != DER_BIT_UNUSED_ZERO) {
01068             ok = false; /* unused bits must be 0 */
01069         }
01070     }
01071     if (ok) {
01072         uint16_t rawSigLen = bsLen - 1U;
01073         if (rawSigLen > 255U) {
01074             ok = false;
01075         } else {
01076             sigPtr = buf + pos + 1U; /* DER ECDSA signature, after unused-bits byte */
01077             sigLen = static_cast<uint8_t>(rawSigLen);
01078         }
01079     }
01080
01081     return ok;
01082 }
01083
01084 bool CW_SecureChannel::parseDerSigToRaw(const uint8_t* der, uint8_t derLen,
01085                                         uint8_t* raw64) {
01086     bool ret = false;
01087
01088     if ((der != NULL) && (raw64 != NULL) && (derLen >= 6U) && (der[0] == 0x30U)) {
01089         /* Validate outer SEQUENCE length against actual buffer (HIGH-04). */
01090         if ((uint8_t)(der[1] + 2U) > derLen) {
01091             return false;
01092         }
01093
01094         uint8_t pos = 2U;  /* skip SEQUENCE tag + length */
01095
01096         if (der[pos] == 0x02U) {
01097             pos++;
01098             uint8_t rLen = der[pos];
01099             pos++;
01100             /* Reject malformed r -- DER r is at most 33 bytes (32 + 1 zero pad) (HIGH-04). */
01101             if (rLen > 33U) {
01102                 return false;
01103             }
01104             if ((pos + rLen) <= derLen) {
01105                 const uint8_t* rPtr = der + pos;
01106                 pos += rLen;
01107
01108                 if ((pos < derLen) && (der[pos] == 0x02U)) {
01109                     pos++;
01110                     uint8_t sLen = der[pos];
01111                     pos++;
01112                     /* Reject malformed s (HIGH-04). */
01113                     if (sLen > 33U) {
01114                         return false;
01115                     }
01116                     if ((pos + sLen) <= derLen) {
01117                         /* M-06: verify sum of both INTEGER fields matches the outer SEQUENCE length.
01118                          * Trailing garbage after s would indicate malformed/crafted DER. */
01119                         if ((pos + sLen) != (uint8_t)(2U + der[1])) {
01120                             return false;
01121                         }
01122                         const uint8_t* sPtr = der + pos;
01123
01124                         memset(raw64, 0U, 64U);
01125
01126                         /* r: strip optional leading 0x00 padding byte */
01127                         if ((rLen == 33U) && (rPtr[0] == 0x00U)) { rPtr++; rLen = 32U; }
01128                         if (rLen <= 32U) {
01129                             (void)CW_Utils::safe_memcpy(raw64 + (32U - rLen), static_cast<size_t>(32U
      + rLen), rPtr, rLen);
01130                         }
01131
01132                         /* s: strip optional leading 0x00 padding byte */
01133                         if ((sLen == 33U) && (sPtr[0] == 0x00U)) { sPtr++; sLen = 32U; }
01134                         if (sLen <= 32U) {
01135                             (void)CW_Utils::safe_memcpy(raw64 + 32U + (32U - sLen),
      static_cast<size_t>(sLen), sPtr, sLen);
01136                         }
01137
01138                         ret = true;
01139                     }
01140                 }
01141             }
01142         }
01143     }
01144
01145     return ret;
01146 }
```

© 2026 Cryptnox SA

```
01147
01148 bool CW_SecureChannel::verifyEcdsaSha256(const uint8_t* pubKey64,
01149                                           const uint8_t* message, uint16_t msgLen,
01150                                           const uint8_t* derSig, uint8_t derSigLen) {
01151     bool result = false;
01152     uint8_t hash[32U] = { 0U };
01153     uint8_t rawSig[64U] = { 0U };
01154
01155     bool hashOk = _crypto.sha256(message, msgLen, hash);
01156
01157     if (hashOk && parseDerSigToRaw(derSig, derSigLen, rawSig)) {
01158         result = _crypto.ecdsaVerify(pubKey64, hash, sizeof(hash), rawSig, CW_CURVE_SECP256R1);
01159     }
01160
01161     return result;
01162 }
01163
01164 bool CW_SecureChannel::getManufacturerCertificate(uint8_t* cert, uint16_t& certLen) {
01165     bool ret = false;
01166     certLen = 0U;
01167
01168     if (cert != NULL) {
01169         const uint8_t APDU_P2_IDX = 3U;  /* P2 field offset in ISO 7816-4 APDU header */
01170         uint8_t apdu[5U] = { 0x80U, 0xF7U, 0x00U, 0x00U, 0x00U };
01171         uint8_t response[RESPONSE_GETMANUFACTURERCERT_PAGE_IN_BYTES] = { 0U };
01172         uint16_t responseLen = static_cast<uint16_t>(sizeof(response));
01173
01174         if (!_driver.sendAPDULarge(apdu, static_cast<uint8_t>(sizeof(apdu)), response,
01175                                    responseLen)) {
01176 #if CW_DEBUG_LOGGING
01177             _logger.println(F("getManufacturerCertificate APDU failed."));
01178 #endif
01179             return false;
01180         }
01181         if (responseLen > static_cast<uint16_t>(sizeof(response))) {
01182 #if CW_DEBUG_LOGGING
01183             _logger.println(F("getManufacturerCertificate: driver reported overflow."));
01184 #endif
01185             return false;
01186         }
01187
01188         if (checkStatusWord(response, responseLen, 0x90U, 0x00U)) {
01189
01190             uint16_t dataBytes = static_cast<uint16_t>(
01191                 responseLen - static_cast<uint16_t>(RESPONSE_STATUS_WORDS_IN_BYTES));
01192
01193             if (dataBytes >= 2U) {
01194                 uint16_t totalCertLen = (static_cast<uint16_t>(response[0]) << 8U)
01195                                       | static_cast<uint16_t>(response[1]);
01196
01197                 if (totalCertLen <= CW_MANUF_CERT_MAX_BYTES) {
01198                     uint16_t certInPage = static_cast<uint16_t>(dataBytes - 2U);
01199                     if (certInPage > totalCertLen) {
01200                         certInPage = totalCertLen;
01201                     }
01202                     (void)CW_Utils::safe_memcpy(cert, CW_MANUF_CERT_MAX_BYTES, response + 2U,
01203                                                 static_cast<size_t>(certInPage));
01204                     certLen = certInPage;
01205
01206                     uint8_t pageIdx = 1U;
01207                     while ((certLen < totalCertLen) && (pageIdx < 8U)) {
01208                         apdu[APDU_P2_IDX] = pageIdx;
01209                         responseLen = static_cast<uint16_t>(sizeof(response));
01210
01211                         if (!_driver.sendAPDULarge(apdu, static_cast<uint8_t>(sizeof(apdu)),
01212                                                    response, responseLen)) {
01213                             break;
01214                         }
01215                         if (responseLen > static_cast<uint16_t>(sizeof(response))) {
01216 #if CW_DEBUG_LOGGING
01217                             _logger.println(F("getManufacturerCertificate: driver reported
      overflow."));
01218 #endif
01219                             return false;
01220                         }
01221                         if (!checkStatusWord(response, responseLen, 0x90U, 0x00U)) {
01222                             break;
01223                         }
01224
01225                         uint16_t pageData = static_cast<uint16_t>(
01226                             responseLen - static_cast<uint16_t>(RESPONSE_STATUS_WORDS_IN_BYTES));
01227                         uint16_t remaining = static_cast<uint16_t>(totalCertLen - certLen);
01228                         if (pageData > remaining) {
01229                             pageData = remaining;
01230                         }
01231
01232                         if ((certLen + pageData) > static_cast<uint16_t>(CW_MANUF_CERT_MAX_BYTES)) {
```

```
01233                             break;
01234                         }
01235                         (void)CW_Utils::safe_memcpy(cert + certLen,
01236                                                     CW_MANUF_CERT_MAX_BYTES -
    static_cast<size_t>(certLen),
01237                                                     response, static_cast<size_t>(pageData));
01238                         certLen = static_cast<uint16_t>(certLen + pageData);
01239                         pageIdx++;
01240                     }
01241
01242                     ret = (certLen == totalCertLen);
01243                     if (!ret) {
01244 #if CW_DEBUG_LOGGING
01245                         _logger.println(F("getManufacturerCertificate: incomplete."));
01246 #endif
01247                     }
01248                 } else {
01249 #if CW_DEBUG_LOGGING
01250                     _logger.println(F("getManufacturerCertificate: cert too large."));
01251 #endif
01252                 }
01253             }
01254         }
01255     }
01256
01257     return ret;
01258 }
01259
01260 bool CW_SecureChannel::preFetchManufacturerCert() {
01261     _cachedMfCertLen = 0U;
01262     bool result = getManufacturerCertificate(s_mfCertBuf, _cachedMfCertLen);
01263     if (!result) {
01264         _cachedMfCertLen = 0U;
01265     }
01266     return result;
01267 }
01268
01293 uint8_t CW_SecureChannel::verifyCertificateChain(const uint8_t* cardCert,
01294                                                  uint8_t cardCertLen) {
01295     uint8_t result = CW_CERT_OK;
01296
01297     if ((cardCert == NULL) || (cardCertLen < 80U) || (cardCert[0] != 0x43U)) {
01298 #if CW_DEBUG_LOGGING
01299         _logger.println(F("verifyCert: invalid card cert."));
01300 #endif
01301         result = CW_CERT_FORMAT_ERROR;
01302     }
01303
01304     /* Consume the pre-fetched cert (filled by preFetchManufacturerCert() before
01305      * getCardCertificate() was called).  Fall back to fetching now if none cached --
01306      * this will fail on cards whose state machine has already advanced past INS=F7. */
01307     uint16_t mfCertLen = _cachedMfCertLen;
01308     _cachedMfCertLen = 0U;
01309
01310     if (result == CW_CERT_OK) {
01311         if (mfCertLen == 0U) {
01312             if (!getManufacturerCertificate(s_mfCertBuf, mfCertLen) || (mfCertLen < 20U)) {
01313 #if CW_DEBUG_LOGGING
01314                 _logger.println(F("verifyCert: failed to get mfr cert."));
01315 #endif
01316                 result = CW_CERT_FORMAT_ERROR;
01317             }
01318         } else if (mfCertLen < 20U) {
01319 #if CW_DEBUG_LOGGING
01320             _logger.println(F("verifyCert: pre-fetched mfr cert too short."));
01321 #endif
01322             result = CW_CERT_FORMAT_ERROR;
01323         } else {
01324             /* Pre-fetched cert in s_mfCertBuf is valid -- no APDU needed. */
01325         }
01326     }
01327
01328     /* MED-05: replace byte-pattern search with a structural DER walker that
01329      * enforces field order.  This prevents attacker-planted OID sequences in
01330      * unsigned extensions from being picked up as the device public key. */
01331     uint16_t       tbsMsgStart = 0U;
01332     uint16_t       tbsMsgLen   = 0U;
01333     const uint8_t* pubKey65Ptr = NULL;
01334     const uint8_t* mfSigPtr    = NULL;
01335     uint8_t        mfSigLen    = 0U;
01336
01337     if (result == CW_CERT_OK) {
01338         if (!derWalkMfCert(s_mfCertBuf, mfCertLen,
01339                            tbsMsgStart, tbsMsgLen,
01340                            pubKey65Ptr,
01341                            mfSigPtr, mfSigLen)) {
01342 #if CW_DEBUG_LOGGING
```

© 2026 Cryptnox SA

```
01343             _logger.println(F("verifyCert: DER walk of mfr cert failed."));
01344 #endif
01345             result = CW_CERT_KEY_NOT_FOUND;
01346         }
01347     }
01348
01349     const uint8_t* devicePubKey64 = NULL;
01350     if (result == CW_CERT_OK) {
01351         devicePubKey64 = pubKey65Ptr + 1U; /* skip the 0x04 uncompressed prefix */
01352
01353         const uint8_t* mfMsg    = s_mfCertBuf + tbsMsgStart;
01354         uint16_t       mfMsgLen = tbsMsgLen;
01355
01356         bool mfVerified = false;
01357         for (uint8_t i = 0U; i < CW_TRUSTED_CA_COUNT; i++) {
01358             if (verifyEcdsaSha256(CW_TRUSTED_CA_KEYS[i],
01359                                   mfMsg, mfMsgLen,
01360                                   mfSigPtr, mfSigLen)) {
01361                 mfVerified = true;
01362                 break;
01363             }
01364         }
01365         if (!mfVerified) {
01366 #if CW_DEBUG_LOGGING
01367             _logger.println(F("verifyCert: mfr cert sig INVALID -- card NOT genuine."));
01368 #endif
01369             result = CW_CERT_MANUF_SIG_INVALID;
01370         }
01371 #if CW_DEBUG_LOGGING
01372         else {
01373             _logger.println(F("Manufacturer cert signature OK."));
01374         }
01375 #endif
01376     }
01377
01378     const uint8_t CARD_CERT_MSG_LEN = 74U;
01379     if (result == CW_CERT_OK) {
01380         if (cardCertLen <= CARD_CERT_MSG_LEN) {
01381 #if CW_DEBUG_LOGGING
01382             _logger.println(F("verifyCert: card cert too short for sig."));
01383 #endif
01384             result = CW_CERT_FORMAT_ERROR;
01385         }
01386         else {
01387             const uint8_t* cardSig    = cardCert + CARD_CERT_MSG_LEN;
01388             uint8_t        cardSigLen = cardCertLen - CARD_CERT_MSG_LEN;
01389
01390             if (!verifyEcdsaSha256(devicePubKey64,
01391                                    cardCert, CARD_CERT_MSG_LEN,
01392                                    cardSig, cardSigLen)) {
01393 #if CW_DEBUG_LOGGING
01394                 _logger.println(F("verifyCert: card cert sig INVALID."));
01395 #endif
01396                 result = CW_CERT_CARD_SIG_INVALID;
01397             }
01398 #if CW_DEBUG_LOGGING
01399             else {
01400                 _logger.println(F("Card cert signature OK."));
01401             }
01402 #endif
01403         }
01404     }
01405
01406     if (result == CW_CERT_OK) {
01407         if ((cardCertLen <= (1U + CW_CERT_NONCE_SIZE)) ||
01408             (!CW_Utils::secure_compare(cardCert + 1U, _lastNonce, CW_CERT_NONCE_SIZE))) { /* M-03:
      constant-time nonce compare */
01409 #if CW_DEBUG_LOGGING
01410             _logger.println(F("verifyCert: nonce mismatch -- possible replay."));
01411 #endif
01412             result = CW_CERT_NONCE_MISMATCH;
01413         }
01414     }
01415
01416 #if CW_DEBUG_LOGGING
01417     if (result == CW_CERT_OK) {
01418         _logger.println(F("Certificate chain OK. Card is genuine."));
01419     }
01420 #endif
01421
01422     /* Wipe manufacturer cert buffer -- it held the card device public key (M-01). */
01423     CW_Utils::secure_wipe(s_mfCertBuf, sizeof(s_mfCertBuf));
01424
01425     return result;
01426 }
```

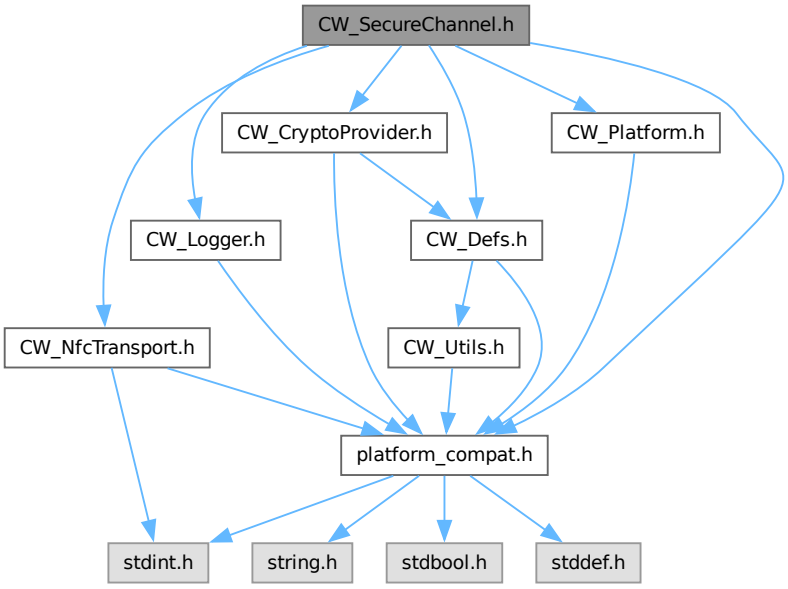
© 2026 Cryptnox SA

5.17 CW_SecureChannel.h File Reference

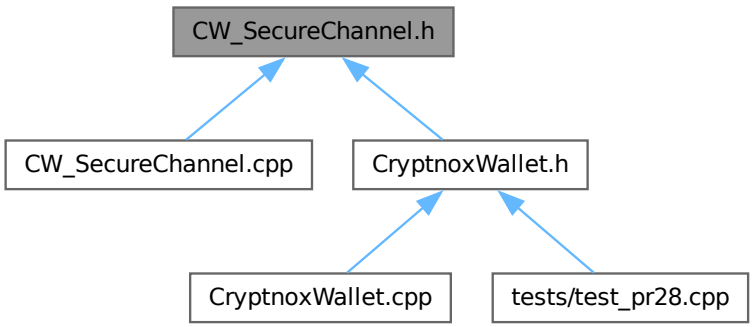
Cryptnox secure channel protocol over NFC.

```
#include "platform_compat.h"
#include "CW_NfcTransport.h"
#include "CW_Logger.h"
#include "CW_CryptoProvider.h"
#include "CW_Platform.h"
#include "CW_Defs.h"
```

Include dependency graph for CW_SecureChannel.h:



This graph shows which files directly or indirectly include this file:



Classes

- class CW_SecureChannel

© 2026 Cryptnox SA

Implements the Cryptnox secure channel protocol over NFC.

Macros

- #define CW_PAIRING_DATA "Cryptnox Basic CommonPairingData"
- #define CW_PAIRING_DATA_BYTES (sizeof(CW_PAIRING_DATA) - 1U)

5.17.1 Detailed Description

Cryptnox secure channel protocol over NFC.
Declares CW_SecureChannel, responsible for every low-level APDU exchange required to establish and use an authenticated, encrypted session with a Cryptnox Hardware Wallet:

- SELECT AID
- Manufacturer + card certificate retrieval and chain verification
- Ephemeral key extraction and ECDH session key derivation
- MUTUALLY AUTHENTICATE
- AES-CBC + MAC secure messaging (encrypt / decrypt / SW check)

The class is composed inside CryptnoxWallet and is not normally used directly by application code.
Definition in file CW_SecureChannel.h.

5.17.2 Macro Definition Documentation

5.17.2.1 CW_PAIRING_DATA

```
#define CW_PAIRING_DATA "Cryptnox Basic CommonPairingData"
```

Definition at line 30 of file CW_SecureChannel.h.

5.17.2.2 CW_PAIRING_DATA_BYTES

```
#define CW_PAIRING_DATA_BYTES (sizeof(CW_PAIRING_DATA) - 1U)
```

Definition at line 31 of file CW_SecureChannel.h.

5.18 CW_SecureChannel.h

Go to the documentation of this file.

```
00001 /*
00002  * SPDX-License-Identifier: LGPL-3.0-or-later
00003  * Copyright (c) 2026 Cryptnox SA
00004  */
00005
00022
00023 #ifndef CW_SECURECHANNEL_H
00024 #define CW_SECURECHANNEL_H
00025
00026 /**********************************************************************
00027  * 1. Public constants
00028  **********************************************************************/
00029
00030 #define CW_PAIRING_DATA       "Cryptnox Basic CommonPairingData"
00031 #define CW_PAIRING_DATA_BYTES (sizeof(CW_PAIRING_DATA) - 1U)
00032
00033 /**********************************************************************
00034  * 2. Included files
00035  **********************************************************************/
00036
00037 #include "platform_compat.h"
00038 #include "CW_NfcTransport.h"
00039 #include "CW_Logger.h"
00040 #include "CW_CryptoProvider.h"
00041 #include "CW_Platform.h"
00042 #include "CW_Defs.h"          /* for CW_SecureSession, CW_Curve, and constants */
00043
```

```
00044 /**************************************************************************
00045  * 2. Class declaration
00046  *************************************************************************/
00047
00064 class CW_SecureChannel {
00065 public:
00074     CW_SecureChannel(CW_NfcTransport& driver, CW_Logger& logger,
00075                      CW_CryptoProvider& crypto, CW_Platform& platform);
00076
00077     CW_SecureChannel(const CW_SecureChannel&) = delete;
00078     CW_SecureChannel& operator=(const CW_SecureChannel&) = delete;
00079
00084     bool begin();
00085
00090     bool inListPassiveTarget();
00091
00095     void resetReader();
00096
00101     bool printFirmwareVersion();
00102
00107     uint8_t appletMajor() const { return _appletMajor; }
00108
00116     bool selectApdu();
00117
00129     bool getCardCertificate(uint8_t* cardCertificate, uint8_t& cardCertificateLength);
00130
00139     static bool extractCardEphemeralKey(const uint8_t* cardCertificate,
00140                                         uint8_t* cardEphemeralPubKey,
00141                                         uint8_t* fullEphemeralPubKey65 = NULL);
00142
00160     bool openSecureChannel(uint8_t* salt,
00161                            uint8_t* clientPublicKey,
00162                            uint8_t* clientPrivateKey,
00163                            CW_Curve sessionCurve);
00164
00194     bool mutuallyAuthenticate(CW_SecureSession& session,
00195                               const uint8_t* salt,
00196                               uint8_t* clientPublicKey,
00197                               const uint8_t* clientPrivateKey,
00198                               CW_Curve sessionCurve,
00199                               const uint8_t* cardEphemeralPubKey);
00200
00208     bool getManufacturerCertificate(uint8_t* cert, uint16_t& certLen);
00209
00220     bool preFetchManufacturerCert();
00221
00249     uint8_t verifyCertificateChain(const uint8_t* cardCert, uint8_t cardCertLen);
00250
00289     bool aesCbcEncrypt(CW_SecureSession& session,
00290                        const uint8_t apdu[], uint16_t apduLength,
00291                        const uint8_t data[], uint16_t dataLength,
00292                        uint8_t* decryptedOutput, uint16_t decryptedOutputSize,
00293                        uint16_t* decryptedOutputLength);
00294
00301     bool aesCbcEncrypt(CW_SecureSession& session,
00302                        const uint8_t apdu[], uint16_t apduLength,
00303                        const uint8_t data[], uint16_t dataLength);
00304
00325     bool aesCbcDecrypt(const CW_SecureSession& session,
00326                        uint8_t* response, size_t responseLen,
00327                        uint8_t* macValue,
00328                        uint8_t* decryptedOutput, uint16_t decryptedOutputSize,
00329                        uint16_t* decryptedOutputLength);
00330
00340     bool checkStatusWord(const uint8_t* response, uint16_t responseLength,
00341                          uint8_t sw1Expected, uint8_t sw2Expected);
00342
00343 private:
00344     CW_NfcTransport&   _driver;
00345     CW_Logger&         _logger;
00346     CW_CryptoProvider& _crypto;
00347     CW_Platform&       _platform;
00348
00350     uint8_t _lastNonce[CW_CERT_NONCE_SIZE] {};
00351
00353     uint16_t _cachedMfCertLen {};
00354
00356     uint8_t _appletMajor {};
00357
00359     bool macIvForCounter(const CW_SecureSession& session, uint8_t* macIv);
00360
00361     static bool parseDerSigToRaw(const uint8_t* der, uint8_t derLen,
00362                                  uint8_t* raw64);
00363
00364     bool verifyEcdsaSha256(const uint8_t* pubKey64,
00365                            const uint8_t* message, uint16_t msgLen,
00366                            const uint8_t* derSig, uint8_t derSigLen);
```

© 2026 Cryptnox SA

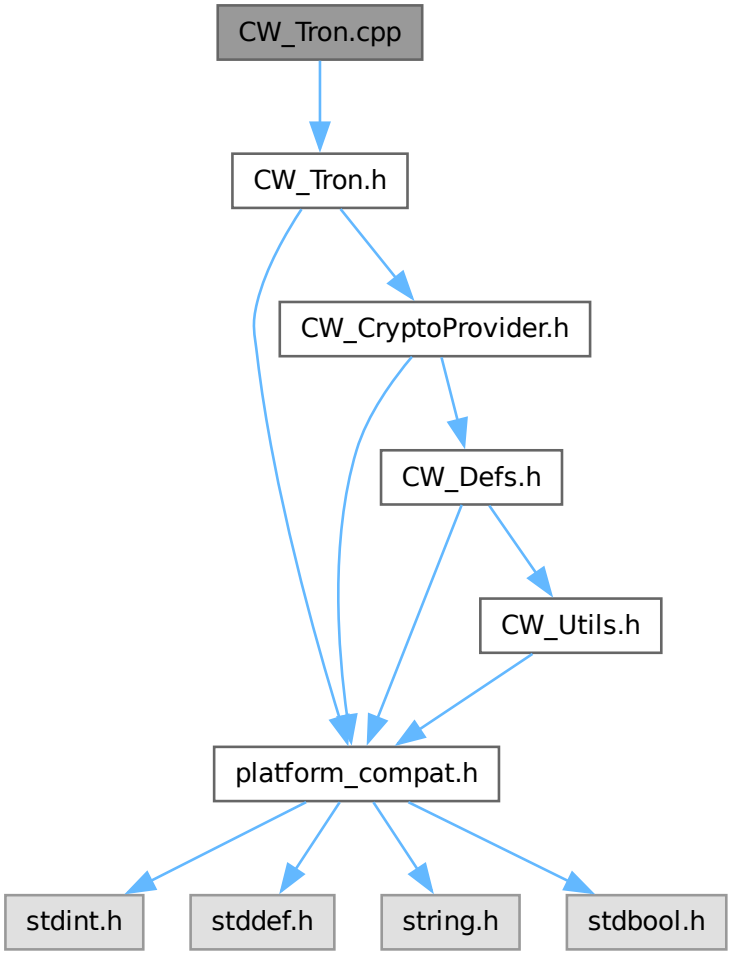
```
00367
00368 #ifdef CW_FUZZ_BUILD
00369     friend struct DerFuzzTarget;
00370 #endif
00371 };
00372
00373 #endif // CW_SECURECHANNEL_H
```

5.19 CW_Tron.cpp File Reference

Implementation of TRON address derivation and Base58Check coding.

`#include "CW_Tron.h"`

Include dependency graph for CW_Tron.cpp:



Macros

- #define BASE58_BASE (58U)
- #define TRON_CHECKSUM_BYTES (4U)
- #define TRON_PAYLOAD_BYTES (CW_TRON_ADDRESS_BYTES + TRON_CHECKSUM_BYTES) /∗ 25 ∗/
- #define KECCAK_DIGEST_BYTES (32U)

© 2026 Cryptnox SA

- #define KECCAK_RATE_BYTES (136U) /* 1600 - 2*256 bits, in bytes */
- #define KECCAK_STATE_LANES (25U)
- #define KECCAK_ROUNDS (24U)
- #define ETH_ADDRESS_BYTES (20U)
- #define PUBLIC_KEY_BYTES (64U)
- #define ROTL64(x, n)

Functions

- static void keccak_f (uint64_t *a)

  The Keccak-f[1600] permutation, in place.
- static void keccak_absorb (uint64_t *a, const uint8_t *data, size_t len)

  Absorb `len` bytes of `data` into the sponge lane-wise.
- static bool base58_encode_payload (const uint8_t *payload, char *out)

  Base58-encode a 25-byte payload into exactly 34 characters + NUL.
- static bool base58_decode_payload (const char *address, uint8_t *payload)

  Base58-decode a 34-character address into a right-aligned 25-byte payload.
- static bool tron_checksum (const uint8_t *addressBytes, CW_CryptoProvider &crypto, uint8_t *checksum4)

  Compute the 4-byte Base58Check checksum: sha256(sha256(payload))[0..4].

Variables

- const uint8_t CW_TRON_DERIVE_PATH [CW_TRON_PATH_LENGTH]

  BIP44 derivation path for TRON: `m/44'/195'/0'/0/0`.
- static const char BASE58_ALPHABET [ ]
- static const uint64_t KECCAK_RC [KECCAK_ROUNDS]
- static const uint8_t KECCAK_ROT [KECCAK_STATE_LANES]

5.19.1 Detailed Description

Implementation of TRON address derivation and Base58Check coding.
Contains a compact Keccak-f[1600] permutation (file-local) because TRON addresses are Ethereum-style `keccak256` digests and no CW_CryptoProvider method provides Keccak. Base58 is done with plain long division / multiplication over the fixed 25-byte payload — no bignum library.
Self-check (no crypto provider needed, Keccak and Base58 only):

```
g++ -DCW_TRON_SELFTEST CW_Tron.cpp CW_Utils.cpp -o cw_tron_selftest
./cw_tron_selftest
```

Definition in file CW_Tron.cpp.

5.19.2 Macro Definition Documentation

5.19.2.1 BASE58_BASE

```
#define BASE58_BASE (58U)
```

Definition at line 43 of file CW_Tron.cpp.
Referenced by base58_decode_payload(), and base58_encode_payload().

5.19.2.2 ETH_ADDRESS_BYTES

```
#define ETH_ADDRESS_BYTES (20U)
```

Definition at line 50 of file CW_Tron.cpp.
Referenced by CW_Tron::addressBytesFromPublicKey().

© 2026 Cryptnox SA

5.19.2.3 KECCAK_DIGEST_BYTES

```
#define KECCAK_DIGEST_BYTES (32U)
```
Definition at line 46 of file CW_Tron.cpp.
Referenced by CW_Tron::addressBytesFromPublicKey(), CW_Tron::keccak256(), and tron_checksum().

5.19.2.4 KECCAK_RATE_BYTES

```
#define KECCAK_RATE_BYTES (136U) /* 1600 - 2*256 bits, in bytes */
```
Definition at line 47 of file CW_Tron.cpp.
Referenced by keccak_absorb().

5.19.2.5 KECCAK_ROUNDS

```
#define KECCAK_ROUNDS (24U)
```
Definition at line 49 of file CW_Tron.cpp.
Referenced by keccak_f().

5.19.2.6 KECCAK_STATE_LANES

```
#define KECCAK_STATE_LANES (25U)
```
Definition at line 48 of file CW_Tron.cpp.
Referenced by CW_Tron::keccak256(), and keccak_f().

5.19.2.7 PUBLIC_KEY_BYTES

```
#define PUBLIC_KEY_BYTES (64U)
```
Definition at line 51 of file CW_Tron.cpp.
Referenced by CW_Tron::addressBytesFromPublicKey().

5.19.2.8 ROTL64

```
#define ROTL64(
                x,
                n)
```
Value:
```
(((x) « (n)) | ((x) » (64U - (n))))
```
Definition at line 57 of file CW_Tron.cpp.
Referenced by keccak_f().

5.19.2.9 TRON_CHECKSUM_BYTES

```
#define TRON_CHECKSUM_BYTES (4U)
```
Definition at line 44 of file CW_Tron.cpp.
Referenced by CW_Tron::decodeAddress(), and tron_checksum().

5.19.2.10 TRON_PAYLOAD_BYTES

```
#define TRON_PAYLOAD_BYTES (CW_TRON_ADDRESS_BYTES + TRON_CHECKSUM_BYTES) /* 25 */
```
Definition at line 45 of file CW_Tron.cpp.
Referenced by base58_decode_payload(), base58_encode_payload(), CW_Tron::decodeAddress(), and CW_Tron::encodeAddress().

5.19.3 Function Documentation

5.19.3.1 base58_decode_payload()

```
bool base58_decode_payload (
            const char * address,
```

© 2026 Cryptnox SA

```
            uint8_t * payload)  [static]
```

Base58-decode a 34-character address into a right-aligned 25-byte payload.

Parameters

in	*address*	34-character NUL-terminated Base58 string.
out	*payload*	25-byte output buffer.

Returns

true on success, false on an invalid character or overflow.

Definition at line 235 of file CW_Tron.cpp.
References BASE58_ALPHABET, BASE58_BASE, CW_TRON_ADDRESS_STR_LEN, and TRON_PAYLOAD_BYTES.
Referenced by CW_Tron::decodeAddress().

5.19.3.2 base58_encode_payload()

```
bool base58_encode_payload (
            const uint8_t * payload,
            char * out)  [static]
```

Base58-encode a 25-byte payload into exactly 34 characters + NUL.

Parameters

in	*payload*	25 bytes, first byte non-zero.
out	*out*	Buffer of at least CW_TRON_ADDRESS_STR_SIZE bytes.

Returns

true on success, false if the encoding did not come out 34 chars.

Definition at line 194 of file CW_Tron.cpp.
References BASE58_ALPHABET, BASE58_BASE, CW_TRON_ADDRESS_STR_LEN, and TRON_PAYLOAD_BYTES.
Referenced by CW_Tron::encodeAddress().

5.19.3.3 keccak_absorb()

```
void keccak_absorb (
            uint64_t * a,
            const uint8_t * data,
            size_t len)  [static]
```

Absorb `len` bytes of `data` into the sponge lane-wise.
Byte-indexed XOR into the lanes, so the result is identical on big- and little-endian hosts.
Definition at line 135 of file CW_Tron.cpp.
References keccak_f(), and KECCAK_RATE_BYTES.
Referenced by CW_Tron::keccak256().

5.19.3.4 keccak_f()

```
void keccak_f (
            uint64_t * a)  [static]
```

The Keccak-f[1600] permutation, in place.

© 2026 Cryptnox SA

Parameters

a	25-lane state.

Definition at line 84 of file CW_Tron.cpp.
References KECCAK_RC, KECCAK_ROT, KECCAK_ROUNDS, KECCAK_STATE_LANES, and ROTL64.
Referenced by keccak_absorb().

5.19.3.5 tron_checksum()

```
bool tron_checksum (
            const uint8_t * addressBytes,
            CW_CryptoProvider & crypto,
            uint8_t * checksum4)  [static]
```

Compute the 4-byte Base58Check checksum: sha256(sha256(payload))[0..4].
Definition at line 266 of file CW_Tron.cpp.
References CW_TRON_ADDRESS_BYTES, KECCAK_DIGEST_BYTES, CW_CryptoProvider::sha256(), and TRON_CHECKSUM_BYTES.
Referenced by CW_Tron::decodeAddress(), and CW_Tron::encodeAddress().

5.19.4 Variable Documentation

5.19.4.1 BASE58_ALPHABET

```
const char BASE58_ALPHABET[]  [static]
```

Initial value:

```
=
    "123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnopqrstuvwxyz"
```

Bitcoin / TRON Base58 alphabet.
Definition at line 40 of file CW_Tron.cpp.
Referenced by base58_decode_payload(), and base58_encode_payload().

5.19.4.2 KECCAK_RC

```
const uint64_t KECCAK_RC[KECCAK_ROUNDS]  [static]
```

Initial value:

```
= {
    0x0000000000000001ULL, 0x0000000000008082ULL, 0x800000000000808AULL,
    0x8000000080008000ULL, 0x000000000000808BULL, 0x0000000080000001ULL,
    0x8000000080008081ULL, 0x8000000000008009ULL, 0x000000000000008AULL,
    0x0000000000000088ULL, 0x0000000080008009ULL, 0x000000008000000AULL,
    0x000000008000808BULL, 0x800000000000008BULL, 0x8000000000008089ULL,
    0x8000000000008003ULL, 0x8000000000008002ULL, 0x8000000000000080ULL,
    0x000000000000800AULL, 0x800000008000000AULL, 0x8000000080008081ULL,
    0x8000000000008080ULL, 0x0000000080000001ULL, 0x8000000080008008ULL
}
```

Iota round constants.
Definition at line 60 of file CW_Tron.cpp.
Referenced by keccak_f().

5.19.4.3 KECCAK_ROT

```
const uint8_t KECCAK_ROT[KECCAK_STATE_LANES]  [static]
```

Initial value:

```
= {
     0U,  1U, 62U, 28U, 27U,
    36U, 44U,  6U, 55U, 20U,
     3U, 10U, 43U, 25U, 39U,
    41U, 45U, 15U, 21U,  8U,
    18U,  2U, 61U, 56U, 14U
}
```

Rho rotation offsets, indexed as `x+5y`.
Definition at line 72 of file CW_Tron.cpp.

© 2026 Cryptnox SA

Referenced by keccak_f().

5.20 CW_Tron.cpp

Go to the documentation of this file.

```
00001 /*
00002  * SPDX-License-Identifier: LGPL-3.0-or-later
00003  * Copyright (c) 2026 Cryptnox SA
00004  */
00005
00022
00023 #include "CW_Tron.h"
00024
00025 /**********************************************************************
00026  * 1. Constants
00027  **********************************************************************/
00028
00029 /* m/44'/195'/0'/0/0 -- five big-endian uint32 levels, hardened levels have
00030  * the high bit set. 195 = 0xC3 is the SLIP-44 coin type for TRON. */
00031 const uint8_t CW_TRON_DERIVE_PATH[CW_TRON_PATH_LENGTH] = {
00032     0x80U, 0x00U, 0x00U, 0x2CU,   /* 44'  */
00033     0x80U, 0x00U, 0x00U, 0xC3U,   /* 195' */
00034     0x80U, 0x00U, 0x00U, 0x00U,   /* 0'   */
00035     0x00U, 0x00U, 0x00U, 0x00U,   /* 0    */
00036     0x00U, 0x00U, 0x00U, 0x00U    /* 0    */
00037 };
00038
00040 static const char BASE58_ALPHABET[] =
00041     "123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnopqrstuvwxyz";
00042
00043 #define BASE58_BASE          (58U)
00044 #define TRON_CHECKSUM_BYTES  (4U)
00045 #define TRON_PAYLOAD_BYTES   (CW_TRON_ADDRESS_BYTES + TRON_CHECKSUM_BYTES) /* 25 */
00046 #define KECCAK_DIGEST_BYTES  (32U)
00047 #define KECCAK_RATE_BYTES    (136U)  /* 1600 - 2*256 bits, in bytes */
00048 #define KECCAK_STATE_LANES   (25U)
00049 #define KECCAK_ROUNDS        (24U)
00050 #define ETH_ADDRESS_BYTES    (20U)
00051 #define PUBLIC_KEY_BYTES     (64U)
00052
00053 /**********************************************************************
00054  * 2. Keccak-256 (file-local)
00055  **********************************************************************/
00056
00057 #define ROTL64(x, n) (((x) << (n)) | ((x) >> (64U - (n))))
00058
00060 static const uint64_t KECCAK_RC[KECCAK_ROUNDS] = {
00061     0x0000000000000001ULL, 0x0000000000008082ULL, 0x800000000000808AULL,
00062     0x8000000080008000ULL, 0x000000000000808BULL, 0x0000000080000001ULL,
00063     0x8000000080008081ULL, 0x8000000000008009ULL, 0x000000000000008AULL,
00064     0x0000000000000088ULL, 0x0000000080008009ULL, 0x000000008000000AULL,
00065     0x000000008000808BULL, 0x800000000000008BULL, 0x8000000000008089ULL,
00066     0x8000000000008003ULL, 0x8000000000008002ULL, 0x8000000000000080ULL,
00067     0x000000000000800AULL, 0x800000008000000AULL, 0x8000000080008081ULL,
00068     0x8000000000008080ULL, 0x0000000080000001ULL, 0x8000000080008008ULL
00069 };
00070
00072 static const uint8_t KECCAK_ROT[KECCAK_STATE_LANES] = {
00073      0U,  1U, 62U, 28U, 27U,
00074     36U, 44U,  6U, 55U, 20U,
00075      3U, 10U, 43U, 25U, 39U,
00076     41U, 45U, 15U, 21U,  8U,
00077     18U,  2U, 61U, 56U, 14U
00078 };
00079
00084 static void keccak_f(uint64_t* a) {
00085     /* Outside the round loop: every round fully rewrites all three before reading them. */
00086     uint64_t c[5] = { 0U };
00087     uint64_t d[5] = { 0U };
00088     uint64_t b[KECCAK_STATE_LANES] = { 0U };
00089
00090     for (unsigned round = 0U; round < KECCAK_ROUNDS; round++) {
00091
00092         /* Theta */
00093         for (unsigned x = 0U; x < 5U; x++) {
00094             c[x] = a[x] ^ a[x + 5U] ^ a[x + 10U] ^ a[x + 15U] ^ a[x + 20U];
00095         }
00096         for (unsigned x = 0U; x < 5U; x++) {
00097             d[x] = c[(x + 4U) % 5U] ^ ROTL64(c[(x + 1U) % 5U], 1U);
00098         }
00099         for (unsigned y = 0U; y < 5U; y++) {
00100             for (unsigned x = 0U; x < 5U; x++) {
00101                 a[x + (5U * y)] ^= d[x];
```

```
00102             }
00103         }
00104
00105         /* Rho + Pi */
00106         for (unsigned y = 0U; y < 5U; y++) {
00107             for (unsigned x = 0U; x < 5U; x++) {
00108                 const unsigned src = x + (5U * y);
00109                 const unsigned dst = y + (5U * (((2U * x) + (3U * y)) % 5U));
00110                 b[dst] = (KECCAK_ROT[src] == 0U) ? a[src]
00111                                                  : ROTL64(a[src], KECCAK_ROT[src]);
00112             }
00113         }
00114
00115         /* Chi */
00116         for (unsigned y = 0U; y < 5U; y++) {
00117             for (unsigned x = 0U; x < 5U; x++) {
00118                 a[x + (5U * y)] = b[x + (5U * y)] ^
00119                                   ((~b[((x + 1U) % 5U) + (5U * y)]) &
00120                                    b[((x + 2U) % 5U) + (5U * y)]);
00121             }
00122         }
00123
00124         /* Iota */
00125         a[0] ^= KECCAK_RC[round];
00126     }
00127 }
00128
00135 static void keccak_absorb(uint64_t* a, const uint8_t* data, size_t len) {
00136     size_t offset = 0U;
00137
00138     while (len >= KECCAK_RATE_BYTES) {
00139         for (size_t i = 0U; i < KECCAK_RATE_BYTES; i++) {
00140             a[i / 8U] ^= ((uint64_t)data[offset + i]) « (8U * (i % 8U));
00141         }
00142         keccak_f(a);
00143         offset += KECCAK_RATE_BYTES;
00144         len    -= KECCAK_RATE_BYTES;
00145     }
00146
00147     /* Final (possibly empty) block plus Keccak padding: 0x01 ... 0x80. */
00148     for (size_t i = 0U; i < len; i++) {
00149         a[i / 8U] ^= ((uint64_t)data[offset + i]) « (8U * (i % 8U));
00150     }
00151     a[len / 8U] ^= ((uint64_t)0x01U) « (8U * (len % 8U));
00152     a[(KECCAK_RATE_BYTES - 1U) / 8U] ^=
00153         ((uint64_t)0x80U) « (8U * ((KECCAK_RATE_BYTES - 1U) % 8U));
00154     keccak_f(a);
00155 }
00156
00157 bool CW_Tron::keccak256(const uint8_t* data, size_t len, uint8_t* out) {
00158     bool ret = false;
00159
00160     if (((data != NULL) || (len == 0U)) && (out != NULL)) {
00161         uint64_t a[KECCAK_STATE_LANES] = { 0U };
00162         for (unsigned i = 0U; i < KECCAK_STATE_LANES; i++) {
00163             a[i] = 0ULL;
00164         }
00165
00166         keccak_absorb(a, data, len);
00167
00168         for (unsigned i = 0U; i < KECCAK_DIGEST_BYTES; i++) {
00169             out[i] = (uint8_t)((a[i / 8U] » (8U * (i % 8U))) & 0xFFU);
00170         }
00171         ret = true;
00172     }
00173
00174     return ret;
00175 }
00176
00177 /*****************************************************************
00178  * 3. Base58 over the fixed 25-byte payload (file-local)
00179  *****************************************************************/
00180
00181 /*
00182  * ponytail: no leading-zero ('1' padding) handling -- a TRON payload always
00183  * starts with 0x41, so the encoding is always exactly 34 chars. Add the
00184  * generic leading-zero loop only if this is ever reused for Bitcoin-style
00185  * payloads, which can start with 0x00.
00186  */
00187
00194 static bool base58_encode_payload(const uint8_t* payload, char* out) {
00195     uint8_t tmp[TRON_PAYLOAD_BYTES] = { 0U };
00196     char    reversed[CW_TRON_ADDRESS_STR_LEN] = {};
00197     size_t  digits = 0U;
00198     size_t  start  = 0U;
00199
00200     memcpy(tmp, payload, sizeof(tmp));
```

© 2026 Cryptnox SA

```
00201
00202     while ((start < TRON_PAYLOAD_BYTES) && (digits < sizeof(reversed))) {
00203         uint32_t remainder = 0U;
00204         for (size_t i = start; i < TRON_PAYLOAD_BYTES; i++) {
00205             const uint32_t acc = (remainder << 8U) | (uint32_t)tmp[i];
00206             tmp[i]    = (uint8_t)(acc / BASE58_BASE);
00207             remainder = acc % BASE58_BASE;
00208         }
00209         reversed[digits] = BASE58_ALPHABET[remainder];
00210         digits++;
00211
00212         while ((start < TRON_PAYLOAD_BYTES) && (tmp[start] == 0U)) {
00213             start++;
00214         }
00215     }
00216
00217     /* Both: 34 digits written AND the value exhausted, else it was truncated. */
00218     const bool ret = (digits == CW_TRON_ADDRESS_STR_LEN) && (start == TRON_PAYLOAD_BYTES);
00219     if (ret) {
00220         for (size_t i = 0U; i < digits; i++) {
00221             out[i] = reversed[digits - 1U - i];
00222         }
00223         out[digits] = '\0';
00224     }
00225
00226     return ret;
00227 }
00228
00235 static bool base58_decode_payload(const char* address, uint8_t* payload) {
00236     memset(payload, 0, TRON_PAYLOAD_BYTES);
00237
00238     for (size_t i = 0U; i < CW_TRON_ADDRESS_STR_LEN; i++) {
00239         const char* found = strchr(BASE58_ALPHABET, address[i]);
00240         /* strchr would also match the terminating NUL of the alphabet. */
00241         if ((found == NULL) || (address[i] == '\0')) {
00242             return false;
00243         }
00244
00245         uint32_t carry = (uint32_t)(found - BASE58_ALPHABET);
00246         for (size_t j = TRON_PAYLOAD_BYTES; j > 0U; j--) {
00247             const uint32_t acc = ((uint32_t)payload[j - 1U] * BASE58_BASE) + carry;
00248             payload[j - 1U] = (uint8_t)(acc & 0xFFU);
00249             carry           = acc >> 8U;
00250         }
00251         if (carry != 0U) {
00252             return false;   /* value wider than 25 bytes */
00253         }
00254     }
00255
00256     return true;
00257 }
00258
00259 /*****************************************************************
00260  * 4. Public API
00261  *****************************************************************/
00262
00266 static bool tron_checksum(const uint8_t* addressBytes, CW_CryptoProvider& crypto,
00267                           uint8_t* checksum4) {
00268     uint8_t first[KECCAK_DIGEST_BYTES] = { 0U };
00269     uint8_t second[KECCAK_DIGEST_BYTES] = { 0U };
00270     bool    ret = false;
00271
00272     if (crypto.sha256(addressBytes, CW_TRON_ADDRESS_BYTES, first) &&
00273         crypto.sha256(first, sizeof(first), second)) {
00274         memcpy(checksum4, second, TRON_CHECKSUM_BYTES);
00275         ret = true;
00276     }
00277
00278     return ret;
00279 }
00280
00281 bool CW_Tron::addressBytesFromPublicKey(const uint8_t* publicKey64, uint8_t* out) {
00282     bool ret = false;
00283
00284     if ((publicKey64 != NULL) && (out != NULL)) {
00285         uint8_t digest[KECCAK_DIGEST_BYTES] = { 0U };
00286         if (CW_Tron::keccak256(publicKey64, PUBLIC_KEY_BYTES, digest)) {
00287             out[0] = CW_TRON_ADDRESS_PREFIX;
00288             memcpy(&out[1], &digest[KECCAK_DIGEST_BYTES - ETH_ADDRESS_BYTES],
00289                    ETH_ADDRESS_BYTES);
00290             ret = true;
00291         }
00292     }
00293
00294     return ret;
00295 }
00296
```

© 2026 Cryptnox SA

```
00297 bool CW_Tron::encodeAddress(const uint8_t* addressBytes, CW_CryptoProvider& crypto,
00298                             char* out, size_t outSize) {
00299     bool ret = false;
00300 
00301     if ((addressBytes != NULL) && (out != NULL) &&
00302         (outSize >= CW_TRON_ADDRESS_STR_SIZE) &&
00303         (addressBytes[0] == CW_TRON_ADDRESS_PREFIX)) {
00304 
00305         uint8_t payload[TRON_PAYLOAD_BYTES] = { 0U };
00306         memcpy(payload, addressBytes, CW_TRON_ADDRESS_BYTES);
00307 
00308         if (tron_checksum(addressBytes, crypto, &payload[CW_TRON_ADDRESS_BYTES])) {
00309             ret = base58_encode_payload(payload, out);
00310         }
00311     }
00312 
00313     return ret;
00314 }
00315 
00316 bool CW_Tron::addressFromPublicKey(const uint8_t* publicKey64,
00317                                    CW_CryptoProvider& crypto,
00318                                    char* out, size_t outSize) {
00319     uint8_t addressBytes[CW_TRON_ADDRESS_BYTES] = { 0U };
00320 
00321     return CW_Tron::addressBytesFromPublicKey(publicKey64, addressBytes) &&
00322            CW_Tron::encodeAddress(addressBytes, crypto, out, outSize);
00323 }
00324 
00325 bool CW_Tron::decodeAddress(const char* address, CW_CryptoProvider& crypto,
00326                             uint8_t* out) {
00327     bool ret = false;
00328 
00329     if ((address != NULL) && (out != NULL) &&
00330         (strlen(address) == CW_TRON_ADDRESS_STR_LEN)) {
00331 
00332         uint8_t payload[TRON_PAYLOAD_BYTES] = { 0U };
00333         uint8_t checksum[TRON_CHECKSUM_BYTES] = { 0U };
00334 
00335         if (base58_decode_payload(address, payload) &&
00336             (payload[0] == CW_TRON_ADDRESS_PREFIX) &&
00337             tron_checksum(payload, crypto, checksum) &&
00338             CW_Utils::secure_compare(checksum, &payload[CW_TRON_ADDRESS_BYTES],
00339                                      TRON_CHECKSUM_BYTES)) {
00340             memcpy(out, payload, CW_TRON_ADDRESS_BYTES);
00341             ret = true;
00342         }
00343     }
00344 
00345     return ret;
00346 }
00347 
00348 /*****************************************************************
00349  * 5. Self-check
00350  *****************************************************************/
00351 
00352 #ifdef CW_TRON_SELFTEST
00353 
00354 #include <assert.h>
00355 #include <stdio.h>
00356 
00357 /* Vectors derived from secp256k1 private keys 1 and
00358  * 0x1234...cdef; cross-checked against pycryptodome Keccak + base58. */
00359 static void check_keccak_empty(void) {
00360     /* The canonical Keccak-256 (not SHA3-256) digest of the empty string. */
00361     static const uint8_t expected[32] = {
00362         0xC5U, 0xD2U, 0x46U, 0x01U, 0x86U, 0xF7U, 0x23U, 0x3CU,
00363         0x92U, 0x7EU, 0x7DU, 0xB2U, 0xDCU, 0xC7U, 0x03U, 0xC0U,
00364         0xE5U, 0x00U, 0xB6U, 0x53U, 0xCAU, 0x82U, 0x27U, 0x3BU,
00365         0x7BU, 0xFAU, 0xD8U, 0x04U, 0x5DU, 0x85U, 0xA4U, 0x70U
00366     };
00367     uint8_t digest[32];
00368     assert(CW_Tron::keccak256(NULL, 0U, digest));
00369     assert(memcmp(digest, expected, sizeof(expected)) == 0);
00370 }
00371 
00372 static void check_address_bytes(void) {
00373     /* Public key for private key 1 (the secp256k1 generator point). */
00374     static const uint8_t pub[64] = {
00375         0x79U,0xBEU,0x66U,0x7EU,0xF9U,0xDCU,0xBBU,0xACU,
00376         0x55U,0xA0U,0x62U,0x95U,0xCEU,0x87U,0x0BU,0x07U,
00377         0x02U,0x9BU,0xFCU,0xDBU,0x2DU,0xCEU,0x28U,0xD9U,
00378         0x59U,0xF2U,0x81U,0x5BU,0x16U,0xF8U,0x17U,0x98U,
00379         0x48U,0x3AU,0xDAU,0x77U,0x26U,0xA3U,0xC4U,0x65U,
00380         0x5DU,0xA4U,0xFBU,0xFCU,0x0EU,0x11U,0x08U,0xA8U,
00381         0xFDU,0x17U,0xB4U,0x48U,0xA6U,0x85U,0x54U,0x19U,
00382         0x9CU,0x47U,0xD0U,0x8FU,0xFBU,0x10U,0xD4U,0xB8U
00383     };
```

© 2026 Cryptnox SA

```
00384     /* 0x41 || keccak256(pub)[12..32] */
00385     static const uint8_t expected[21] = {
00386         0x41U,0x7EU,0x5FU,0x45U,0x52U,0x09U,0x1AU,0x69U,0x12U,0x5DU,0x5DU,
00387         0xFCU,0xB7U,0xB8U,0xC2U,0x65U,0x90U,0x29U,0x39U,0x5BU,0xDFU
00388     };
00389     uint8_t addr[21];
00390     assert(CW_Tron::addressBytesFromPublicKey(pub, addr));
00391     assert(memcmp(addr, expected, sizeof(expected)) == 0);
00392 }
00393
00394 static void check_base58(void) {
00395     /* addr21 above || sha256(sha256(addr21))[0..4] == 0xB164FB1F */
00396     static const uint8_t payload[25] = {
00397         0x41U,0x7EU,0x5FU,0x45U,0x52U,0x09U,0x1AU,0x69U,0x12U,0x5DU,0x5DU,
00398         0xFCU,0xB7U,0xB8U,0xC2U,0x65U,0x90U,0x29U,0x39U,0x5BU,0xDFU,
00399         0xB1U,0x64U,0xFBU,0x1FU
00400     };
00401     static const char expected[] = "TMVQGm1qAQYVdetCeGRRkTWYYrLXuHK2HC";
00402
00403     char text[CW_TRON_ADDRESS_STR_SIZE];
00404     assert(base58_encode_payload(payload, text));
00405     assert(strcmp(text, expected) == 0);
00406
00407     uint8_t roundTrip[25];
00408     assert(base58_decode_payload(expected, roundTrip));
00409     assert(memcmp(roundTrip, payload, sizeof(payload)) == 0);
00410
00411     /* A second, independent vector (private key 0x1234...cdef). */
00412     static const uint8_t payload2[25] = {
00413         0x41U,0x1BU,0xE3U,0x1AU,0x94U,0x36U,0x1AU,0x39U,0x1BU,0xBAU,0xFBU,
00414         0x2AU,0x4CU,0xCDU,0x70U,0x4FU,0x57U,0xDCU,0x04U,0xD4U,0xBBU,
00415         0x11U,0xAFU,0x6DU,0xEEU
00416     };
00417     assert(base58_encode_payload(payload2, text));
00418     assert(strcmp(text, "TCWfJguCLonUkvmsptQKHUyUJ713pU7XQ1") == 0);
00419
00420     /* Characters outside the Base58 alphabet must be rejected. */
00421     char bad[CW_TRON_ADDRESS_STR_SIZE];
00422     memcpy(bad, expected, sizeof(bad));
00423     bad[5] = '0';   /* '0' is not in the Base58 alphabet */
00424     uint8_t ignored[25];
00425     assert(!base58_decode_payload(bad, ignored));
00426 }
00427
00428 int main(void) {
00429     check_keccak_empty();
00430     check_address_bytes();
00431     check_base58();
00432     printf("CW_Tron self-check OK\n");
00433     return 0;
00434 }
00435
00436 #endif /* CW_TRON_SELFTEST */
```

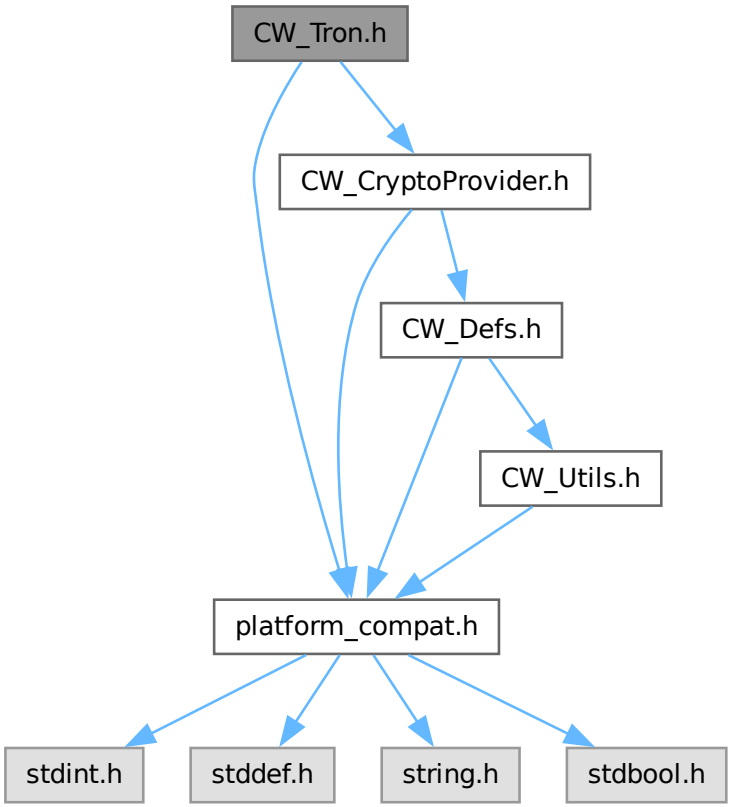
5.21 CW_Tron.h File Reference

TRON (TRX) network support — BIP44 path and Base58Check addresses.

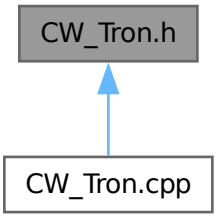
```
#include "platform_compat.h"
#include "CW_CryptoProvider.h"
```

© 2026 Cryptnox SA

Include dependency graph for CW_Tron.h:



This graph shows which files directly or indirectly include this file:



Classes

- class CW_Tron

  TRON address derivation and Base58Check encoding.

© 2026 Cryptnox SA

Macros

- #define CW_TRON_PATH_LENGTH (20U)

  Length of the wire-format TRON BIP44 derivation path in bytes.
- #define CW_TRON_ADDRESS_BYTES (21U)

  Raw TRON address length: `0x41` prefix + 20 address bytes.
- #define CW_TRON_ADDRESS_PREFIX (0x41U)

  TRON mainnet address prefix byte (renders as a leading `'T'`).
- #define CW_TRON_ADDRESS_STR_LEN (34U)

  Base58Check address length in characters, excluding the NUL.
- #define CW_TRON_ADDRESS_STR_SIZE (CW_TRON_ADDRESS_STR_LEN + 1U)

  Minimum buffer size for a Base58Check address string (34 + NUL).

Variables

- const uint8_t CW_TRON_DERIVE_PATH [CW_TRON_PATH_LENGTH]

  BIP44 derivation path for TRON: `m/44'/195'/0'/0/0`.

5.21.1 Detailed Description

TRON (TRX) network support — BIP44 path and Base58Check addresses.
TRON reuses Ethereum's key material and hashing: the account address is `keccak256(publicKey)` [12..32], but it is prefixed with `0x41` and presented as Base58Check instead of hex — so every address starts with a capital `T` and is exactly 34 characters long.
Provides:

- CW_TRON_DERIVE_PATH — the wire-format BIP44 path for `CW_SIGN_DERIVE_K1`
- public key → address (raw 21 bytes, or Base58Check text)
- Base58Check address → raw 21 bytes, with checksum validation

Signing itself is plain secp256k1 ECDSA over a 32-byte digest, so CryptnoxWallet::sign needs no TRON-specific handling. Transaction serialisation (protobuf) is out of scope here — TRON full nodes build the unsigned transaction over HTTP, and the caller verifies it locally before signing (see the ESP32 `TronSigning` example).
Definition in file CW_Tron.h.

5.21.2 Macro Definition Documentation

5.21.2.1 CW_TRON_ADDRESS_BYTES

```
#define CW_TRON_ADDRESS_BYTES (21U)
```

Raw TRON address length: `0x41` prefix + 20 address bytes.
Definition at line 45 of file CW_Tron.h.
Referenced by CW_Tron::addressFromPublicKey(), CW_Tron::decodeAddress(), CW_Tron::encodeAddress(), and tron_checksum().

5.21.2.2 CW_TRON_ADDRESS_PREFIX

```
#define CW_TRON_ADDRESS_PREFIX (0x41U)
```

TRON mainnet address prefix byte (renders as a leading `'T'`).
Definition at line 48 of file CW_Tron.h.
Referenced by CW_Tron::addressBytesFromPublicKey(), CW_Tron::decodeAddress(), and CW_Tron::encodeAddress().

© 2026 Cryptnox SA

5.21.2.3 CW_TRON_ADDRESS_STR_LEN

```
#define CW_TRON_ADDRESS_STR_LEN (34U)
```

Base58Check address length in characters, excluding the NUL.
Definition at line 51 of file CW_Tron.h.
Referenced by base58_decode_payload(), base58_encode_payload(), and CW_Tron::decodeAddress().

5.21.2.4 CW_TRON_ADDRESS_STR_SIZE

```
#define CW_TRON_ADDRESS_STR_SIZE (CW_TRON_ADDRESS_STR_LEN + 1U)
```

Minimum buffer size for a Base58Check address string (34 + NUL).
Definition at line 54 of file CW_Tron.h.
Referenced by CW_Tron::encodeAddress().

5.21.2.5 CW_TRON_PATH_LENGTH

```
#define CW_TRON_PATH_LENGTH (20U)
```

Length of the wire-format TRON BIP44 derivation path in bytes.
Definition at line 42 of file CW_Tron.h.

5.22 CW_Tron.h

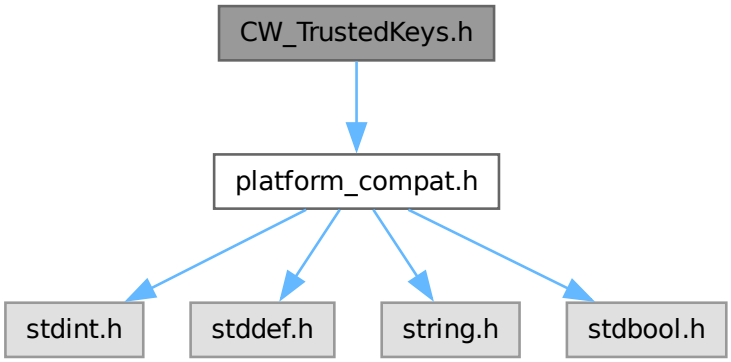
Go to the documentation of this file.

```
00001 /*
00002  * SPDX-License-Identifier: LGPL-3.0-or-later
00003  * Copyright (c) 2026 Cryptnox SA
00004  */
00005
00026
00027 #ifndef CW_TRON_H
00028 #define CW_TRON_H
00029
00030 /**************************************************************************
00031  * 1. Included files
00032  *************************************************************************/
00033
00034 #include "platform_compat.h"
00035 #include "CW_CryptoProvider.h"
00036
00037 /**************************************************************************
00038  * 2. Constants / define declarations
00039  *************************************************************************/
00040
00042 #define CW_TRON_PATH_LENGTH      (20U)
00043
00045 #define CW_TRON_ADDRESS_BYTES    (21U)
00046
00048 #define CW_TRON_ADDRESS_PREFIX   (0x41U)
00049
00051 #define CW_TRON_ADDRESS_STR_LEN  (34U)
00052
00054 #define CW_TRON_ADDRESS_STR_SIZE (CW_TRON_ADDRESS_STR_LEN + 1U)
00055
00064 extern const uint8_t CW_TRON_DERIVE_PATH[CW_TRON_PATH_LENGTH];
00065
00066 /**************************************************************************
00067  * 3. Class declaration
00068  *************************************************************************/
00069
00080 class CW_Tron {
00081 public:
00091     static bool addressBytesFromPublicKey(const uint8_t* publicKey64,
00092                                           uint8_t* out);
00093
00107     static bool addressFromPublicKey(const uint8_t* publicKey64,
00108                                      CW_CryptoProvider& crypto,
00109                                      char* out, size_t outSize);
00110
00125     static bool encodeAddress(const uint8_t* addressBytes,
00126                               CW_CryptoProvider& crypto,
00127                               char* out, size_t outSize);
00128
00144     static bool decodeAddress(const char* address,
00145                               CW_CryptoProvider& crypto,
```

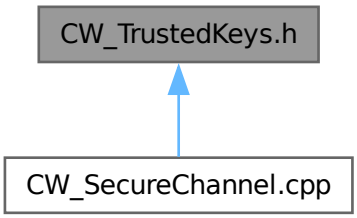
```
00146                                uint8_t* out);
00147
00160     static bool keccak256(const uint8_t* data, size_t len, uint8_t* out);
00161 };
00162
00163 #endif // CW_TRON_H
```

5.23 CW_TrustedKeys.h File Reference

Cryptnox CA public keys used for offline certificate verification.
`#include "platform_compat.h"`
Include dependency graph for CW_TrustedKeys.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define CW_TRUSTED_CA_COUNT (1U)

Variables

- static const uint8_t CW_CA_DLT_PUBKEY [64]
- static const uint8_t *const CW_TRUSTED_CA_KEYS [CW_TRUSTED_CA_COUNT]

© 2026 Cryptnox SA

5.23.1 Detailed Description

Cryptnox CA public keys used for offline certificate verification.
Holds the trusted secp256r1 public keys that CW_SecureChannel::verifyCertificateChain checks the manufacturer certificate against. Each key is stored as 64 raw bytes (X[32] || Y[32], no 0x04 prefix) so it can be passed directly to `uECC_verify()`.
Source: https://verify.cryptnox.tech/certificates/
To update a key: download the .crt file from the URL above and extract the EC public-key coordinates with:

```
python -c "from cryptography import x509; \
  c=x509.load_pem_x509_certificate(open('CERT.crt','rb').read()); \
  n=c.public_key().public_numbers(); \
  print(hex(n.x)); print(hex(n.y))"
```

Definition in file CW_TrustedKeys.h.

5.23.2 Macro Definition Documentation

5.23.2.1 CW_TRUSTED_CA_COUNT

```
#define CW_TRUSTED_CA_COUNT (1U)
```

Definition at line 46 of file CW_TrustedKeys.h.
Referenced by CW_SecureChannel::verifyCertificateChain().

5.23.3 Variable Documentation

5.23.3.1 CW_CA_DLT_PUBKEY

```
const uint8_t CW_CA_DLT_PUBKEY[64]  [static]
```

Initial value:

```
= {

    0x64, 0x7a, 0x11, 0x2c, 0x2a, 0xcf, 0x5a, 0x49,
    0xb5, 0x2b, 0x32, 0x1c, 0xbd, 0xcf, 0x5a, 0x9e,
    0xa3, 0x07, 0xfc, 0x4c, 0xcd, 0x33, 0xaa, 0x78,
    0xb9, 0xb8, 0x05, 0x5d, 0xd8, 0x4d, 0x03, 0xae,

    0xda, 0xb0, 0x2c, 0xc7, 0x20, 0xa7, 0x80, 0xcb,
    0x1f, 0xf2, 0x80, 0xaf, 0x50, 0x77, 0x4a, 0x6c,
    0xdc, 0x15, 0x7e, 0xfa, 0x23, 0x5e, 0xa2, 0x53,
    0x11, 0xa4, 0x2b, 0x4c, 0xf5, 0x7d, 0x88, 0x61
}
```

Definition at line 32 of file CW_TrustedKeys.h.

5.23.3.2 CW_TRUSTED_CA_KEYS

```
const uint8_t* const CW_TRUSTED_CA_KEYS[CW_TRUSTED_CA_COUNT]  [static]
```

Initial value:

```
= {
    CW_CA_DLT_PUBKEY
}
```

Definition at line 50 of file CW_TrustedKeys.h.
Referenced by CW_SecureChannel::verifyCertificateChain().

5.24 CW_TrustedKeys.h

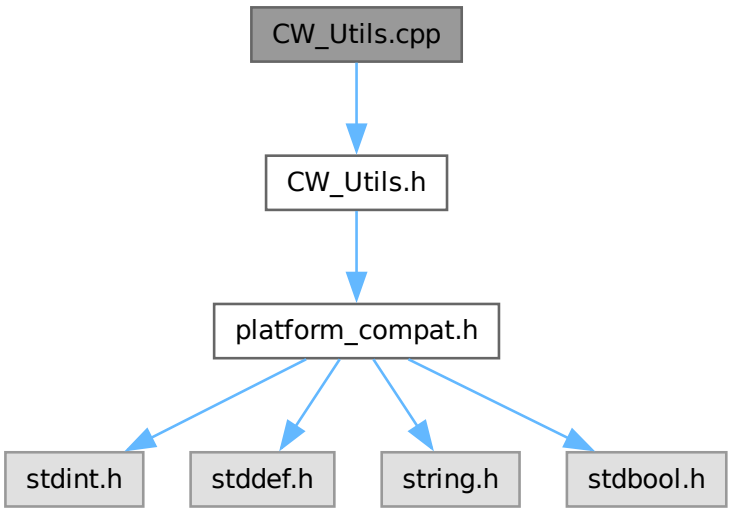
Go to the documentation of this file.

```
00001 /*
00002  * SPDX-License-Identifier: LGPL-3.0-or-later
00003  * Copyright (c) 2026 Cryptnox SA
00004  */
00005
00026
00027 #ifndef CW_TRUSTEDKEYS_H
00028 #define CW_TRUSTEDKEYS_H
00029
00030 #include "platform_compat.h"
```

```
00031
00032 /* CRYPTNOX_DLT_CARDS_CA -- signs manufacturer certificates for DLT cards.
00033  * Curve: secp256r1. Issued by CRYPTNOX INTERMEDIATE CA #2. */
00034 static const uint8_t CW_CA_DLT_PUBKEY[64] = {
00035     /* X */
00036     0x64, 0x7a, 0x11, 0x2c, 0x2a, 0xcf, 0x5a, 0x49,
00037     0xb5, 0x2b, 0x32, 0x1c, 0xbd, 0xcf, 0x5a, 0x9e,
00038     0xa3, 0x07, 0xfc, 0x4c, 0xcd, 0x33, 0xaa, 0x78,
00039     0xb9, 0xb8, 0x05, 0x5d, 0xd8, 0x4d, 0x03, 0xae,
00040     /* Y */
00041     0xda, 0xb0, 0x2c, 0xc7, 0x20, 0xa7, 0x80, 0xcb,
00042     0x1f, 0xf2, 0x80, 0xaf, 0x50, 0x77, 0x4a, 0x6c,
00043     0xdc, 0x15, 0x7e, 0xfa, 0x23, 0x5e, 0xa2, 0x53,
00044     0x11, 0xa4, 0x2b, 0x4c, 0xf5, 0x7d, 0x88, 0x61
00045 };
00046
00047 /* Number of trusted CA keys in the table below. */
00048 #define CW_TRUSTED_CA_COUNT (1U)
00049
00050 /* Table of all trusted CA public keys (each 64 bytes, X||Y).
00051  * verifyCertificateChain() tries every entry until one succeeds. */
00052 static const uint8_t* const CW_TRUSTED_CA_KEYS[CW_TRUSTED_CA_COUNT] = {
00053     CW_CA_DLT_PUBKEY
00054 };
00055
00056 #endif /* CW_TRUSTEDKEYS_H */
```

5.25 CW_Utils.cpp File Reference

Implementation of the platform-independent utility helpers.
`#include "CW_Utils.h"`
Include dependency graph for CW_Utils.cpp:



5.25.1 Detailed Description

Implementation of the platform-independent utility helpers.
CW_Utils::secure_compare iterates over the full length to avoid leaking byte-position information through timing. CW_Utils::secure_wipe uses a volatile pointer so the writes cannot be optimised away. CW_Utils::safe_memcpy validates pointers, length, and source/destination overlap before delegating to memcpy.
Definition in file CW_Utils.cpp.

© 2026 Cryptnox SA

5.26 CW_Utils.cpp

Go to the documentation of this file.

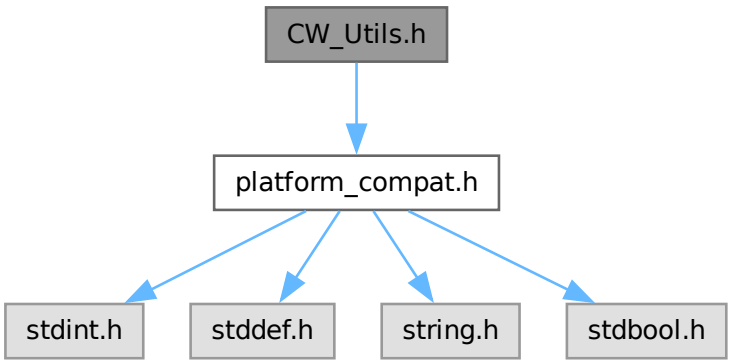
```
00001 /*
00002  * SPDX-License-Identifier: LGPL-3.0-or-later
00003  * Copyright (c) 2026 Cryptnox SA
00004  */
00005
00016
00017 #include "CW_Utils.h"
00018
00022 bool CW_Utils::secure_compare(const uint8_t* a, const uint8_t* b, size_t len) {
00023     bool ret = false;
00024     if ((a != NULL) && (b != NULL) && (len > 0U)) {
00025         uint8_t diff = 0U;
00026         for (size_t i = 0U; i < len; i++) {
00027             diff |= a[i] ^ b[i];
00028         }
00029         ret = (diff == 0U);
00030     }
00031     return ret;
00032 }
00033
00037 void CW_Utils::secure_wipe(uint8_t* buf, size_t len) {
00038     if ((buf != NULL) && (len > 0U)) {
00039         volatile uint8_t* p = buf;
00040         for (size_t i = 0U; i < len; i++) {
00041             p[i] = 0U;
00042         }
00043     }
00044 }
00045
00050 bool CW_Utils::safe_memcpy(uint8_t* dst, size_t dstSize,
00051                                 const uint8_t* src, size_t count) {
00052     bool ret = false;
00053     if ((dst != NULL) && (src != NULL) && (count > 0U) && (count <= dstSize)) {
00054         bool overlap = (dst < (src + count)) && (src < (dst + dstSize));
00055         if (!overlap) {
00056             memcpy(dst, src, count);
00057             ret = true;
00058         }
00059     }
00060     return ret;
00061 }
```

5.27 CW_Utils.h File Reference

Platform-independent security and memory utilities.

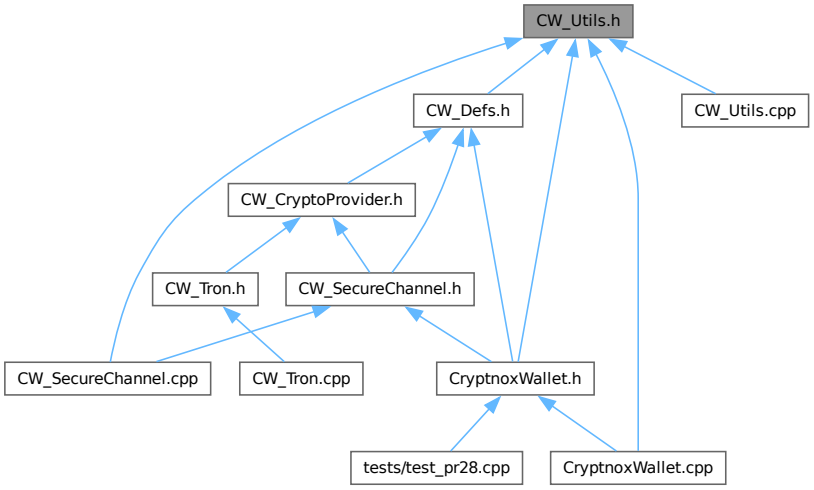
`#include "platform_compat.h"`

Include dependency graph for CW_Utils.h:



© 2026 Cryptnox SA

This graph shows which files directly or indirectly include this file:



Classes

- class CW_Utils

  Portable utility functions for cryptographic and security operations.

5.27.1 Detailed Description

Platform-independent security and memory utilities.
Declares CW_Utils, a small collection of helpers used throughout the SDK that have no platform dependencies:

- constant-time buffer comparison (timing-side-channel safe)
- guaranteed-not-elided secure wipe of sensitive buffers
- bounds-checked, overlap-checked memcpy
- secure random byte generation (delegated to a platform impl)

Hardware-specific helpers (e.g. TRNG bring-up) live in the concrete crypto provider rather than here.
Definition in file CW_Utils.h.

5.28 CW_Utils.h

Go to the documentation of this file.

```
00001 /*
00002  * SPDX-License-Identifier: LGPL-3.0-or-later
00003  * Copyright (c) 2026 Cryptnox SA
00004  */
00005
00020
00021 #ifndef CW_UTILS_H
00022 #define CW_UTILS_H
00023
00024 /**************************************************************
00025  * 1. Included files
00026  **************************************************************/
00027
00028 #include "platform_compat.h"
00029
```

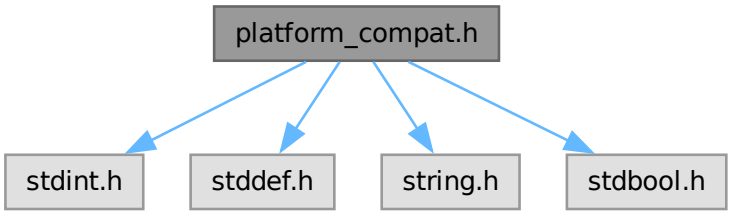
```
00030 /*****************************************************************
00031  * 2. Class declaration
00032  *****************************************************************/
00033
00045 class CW_Utils {
00046 public:
00059     static bool secure_compare(const uint8_t* a, const uint8_t* b, size_t len);
00060
00070     static void secure_wipe(uint8_t* buf, size_t len);
00071
00081     static bool safe_memcpy(uint8_t* dst, size_t dstSize,
00082                             const uint8_t* src, size_t count);
00083
00095     static bool fill_secure_random(uint8_t* dest, size_t len);
00096 };
00097
00098 #endif // CW_UTILS_H
```

5.29 platform_compat.h File Reference

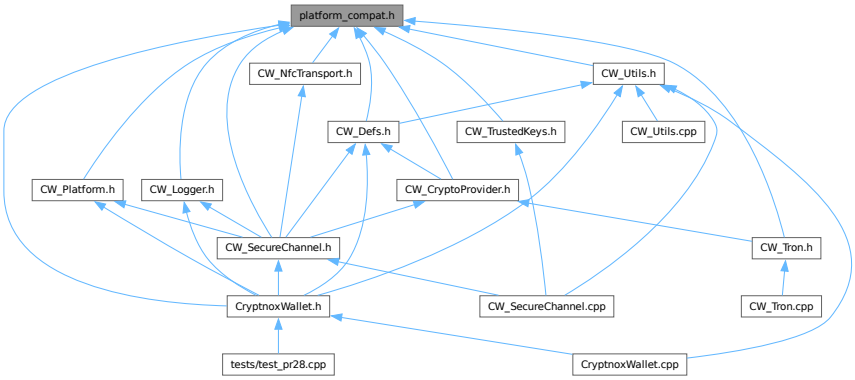
Arduino compatibility shims for non-Arduino (plain C++) builds.

```
#include <stdint.h>
#include <stddef.h>
#include <string.h>
#include <stdbool.h>
```

Include dependency graph for platform_compat.h:



This graph shows which files directly or indirectly include this file:



Classes

- class __FlashStringHelper

© 2026 Cryptnox SA

Macros

- #define DEC 10
- #define HEX 16
- #define OCT 8
- #define BIN 2
- #define F(string_literal)

5.29.1 Detailed Description

Arduino compatibility shims for non-Arduino (plain C++) builds.
Include this file instead of <Arduino.h> in platform-independent headers. When building for Arduino, ARDUINO is already defined by the toolchain and this file is a no-op — Arduino.h has already provided all these definitions. When building outside Arduino (desktop, ESP-IDF, CI) this file provides the minimum set of defines/types needed so the SDK headers compile cleanly.
Definition in file platform_compat.h.

5.29.2 Macro Definition Documentation

5.29.2.1 BIN

```
#define BIN 2
```

Definition at line 37 of file platform_compat.h.

5.29.2.2 DEC

```
#define DEC 10
```

Definition at line 34 of file platform_compat.h.
Referenced by CW_Logger::print(), CW_Logger::print(), CW_Logger::print(), CW_Logger::print(), CW_Logger::println(), CW_Logger::println(), CW_Logger::println(), and CW_Logger::println().

5.29.2.3 F

```
#define F(
            string_literal)
```

Value:

```
(string_literal)
```

Definition at line 42 of file platform_compat.h.
Referenced by CW_SecureChannel::aesCbcDecrypt(), CW_SecureChannel::aesCbcEncrypt(), CW_SecureChannel::checkStatusWord(), CryptnoxWallet::connect(), CryptnoxWallet::debugPrintSignature(), CryptnoxWallet::establishSecureChannel(), CryptnoxWallet::extractRawSignature(), CW_SecureChannel::getCardCertif CryptnoxWallet::getCardInfo(), CW_SecureChannel::getManufacturerCertificate(), CryptnoxWallet::getPublicKey(), CryptnoxWallet::getPublicKey(), CW_SecureChannel::mutuallyAuthenticate(), CW_SecureChannel::openSecureChanne CW_SecureChannel::selectApdu(), CryptnoxWallet::sendSignApdu(), CryptnoxWallet::sign(), CryptnoxWallet::validateSignRequest(), CW_SecureChannel::verifyCertificateChain(), CryptnoxWallet::verifyPin(), and CryptnoxWallet::writeUserData().

5.29.2.4 HEX

```
#define HEX 16
```

Definition at line 35 of file platform_compat.h.
Referenced by CW_SecureChannel::aesCbcDecrypt(), CW_SecureChannel::aesCbcEncrypt(), CW_SecureChannel::checkStatusWord(), CryptnoxWallet::debugPrintSignature(), and CryptnoxWallet::establishSecure

5.29.2.5 OCT

```
#define OCT 8
```

Definition at line 36 of file platform_compat.h.

© 2026 Cryptnox SA

5.30 platform_compat.h

Go to the documentation of this file.

```
00001 /*
00002  * SPDX-License-Identifier: LGPL-3.0-or-later
00003  * Copyright (c) 2026 Cryptnox SA
00004  */
00005
00006 #ifndef PLATFORM_COMPAT_H
00007 #define PLATFORM_COMPAT_H
00008
00019
00020 #ifdef ARDUINO
00021 /* On Arduino, pull in Arduino.h so SDK headers that reference __FlashStringHelper,
00022  * DEC/HEX/OCT/BIN, F(), or delay() compile regardless of whether the including
00023  * translation unit already included <Arduino.h>. Arduino.h transitively provides
00024  * <stdint.h>, <stddef.h>, <string.h>, etc., so no need to include them here. */
00025 #  include <Arduino.h>
00026 #else
00027 /* Off Arduino, include the C standard headers explicitly so the SDK compiles
00028  * without depending on what the including TU may or may not have pulled in. */
00029 #  include <stdint.h>
00030 #  include <stddef.h>
00031 #  include <string.h>
00032 #  include <stdbool.h>
00033
00034 #  define DEC  10
00035 #  define HEX  16
00036 #  define OCT   8
00037 #  define BIN   2
00038
00039 /* F("...") on Arduino returns __FlashStringHelper* to keep string literals in
00040  * flash.  On non-Arduino it is the identity macro, resolving to const char*. */
00041 class __FlashStringHelper {};
00042 #  define F(string_literal) (string_literal)
00043
00044 #endif /* ARDUINO / !ARDUINO */
00045
00046 #endif /* PLATFORM_COMPAT_H */
```

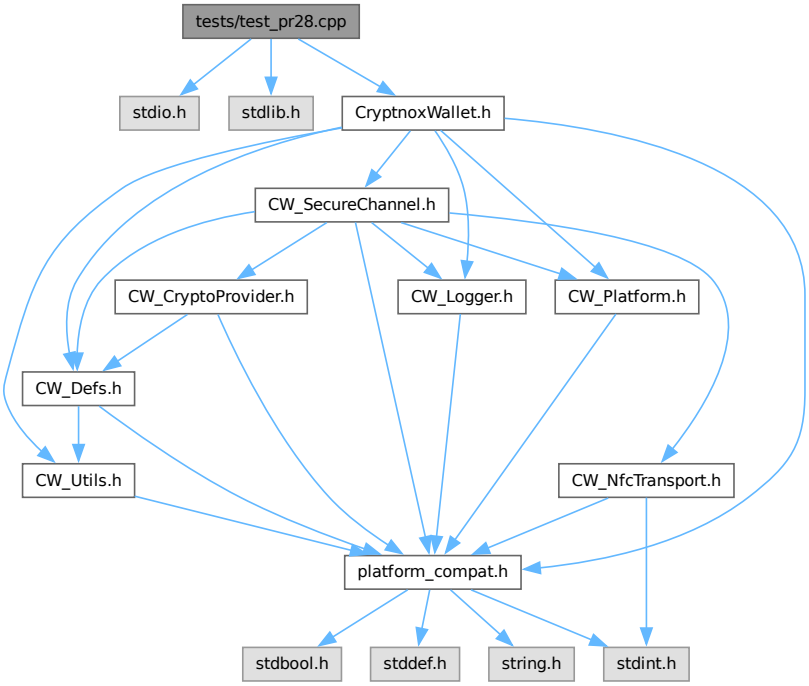
5.31 README.md File Reference

5.32 tests/test_pr28.cpp File Reference

```
#include <stdio.h>
#include <stdlib.h>
#include "CryptnoxWallet.h"
```

© 2026 Cryptnox SA

Include dependency graph for test_pr28.cpp:



Classes

- class FakeNfc
- class FakeCrypto
- class NullLogger
- class NullPlatform
- struct Rig

Macros

- #define CHECK(cond)

Functions

- static void openSession (CW_SecureSession &s)
- static bool isCleared (const CW_SecureSession &s)
- static void check_session_clearing (void)
- static void selectOnly (CryptnoxWallet &wallet, FakeNfc &nfc, CW_SecureSession &s)
- static bool pathFor (uint8_t keyType)
- static void check_pubkey_allow_list (void)
- static CW_SignResult signWith (CryptnoxWallet &wallet, CW_SecureSession &s, uint8_t key↩ Type)
- static void check_ed25519_applet_gate (void)
- int main (void)

© 2026 Cryptnox SA

Variables

- static NullLogger logger
- static NullPlatform platform
- static const uint8_t HDR [ ] = { 0x80U, 0xCAU, 0x00U, 0x00U }
- static const uint8_t DATA [ ] = { 1U, 2U, 3U }
- static const uint8_t PATH [16]

5.32.1 Macro Definition Documentation

5.32.1.1 CHECK

```
#define CHECK(
              cond)
```

Value:

```
    do { if (!(cond)) { \
    printf("FAIL %s:%d: %s\n", __FILE__, __LINE__, #cond); exit(1); } } while (0)
```

Definition at line 21 of file test_pr28.cpp.
Referenced by check_ed25519_applet_gate(), check_pubkey_allow_list(), check_session_clearing(), Rig::Rig(), and selectOnly().

5.32.2 Function Documentation

5.32.2.1 check_ed25519_applet_gate()

```
void check_ed25519_applet_gate (
            void )  [static]
```

Definition at line 295 of file test_pr28.cpp.
References FakeNfc::appletMajor, CHECK, CW_SIGN_CURR_ED25519, CW_SIGN_CURR_K1, CW_SIGN_SIG_EDDSA, CW_SIGN_UNSUPPORTED_KEY_TYPE, CryptnoxWallet::getPublicKey(), FakeNfc::last, logger, openSession(), platform, selectOnly(), FakeNfc::sent, and signWith().
Referenced by main().

5.32.2.2 check_pubkey_allow_list()

```
void check_pubkey_allow_list (
            void )  [static]
```

Definition at line 249 of file test_pr28.cpp.
References CHECK, CW_SIGN_CURR_ED25519, CW_SIGN_CURR_K1, CW_SIGN_CURR_R1, CW_SIGN_DERIVE_ED25519, CW_SIGN_DERIVE_K1, CW_SIGN_DERIVE_R1, CryptnoxWallet::getPublicKey(), isCleared(), FakeNfc::last, logger, openSession(), PATH, pathFor(), platform, selectOnly(), and FakeNfc::sent.
Referenced by main().

5.32.2.3 check_session_clearing()

```
void check_session_clearing (
            void )  [static]
```

Definition at line 169 of file test_pr28.cpp.
References CHECK, Rig::crypto, CW_USER_DATA_PAGE_SIZE, DATA, FakeCrypto::failMac, FakeCrypto::failMacIv, isCleared(), CW_SecureSession::macCounter, Rig::nfc, FakeNfc::resp, FakeNfc::respLen, Rig::s, Rig::send(), FakeNfc::sendOk, and FakeNfc::sent.
Referenced by main().

5.32.2.4 isCleared()

```
bool isCleared (
            const CW_SecureSession & s)  [static]
```

Definition at line 140 of file test_pr28.cpp.

© 2026 Cryptnox SA

References CW_SecureSession::aesKey, CW_SecureSession::macCounter, and CW_SecureSession::macKey.
Referenced by check_pubkey_allow_list(), and check_session_clearing().

5.32.2.5 main()

```
int main (
            void )
```

Definition at line 336 of file test_pr28.cpp.
References check_ed25519_applet_gate(), check_pubkey_allow_list(), and check_session_clearing().

5.32.2.6 openSession()

```
void openSession (
            CW_SecureSession & s)  [static]
```

Definition at line 133 of file test_pr28.cpp.
References CW_SecureSession::aesKey, CW_SecureSession::clear(), CW_SecureSession::iv, and CW_SecureSession::macKey.
Referenced by check_ed25519_applet_gate(), check_pubkey_allow_list(), and Rig::Rig().

5.32.2.7 pathFor()

```
bool pathFor (
            uint8_t keyType)  [static]
```

Definition at line 245 of file test_pr28.cpp.
Referenced by check_pubkey_allow_list().

5.32.2.8 selectOnly()

```
void selectOnly (
            CryptnoxWallet & wallet,
            FakeNfc & nfc,
            CW_SecureSession & s)  [static]
```

Definition at line 239 of file test_pr28.cpp.
References CHECK, CryptnoxWallet::connect(), and FakeNfc::sent.
Referenced by check_ed25519_applet_gate(), and check_pubkey_allow_list().

5.32.2.9 signWith()

```
CW_SignResult signWith (
            CryptnoxWallet & wallet,
            CW_SecureSession & s,
            uint8_t keyType)  [static]
```

Definition at line 286 of file test_pr28.cpp.
References CW_SIGN_SIG_ECDSA_LOW_S, CW_SIGN_WITH_PIN, CW_SignRequest::hash, CW_SignRequest::hashLength, CW_SignRequest::pin, and CryptnoxWallet::sign().
Referenced by check_ed25519_applet_gate().

5.32.3 Variable Documentation

5.32.3.1 DATA

```
const uint8_t DATA[] = { 1U, 2U, 3U }  [static]
```

Definition at line 148 of file test_pr28.cpp.
Referenced by check_session_clearing().

5.32.3.2 HDR

```
const uint8_t HDR[] = { 0x80U, 0xCAU, 0x00U, 0x00U }  [static]
```

Definition at line 147 of file test_pr28.cpp.
Referenced by Rig::send().

© 2026 Cryptnox SA

5.32.3.3 logger

```
NullLogger logger  [static]
```

Definition at line 130 of file test_pr28.cpp.
Referenced by check_ed25519_applet_gate(), check_pubkey_allow_list(), CryptnoxWallet::CryptnoxWallet(), CW_SecureChannel::CW_SecureChannel(), and Rig::Rig().

5.32.3.4 PATH

```
const uint8_t PATH[16]  [static]
```

Initial value:

```
= {
    0x80U, 0x00U, 0x00U, 0x2CU, 0x80U, 0x00U, 0x01U, 0xF5U,
    0x80U, 0x00U, 0x00U, 0x00U, 0x80U, 0x00U, 0x00U, 0x00U
}
```

Definition at line 232 of file test_pr28.cpp.
Referenced by check_pubkey_allow_list().

5.32.3.5 platform

```
NullPlatform platform  [static]
```

Definition at line 131 of file test_pr28.cpp.
Referenced by check_ed25519_applet_gate(), check_pubkey_allow_list(), CryptnoxWallet::CryptnoxWallet(), CW_SecureChannel::CW_SecureChannel(), and Rig::Rig().

5.33 test_pr28.cpp

Go to the documentation of this file.

```
00001 /*
00002  * SPDX-License-Identifier: LGPL-3.0-or-later
00003  * Copyright (c) 2026 Cryptnox SA
00004  */
00005
00006 /* Host tests for the PR #28 review fixes: session clearing in aesCbcEncrypt(),
00007  * the getPublicKey() key-type allow-list and the Ed25519 applet check.
00008  *
00009  * Build and run from the repository root:
00010  *   g++ -std=c++17 -I. tests/test_pr28.cpp CryptnoxWallet.cpp CW_SecureChannel.cpp CW_Utils.cpp -o
      test_pr28 && ./test_pr28
00011  *
00012  * Fakes only: the reader answers SELECT with a chosen applet version and records
00013  * every other APDU; AES is a XOR "cipher" in CBC mode, which is enough for the
00014  * framing and failure paths tested here (no real card or crypto library). */
00015
00016 #include <stdio.h>
00017 #include <stdlib.h>
00018 #include "CryptnoxWallet.h"
00019
00020 /* Not assert(): it vanishes under NDEBUG and would skip the calls under test. */
00021 #define CHECK(cond) do { if (!(cond)) { \
00022     printf("FAIL %s:%d: %s\n", __FILE__, __LINE__, #cond); exit(1); } } while (0)
00023
00024 /*****************************************************************
00025  * Fakes
00026  *****************************************************************/
00027
00028 class FakeNfc : public CW_NfcTransport {
00029 public:
00030     uint8_t appletMajor = 2U;        /* version byte returned by SELECT */
00031     bool    sendOk = false;          /* result for every non-SELECT APDU */
00032     uint8_t resp[255] = { 0U };      /* response for every non-SELECT APDU */
00033     uint8_t respLen = 0U;
00034     unsigned sent = 0U;              /* non-SELECT APDUs sent */
00035     uint8_t last[255] = { 0U };      /* last non-SELECT APDU */
00036
00037     bool begin() override { return true; }
00038     bool inListPassiveTarget() override { return true; }
00039     void resetReader() override { }
00040     bool printFirmwareVersion() override { return true; }
00041
00042     bool sendAPDU(const uint8_t* apdu, uint8_t apduLen,
00043                   uint8_t* response, uint8_t& responseLen) override {
00044         if ((apdu[0] == 0x00U) && (apdu[1] == 0xA4U)) {
```

```
00045             const uint8_t sel[] = { 0x42U, appletMajor, 0x00U, 0x00U, 0x00U, 0x50U, 0x90U, 0x00U };
00046             memcpy(response, sel, sizeof(sel));
00047             responseLen = sizeof(sel);
00048             return true;
00049         }
00050         sent++;
00051         memcpy(last, apdu, apduLen);
00052         memcpy(response, resp, respLen);
00053         responseLen = respLen;
00054         return sendOk;
00055     }
00056 };
00057
00058 class FakeCrypto : public CW_CryptoProvider {
00059 public:
00060     bool failMacIv = false;   /* fail the 16-byte counter block encryption (v2 MAC IV) */
00061     bool failMac   = false;   /* fail the MAC computation itself */
00062
00063     bool sha256(const uint8_t*, size_t, uint8_t* out) override { memset(out, 0, 32); return true; }
00064     bool sha512(const uint8_t*, size_t, uint8_t* out) override { memset(out, 0, 64); return true; }
00065     bool ecdh(const uint8_t*, const uint8_t*, uint8_t* s, CW_Curve) override { memset(s, 1, 32);
      return true; }
00066     bool makeKey(uint8_t* pub, uint8_t* priv, CW_Curve) override { memset(pub, 2, 64); memset(priv, 3,
      32); return true; }
00067     bool random(uint8_t* d, unsigned n) override { memset(d, 0x5A, n); return true; }
00068     bool ecdsaVerify(const uint8_t*, const uint8_t*, size_t, const uint8_t*, CW_Curve) override {
      return false; }
00069
00070     uint16_t aesCbcEncrypt(const uint8_t* in, uint16_t len, uint8_t* out,
00071                            const uint8_t* key, uint8_t, uint8_t* iv, bool bitPadding) override {
00072         if (!bitPadding && (len == 16U) && failMacIv) { return 0U; }
00073         if (!bitPadding && (len > 16U) && failMac) { return 0U; }
00074         uint8_t buf[512] = { 0U };
00075         memcpy(buf, in, len);
00076         uint16_t total = len;
00077         if (bitPadding) {
00078             buf[total++] = 0x80U;
00079             while ((total % 16U) != 0U) { buf[total++] = 0U; }
00080         }
00081         for (uint16_t b = 0U; b < total; b += 16U) {
00082             for (uint8_t i = 0U; i < 16U; i++) { iv[i] = (uint8_t)(buf[b + i] ^ iv[i] ^ key[i]); }
00083             memcpy(out + b, iv, 16U);
00084         }
00085         return total;
00086     }
00087
00088     uint16_t aesCbcDecrypt(uint8_t* in, uint16_t len, uint8_t* out,
00089                            const uint8_t* key, uint8_t, uint8_t* iv, bool bitPadding) override {
00090         for (uint16_t b = 0U; b < len; b += 16U) {
00091             for (uint8_t i = 0U; i < 16U; i++) {
00092                 out[b + i] = (uint8_t)(in[b + i] ^ key[i] ^ iv[i]);
00093                 iv[i] = in[b + i];
00094             }
00095         }
00096         uint16_t n = len;
00097         if (bitPadding) {
00098             while ((n > 0U) && (out[n - 1U] == 0U)) { n--; }
00099             if ((n > 0U) && (out[n - 1U] == 0x80U)) { n--; }
00100         }
00101         return n;
00102     }
00103 };
00104
00105 class NullLogger : public CW_Logger {
00106 public:
00107     bool begin(unsigned long) override { return true; }
00108     void print(const __FlashStringHelper*) override { }
00109     void print(const char*) override { }
00110     void print(char) override { }
00111     void print(uint8_t, int) override { }
00112     void print(uint16_t, int) override { }
00113     void print(uint32_t, int) override { }
00114     void print(int, int) override { }
00115     void println() override { }
00116     void println(const __FlashStringHelper*) override { }
00117     void println(const char*) override { }
00118     void println(char) override { }
00119     void println(uint8_t, int) override { }
00120     void println(uint16_t, int) override { }
00121     void println(uint32_t, int) override { }
00122     void println(int, int) override { }
00123 };
00124
00125 class NullPlatform : public CW_Platform {
00126 public:
00127     void sleep_ms(uint32_t) override { }
00128 };
```

© 2026 Cryptnox SA

```
00129
00130 static NullLogger   logger;
00131 static NullPlatform platform;
00132
00133 static void openSession(CW_SecureSession& s) {
00134     s.clear();
00135     memset(s.aesKey, 0x11, sizeof(s.aesKey));
00136     memset(s.macKey, 0x22, sizeof(s.macKey));
00137     memset(s.iv,     0x33, sizeof(s.iv));
00138 }
00139
00140 static bool isCleared(const CW_SecureSession& s) {
00141     uint8_t acc = 0U;
00142     for (uint8_t i = 0U; i < sizeof(s.aesKey); i++) { acc |= s.aesKey[i]; }
00143     for (uint8_t i = 0U; i < sizeof(s.macKey); i++) { acc |= s.macKey[i]; }
00144     return (acc == 0U) && (s.macCounter == 0U);
00145 }
00146
00147 static const uint8_t HDR[] = { 0x80U, 0xCAU, 0x00U, 0x00U };
00148 static const uint8_t DATA[] = { 1U, 2U, 3U };
00149
00150 /****************************************************************
00151  * aesCbcEncrypt(): which failures clear the session (review point 4)
00152  ****************************************************************/
00153
00154 /* Fresh fakes + a v2 channel + an open session for each case. */
00155 struct Rig {
00156     FakeNfc nfc;
00157     FakeCrypto crypto;
00158     CW_SecureChannel ch;
00159     CW_SecureSession s;
00160     Rig() : ch(nfc, logger, crypto, platform) {
00161         CHECK(ch.selectApdu() && (ch.appletMajor() == 2U));
00162         openSession(s);
00163     }
00164     bool send(const uint8_t* data, uint16_t len) {
00165         return ch.aesCbcEncrypt(s, HDR, sizeof(HDR), data, len);
00166     }
00167 };
00168
00169 static void check_session_clearing(void) {
00170     { /* Exhausted counter: cleared, nothing sent. */
00171         Rig r;
00172         r.s.macCounter = UINT32_MAX;
00173         CHECK(!r.send(DATA, sizeof(DATA)));
00174         CHECK(isCleared(r.s) && (r.nfc.sent == 0U));
00175     }
00176     { /* MAC IV derivation failure: cleared, nothing sent. */
00177         Rig r;
00178         r.crypto.failMacIv = true;
00179         CHECK(!r.send(DATA, sizeof(DATA)));
00180         CHECK(isCleared(r.s) && (r.nfc.sent == 0U));
00181     }
00182     { /* MAC computation failure: cleared, nothing sent. */
00183         Rig r;
00184         r.crypto.failMac = true;
00185         CHECK(!r.send(DATA, sizeof(DATA)));
00186         CHECK(isCleared(r.s) && (r.nfc.sent == 0U));
00187     }
00188     { /* Transport failure: cleared. */
00189         Rig r;
00190         CHECK(!r.send(DATA, sizeof(DATA)));
00191         CHECK(isCleared(r.s) && (r.nfc.sent == 1U));
00192     }
00193     { /* Bad response MAC (SW 9000, junk MAC): cleared. */
00194         Rig r;
00195         r.nfc.sendOk = true;
00196         memset(r.nfc.resp, 0xEEU, 32U);
00197         r.nfc.resp[32] = 0x90U;
00198         r.nfc.resp[33] = 0x00U;
00199         r.nfc.respLen = 34U;
00200         CHECK(!r.send(DATA, sizeof(DATA)));
00201         CHECK(isCleared(r.s));
00202     }
00203     { /* Bare error SW (unauthenticated, may be injected): cleared, so the next
00204        * command cannot go out with a stale IV. */
00205         Rig r;
00206         r.nfc.sendOk = true;
00207         r.nfc.resp[0] = 0x6FU;
00208         r.nfc.resp[1] = 0x00U;
00209         r.nfc.respLen = 2U;
00210         CHECK(!r.send(DATA, sizeof(DATA)));
00211         CHECK(isCleared(r.s));
00212     }
00213     { /* Response too short to hold an SW: cleared. */
00214         Rig r;
00215         r.nfc.sendOk = true;
```

© 2026 Cryptnox SA

```
00216         r.nfc.respLen = 1U;
00217         CHECK(!r.send(DATA, sizeof(DATA)));
00218         CHECK(isCleared(r.s));
00219     }
00220     { /* Oversized payload, rejected before anything is sent: session kept. */
00221         static const uint8_t big[CW_USER_DATA_PAGE_SIZE + 1U] = { 0U };
00222         Rig r;
00223         CHECK(!r.send(big, sizeof(big)));
00224         CHECK(!isCleared(r.s) && (r.s.macCounter == 0U) && (r.nfc.sent == 0U));
00225     }
00226 }
00227
00228 /*****************************************************************
00229  * getPublicKey(): key-type allow-list (review point 1)
00230  *****************************************************************/
00231
00232 static const uint8_t PATH[16] = {
00233     0x80U, 0x00U, 0x00U, 0x2CU, 0x80U, 0x00U, 0x01U, 0xF5U,
00234     0x80U, 0x00U, 0x00U, 0x00U, 0x80U, 0x00U, 0x00U, 0x00U
00235 };
00236
00237 /* connect() fails after SELECT (the fake answers nothing else), which leaves the
00238  * wallet's applet version set; the session is then opened by hand. */
00239 static void selectOnly(CryptnoxWallet& wallet, FakeNfc& nfc, CW_SecureSession& s) {
00240     const bool connected = wallet.connect(s);
00241     CHECK(!connected);
00242     nfc.sent = 0U;
00243 }
00244
00245 static bool pathFor(uint8_t keyType) {
00246     return ((keyType & 0x0FU) == 1U) || ((keyType & 0x0FU) == 2U);
00247 }
00248
00249 static void check_pubkey_allow_list(void) {
00250     FakeNfc nfc;
00251     FakeCrypto crypto;
00252     CryptnoxWallet wallet(nfc, logger, crypto, platform);
00253     CW_SecureSession s;
00254     selectOnly(wallet, nfc, s);
00255
00256     uint8_t pk[65] = { 0U };
00257     uint16_t pkLen = 0U;
00258
00259     /* Refused locally: derive-and-make-current, PIN-less, unknown curve. */
00260     const uint8_t refused[] = { 0x02U, 0x12U, 0x22U, 0x03U, 0x13U, 0x30U, 0x31U, 0xF0U };
00261     for (uint8_t i = 0U; i < sizeof(refused); i++) {
00262         const uint8_t t = refused[i];
00263         openSession(s);
00264         CHECK(!wallet.getPublicKey(s, t, pathFor(t) ? PATH : NULL, pathFor(t) ? sizeof(PATH) : 0U,
00265                                    pk, sizeof(pk), pkLen));
00266         CHECK((nfc.sent == 0U) && !isCleared(s));
00267     }
00268
00269     /* Allowed: reaches the card with P1 = keyType (the fake then fails the exchange). */
00270     const uint8_t allowed[] = { CW_SIGN_CURR_K1, CW_SIGN_CURR_R1, CW_SIGN_DERIVE_K1,
00271                                 CW_SIGN_DERIVE_R1, CW_SIGN_CURR_ED25519, CW_SIGN_DERIVE_ED25519 };
00272     for (uint8_t i = 0U; i < sizeof(allowed); i++) {
00273         const uint8_t t = allowed[i];
00274         openSession(s);
00275         nfc.sent = 0U;
00276         (void)wallet.getPublicKey(s, t, pathFor(t) ? PATH : NULL, pathFor(t) ? sizeof(PATH) : 0U,
00277                                   pk, sizeof(pk), pkLen);
00278         CHECK((nfc.sent == 1U) && (nfc.last[1] == 0xC2U) && (nfc.last[2] == t));
00279     }
00280 }
00281
00282 /*****************************************************************
00283  * Ed25519 needs applet 2.0+ (review point 2)
00284  *****************************************************************/
00285
00286 static CW_SignResult signWith(CryptnoxWallet& wallet, CW_SecureSession& s, uint8_t keyType) {
00287     static const uint8_t msg[32] = { 0x01U };
00288     CW_SignRequest req(s, keyType, CW_SIGN_SIG_ECDSA_LOW_S, CW_SIGN_WITH_PIN);
00289     req.hash = msg;
00290     req.hashLength = sizeof(msg);
00291     memcpy(req.pin, "000000000", 9U);
00292     return wallet.sign(req);
00293 }
00294
00295 static void check_ed25519_applet_gate(void) {
00296     /* Applet 1.6.x: refused before anything is sent. */
00297     {
00298         uint8_t pk[65] = { 0U };
00299         uint16_t pkLen = 0U;
00300         FakeNfc nfc;
00301         nfc.appletMajor = 1U;
00302         FakeCrypto crypto;
```

© 2026 Cryptnox SA

```
00303         CryptnoxWallet wallet(nfc, logger, crypto, platform);
00304         CW_SecureSession s;
00305         selectOnly(wallet, nfc, s);
00306 
00307         openSession(s);
00308         CHECK(!wallet.getPublicKey(s, CW_SIGN_CURR_ED25519, NULL, 0U, pk, sizeof(pk), pkLen));
00309         CHECK(nfc.sent == 0U);
00310 
00311         openSession(s);
00312         CHECK(signWith(wallet, s, CW_SIGN_CURR_ED25519).errorCode == CW_SIGN_UNSUPPORTED_KEY_TYPE);
00313         CHECK(nfc.sent == 0U);
00314 
00315         /* k1 is not affected. */
00316         openSession(s);
00317         (void)signWith(wallet, s, CW_SIGN_CURR_K1);
00318         CHECK(nfc.sent == 1U);
00319     }
00320 
00321     /* Applet 2.x: sent, with P2 forced to EdDSA whatever signatureType says. */
00322     {
00323         FakeNfc nfc;
00324         FakeCrypto crypto;
00325         CryptnoxWallet wallet(nfc, logger, crypto, platform);
00326         CW_SecureSession s;
00327         selectOnly(wallet, nfc, s);
00328 
00329         openSession(s);
00330         CHECK(signWith(wallet, s, CW_SIGN_CURR_ED25519).errorCode != CW_SIGN_UNSUPPORTED_KEY_TYPE);
00331         CHECK((nfc.sent == 1U) && (nfc.last[1] == 0xC0U) &&
00332                (nfc.last[2] == CW_SIGN_CURR_ED25519) && (nfc.last[3] == CW_SIGN_SIG_EDDSA));
00333     }
00334 }
00335 
00336 int main(void) {
00337     check_session_clearing();
00338     check_pubkey_allow_list();
00339     check_ed25519_applet_gate();
00340     printf("test_pr28: OK\n");
00341     return 0;
00342 }
```

© 2026 Cryptnox SA